

SIGA MAS DOCUMENTATION CODE



Controllers.....	4
Agenda.php.....	4
Archivos.php	7
Autos.php.....	9
Bitacora.php.....	23
Cafeteria.php.....	24
Clientes.php	28
Compras.php.....	30
Configuracion.php.....	94
Correo.php	108
Cotizaciones.php.....	111
Descargas.php.....	172
Documentacion.php.....	174
Empresas.php.....	174
Equipos.php	204
Facturas.php.....	213
Inicio.php	249
Login.php.....	252
Logistica.php	256
Ordenes_compra.php	286
Ordebes_trabajo.php.....	343
Privilegios.php.....	393
Recursos.php.....	398
Reloj_checador.php.....	402
Requerimientos.php	405
Seguridad.php.....	410
Servicios.php.....	413
Solicitudes_pago.php.....	417

Test.php.....	432
Tickets_AT.php.....	436
Tickets_ED.php.....	448
Tickets_IT.php.....	457
Tickets.php	475
Toolcrib.php	484
Usuarios.php	505

Controllers

Agenda.php

```
<?php
defined('BASEPATH') OR exit('No direct script access allowed');

// Controlador Agenda para manejar eventos y reuniones
class Agenda extends CI_Controller {

    // Constructor: carga modelos y librerías necesarias
    function __construct() {
        parent::__construct();
        $this->load->model('agenda_model','Modelo');    // Modelo para agenda
        $this->load->model('tickets_IT_model', 'ITModelo'); // Modelo para tickets IT
        $this->load->library('correos');                // Librería para envío de correos
        $this->load->model('conexion_model', 'Conexion'); // Modelo para conexiones DB
    }

    // Página principal: muestra el calendario
    public function index() {
        $this->load->view('header');                // Carga el header
        $this->load->view('agenda/calendario');      // Carga la vista del calendario
    }

    // Obtiene eventos para mostrarlos en el calendario
    function getEventos()
    {
        $data = $this->Modelo->getEventos(); // Obtiene eventos del modelo
        echo json_encode($data);           // Devuelve datos en formato JSON
    }

    // Crea un nuevo evento o reunión
    function crearEvento()
    {
        $reunion = $_POST['reunion']; // Determina si es reunión (1) o evento normal

        // Datos básicos del evento
```

```

$datos = array(
    'usuario' => $this->session->id,    // ID del usuario de sesión
    'titulo' => $_POST['titulo'],       // Título del evento
    'inicia' => $_POST['inicia'],       // Fecha/hora de inicio
    'termina' => $_POST['termina'],     // Fecha/hora de fin
    'descripcion' => $_POST['descripcion'], // Descripción
    'sala' => $_POST['sala'],           // Sala asignada
);

// Si es una reunión (valor 1)
if ($reunion == 1) {

    $datos['equipo'] = 1;               // Marca como reunión de equipo
    $datos['correos'] = $_POST['tags_1']; // Correos de invitados

    echo $this->Modelo->insertEvent($datos); // Inserta el evento

    // Crea un ticket asociado a la reunión
    $data = array(
        'usuario' => $this->session->id,
        'tipo' => 'Reunion',           // Tipo de ticket
        'titulo' => $_POST['titulo'] . ' -- Fecha/Hora: ' . $_POST['inicia'] . ' hasta ' .
$_POST['termina'],
        'descripcion' => $_POST['descripcion'] . ' -- Fecha/Hora: ' . $_POST['inicia'] . ' hasta ' .
$_POST['termina'] . ' agregar correos: ' . $_POST['tags_1'],
        'estatus' => 'ABIERTO',        // Estado inicial
        'cierre' => '0',               // No cerrado
    );

    $last_id = $this->ITModelo->crear_ticket($data); // Crea ticket y obtiene ID

    // Prepara datos para correo de notificación
    $datosCorreo = array(
        'id' => $last_id,
        'prefijo' => 'IT',
        'titulo' => $data['titulo'],
        'fecha' => date('d/m/Y h:i A'),
        'usuario' => $this->session->nombre,

```

```

        'correo' => $this->session->correo,
    );

    $this->correos->creacionTicket($datosCorreo); // Envía correo

    // Redirecciona a la página de archivos del ticket
    redirect(base_url('tickets_IT/archivos/') . $last_id);
} else {
    // Si no es reunión, solo inserta el evento normal
    echo $this->Modelo->insertEvent($datos);
}
}

// Elimina un evento
function borrarEvento()
{
    $id = $_POST['id']; // ID del evento a borrar

    // Intenta borrar y devuelve resultado (1=éxito, 0=fallo)
    if($this->Modelo->deleteEvent($id) > 0)
    {
        echo "1";
    }
    else {
        echo "0";
    }
}

// Valida disponibilidad de sala en un horario específico
function validacion()
{
    $inicia=$this->input->post('inicia'); // Hora de inicio
    $termina=$this->input->post('termina'); // Hora de fin
    $sala=$this->input->post('sala'); // Sala a verificar

    // Consulta eventos que se solapen con el horario solicitado
    $res = $this->Conexion->consultar("SELECT * FROM `agenda` WHERE inicia <= '". $termina ."
    AND termina >= '". $inicia ." and sala='". $sala );

```

```

if($res)
{
    echo json_encode($res); // Devuelve eventos conflictivos
}
}
}

```

Archivos.php

```
<?php defined('BASEPATH') OR exit('No direct script access allowed');
```

```
// Controlador para manejo de archivos (PDF, documentos y descargas)
```

```
class Archivos extends CI_Controller {
```

```
    // Constructor: carga el modelo de conexión a base de datos
```

```
    function __construct() {
```

```
        parent::__construct();
```

```
        $this->load->model('conexion_model', 'Conexion');
```

```
    }
```

```
    // Muestra un archivo PDF en línea desde la base de datos
```

```
    function pdf() {
```

```
        // Verifica que existan los parámetros requeridos
```

```
        if(isset($_POST['tabla']) && isset($_POST['id'])) {
```

```
            $tabla = $this->input->post("tabla"); // Tabla donde está almacenado
```

```
            $id = $this->input->post("id"); // ID del registro
```

```
            // Obtiene el archivo y su nombre desde la base de datos
```

```
            $res = $this->Conexion->consultar("SELECT nombre, archivo from $tabla where id = $id",
TRUE);
```

```
            $nombre = $res->nombre; // Nombre original del archivo
```

```
            $file = $res->archivo; // Contenido binario del PDF
```

```
            // Configura headers para visualización en navegador
```

```
            header('Content-type: application/pdf');
```

```
            header('Content-Disposition: inline; filename="' . $nombre . '');
```

```
            header('Content-Transfer-Encoding: binary');
```

```
            header('Accept-Ranges: bytes');
```

```

        echo $file; // Imprime el contenido del PDF
    } else {
        // Redirecciona si faltan parámetros
        redirect(base_url('inicio'));
    }
}

// Muestra un documento genérico (similar a pdf() pero con campo personalizable)
function documento() {
    // Verifica parámetros requeridos (incluyendo el campo específico)
    if(isset($_POST['tabla']) && isset($_POST['id']) && isset($_POST['campo'])) {
        $id = $this->input->post("id"); // ID del registro
        $tabla = $this->input->post("tabla"); // Tabla de origen
        $campo = $this->input->post("campo"); // Campo que contiene el archivo

        // Obtiene el archivo desde la base de datos
        $res = $this->Conexion->consultar("SELECT $campo from $tabla where id = $id",
TRUE);
        $file = $res->$campo; // Contenido binario del documento

        // Configura headers para visualización en navegador
        header('Content-type: application/pdf');
        header('Content-Disposition: inline; filename="" . $campo . ""');
        header('Content-Transfer-Encoding: binary');
        header('Accept-Ranges: bytes');

        echo $file; // Imprime el contenido del documento
    } else {
        // Redirecciona si faltan parámetros
        redirect(base_url('inicio'));
    }
}

// Descarga un archivo XML desde la base de datos
function descarga() {
    // Verifica parámetros requeridos
    if(isset($_POST['tabla']) && isset($_POST['id']) && isset($_POST['campo'])) {

```



```

        $id = $this->input->post("id");    // ID del registro
        $tabla = $this->input->post("tabla"); // Tabla de origen
        $campo = $this->input->post("campo"); // Campo que contiene el XML

        // Obtiene el archivo XML desde la base de datos
        $res = $this->Conexion->consultar("SELECT $campo from $tabla where id = $id",
TRUE);
        $file = $res->$campo; // Contenido del XML

        // Configura headers para descarga
        header('Content-type: text/xml');
        echo $file; // Imprime el contenido XML
    } else {
        // Redirecciona si faltan parámetros
        redirect(base_url('inicio'));
    }
}
}

```

Autos.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');
// Controlador para gestión de vehículos (autos)
class Autos extends CI_Controller {

    // Constructor: carga modelos y librerías necesarias
    function __construct() {
        parent::__construct();
        $this->load->model('autos_model','Modelo');    // Modelo principal de autos
        $this->load->model('privilegios_model');        // Modelo de privilegios
        $this->load->model('usuarios_model');           // Modelo de usuarios
        $this->load->model('descargas_model');          // Modelo para descargas
        $this->load->library('correos');                // Librería de envío de correos
        $this->load->model('conexion_model', 'Conexion'); // Modelo de conexión a DB
    }

    // Muestra el catálogo principal de vehículos

```

```

function index() {
    $datos['autos'] = $this->Modelo->getCatalogo(); // Obtiene listado de autos
    $datos['titulo'] = "Autos";                // Título para la vista
    $this->load->view('header');                // Carga el header
    $this->load->view('autos/catalogo',$datos); // Carga la vista principal
}

// Muestra detalles de un vehículo específico
function ver($idauto) {
    // Actualiza responsable si se envió el formulario
    $responsable = $this->input->post('responsable');
    if(isset($responsable)) {
        $this->Modelo->updateAuto($idauto, array('responsable' => $responsable));
    }

    // Obtiene datos del auto y usuarios disponibles
    $datos['auto'] = $this->Modelo->getAuto($idauto);
    $datos['responsable'] = $this->usuarios_model->getUsuario($datos['auto']->responsable);
    $datos['usuarios'] = $this->usuarios_model->getUsuarios();

    $this->load->view('header');
    $this->load->view('autos/ver_auto', $datos);
}

// Muestra vista de seguimiento GPS
function gps() {
    $datos['autos'] = $this->Modelo->gpsAutos(); // Obtiene autos con GPS
    $datos['titulo'] = "GPS";                // Título para la vista
    $this->load->view('header');
    $this->load->view('autos/gps',$datos);
}

// Muestra formulario para registrar GPS de un vehículo
function alta_gps($idauto) {
    $datosCoche = $this->Modelo->getAuto($idauto);
    $datos['auto_foto'] = $datosCoche->foto;
    $datos['auto'] = $datosCoche->id;
    $datos['auto_marca'] = $datosCoche->marca;
}

```

```

    $datos['auto_combustible'] = $datosCoche->combustible;
    $datos['auto_placas'] = $datosCoche->placas;
    $datos['gps'] = $this->Modelo->getGPS($idauto);
    $datos['titulo'] = "Autos";
    $this->load->view('header');
    $this->load->view('autos/alta_gps',$datos);
}

// Muestra formulario para editar vehículo
function editar($idauto) {
    $datos['auto'] = $this->Modelo->getAutos($idauto);
    $datos['usuarios'] = $this->privilegios_model->listadoJefes();
    $this->load->view('header');
    $this->load->view('autos/editar',$datos);
}

// Muestra vista de uso de vehículos
function uso_autos() {
    $this->load->view('header');
    $this->load->view('autos/uso_autos');
}

// Muestra formulario para registrar nuevo vehículo
function alta_autos() {
    $datos['usuarios'] = $this->privilegios_model->listadoJefes();
    $this->load->view('header');
    $this->load->view('autos/alta_autos', $datos);
}

// Muestra revisiones de un vehículo específico
function revisiones_ANT($auto) {
    $datos['auto'] = $auto;
    $datos['rev'] = $this->Modelo->getRevisiones($auto);
    $this->load->view('header');
    $this->load->view('autos/revisiones', $datos);
}

// Muestra opciones de revisiones

```

```

function revisiones() {
    $this->load->view('header');
    $this->load->view('autos/revisiones_opciones');
}

// Muestra revisiones pendientes
function revisiones_pendientes() {
    $datos['autos'] = $this->Modelo->getRevPendientes();
    $datos['titulo'] = "Revisiones Pendientes";
    $this->load->view('header');
    $this->load->view('autos/catalogo',$datos);
}

// Muestra revisiones programadas para hoy
function revisiones_hoy() {
    $datos['autos'] = $this->Modelo->getRevHoy();
    $datos['titulo'] = "Revisiones de Hoy";
    $this->load->view('header');
    $this->load->view('autos/catalogo',$datos);
}

// Muestra próximos mantenimientos
function proximos_mttos() {
    $head['url_actual'] = current_url();
    $datos['autos'] = $this->Modelo->getProxMttos();
    $datos['titulo'] = "Proximos Mantenimientos";
    $this->load->view('header', $head);
    $this->load->view('autos/proximos_mttos',$datos);
}

// Muestra formulario para registrar revisión
function registrar_revision($auto) {
    $head['url_actual'] = current_url();
    $datos['iu'] = uniqid();
    $datos['auto'] = $auto;
    $res = $this->Modelo->getMotorPlacasKm($auto);
    $datos['combustible'] = $res->combustible;
    $datos['placas'] = $res->placas;
}

```

```

    $datos['kilometraje'] = $res->kilometraje;
    $this->load->view('header', $head);
    $this->load->view('autos/registrar_revision',$datos);
}

// Muestra checklist de revisión en PDF
function ver_checklist($id) {
    $res = $this->Modelo->getChecklist($id);
    $pdf = $res->checklist;
    $datos['data'] = $pdf;
    $this->load->view('pdf', $datos);
}

// Muestra foto de hallazgo específico
function hallazgo_foto($id_foto) {
    $photo = $this->Modelo->getHallazgoFoto($id_foto);
    if($photo) {
        header("Content-type: image/png");
        echo $photo->foto;
    } else {
        echo "ERROR";
    }
}

// Muestra hallazgos de una revisión específica
function hallazgos($id_rev) {
    // Datos de la revisión
    $datosRevision = $this->Modelo->getRevision($id_rev);
    $auto = $datosRevision->auto;
    $datos['rev_id'] = $datosRevision->id;
    $datos['rev_fecha'] = $datosRevision->fecha;
    $datos['rev_User'] = $datosRevision->User;
    $datos['rev_kilometraje'] = $datosRevision->kilometraje;
    $datos['rev_combustible'] = $datosRevision->combustible;
    $datos['rev_placas'] = $datosRevision->placas;
    $datos['rev_vencimiento_poliza'] = $datosRevision->vencimiento_poliza;
    $datos['rev_ecologico'] = $datosRevision->vencimiento_ecologico;

```

```

// Datos del vehículo
$datosCoche = $this->Modelo->getAuto($auto);
$datos['auto_foto'] = $datosCoche->foto;
$datos['auto_marca'] = $datosCoche->marca;
$datos['auto_combustible'] = $datosCoche->combustible;
$datos['auto_placas'] = $datosCoche->placas;

// Hallazgos encontrados
$datos['carroceria'] = $this->Modelo->getHallazgosCarroceria($id_rev);
$datos['otros'] = $this->Modelo->getHallazgosOtros($id_rev);

$this->load->view('header');
$this->load->view('autos/hallazgos', $datos);
}

// Guarda fotos temporales de hallazgos
function temp_photos() {
    $datos['iu'] = $this->input->post('iu');
    $datos['tipo'] = 'REVISION';
    $datos['usuario'] = $this->session->id;
    $datos['objeto'] = $this->input->post('auto');
    $datos['texto'] = $this->input->post('texto');
    $fotoB64 = $this->input->post('archivo');
    $datos['archivo'] = base64_decode($fotoB64);

    $res = $this->Modelo->saveTempPhotos($datos);
    if($res > 0) {
        $respuesta = array('id' => $res, 'file' => $fotoB64);
        echo json_encode($respuesta);
    }
}

// Elimina fotos temporales
function delete_temp_photos() {
    $id_temp = $this->input->post('id');
    $res = $this->Modelo->deleteTempPhotos($id_temp);
    if($res) {
        echo "1";
    }
}

```

```

    }
}

// Procesa el registro de una revisión completa
function registrarRev() {
    $ACIERTOS = array();
    $ERRORES = array();
    $VARIABLES = array();

    // Datos básicos de la revisión
    $descripcion = $this->input->post('texto');
    $usuario = $this->session->nombre;
    $AUTO = $this->input->post('auto');
    $IU = $this->input->post('iu');

    // Datos del vehículo
    $KILOMETRAJE = $this->input->post('kilometraje');
    $COMBUSTIBLE = $this->input->post('combustible');
    $PLACAS = $this->input->post('placas');
    $timePoliza = strtotime($this->input->post('vencimientoPoliza'));
    $VENCIMIENTO_POLIZA = date("Y-m-d", $timePoliza);
    $timeEco = strtotime($this->input->post('vencimientoEcologico'));
    $VENCIMIENTO_ECOLOGICO = date("Y-m-d", $timeEco);

    // Recolección de variables de revisión
    $VARIABLES['Kilometraje'] = array('Kilometraje' => $this->input->post('kilometraje'),
    'comentario' => $this->input->post('com_kilometraje'));
    $VARIABLES['Aceite del motor'] = array('Aceite del motor' => $this->input-
    >post('aceiteMotor'), 'comentario' => $this->input->post('com_aceiteMotor'));
    $VARIABLES['Condiciones del aceite'] = array('Condiciones del aceite' => $this->input-
    >post('condicionesAceite'), 'comentario' => $this->input->post('com_condicionesAceite'));
    $VARIABLES['Limpia parabrisas delantero'] = array('Limpia parabrisas delantero' => $this-
    >input->post('llpbDelantero'), 'comentario' => $this->input->post('com_llpbDelantero'));
    $VARIABLES['Limpia parabrisas trasero'] = array('Limpia parabrisas trasero' => $this->input-
    >post('llpbTrasero'), 'comentario' => $this->input->post('com_llpbTrasero'));
    $VARIABLES['Deposito de refrigerante'] = array('Deposito de refrigerante' => $this->input-
    >post('refrigeranteDeposito'), 'comentario' => $this->input->post('com_refrigeranteDeposito'));

```

```

$VARIABLES['Radiador'] = array('Radiador' => $this->input->post('refrigeranteRadiador'),
'comentario' => $this->input->post('com_refrigeranteRadiador'));
$VARIABLES['Liquido de frenos'] = array('Liquido de frenos' => $this->input-
>post('liquidoFrenos'), 'comentario' => $this->input->post('com_liquidoFrenos'));
$VARIABLES['Direccion hidraulica'] = array('Direccion hidraulica' => $this->input-
>post('direccionH'), 'comentario' => $this->input->post('com_direccionH'));
$VARIABLES['Direccional izquierda'] = array('Direccional izquierda' => $this->input-
>post('direccionalIzq'), 'comentario' => $this->input->post('com_direccionalIzq'));
$VARIABLES['Direccional derecha'] = array('Direccional derecha' => $this->input-
>post('direccionalDer'), 'comentario' => $this->input->post('com_direccionalDer'));
$VARIABLES['Lampara izquierda'] = array('Lampara izquierda' => $this->input-
>post('lamparaIzq'), 'comentario' => $this->input->post('com_lamparaIzq'));
$VARIABLES['Lampara derecha'] = array('Lampara derecha' => $this->input-
>post('lamparaDer'), 'comentario' => $this->input->post('com_lamparaDer'));
$VARIABLES['Lampara trasera izquierda'] = array('Lampara trasera izquierda' => $this-
>input->post('traseraIzq'), 'comentario' => $this->input->post('com_traseraIzq'));
$VARIABLES['Lampara trasera derecha'] = array('Lampara trasera derecha' => $this->input-
>post('traseraDer'), 'comentario' => $this->input->post('com_traseraDer'));
$VARIABLES['Lampara de emergencia'] = array('Lampara de emergencia' => $this->input-
>post('emergencia'), 'comentario' => $this->input->post('com_emergencia'));
$VARIABLES['Reversa'] = array('Reversa' => $this->input->post('reversa'), 'comentario' =>
$this->input->post('com_reversa'));
$VARIABLES['Tapiceria'] = array('Tapiceria' => $this->input->post('tapiceria'), 'comentario'
=> $this->input->post('com_tapiceria'));
$VARIABLES['Controles'] = array('Controles' => $this->input->post('controles'), 'comentario'
=> $this->input->post('com_controles'));
$VARIABLES['Kit de herramientas'] = array('Kit de herramientas' => $this->input-
>post('kitHerramientas'), 'comentario' => $this->input->post('com_kitHerramientas'));
$VARIABLES['Tapetes'] = array('Tapetes' => $this->input->post('tapetes'), 'comentario' =>
$this->input->post('com_tapetes'));
$VARIABLES['Placa delantera'] = array('Placa delantera' => $this->input-
>post('placaDelantera'), 'comentario' => $this->input->post('com_placaDelantera'));
$VARIABLES['Placa trasera'] = array('Placa trasera' => $this->input->post('placaTrasera'),
'comentario' => $this->input->post('com_placaTrasera'));
$VARIABLES['Tarjeta de combustible'] = array('Tarjeta de combustible' => $this->input-
>post('tarjetaCombustible'), 'comentario' => $this->input->post('com_tarjetaCombustible'));
$VARIABLES['Tarjeta de circulacion'] = array('Tarjeta de circulacion' => $this->input-
>post('tarjetaCirculacion'), 'comentario' => $this->input->post('com_tarjetaCirculacion'));

```



```
$VARIABLES['Poliza'] = array('Poliza' => $this->input->post('poliza'), 'comentario' => $this->input->post('com_poliza'));
```

```
// Identificación de fallas
```

```
$FALLAS = array();
```

```
foreach ($VARIABLES as $key => $value) {  
    if($value[$key] == "NG") {  
        $FALLAS[$key] = $value['comentario'];  
    }  
}
```

```
// Guarda en base de datos
```

```
$datos = array(  
    'auto' => $AUTO,  
    'usuario' => $this->session->id,  
    'kilometraje' => $KILOMETRAJE,  
    'combustible' => $COMBUSTIBLE,  
    'placas' => $PLACAS,  
    'vencimiento_poliza' => $VENCIMIENTO_POLIZA,  
    'vencimiento_ecologico' => $VENCIMIENTO_ECOLOGICO  
);
```

```
$db_response = $this->Modelo->saveChecklist($datos);
```

```
if($db_response['EXITO']) {  
    // Actualiza datos del vehículo  
    $AutoData = array(  
        'kilometraje' => $KILOMETRAJE,  
        'vencimiento_poliza' => $VENCIMIENTO_POLIZA,  
        'vencimiento_ecologico' => $VENCIMIENTO_ECOLOGICO  
    );  
    $this->Modelo->updateAuto($AUTO, $AutoData);
```

```
// Registra hallazgos si existen
```

```
if(isset($descripcion)) {  
    $arreglo = array('revision' => $db_response['ID'], 'iu' => $IU);  
    $this->Modelo->saveHallazgosCarroceria($arreglo);  
}
```

```

// Registra fallas encontradas
if(isset($FALLAS)) {
    foreach ($FALLAS as $key => $value) {
        $arreglo = array('revision' => $db_response['ID'], 'tipo' => $key, 'descripcion' =>
$value);
        $this->Modelo->saveHallazgos($arreglo);
    }

    $acierto = array('titulo' => 'Registro de Revisión', 'detalle' => 'Se ha registrado Revisión
con Éxito');
    array_push($ACIERTOS, $acierto);
}

// Notifica si está próximo el mantenimiento
$prox = $this->Modelo->getMotorPlacasKm($AUTO);
if(($prox->kilometraje + 200) >= $prox->proximo_mtto) {
    $Auto = $this->Modelo->getAuto($AUTO);
    $Resp = $this->Modelo->getResponsable($Auto->responsable);

    $data['auto'] = $Auto->fabricante . ' ' . $Auto->marca . ' ' . $Auto->modelo;
    $data['placas'] = $Auto->placas;
    $data['serie'] = $Auto->serie;
    $data['kmActual'] = number_format($KILOMETRAJE);
    $data['proxMtto'] = number_format($Auto->proximo_mtto);

    if($Resp) {
        $data['correoResponsable'] = $Resp->correo;
        $this->correos->proxMtto200($data);
    }
} else {
    $error = array('titulo' => 'ERROR', 'detalle' => 'Error al registrar revisión');
    array_push($ERRORES, $error);
}

$this->session->aciertos = $ACIERTOS;
$this->session->errores = $ERRORES;

```

```

    redirect(base_url('inicio'));
}

// Obtiene lista de vehículos para AJAX
function ajax_getAutos() {
    $res = $this->Conexion->consultar("SELECT id, marca, placas from autos order by marca");
    if($res) {
        echo json_encode($res);
    }
}

// Obtiene eventos de vehículos para calendario
function ajax_getEventos() {
    $res = $this->Conexion->consultar("SELECT BA.id, BA.inicio as start, BA.final as end,
BA.usuario, BA.usuarios, BA.auto, concat(A.placas, ' - ', A.marca) as title, BA.destino, BA.visita,
BA.rsi, BA.comentarios, BA.equipo, BA.color from bitacora_autos BA inner join autos A on A.id =
BA.auto");
    if($res) {
        echo json_encode($res);
    }
}

// Crea evento para vehículo (AJAX)
function ajax_crearEvento() {
    $data = json_decode($this->input->post('data'));
    $data->usuario = $this->session->id;
    $f = array('fecha' => 'CURRENT_TIMESTAMP()');
    $res = $this->Conexion->insertar('bitacora_autos', $data, $f);
    echo $res;
}

// Elimina evento de vehículo (AJAX)
function ajax_borrarEvento() {
    $where['id'] = $this->input->post('id');
    if($this->Conexion->eliminar('bitacora_autos', $where)) {
        echo "1";
    }
}

```

```

// Muestra foto de vehículo
function photo($id) {
    $res = $this->Conexion->consultar("SELECT ifnull(A.foto, '') as Photo from autos A where
A.id = $id", TRUE);
    header("Content-type: image/png");
    echo $res->Photo;
}

// Registra o actualiza GPS de vehículo
function registrarGPS() {
    $auto = intval($this->input->post('idAuto'));
    $res = $this->Conexion->consultar("SELECT * from gpsAutos where idAuto = ".$auto);

    $data = array(
        'idAuto' => $auto,
        'marca' => $this->input->post('marca'),
        'imei' => $this->input->post('imei'),
        'modelo' => $this->input->post('modelo'),
        'chip' => $this->input->post('chip'),
        'telefono' => $this->input->post('nchip'),
    );

    if($res) {
        $id_inserted = $this->Modelo->updateGps($auto, $data);
    } else {
        $id_inserted = $this->Modelo->registrar_GPS($data);
    }

    redirect(base_url('autos/alta_gps/'.$auto));
}

// Registra nuevo vehículo
function registrar() {
    $foto = file_get_contents($_FILES['foto']['tmp_name']);
    $poliza = file_get_contents($_FILES['poliza']['tmp_name']);

    $data = array(

```

```

'serie' => $this->input->post('serie'),
'fabricante' => $this->input->post('fabricante'),
'marca' => $this->input->post('marca'),
'modelo' => $this->input->post('modelo'),
'combustible' => $this->input->post('combustible'),
'kilometraje' => $this->input->post('kilometraje'),
'placas' => $this->input->post('placas'),
'no_poliza' => $poliza,
'vencimiento_poliza' => $this->input->post('vencimiento_poliza'),
'vencimiento_ecologico' => $this->input->post('vencimiento_ecologico'),
'tarjeta_combustible' => $this->input->post('tarjeta_combustible'),
'foto' => $foto,
'responsable' => $this->input->post('responsable'),
);

$id_inserted = $this->Modelo->registrar_auto($data);
redirect(base_url('autos'));
}

```

// Actualiza datos de vehículo

```

function updateAuto() {
    $idauto = $this->input->post('idAuto');
    $data = array(
        'serie' => $this->input->post('serie'),
        'fabricante' => $this->input->post('fabricante'),
        'marca' => $this->input->post('marca'),
        'modelo' => $this->input->post('modelo'),
        'placas' => $this->input->post('placas'),
        'vencimiento_ecologico' => $this->input->post('vencimiento_ecologico'),
        'tarjeta_combustible' => $this->input->post('tarjeta_combustible'),
        'vencimiento_poliza' => $this->input->post('vencimiento_poliza'),
        'activo' => $this->input->post('activo'),
        'responsable' => $this->input->post('responsable'),
    );

    $this->Modelo->updateAuto($idauto, $data);
    redirect(base_url('autos/'));
}

```

```
// Obtiene vehículos filtrados (AJAX)
```

```
function ajaxgetAutos() {
```

```
    $activo = $this->input->post('activo');
```

```
    $parametro = $this->input->post('parametro');
```

```
    $texto = $this->input->post('texto');
```

```
    $query = "SELECT A.id, A.fabricante, A.marca, A.serie, A.placas, A.responsable, A.modelo,
A.activo,ifnull(concat(U.nombre, ' ', U.paterno), 'N/A') as Responsable, (SELECT max(fecha) from
autos_revisiones where auto=A.id) as Ultrev, (SELECT max(id) from autos_revisiones where
auto=A.id) as IdUltrev FROM autos A LEFT JOIN usuarios U ON A.responsable = U.id where 1=1
";
```

```
    if($activo == "0") {
```

```
        $query .= " and A.activo = '1'";
```

```
    } else {
```

```
        $query .= " and A.activo = '0'";
```

```
    }
```

```
    if(!empty($texto)) {
```

```
        if($parametro == 'placas') {
```

```
            $query .= " and A.placas = '" . $texto . "'";
```

```
        } elseif($parametro == 'serie') {
```

```
            $query .= " and A.serie = '" . $texto . "'";
```

```
        }
```

```
    }
```

```
    $res = $this->Conexion->consultar($query);
```

```
    if($res) {
```

```
        echo json_encode($res);
```

```
    }
```

```
}
```

```
// Muestra archivo de póliza
```

```
function getFile($id) {
```

```
    $row = $this->descargas_model->getFile($id, 'autos');
```

```
    $file = $row->no_poliza;
```

```
    header('Content-type: application/pdf');
```

```

        header('Content-Transfer-Encoding: binary');
        header('Accept-Ranges: bytes');
        echo $file;
    }
    // Sube foto de vehículo
    function uploadFoto() {
        $id = $this->input->post('id');
        $datos['foto'] = file_get_contents($_FILES['file']['tmp_name']);
        $this->Modelo->updateAuto($id,$datos);
    }

    // Sube póliza de vehículo
    function uploadPoliza() {
        $id = $this->input->post('id');
        $datos['no_poliza'] = file_get_contents($_FILES['file']['tmp_name']);
        if(!$this->Modelo->updateAuto($id,$datos)) {
            trigger_error("Error al subir archivo", E_USER_ERROR);
        }
    }
}

```

Bitacora.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');
// Controlador Bitacora para gestionar registros de software
class Bitacora extends CI_Controller {
    // Constructor del controlador
    function __construct() {
        parent::__construct();
    }
    // Página principal: carga la vista de la bitácora de software
    public function index() {
        $this->load->view('header');          // Carga el encabezado común
        $this->load->view('bitacora/software'); // Carga la vista específica de bitácora de
software
    }

    // Obtiene los registros de la bitácora (vía Ajax)

```

```

function ajax_getLog() {
    $res = $this->Conexion->consultar("SELECT * from bitacora_software"); // Consulta todos
    los registros
    echo json_encode($res); // Devuelve los datos en formato JSON
}
}

```

Cafeteria.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');
// Controlador Cafeteria para gestionar incidencias relacionadas con el área de cafetería
class Cafeteria extends CI_Controller {

    // Constructor: carga modelos y librerías necesarias
    function __construct() {
        parent::__construct();
        $this->load->model('cafeteria_model','Modelo'); // Modelo principal para cafetería
        $this->load->library('correos'); // Librería para enviar correos
        $this->load->library('AOS_funciones'); // Librería adicional (propietaria)
    }

    // Carga la vista principal para generar tickets de cafetería
    public function generar() {
        $this->load->view('header'); // Encabezado
        $this->load->view('cafeteria/cafeteria'); // Vista principal del formulario
        // $this->load->view('test/chat'); // Vista de prueba (comentada)
    }

    // Registra un nuevo ticket
    function registrar() {
        $data = array(
            'id_user' => $this->session->id, // Usuario que registra
            'tipo' => $this->input->post('opCategoria'), // Categoría seleccionada
            'titulo' => $this->input->post('titulo'), // Título del ticket
            'descripcion' => $this->input->post('descripcion'), // Descripción del problema
            'estatus' => 'ABIERTO', // Estado inicial
            'cierre' => '0', // Aún no cerrado
            'fecha_incidencia' => $this->input->post('fecha_incidencia'), // Fecha de la incidencia

```



```

);

$last_id = $this->Modelo->crear_comentario($data);           // Guarda el ticket y
obtiene ID

// Datos para notificación por correo
$datosCorreo['id'] = $last_id;
$datosCorreo['prefijo'] = substr($this->router->fetch_class(), 8);
$datosCorreo['titulo'] = $data['titulo'];
$datosCorreo['fecha'] = date('d/m/Y h:i A');
$datosCorreo['usuario'] = $this->session->nombre;
$datosCorreo['correo'] = $this->session->correo;
$datosCorreo['categoria'] = $this->input->post('opCategoria');
$datosCorreo['fecha_incidencia'] = $this->input->post('fecha_incidencia');

// $this->correos->comentarios_cafeteria($datosCorreo); // Envío de correo (comentado)

redirect(base_url('cafeteria/archivos/') . $last_id); // Redirecciona a la carga de archivos
}

// Carga vista para subir archivos relacionados a un ticket
function archivos($id_ticket) {
    $datos['id_ticket'] = $id_ticket;
    // $datos['controlador'] = 'tickets_IT';           // Referencia a otro controlador (comentado)
    $this->load->view('header');
    $this->load->view('cafeteria/subir_archivos', $datos);
}

// Procesa subida de archivos al ticket
function subir_archivos() {
    $idTicket = $this->input->post('id_ticket');

    for ($i = 0; $i < count($_FILES['file']['tmp_name']); $i++) {
        $datos = array(
            'id_comentario' => $idTicket,
            'nombre' => $_FILES['file']['name'][$i],
            'archivo' => file_get_contents($_FILES['file']['tmp_name'][$i])
        );
    }
}

```

```

        if (!$this->Modelo->subir_archivos($datos)) {
            trigger_error("Error al subir archivo", E_USER_ERROR);
        }
    }
}

// Muestra detalles del ticket, comentarios y archivos
public function ver($id) {
    // $this->output->enable_profiler(TRUE);          // Habilita profiler (comentado)
    $Renglon = $this->Modelo->verTicket($id);
    $datos['ticket'] = $Renglon->row();              // Obtiene un solo renglón
    $datos['comentarios'] = $this->Modelo->verTicket_comentarios($id);    // Comentarios
    $datos['comentarios_fotos'] = $this->Modelo->verTicket_comentarios_fotos($id); // Fotos
    $datos['archivos'] = $this->Modelo->verTicketArchivos($id);           // Archivos adjuntos
    // $datos['controlador'] = $this->router->fetch_class();              // Controlador actual
    (comentado)
    $this->load->view('header');
    $this->load->view('cafeteria/ver', $datos);
}

// Muestra una imagen adjunta a un comentario
function ver_foto($id) {
    $photo = $this->Modelo->getFoto($id);
    if ($photo) {
        header("Content-type: image/png");
        echo $photo->archivo;
    } else {
        echo "ERROR";
    }
}

// Agrega un comentario a un ticket existente
public function agregarComentario() {
    $idTicket = $this->input->post('idticket');
    $data = array(
        'id_comentario' => $idTicket,
        'usuario' => $this->session->id,

```

```

        'comentario' => $this->input->post('comentario'),
    );
    $this->Modelo->agregar_comentario($data);
    redirect(base_url('cafeteria/ver/' . $idTicket));
}

```

// Cambia el estatus del ticket (ABIERTO, EN CURSO, etc.)

```

public function estatus($idTicket, $estatus) {
    $Res = $this->Modelo->getUsuarioTicket($idTicket);

    switch ($estatus) {
        case '1': $Stat = 'ABIERTO'; break;
        case '2': $Stat = 'EN CURSO'; break;
        case '3': $Stat = 'DETENIDO'; break;
        case '4': $Stat = 'CANCELADO'; break;
        case '5': $Stat = 'SOLUCIONADO'; break;
        case '6': $Stat = 'CERRADO'; break;
        default:
            redirect(base_url('inicio')); exit(); break;
    }

    $correo = $Res->correo;
    $usuario = $Res->User;
    $data = array('estatus' => $Stat);
    $this->Modelo->update($idTicket, $data);
    redirect(base_url('cafeteria/ver/' . $idTicket));
}

```

// Muestra los tickets creados por el usuario actual

```

public function mis_tickets() {
    $datos['tickets'] = $this->Modelo->getMis_tickets($this->session->id);
    $this->load->view('header');
    $this->load->view('cafeteria/mis_tickets', $datos);
}

```

// Muestra todos los tickets disponibles para el usuario administrador

```

public function administrar() {
    $datos['tickets'] = $this->Modelo->get_tickets($this->session->id);
}

```

```

        $this->load->view('header');
        $this->load->view('cafeteria/tickets', $datos);
    }

    // Cierra un ticket desde una petición AJAX
    function ajax_cerrarTicket() {
        $id = $this->input->post('id');
        $comentario = $this->input->post('comentario');

        $t->calificacion = $this->input->post('calificacion'); // Calificación del servicio
        $t->estatus = "CERRADO"; // Marca como cerrado

        $this->Conexion->modificar('comentarios_cafeteria', $t, null, array('id' => $id)); // Actualiza
        en BD

        if (!empty($comentario)) {
            $data = array(
                'ticket' => $id,
                'usuario' => $this->session->id,
                'comentario' => $comentario
            );
            $this->Modelo->agregar_comentario($data); // Agrega comentario final
        }
    }
}

```

Cientes.php

```
<?php
```

```
defined('BASEPATH') OR exit('No direct script access allowed');
```

```
// Controlador Clientes para gestionar datos de empresas y contactos
```

```
class Clientes extends CI_Controller {
```

```
    // Constructor: carga modelo de conexión a base de datos
```

```
    function __construct() {
```

```
        parent::__construct();
```

```

        $this->load->model('conexion_model', 'Conexion'); // Modelo de conexión a la base de
datos
    }

```

```

// Obtiene clientes desde la tabla 'empresas' (filtrados por ID o nombre) vía Ajax

```

```

function ajax_getClientes() {

```

```

    $id = $this->input->post('id');      // ID del cliente (opcional)

```

```

    $nombre = $this->input->post('nombre'); // Nombre del cliente (opcional)

```

```

// Consulta base para obtener clientes

```

```

$query = "SELECT id, nombre, razon_social, rfc, calle, numero, numero_interior, colonia,
        credito_cliente, credito_cliente_plazo, horario_facturas,
        ultimo_dia_facturas, documentos_facturacion, codigo_impresion
        FROM empresas
        WHERE cliente = 1";

```

```

// Filtro por ID si se proporciona

```

```

if ($id) {

```

```

    $query .= " AND id = '$id'";

```

```

} else {

```

```

    // Filtro por nombre si se proporciona

```

```

    if ($nombre) {

```

```

        $query .= " AND nombre LIKE '%$nombre%'";

```

```

    }

```

```

}

```

```

$res = $this->Conexion->consultar($query, $id); // Ejecuta consulta

```

```

if ($res) {

```

```

    echo json_encode($res);          // Devuelve resultados en JSON

```

```

} else {

```

```

    echo "";                          // Si no hay resultados, devuelve vacío

```

```

}

```

```

}

```

```

// Obtiene contactos de empresas (por ID o por empresa) vía Ajax

```

```

function ajax_getContactos() {

```

```

    $id = $this->input->post('id');      // ID del contacto (opcional)

```

```

    $empresa = $this->input->post('id_cliente'); // ID de la empresa (opcional)

```

```

// Consulta base para obtener contactos activos
$query = "SELECT * FROM empresas_contactos WHERE activo = 1";

// Filtro por ID si se proporciona
if ($id) {
    $query .= " AND id = '$id'";
} else {
    // Filtro por empresa si se proporciona
    if ($empresa) {
        $query .= " AND empresa = '$empresa'";
    }
}

$res = $this->Conexion->consultar($query, $id); // Ejecuta consulta
if ($res) {
    echo json_encode($res);          // Devuelve resultados en JSON
}
}
}

```

Compras.php

```
<?php
```

```
defined('BASEPATH') OR exit('No direct script access allowed');
```

```
// Controlador Compras para gestionar requisiciones, QR, y flujos relacionados
```

```
class Compras extends CI_Controller {
```

```
    // Constructor: carga modelos, librerías y helpers necesarios
```

```
    function __construct() {
```

```
        parent::__construct();
```

```
        $this->load->model('compras_model', 'Modelo');          // Modelo principal de compras
```

```
        $this->load->model('MLConexion_model', 'MLConexion');    // Modelo para conexión con
```

```
ML
```

```
        $this->load->model('descargas_model');                  // Modelo para descargas
```

```
        $this->load->model('conexion_model', 'Conexion');      // Modelo de conexión general
```

```

$this->load->helper('download');          // Helper para descargas
$this->load->library('correos');           // Librería de correos (general)
$this->load->library('correos_pr');        // Librería de correos para PR
$this->load->library('AOS_funciones');     // Funciones auxiliares AOS
}

// Carga vista para generar una nueva QR
function generar_qr() {
    $data['intervalo'] = $this->Modelo->intervalos(); // Obtiene intervalos disponibles
    $this->load->view('header');
    $this->load->view('compras/generar_qr', $data);    // Vista para generación de QR
}

// Muestra los detalles de un QR específico
function ver_qr($id) {
    $data['comentarios'] = $this->Modelo->verQr_comentarios($id); // Comentarios del
QR
    $data['comentarios_fotos'] = $this->Modelo->verQr_comentarios_fotos($id); // Fotos
relacionadas
    $data['conceptos'] = $this->Modelo->conceptos();           // Catálogo de conceptos
    $data['qr'] = $this->Modelo->getDetalleQR($id);           // Detalle del QR

    // Consulta la fecha del rechazo si aplica
    $fecha_rechazo = $this->Conexion->consultar(
        "SELECT b.fecha FROM requisiciones_cotizacion qr
        JOIN bitacora_qrs b ON qr.id = b.qr
        WHERE b.estatus = 'RECHAZADO' AND b.qr = " . $id, true
    );
    $data['fecha_rechazo'] = $fecha_rechazo;

    // Define estilo del botón según estatus del QR
    switch ($data['qr']->estatus) {
        case 'ABIERTO':
            $data['btn_estatus'] = "btn-primary";
            break;
        case 'LIBERADO':
        case 'COMPRA APROBADA':
    }

```

```

        $data['btn_estatus'] = "btn-success";
        break;
    case 'COTIZANDO':
        $data['btn_estatus'] = "btn-warning";
        break;
    case 'CANCELADO':
        $data['btn_estatus'] = "btn-default";
        break;
    case 'RECHAZADO':
    case 'COMPRA RECHAZADA':
        $data['btn_estatus'] = "btn-danger";
        break;
    default:
        break;
}

$this->load->view('header');
$this->load->view('compras/ver_qr', $data);        // Vista para ver QR
}

// Carga la vista para editar un QR específico
function editar_qr($id) {
    $data['qr'] = $this->Modelo->getDetalleQR($id);    // Datos del QR a editar
    $data['intervalo'] = $this->Modelo->intervalos();    // Intervalos disponibles
    // $this->load->view('debug', $data); // Para depuración (comentado)
    $this->load->view('header');
    $this->load->view('compras/editar_qr', $data);
}

// Carga la vista para clonar un QR específico
function clonar_qr($id) {
    $data['qr'] = $this->Modelo->getDetalleQR($id);    // Datos del QR original
    $data['intervalo'] = $this->Modelo->intervalos();    // Intervalos disponibles
    $this->load->view('header');
    $this->load->view('compras/clonar_qr', $data);
}

// Muestra listado de requisiciones filtradas por estatus o ID

```



```

function requisiciones($estatus = 'TODO', $id = "") {
    $data['estatus'] = strtoupper($estatus);           // Estatus seleccionado
    $data['qr'] = $id;                                // ID del QR (si aplica)
    $data['asignado'] = $this->Modelo->usuariosCompras(); // Usuarios del área de
compras
    $data['otros_aprobadores'] = $data['estatus'] == 'TODO' ? " : 'unchecked'; // Filtro de vista

    $this->load->view('header');
    $this->load->view('compras/catalogo_qr', $data);      // Vista del catálogo
}

// Muestra la vista con las requisiciones del usuario actual
function mis_qrs() {
    $this->load->view('header');
    $this->load->view('compras/mis_qrs');
}

// Genera una nueva QR con datos recibidos vía POST (JSON + atributos)
function ajax_generarQR() {
    $_info = json_decode($this->input->post('info')); // Información general en formato
JSON
    $_atributos = $this->input->post('atributos');     // Atributos específicos

    $unidad = "";
    $clave_unidad = "";
    $intervalo = null;

    // Si hay intervalo definido, se asigna
    if (!empty($_info->intervalo)) {
        $intervalo = $_info->intervalo;
    }

    // Si el tipo es 'PRODUCTO', se extraen unidad y clave, y se ignora el intervalo
    if ($_info->tipo == 'PRODUCTO') {
        $unidad = $_info->unidad;
        $clave_unidad = $_info->clave_unidad;
        $intervalo = null;
    }
}

```

```

// Preparación de datos para guardar la QR
$info = array(
    'usuario' => $this->session->id,
    'archivo' => "0",
    'nombre_archivo' => "",
    'tipo' => $_info->tipo,
    'subtipo' => $_info->subtipo,
    'cantidad' => $_info->cantidad,
    'cantidad_aprobada' => 0,
    'unidad' => $unidad,
    'clave_unidad' => $clave_unidad,
    'descripcion' => $_info->descripcion,
    'prioridad' => $_info->prioridad,
    'lugar_entrega' => $_info->lugar_entrega,
    'comentarios' => $_info->comentarios,
    'critico' => $_info->critico,
    'destino' => $_info->destino,
    'atributos' => $_atributos,
    'notificaciones' => $_info->notificaciones,
    'estatus' => 'ABIERTO',
    'especificos' => $_info->especificos,
    'intervalo' => $intervalo
);

// Si es tipo PRODUCTO y tiene tipo de calibración, se añade
if ($_info->tipo == 'PRODUCTO' && isset($_info->tipocalibracion)) {
    $info['tipocalibracion'] = $_info->tipocalibracion;
}

$res = $this->Modelo->generarQR($info); // Se crea la QR en la base de datos

// Registra movimiento en bitácora de QR
$bitacoraQR = array(
    'qr' => intval($res),
    'user' => $this->session->id,
    'estatus' => 'ABIERTO'
);

```

```

$this->Modelo->estatusQR($bitacoraQR);

// Si la creación fue exitosa
if ($res) {
    $datos = array(
        'id' => $res,
        'fecha' => date('d/m/Y h:i A'),
        'usuario' => $this->session->nombre,
        'cantidad' => $_info->cantidad,
        'unidad' => $unidad,
        'descripcion' => $_info->descripcion,
        'atributos' => $_atributos,
        'prioridad' => $_info->prioridad,
        'comentarios' => $_info->comentarios,
        'correos' => array_merge(
            array($this->session->correo),
            $this->Modelo->getCorreosQR()
        )
    );

    // Solo se envía correo si la prioridad no es NORMAL
    if ($_info->prioridad != 'NORMAL') {
        $this->correos->creacionQR($datos);
    }

    echo $res; // Devuelve el ID de la nueva QR
}

// Prueba: imprime lista de correos configurados para QR
function test() {
    print_r($this->Modelo->getCorreosQR());
}

// Edita una QR existente con nueva información recibida vía POST
function ajax_editarQR() {
    $_info = json_decode($this->input->post('info')); // Datos principales
    $_atributos = $this->input->post('atributos'); // Atributos editados

```

```

$unidad = "";
$clave_unidad = "";
$intervalo = null;
$especificos = null;

// Registra edición en bitácora
$bitacoraQR = array(
    'qr' => intval($_info->id),
    'user' => $this->session->id,
    'estatus' => 'EDICION'
);
$this->Modelo->estatusQR($bitacoraQR);

// Asigna intervalo si existe
if (!empty($_info->intervalo)) {
    $intervalo = $_info->intervalo;
}

// Asigna específicos si existen
if (!empty($_info->especificos)) {
    $especificos = $_info->especificos;
}

// Si el tipo es PRODUCTO, se ajustan los valores y se ignora el intervalo
if ($_info->tipo == 'PRODUCTO') {
    $unidad = $_info->unidad;
    $clave_unidad = $_info->clave_unidad;
    $intervalo = null;
}

// Datos a actualizar en la QR
$info = array(
    'id' => $_info->id,
    'usuario' => $this->session->id,
    'tipo' => $_info->tipo,
    'subtipo' => $_info->subtipo,
    'cantidad' => $_info->cantidad,

```

```

'unidad' => $unidad,
'clave_unidad' => $clave_unidad,
'descripcion' => $_info->descripcion,
'prioridad' => $_info->prioridad,
'lugar_entrega' => $_info->lugar_entrega,
'comentarios' => $_info->comentarios,
'critico' => $_info->critico,
'destino' => $_info->destino,
'atributos' => $_atributos,
'notificaciones' => $_info->notificaciones,
'estatus' => 'ABIERTO',
'especificos' => $especificos,
'intervalo' => $intervalo
);

// Si tiene tipo de calibración y es producto, lo añade
if ($_info->tipo == 'PRODUCTO' && isset($_info->tipocalibracion)) {
    $info['tipocalibracion'] = $_info->tipocalibracion;
}

// Si se escribió un comentario durante la edición, lo guarda
if ($_info->coments) {
    $coments = array(
        'comentario' => $_info->coments,
        'usuario' => $this->session->id,
        'qr' => $_info->id
    );
    $this->Modelo->agregar_comentario($coments);
}

$res = $this->Modelo->editarQR($info); // Ejecuta la actualización

// Si fue exitosa la edición
if ($res) {
    $datos = array(
        'id' => $_info->id,
        'fecha' => date('d/m/Y h:i A'),
        'usuario' => $this->session->nombre,

```

```

        'cantidad' => $_info->cantidad,
        'unidad' => $unidad,
        'descripcion' => $_info->descripcion,
        'atributos' => $_atributos,
        'prioridad' => $_info->prioridad,
        'comentarios' => $_info->comentarios,
        'correos' => array_merge(
            array($this->session->correo),
            $this->Modelo->getCorreosQR()
        )
    );

    // Solo si la prioridad no es NORMAL se notifica por correo
    if ($_info->prioridad != 'NORMAL') {
        $this->correos->edicionQR($datos);
    }

    echo "1"; // Confirma éxito
}
}

// Filtra y devuelve QRs según múltiples criterios recibidos vía POST (prioridad, tipo,
estatus, fechas, texto, etc.)
function ajax_getQRs() {
    $prioridad = json_decode($this->input->post('prioridad'));
    $tipo = json_decode($this->input->post('tipo'));

    $estatus = $this->input->post('estatus');
    $texto = $this->input->post('texto');
    $parametro = $this->input->post('parametro');
    $usuario = $this->input->post('usuario');
    $asignado = $this->input->post('asignado');
    $archivo = $this->input->post('archivo');
    $fecha1 = $this->input->post('fecha1');
    $fecha2 = $this->input->post('fecha2');

    $f1 = strval($fecha1) . ' 00:00:00';
    $f2 = strval($fecha2) . ' 23:59:59';

```

```
// Consulta base con JOIN para obtener nombre del usuario
$query = "SELECT R.id, R.fecha, R.usuario, R.prioridad, R.tipo, R.subtipo, R.cantidad,
        R.cantidad_aprobada, R.unidad, R.clave_unidad, R.descripcion, R.atributos,
        R.critico, R.destino, R.lugar_entrega, R.comentarios, R.estatus, R.asignado,
        CONCAT(U.nombre, ' ', U.paterno) AS User
        FROM requisiciones_cotizacion R
        LEFT JOIN usuarios U ON R.usuario = U.id
        WHERE 1 = 1";
```

```
// Filtros opcionales según campos recibidos
```

```
if ($estatus != 'TODO') {
    $query .= " AND R.estatus = '$estatus'";
}
```

```
if (!empty($asignado)) {
    $query .= " AND R.asignado = '$asignado'";
}
```

```
if ($usuario == '1') {
    $idUser = $this->session->id;
    $query .= " AND R.usuario = '$idUser'";
}
```

```
if (count($prioridad) > 0) {
    $query .= " AND (1 = 0";
    foreach ($prioridad as $value) {
        $query .= " OR R.prioridad = '$value'";
    }
    $query .= ")";
}
```

```
if (isset($tipo) && count($tipo) > 0) {
    $query .= " AND (1 = 0";
    foreach ($tipo as $value) {
        $query .= " OR R.tipo = '$value'";
    }
    $query .= ")";
}
```

```

    }

    // Control de acceso según privilegios para consumo interno o venta
    if ($this->session->privilegios['crear_qr_interno'] != $this->session-
>privilegios['crear_qr_venta']) {
        if ($this->session->privilegios['editar_qr'] == "0" && $this->session-
>privilegios['liberar_qr'] == "0") {
            if ($this->session->privilegios['crear_qr_interno'] == "1") {
                $query .= " AND R.destino = 'CONSUMO INTERNO'";
            } else {
                $query .= " AND R.destino = 'VENTA'";
            }
        }
    }
}

// Filtro por texto libre según el parámetro seleccionado
if (!empty($texto)) {
    switch ($parametro) {
        case 'folio':
            $query .= " AND R.id = '$texto'";
            break;
        case 'usuario':
            $query .= " AND CONCAT(U.nombre, ' ', U.paterno) LIKE '%$texto%'";
            break;
        case 'contenido':
            $query .= " AND (R.descripcion LIKE '%$texto%'
                OR UPPER(R.atributos->'$.marca') LIKE UPPER('%$texto%')
                OR UPPER(R.atributos->'$.modelo') LIKE UPPER('%$texto%'))";
            break;
    }
}

// Filtro por rango de fechas
if (!empty($fecha1) && !empty($fecha2)) {
    $query .= " AND R.fecha BETWEEN '$f1' AND '$f2'";
}

// Si no es consulta de archivo histórico

```



```

if ($archivo != 1) {
    $query .= " AND R.fecha > '2021-01-01 00:00:00'";
}

$query .= " ORDER BY R.fecha DESC";

// Ejecuta la consulta
$res = $this->Modelo->consulta($query);
if ($res) {
    echo json_encode($res);
} else {
    echo "";
}
}

// Devuelve todas las QRs creadas por un usuario específico
function ajax_getMisQRs() {
    $user = $this->input->post('usuario');
    $res = $this->Modelo->getMisQrs($user);

    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

// Devuelve los comentarios de una QR específica
function ajax_getQRComentarios() {
    $qr = $this->input->post('qr');
    $res = $this->Conexion->consultar(
        "SELECT C.*, CONCAT(U.nombre, ' ', U.paterno, ' ', U.materno) AS User
        FROM qr_comentarios C
        INNER JOIN usuarios U ON U.id = C.usuario
        WHERE C.qr = $qr"
    );

    if ($res) {

```

```

        echo json_encode($res);
    }
}

// Devuelve el detalle completo de una QR específica
function ajax_getDetalleQR() {
    $id = $this->input->post('idQR');
    $res = $this->Modelo->getDetalleQR($id);

    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

// Devuelve usuarios con privilegios específicos para recibir notificaciones de QR
function ajax_getUsuariosQRNotificaciones() {
    $privilegio = $this->input->post('privilegio');
    $id = $this->input->post('id');

    // Consulta base
    $query = "SELECT U.id, CONCAT(U.nombre, ' ', U.paterno) AS Nombre, P.puesto AS Puesto,
U.correo
        FROM usuarios U
        INNER JOIN puestos P ON U.puesto = P.id
        INNER JOIN privilegios PR ON PR.usuario = U.id
        WHERE U.activo = 1";

    // Filtro por ID o privilegio específico
    if ($id) {
        $query .= " AND U.id = '$id'";
    } else {
        $query .= " AND PR.$privilegio = 1";
    }

    $query .= " ORDER BY Nombre ASC";
}

```

```

$res = $this->Conexion->consultar($query, $id);
if ($res) {
    echo json_encode($res);
}
}

// Devuelve los proveedores asignados a una QR específica
function ajax_getProveedoresAsignados() {
    $idQR = $this->input->post('idQR');

    // Consulta con información de empresa, propuesta y asignador
    $query = "SELECT E.id, QP.id as idQP, P.entrega, E.nombre, QP.monto, QP.total, QP.moneda,
        QP.tiempo_entrega, QP.dias_habiles, QP.comentarios, QP.nominado,
        QP.seleccionado, QP.nombre_archivo, QP.vencimiento, QP.fechaAsignacion,
        CONCAT(U.nombre, ' ', U.paterno) AS Asignador
    FROM qr_proveedores QP
    INNER JOIN empresas E ON E.id = QP.empresa
    INNER JOIN proveedores P ON P.empresa = E.id
    LEFT JOIN usuarios U ON U.id = QP.asignador
    WHERE QP.qr = '$idQR'";

    $res = $this->Modelo->consulta($query);
    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

// Devuelve las propuestas enviadas por proveedores para una QR específica
function ajax_getPropuestas() {
    $idQR = $this->input->post('idQR');

    $query = "SELECT E.id, QP.id as idQP, P.entrega, P.rma_requerido, E.nombre, QP.monto,
QP.total,
        QP.moneda, QP.tiempo_entrega, QP.dias_habiles, QP.comentarios, QP.nominado,
        QP.seleccionado, QP.nombre_archivo, QP.vencimiento, QR.cantidad,
        QR.descripcion,

```

```

        QR.tipo, QR.subtipo, IFNULL(JSON_UNQUOTE(attributes->'$.serie'), '') AS Serie,
        QR.nombre_archivoEjemplo, QR.id as QR
    FROM qr_proveedores QP
    INNER JOIN requisiciones_cotizacion QR ON QR.id = QP.qr
    INNER JOIN empresas E ON E.id = QP.empresa
    INNER JOIN proveedores P ON P.empresa = E.id
    WHERE QP.qr = '$idQR';

```

```

$res = $this->Modelo->consulta($query);
if ($res) {
    echo json_encode($res);
} else {
    echo "";
}
}

```

// Asigna un proveedor a una QR (registro inicial con valores por defecto)

```

function ajax_setProveedor() {
    $data['qr'] = $this->input->post('idQR');
    date_default_timezone_set('America/Chihuahua');
    $data['fechaAsignacion'] = date('Y-m-d h:i:s');

    $data['empresa'] = $this->input->post('idProv');
    $data['total'] = "0";
    $data['dias_habiles'] = "1";
    $data['nominado'] = "0";
    $data['seleccionado'] = "0";
    $data['comentarios'] = "";
    $data['factor'] = "0";
    $data['asignador'] = $this->session->id;

    $res = $this->Modelo->setProveedor($data);
    if ($res) {
        echo "1";
    }
}

```

// Marca un proveedor como el seleccionado para una QR

```

function ajax_setProveedorSugerido() {
    $qr = $this->input->post('qr');
    $qr_prov = $this->input->post('qr_prov');

    // Primero desmarca todos los proveedores de esa QR
    $query = "UPDATE qr_proveedores SET seleccionado = '0' WHERE qr = '$qr'";
    $this->Modelo->update($query);

    // Luego marca como seleccionado al proveedor indicado
    $query2 = "UPDATE qr_proveedores SET seleccionado = '1' WHERE id = '$qr_prov'";
    $this->Modelo->update($query2);

    echo "1";
}

// Elimina un proveedor asignado a una QR
function ajax_eliminarProveedor() {
    $data['qr'] = $this->input->post('qr');
    $data['empresa'] = $this->input->post('empresa');

    $res = $this->Modelo->deleteProveedor($data);
    if ($res) {
        echo "1";
    }
}

// Guarda información actualizada de todos los proveedores de una QR
function guardarProveedores() {
    $datos = json_decode($this->input->post('datos'), TRUE);

    // Se actualiza cada proveedor uno por uno
    foreach ($datos as $value) {
        $this->Modelo->updateProveedor($value);
    }

    $id = $datos[0]["id"];

    // Se actualiza el campo de máximo vencimiento en la tabla de requisiciones

```

```

$this->Conexion->comando(
    "UPDATE requisiciones_cotizacion
    SET maximo_vencimiento = (
        SELECT MAX(vencimiento)
        FROM qr_proveedores
        WHERE qr = (
            SELECT qr
            FROM qr_proveedores
            WHERE id = $id
        )
    )"
);

echo "1";
}

// Sugiere proveedores según coincidencia en palabras clave (tags)
function proveedoresSugeridos() {
    $tags = $this->input->post('tags');
    $arreglo = explode(" ", trim($tags));
    $arreglo = array_diff($arreglo, array("")); // Elimina vacíos

    // Consulta proveedores con coincidencias en tags
    $query = "SELECT E.id, E.nombre
        FROM empresas E
        INNER JOIN proveedores P ON P.empresa = E.id
        WHERE E.proveedor = 1 AND (1 != 1";

    foreach ($arreglo as $value) {
        $query .= " OR tags LIKE '%," . $value . ",%";
    }

    $query .= ")";

    $res = $this->Modelo->consulta($query);
    if ($res) {
        echo json_encode($res);
    } else {

```

```

        echo "";
    }
}

```

```

// Sugiere proveedores según coincidencia con la marca en atributos
function proveedoresSugeridosMarca() {
    $tags = $this->input->post('tags');

```

```

    $query = "SELECT E.id, E.nombre, IFNULL((
        SELECT COUNT(QP.id)
        FROM qr_proveedores QP
        INNER JOIN requisiciones_cotizacion QR ON QR.id = QP.qr
        WHERE QP.monto > 0
        AND QP.empresa = E.id
        AND UPPER(QR.atributos->'$.marca') = UPPER(\"$tags\")
    ), 0) AS QtyQr
    FROM empresas E
    INNER JOIN proveedores P ON P.empresa = E.id
    WHERE E.proveedor = 1
        AND (1 != 1 OR tags LIKE ',$tags,')
        AND !ISNULL(P.entrega)
        AND JSON_LENGTH(P.tipo) > 0
        AND JSON_LENGTH(P.formas_pago) > 0";

```

```

    $query .= " UNION ";

```

```

    $query .= "SELECT E.id, E.nombre, IFNULL((
        SELECT COUNT(QP.id)
        FROM qr_proveedores QP
        INNER JOIN requisiciones_cotizacion QR ON QR.id = QP.qr
        WHERE QP.monto > 0
        AND QP.empresa = E.id
        AND UPPER(QR.atributos->'$.marca') = UPPER(\"$tags\")
    ), 0) AS QtyQr
    FROM qr_proveedores QP
    INNER JOIN requisiciones_cotizacion QR ON QP.qr = QR.id
    INNER JOIN empresas E ON E.id = QP.empresa
    WHERE E.proveedor = 1

```

```

        AND QP.monto > 0
        AND UPPER(QR.atributos->'$.marca') = UPPER('\ "$tags\ "'');

$res = $this->Modelo->consulta($query);
echo $res ? json_encode($res) : "";
}

```

// Sugiere proveedores según coincidencia con modelo en atributos

```

function proveedoresSugeridosModelo() {
    $tags = $this->input->post('tags');

    $query = "SELECT E.id, E.nombre, IFNULL(
        SELECT COUNT(QP.id)
        FROM qr_proveedores QP
        INNER JOIN requisiciones_cotizacion QR ON QR.id = QP.qr
        WHERE QP.monto > 0
        AND QP.empresa = E.id
        AND UPPER(QR.atributos->'$.modelo') = UPPER('\ "$tags\ "')
    ), 0) AS QtyQr
    FROM qr_proveedores QP
    INNER JOIN requisiciones_cotizacion QR ON QP.qr = QR.id
    INNER JOIN empresas E ON E.id = QP.empresa
    WHERE E.proveedor = 1
    AND QP.monto > 0
    AND UPPER(QR.atributos->'$.modelo') = UPPER('\ "$tags\ "')";

    $res = $this->Modelo->consulta($query);
    echo $res ? json_encode($res) : "";
}

```

// Obtiene información detallada de un proveedor asignado a una QR

```

function ajax_getProveedor() {
    $id = $this->input->post('id');

    $query = "SELECT QP.id, QP.qr, QP.empresa, QP.costos, QP.monto, QP.moneda, QP.total,
        QP.tiempo_entrega, QP.dias_habiles, QP.comentarios, QP.nominado,
        QP.seleccionado, QP.vencimiento, QP.nombre_archivo, E.nombre,
        P.entrega, QR.nombre_archivo";
}

```



```

FROM qr_proveedores QP
INNER JOIN empresas E ON E.id = QP.empresa
INNER JOIN proveedores P ON P.empresa = E.id
JOIN requisiciones_cotizacion QR ON QR.id = QP.qr
WHERE QP.id = '$id';

```

```

$res = $this->Modelo->consulta($query, TRUE);
echo $res ? json_encode($res) : "";
}

```

// Obtiene lista de proveedores filtrando por texto y condiciones necesarias

```

function ajax_getProveedores() {
    $texto = $this->input->post('texto');

    $query = "SELECT E.*
FROM empresas E
INNER JOIN proveedores P ON E.id = P.empresa
WHERE E.proveedor = 1
AND (P.tags LIKE ',$texto,%' OR E.nombre LIKE '%$texto%')
AND !ISNULL(P.entrega)
AND JSON_LENGTH(P.tipo) > 0
AND JSON_LENGTH(P.formas_pago) > 0";

```

```

$res = $this->Modelo->consulta($query);
echo $res ? json_encode($res) : "";
}

```

// Cambia el estatus de una QR, registra en bitácora y envía correo si aplica

```

function ajax_setEstatusQR() {
    $estatus = $this->input->post('estatus');
    $idqr = $this->input->post('idqr');
    $liberador = $this->session->id;

    if ($estatus == "LIBERADO") {
        $query = "UPDATE requisiciones_cotizacion
SET estatus = '$estatus',
    fecha_liberacion = CURRENT_TIMESTAMP(),
    liberador = $liberador

```

```

        WHERE id = '$idqr';
    } else {
        $query = "UPDATE requisiciones_cotizacion
            SET estatus = '$estatus'
            WHERE id = '$idqr';
    }

$this->Modelo->update($query);

// Registra cambio en bitácora
$bitacoraQR = array(
    'qr' => intval($idqr),
    'user' => $this->session->id,
    'estatus' => $estatus
);
$this->Modelo->estatusQR($bitacoraQR);

// Si fue exitoso, enviar correo si es necesario
if (in_array($estatus, ["LIBERADO", "COMPRA APROBADA"])) {
    $qr = $this->Modelo->getDetalleQR($idqr);
    $datos = array(
        'id' => $idqr,
        'fecha' => date('d/m/Y h:i A'),
        'usuario' => $qr->User,
        'cliente' => $qr->Client,
        'prioridad' => $qr->prioridad,
        'unidad' => $qr->unidad,
        'cantidad' => $qr->cantidad,
        'descripcion' => $qr->descripcion,
        'atributos' => $qr->atributos,
        'comentarios' => $qr->comentarios,
        'correos' => array($qr->correo),
        'estatus' => $estatus
    );

    // Añade notificados adicionales
    $Notificar = json_decode($qr->notificaciones);
    $query = "SELECT U.correo FROM usuarios U WHERE 1 != 1";

```

```

        foreach ($Notificar as $value) {
            $query .= " OR U.id = $value";
        }
        $res = $this->Conexion->consultar($query);
        foreach ($res as $value) {
            array_push($datos['correos'], $value->correo);
        }

        $this->correos->liberarQR($datos);
    }

    echo "1";
}

// Cambia el estatus de una QR e incluye un comentario con notificación por correo
function ajax_setEstatusMsjQR() {
    $idqr = $this->input->post('idqr');
    $estatus = $this->input->post('estatus');
    $comentario_original = $this->input->post('comentario');
    $comentario = "<b><font color='red'>$estatus:</font></b> " . $comentario_original;
    $tags = $this->input->post('txtTags');
    $correos = explode(",", $tags);

    $query = "UPDATE requisiciones_cotizacion SET estatus = '$estatus' WHERE id = '$idqr'";
    $this->Modelo->update($query);

    // Bitácora
    $bitacoraQR = array(
        'qr' => intval($idqr),
        'user' => $this->session->id,
        'estatus' => $estatus
    );
    $this->Modelo->estatusQR($bitacoraQR);

    // Guarda el comentario del cambio de estatus
    $data = array(
        'qr' => $idqr,
        'usuario' => $this->session->id,

```

```

        'comentario' => $comentario
    );
    $this->Modelo->agregar_comentario($data);

    $qr = $this->Modelo->getDetalleQR($idqr);
    $datos = array(
        'id' => $idqr,
        'fecha' => date('d/m/Y h:i A'),
        'usuario' => $qr->User,
        'cliente' => $qr->Client,
        'prioridad' => $qr->prioridad,
        'unidad' => $qr->unidad,
        'cantidad' => $qr->cantidad,
        'descripcion' => $qr->descripcion,
        'atributos' => $qr->atributos,
        'comentarios' => $comentario_original,
        'correos' => array($qr->correo)
    );

    // Envía correo según tipo de estatus
    if ($estatus == "RECHAZADO" || $estatus == "COMPRA RECHAZADA") {
        $this->correos->rechazoQR($datos);
    } else if ($estatus == "LIBERADO") {
        $this->correos->liberarQR($datos);
    }

    // Si se agregaron correos adicionales manualmente
    if (count($correos) > 0) {
        $datos2 = array(
            'id' => $idqr,
            'comentario' => $comentario,
            'correos' => $correos
        );
        $this->correos->comentarioQR($datos2);
    }

    echo $comentario;
}

```

```

// Marca uno o varios proveedores como nominados para una QR
function ajax_setProveedoresNominados() {
    $res = TRUE;
    $qr_proveedor = json_decode($this->input->post('qr_proveedores')); // IDs de proveedores
a nominar
    $qr = $this->input->post('qr'); // ID de la QR

    // Primero se desmarcan todos
    $this->Modelo->update("UPDATE qr_proveedores SET nominado = 0 WHERE qr = $qr");

    // Luego se marcan los seleccionados
    foreach ($qr_proveedor as $value) {
        if (!$this->Modelo->update("UPDATE qr_proveedores SET nominado = 1 WHERE id =
$value")) {
            $res = FALSE;
        }
    }

    echo $res ? "1" : "";
}

// Sube archivo PDF asociado a una QR (como archivo principal)
function ajax_subirArchivoQR() {
    $datos['id'] = $this->input->post('qr');
    $datos['archivo'] = file_get_contents($_FILES['file']['tmp_name']);
    $datos['nombre_archivo'] = str_pad($datos['id'], 6, "0", STR_PAD_LEFT) . ".pdf";

    if (!$this->Modelo->setQRFile($datos)) {
        trigger_error("Error al subir archivo", E_USER_ERROR);
    } else {
        echo "1";
    }
}

// Elimina el archivo PDF asociado a una QR
function ajax_borrarArchivoQR() {
    $datos['id'] = $this->input->post('qr');

```

```

$datos['archivo'] = "";
$datos['nombre_archivo'] = "";

if (!$this->Modelo->setQRFile($datos)) {
    trigger_error("Error al borrar archivo", E_USER_ERROR);
} else {
    echo "1";
}
}

// Sube archivo de evidencia (como propuesta de proveedor)
function ajax_subirEvidencia() {
    $datos['id'] = $this->input->post('qr_prov');
    $datos['archivo'] = file_get_contents($_FILES['file']['tmp_name']);
    $datos['nombre_archivo'] = $_FILES['file']['name'];

    if (!$this->Modelo->updateProveedor($datos)) {
        trigger_error("Error al subir evidencia", E_USER_ERROR);
    } else {
        echo $datos['nombre_archivo'];
    }
}

// Elimina archivo de evidencia cargado por proveedor
function ajax_eliminarEvidencia() {
    $datos['id'] = $this->input->post('qr_prov');
    $datos['archivo'] = null;
    $datos['nombre_archivo'] = null;

    if (!$this->Modelo->updateProveedor($datos)) {
        trigger_error("Error al eliminar evidencia", E_USER_ERROR);
    } else {
        echo "1";
    }
}

// Muestra un archivo de evidencia en el navegador (inline PDF)
function getEvidencia($qr_prov) {

```

```

$row = $this->descargas_model->getFile($qr_prov, 'qr_proveedores');
$file = $row->archivo;
$nombre = $row->nombre_archivo;

header('Content-type: application/pdf');
header('Content-Disposition: inline; filename="" . $nombre . "");
header('Content-Transfer-Encoding: binary');
header('Accept-Ranges: bytes');

echo $file;
}

// Muestra el archivo principal PDF de una QR
function getQrFile($qr) {
    $row = $this->descargas_model->getFile($qr, 'requisiciones_cotizacion');
    $file = $row->archivo;
    $nombre = $row->nombre_archivo;

    header('Content-type: application/pdf');
    header('Content-Disposition: inline; filename="" . $nombre . "");
    header('Content-Transfer-Encoding: binary');
    header('Accept-Ranges: bytes');

    echo $file;
}

// Agrega un comentario a una QR y notifica a usuarios relacionados
function agregarComentario() {
    $idQr = $this->input->post('idQr');
    $comentario = $this->input->post('comentario');
    $tags = $this->input->post('txtTags');
    $correos = explode(",", $tags);

    // Obtiene correos del solicitante y del asignado
    $query = "SELECT ur.correo AS correoReq, ua.correo AS correoA
              FROM requisiciones_cotizacion qr
              JOIN usuarios ur ON ur.id = qr.usuario
              LEFT JOIN usuarios ua ON ua.id = qr.asignado";

```

```
WHERE qr.id = $idQr";
```

```
$res = $this->Modelo->consulta($query, TRUE);
```

```
$mails = explode(',', $res->correoReq . ',' . $res->correoA);
```

```
// Enviar correo principal de comentario
```

```
$correoData = array(
```

```
    'id' => $idQr,
```

```
    'comentario' => $comentario,
```

```
    'correos' => $mails,
```

```
    'usuario' => $this->session->nombre
```

```
);
```

```
$this->correos->comentarioQR($correoData);
```

```
// Guarda comentario en base de datos
```

```
$data = array(
```

```
    'qr' => $idQr,
```

```
    'usuario' => $this->session->id,
```

```
    'comentario' => $comentario
```

```
);
```

```
$this->Modelo->agregar_comentario($data);
```

```
// Enviar notificaciones a correos adicionales si existen
```

```
if (count($correos) > 0) {
```

```
    $datos = array(
```

```
        'id' => $idQr,
```

```
        'comentario' => $comentario,
```

```
        'correos' => $correos,
```

```
        'usuario' => $this->session->nombre
```

```
    );
```

```
    $this->correos->comentarioQR($datos);
```

```
}
```

```
redirect(base_url('compras/ver_qr/' . $idQr));
```

```
}
```

```
// Obtiene resumen de propuestas por marca para un proveedor específico
```



```

function ajax_getResumenMarca() {
    $prov = $this->input->post('prov');
    $marca = $this->input->post('marca');

    $query = "SELECT QP.id AS idQP, QP.qr, QR.descripcion, QP.total, QP.moneda,
                QP.tiempo_entrega, QP.dias_habiles, QP.nombre_archivo, QP.vencimiento
            FROM qr_proveedores QP
            INNER JOIN requisiciones_cotizacion QR ON QR.id = QP.qr
            WHERE QP.monto > 0
            AND QP.empresa = '$prov'
            AND UPPER(QR.atributos->'$.marca') = UPPER('$marca')";

    $res = $this->Modelo->consulta($query);
    echo $res ? json_encode($res) : "";
}

// Obtiene resumen de propuestas por modelo para un proveedor específico
function ajax_getResumenModelo() {
    $prov = $this->input->post('prov');
    $modelo = $this->input->post('modelo');

    $query = "SELECT QP.id AS idQP, QP.qr, QR.descripcion, QP.total, QP.moneda,
                QP.tiempo_entrega, QP.dias_habiles, QP.nombre_archivo, QP.vencimiento
            FROM qr_proveedores QP
            INNER JOIN requisiciones_cotizacion QR ON QR.id = QP.qr
            WHERE QP.monto > 0
            AND QP.empresa = '$prov'
            AND UPPER(QR.atributos->'$.modelo') = UPPER('$modelo')";

    $res = $this->Modelo->consulta($query);
    echo $res ? json_encode($res) : "";
}

// Consulta PRs activas relacionadas a una QR específica
function ajax_getPRS_QR() {
    $idQR = $this->input->post('id_qr');

    $query = "SELECT PR.id, PR.fecha, PR.estatus, CONCAT(U.nombre, ' ', U.paterno) AS User

```

```

        FROM prs PR
        INNER JOIN usuarios U ON PR.usuario = U.id
        WHERE (PR.estatus != 'CANCELADO' AND PR.estatus != 'CERRADO')
        AND PR.qr = $idQR";

$res = $this->Conexion->consultar($query);
echo $res ? json_encode($res) : "";
}

// Carga el dashboard general de compras
function dashboard() {
    $data['asignado'] = $this->Modelo->usuariosCompras(); // Usuarios del área de compras
    $this->load->view('header');
    $this->load->view('compras/dashboard', $data);
}

// Genera reporte resumen de QRs activas para el dashboard
function ajax_getReporteQR() {
    $asignado = $this->input->post('asignado');
    $us = ($asignado != 'TODO') ? " AND asignado = $asignado" : "";

    $query = "SELECT
        COUNT(*) AS Total,
        (SELECT COUNT(*) FROM requisiciones_cotizacion WHERE estatus = 'ABIERTO' AND
        fecha > '2021-01-01 00:00:00' $us) AS Abiertos,
        (SELECT MIN(fecha) FROM requisiciones_cotizacion WHERE estatus = 'ABIERTO' AND
        fecha > '2021-01-01 00:00:00' $us) AS ultAbiertos,
        (SELECT COUNT(*) FROM requisiciones_cotizacion WHERE estatus = 'RECHAZADO'
        AND fecha > '2021-01-01 00:00:00' $us) AS Rechazados,
        (SELECT MIN(fecha) FROM requisiciones_cotizacion WHERE estatus = 'RECHAZADO'
        AND fecha > '2021-01-01 00:00:00' $us) AS ultRechazados,
        (SELECT COUNT(*) FROM requisiciones_cotizacion WHERE estatus = 'COTIZANDO'
        AND fecha > '2021-01-01 00:00:00' $us) AS Cotizando,
        (SELECT MIN(fecha) FROM requisiciones_cotizacion WHERE estatus = 'COTIZANDO'
        AND fecha > '2021-01-01 00:00:00' $us) AS ultCotizando
        FROM requisiciones_cotizacion
        WHERE fecha > '2021-01-01 00:00:00' $us";

```

```

$res = $this->Conexion->consultar($query, TRUE);
echo json_encode($res);
}

// Genera reporte resumen de PRs activas para el dashboard
function ajax_getReportePR() {
    $query = "SELECT
        COUNT(*) AS Total,
        (SELECT COUNT(*) FROM prs WHERE estatus = 'PENDIENTE') AS Pendientes,
        (SELECT MIN(fecha) FROM prs WHERE estatus = 'PENDIENTE') AS ultPendientes,
        (SELECT COUNT(*) FROM prs WHERE estatus = 'APROBADO') AS Aprobados,
        (SELECT MIN(fecha) FROM prs WHERE estatus = 'APROBADO') AS ultAprobados,
        (SELECT COUNT(*) FROM prs WHERE estatus = 'RECHAZADO') AS Rechazados,
        (SELECT MIN(fecha) FROM prs WHERE estatus = 'RECHAZADO') AS ultRechazados,
        (SELECT COUNT(*) FROM prs WHERE estatus = 'EN SELECCION') AS Seleccion,
        (SELECT MIN(fecha) FROM prs WHERE estatus = 'EN SELECCION') AS ultSeleccion,
        (SELECT COUNT(*) FROM prs WHERE estatus = 'PO AUTORIZADA') AS PO_Autorizada,
        (SELECT MIN(fecha) FROM prs WHERE estatus = 'PO AUTORIZADA') AS
ultPO_Autorizada,
        (SELECT COUNT(*) FROM prs WHERE estatus = 'EN PO') AS En_PO,
        (SELECT MIN(fecha) FROM prs WHERE estatus = 'EN PO') AS ultEn_PO,
        (SELECT COUNT(*) FROM prs WHERE estatus = 'POR RECIBIR') AS Por_Recibir,
        (SELECT MIN(fecha) FROM prs WHERE estatus = 'POR RECIBIR') AS ultPor_Recibir
    FROM prs";

    $res = $this->Conexion->consultar($query, TRUE);
    echo json_encode($res);
}

// Reporte de órdenes de compra por estatus
function ajax_getReportePO() {
    $requisitor = $this->input->post('requisitor');
    $us = ($requisitor != 'TODO') ? " and usuario = $requisitor" : "";

    $query = 'SELECT count(*) as Total, ';

    // Subconsultas para contar y obtener la fecha más antigua de cada estatus
    $estados = [

```

```

'EN PROCESO' => 'EnProceso',
'PENDIENTE AUTORIZACION' => 'PendienteAutorizacion',
'AUTORIZADA' => 'Autorizada',
'RECHAZADA' => 'Rechazada',
'ORDENADA' => 'Ordenada',
'RECIBIDA' => 'Recibida',
'RECIBIDA PARCIAL' => 'Parcial',
'RECIBIDA TOTAL' => 'RecibidaTotal',
'LISTA PARA CERRAR' => 'Lista'
];

foreach ($estados as $estatus => $alias) {
    $query .= "(SELECT count(*) FROM ordenes_compra WHERE estatus = \"\$estatus\" AND
publish = 1 AND fecha > \"2022-01-01 00:00:00\"\$us) AS $alias, ";
    $query .= "(SELECT MIN(fecha) FROM ordenes_compra WHERE estatus = \"\$estatus\" AND
publish = 1 AND fecha > \"2022-01-01 00:00:00\"\$us) AS ult$alias, ";
}

// Remueve la última coma
$query = rtrim($query, ', ');

// Consulta principal
$query .= " FROM ordenes_compra WHERE publish = 1 AND fecha > \"2022-01-01
00:00:00\"\$us";

$res = $this->Conexion->consultar($query, TRUE);
echo json_encode($res);
}

// Carga vista para solicitudes de compra (PR)
function solicitudes_compra($estatus = 'TODO') {
    $estatus = strtoupper($estatus);
    $data['estatus'] = str_replace('_', ' ', $estatus);
    $data['otros_aprobadores'] = ($data['estatus'] == 'TODO') ? '' : 'checked';

    $this->load->view('header');
    $this->load->view('compras/catalogo_pr', $data);
}

```

```

// Vista detallada de una PR
function ver_pr($id) {
    $data['comentarios'] = $this->Modelo->verPr_comentarios($id);
    $data['comentarios_fotos'] = $this->Modelo->verPr_comentarios_fotos($id);
    $data['surtidorPR'] = $this->Modelo->surtidorPR($id);
    $data['atributos'] = $this->Modelo->atributosPr($id);
    $data['pr'] = $this->Modelo->getPR($id);

    // Determina color de botón según estatus
    switch ($data['pr']->estatus) {
        case 'APROBADO':
        case 'PO AUTORIZADA':
            $data['btn_estatus'] = "btn-success"; break;
        case 'PENDIENTE':
        case 'EN SELECCION':
        case 'POR RECIBIR':
            $data['btn_estatus'] = "btn-warning"; break;
        case 'EN PO':
        case 'CERRADO':
            $data['btn_estatus'] = "btn-primary"; break;
        case 'RECHAZADO':
            $data['btn_estatus'] = "btn-danger"; break;
        case 'PROCESADO':
        case 'CANCELADO':
            $data['btn_estatus'] = "btn-default"; break;
        default:
            $data['btn_estatus'] = "btn-warning"; break;
    }

    $this->load->view('header');
    $this->load->view('compras/ver_pr', $data);
}

// Vista de PRs del usuario actual
function mis_prs() {
    $this->load->view('header');
    $this->load->view('compras/mis_prs');
}

```

```

}

// Genera una nueva PR desde una QR
function ajax_generarPR() {
    $this->load->model('compras_model');

    $qrt = $this->input->post('qr');
    $qty = $this->input->post('qty');
    $precio = $this->input->post('precio');
    $descripcion = $this->input->post('descripcion');
    $serie = $this->input->post('serie');
    $qr_prov = $this->input->post('qr_prov');
    $item = $this->input->post('item');
    $id = $this->input->post('id');

    $qr = $this->Modelo->getDetalleQR($qrt);
    $qr_prov = $this->Modelo->getQRProv($qr_prov);

    // Modifica atributos si existe clave 'serie'
    $att = json_decode($qr->atributos, TRUE);
    if (array_key_exists('serie', $att)) {
        $att['serie'] = $serie ? 'N/A';
        if ($id) $att['id'] = $id;
        if ($item) $att['item'] = $item;
    }

    // Datos para la nueva PR
    $datos = [
        'qr' => $qr->id,
        'qr_proveedor' => $qr_prov->id,
        'usuario' => $this->session->id,
        'prioridad' => $qr->prioridad,
        'tipo' => $qr->tipo,
        'subtipo' => $qr->subtipo,
        'cantidad' => $qty,
        'precio_unitario' => $qr_prov->monto,
        'importe' => $qr_prov->monto * $qty,
        'moneda' => $qr_prov->moneda,
    ];
}

```

```

        'unidad' => $qr->unidad,
        'clave_unidad' => $qr->clave_unidad,
        'descripcion' => $descripcion,
        'atributos' => json_encode($att),
        'critico' => $qr->critico,
        'destino' => $qr->destino,
        'lugar_entrega' => $qr->lugar_entrega,
        'comentarios' => "",
        'estatus' => "PENDIENTE"
    ];

    $funciones['fecha'] = 'CURRENT_TIMESTAMP()';
    $res = $this->Conexion->insertar('prs', $datos, $funciones);

    // Guarda bitácora de estatus inicial
    $this->compras_model->estatusPR([
        'pr' => intval($res),
        'user' => $this->session->id,
        'estatus' => 'PENDIENTE'
    ]);

    // Si es SERVICIO, transfiere atributos temporales
    if ($qr->tipo == "SERVICIO" || ($qr->tipo == "PRODUCTO" && $qr->destino == "VENTA")) {
        $result = $this->Conexion->consultar("SELECT * FROM qr_atributos_temp WHERE idQr = $qr->id");
        if ($result) {
            foreach ($result as $elem) {
                $atributosPR = [
                    'idPr' => intval($res),
                    'item' => $elem->item,
                    'asignado' => $elem->asignado
                ];
                if ($qr->tipo == "SERVICIO") {
                    $atributosPR['equipo'] = $elem->equipo;
                    $atributosPR['serie'] = $elem->serie;
                    $atributosPR['modelo'] = $elem->modelo;
                    $atributosPR['fabricante'] = $elem->fabricante;
                }
            }
        }
    }

```

```

        $this->Conexion->insertar('pr_atributos', $atributosPR);
    }
    $this->Conexion->eliminar('qr_atributos_temp', ['idQr' => $qrt]);
}
}

// Enviar correo si se generó correctamente
if ($res > 0) {
    $correoData = [
        'id' => $res,
        'fecha' => date('d/m/Y h:i A'),
        'usuario' => $this->session->nombre,
        'cantidad' => $qty,
        'unidad' => $qr->unidad,
        'descripcion' => $qr->descripcion,
        'atributos' => $qr->atributos,
        'prioridad' => $qr->prioridad,
        'comentarios' => "",
        'correos' => array_merge([$this->session->correo], $this->Modelo-
>getAprobadorPR($this->session->id, $qr->destino))
    ];
    $this->correos_pr->creacionPR($correoData);
    echo $res;
}
}

// Edita cantidad e importe de una PR, reinicia estatus a 'PENDIENTE'
function ajax_editarPR() {
    $id = $this->input->post('id');
    $qty = $this->input->post('qty');

    $query = "UPDATE prs SET cantidad = $qty, importe = precio_unitario * cantidad, estatus =
'PENDIENTE' WHERE id = $id";
    $this->Conexion->comando($query);

    echo "1";
}

```



```
// Recupera lista de PRs filtradas por múltiples criterios
function ajax_getPRs() {
    $curUser = $this->session->id;
    $misprs = $this->input->post('misprs');
    $prioridad = json_decode($this->input->post('prioridad'));
    $estatus = $this->input->post('estatus');
    $texto = $this->input->post('texto');
    $parametro = $this->input->post('parametro');
    $fecha1 = $this->input->post('fecha1');
    $fecha2 = $this->input->post('fecha2');
    $f1 = $fecha1 . ' 00:00:00';
    $f2 = $fecha2 . ' 23:59:59';
    $tipo = json_decode($this->input->post('tipo'));
    $stock = $this->input->post('stock');
    $archivo = $this->input->post('archivo');

    $query = "SELECT
        PR.id, PR.fecha, PR.usuario, PR.prioridad, PR.tipo, PR.subtipo,
        PR.cantidad, PR.unidad, PR.clave_unidad, PR.descripcion, PR.atributos,
        PR.critico, PR.destino, PR.lugar_entrega, PR.comentarios, PR.estatus,
        CONCAT(U.nombre, ' ', U.paterno) AS User,
        U.autorizador_compras, U.autorizador_compras_venta, PR.stock,
        IFNULL(
            SELECT OCC.po
            FROM ordenes_compra_conceptos OCC
            INNER JOIN ordenes_compra OC ON OCC.po = OC.id
            WHERE OCC.pr = PR.id AND OC.estatus != 'CANCELADA' LIMIT 1
        ), 0) AS POActual
    FROM prs PR
    LEFT JOIN usuarios U ON PR.usuario = U.id
    WHERE 1 = 1";

    if ($estatus != 'TODO') {
        $query .= " AND PR.estatus = '$estatus'";
    }

    if (count($prioridad) > 0) {
        $query .= " AND (0";
    }
}
```

```

foreach ($prioridad as $value) {
    $query .= " OR PR.prioridad = '$value'";
}
$query .= " ";
}

if ($misprs == "1") {
    $query .= " AND IF(PR.destino = 'VENTA', U.autorizador_compras_venta = $curUser,
U.autorizador_compras = $curUser)";
}

if ($stock == "1") {
    $query .= " AND PR.stock = 1";
}

if (!empty($texto)) {
    switch ($parametro) {
        case "folio":
            $query .= " AND PR.id = '$texto'";
            break;
        case "usuario":
            $query .= " HAVING User LIKE '%$texto%'";
            break;
        case "contenido":
            $query .= " AND (
                PR.descripcion LIKE '%$texto%' OR
                UPPER(PR.atributos->'$.marca') LIKE UPPER('%$texto%') OR
                UPPER(PR.atributos->'$.modelo') LIKE UPPER('%$texto%')
            )";
            break;
        case "productos":
            $query .= " AND PR.tipo LIKE '%$texto%'";
            break;
        case "servicios":
            $query .= " AND PR.subtipo LIKE '%$texto%'";
            break;
    }
}
}

```

```

if (isset($tipo) && count($tipo) > 0) {
    $query .= " AND (0";
    foreach ($tipo as $value) {
        $query .= " OR PR.tipo = '$value'";
    }
    $query .= ")";
}

if (!empty($fecha1) && !empty($fecha2)) {
    $query .= " AND PR.fecha BETWEEN '$f1' AND '$f2'";
}

if (true) {
    if ($archivo != 1) {
        $query .= " AND PR.fecha > '2022-01-01 00:00:00'";
    }
    $query .= " ORDER BY PR.fecha DESC";
}

$res = $this->Modelo->consulta($query);

if ($res) {
    echo json_encode($res);
}
}

// Obtiene PRs del usuario actual con filtros avanzados
function ajax_getMisPRs() {
    $user = $this->session->id;

    $prioridad = json_decode($this->input->post('prioridad'));
    $estatus = $this->input->post('estatus');
    $texto = $this->input->post('texto');
    $parametro = $this->input->post('parametro');
    $misprs = $this->input->post('misprs');

    $query = "SELECT

```

```

PR.id, PR.fecha, PR.usuario,
CONCAT(U.nombre, ' ', U.paterno, ' ', U.materno) AS User,
PR.prioridad, PR.tipo, PR.subtipo, PR.cantidad, PR.unidad, PR.clave_unidad,
PR.descripcion, PR.atributos, PR.critico, PR.destino, PR.lugar_entrega,
PR.comentarios, PR.estatus,
IFNULL((
    SELECT OCC.po
    FROM ordenes_compra_conceptos OCC
    INNER JOIN ordenes_compra OC ON OCC.po = OC.id
    WHERE OCC.pr = PR.id AND OC.estatus != 'CANCELADA' LIMIT 1
), 0) AS POActual
FROM prs PR
INNER JOIN usuarios U ON U.id = PR.usuario
WHERE 1 = 1";

```

```

if ($misprs == "1") {
    $query .= " AND PR.usuario = $user";
}

```

```

if ($estatus != 'TODO') {
    $query .= " AND PR.estatus = '$estatus'";
}

```

```

if (count($prioridad) > 0) {
    $query .= " AND (0";
    foreach ($prioridad as $value) {
        $query .= " OR PR.prioridad = '$value'";
    }
    $query .= ")";
}

```

```

if (!empty($texto)) {
    if ($parametro == "folio") {
        $query .= " AND PR.id = '$texto'";
    }
    if ($parametro == "contenido") {
        $query .= " AND (
            PR.descripcion LIKE '%$texto%' OR

```

```

        UPPER(PR.atributos->'$.marca') LIKE UPPER('%$texto%') OR
        UPPER(PR.atributos->'$.modelo') LIKE UPPER('%$texto%')
    );
}
}

$query = " ORDER BY PR.fecha DESC";

$res = $this->Modelo->consulta($query);
if ($res) {
    echo json_encode($res);
} else {
    echo "";
}
}

// Obtiene información detallada de una PR específica
function ajax_getPR() {
    $id = $this->input->post('id');

    $query = "SELECT PR.*, CONCAT(U.nombre, ' ', U.paterno) AS User, U.correo
    FROM prs PR
    INNER JOIN usuarios U ON U.id = PR.usuario
    WHERE PR.id = $id";

    $res = $this->Conexion->consultar($query, TRUE);

    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

// Obtiene detalles del proveedor relacionado a una PR
function ajax_getProveedorPR() {
    $id = $this->input->post('id');

```

```

$query = "SELECT
    E.id, QP.id AS idQP, P.entrega, E.nombre,
    QP.monto, QP.total, QP.moneda, QP.tiempo_entrega,
    QP.dias_habiles, QP.comentarios, QP.nominado, QP.seleccionado,
    QP.nombre_archivo, QP.vencimiento
FROM qr_proveedores QP
INNER JOIN empresas E ON E.id = QP.empresa
INNER JOIN proveedores P ON P.empresa = E.id
WHERE QP.id = '$id'";

$res = $this->Modelo->consulta($query);
if ($res) {
    echo json_encode($res);
} else {
    echo "";
}
}

// Cambia el estatus de una PR y envía correo si aplica
function ajax_setEstatusPR() {
    $this->load->model('compras_model');

    $estatus = $this->input->post('estatus');
    $id = $this->input->post('id');
    $aprobador = $this->session->id;

    // Bitácora del cambio de estatus (no se almacena en esta función, pero se construye)
    $bitacora = [$estatus, date("Y-m-d h:i:sa"), $aprobador];
    $estatusBit = json_encode([$bitacora]);

    $query = "UPDATE prs SET estatus = '$estatus' WHERE id = '$id'";
    $data['pr'] = $id;
    $data['user'] = $aprobador;
    $data['estatus'] = $estatus;

    // Si fue aprobado, agrega fecha y aprobador
    if ($estatus == "APROBADO") {
        $query = "UPDATE prs SET estatus = '$estatus', fecha_aprobacion =
CURRENT_TIMESTAMP(), aprobador = $aprobador WHERE id = '$id'";
    }
}

```

```

    $this->compras_model->estatusPR($data);
}

// Si fue cerrado, agrega fecha de entrega
if ($estatus == "CERRADO") {
    $query = "UPDATE prs SET estatus = '$estatus', entregado = CURRENT_TIMESTAMP()
WHERE id = '$id'";
    if ($this->Modelo->update($query)) {
        echo "1";
    }
    exit();
}

// Ejecuta la actualización de estatus
$res = $this->Modelo->update($query);

if ($res) {
    $pr = $this->Modelo->getPR($id);

    $datos['id'] = $id;
    $datos['fecha'] = date('d/m/Y h:i A');
    $datos['usuario'] = $pr->User;
    $datos['prioridad'] = $pr->prioridad;
    $datos['unidad'] = $pr->unidad;
    $datos['cantidad'] = $pr->cantidad;
    $datos['descripcion'] = $pr->descripcion;
    $datos['atributos'] = $pr->atributos;
    $datos['comentarios'] = $pr->comentarios;
    $datos['estatus'] = $estatus;

    // Si fue aprobado, notifica por correo
    if ($estatus == "APROBADO") {
        $datos['correos'] = array_merge([$qr->correo], $this->Modelo->getCorreosQR());
        $this->correos_pr->liberarPR($datos);
    }

    echo "1";
} else {

```

```

        echo "";
    }
}

// Agrega comentario a una PR y notifica a los correos relacionados
function agregarComentarioPR() {
    $id = $this->input->post('id');
    $comentario = $this->input->post('comentario');
    $tags = $this->input->post('txtTags');
    $correos = explode(",", $tags);

    // Correos del solicitante y del asignado a la QR relacionada
    $query = "SELECT u.correo AS correoReq, ua.correo AS correoA
        FROM prs pr
        JOIN usuarios u ON u.id = pr.usuario
        JOIN requisiciones_cotizacion qr ON qr.id = pr.qr
        JOIN usuarios ua ON qr.asignado = ua.id
        WHERE pr.id = $id";
    $res = $this->Modelo->consulta($query, TRUE);

    $mails = explode(',', $res->correoReq . ',' . $res->correoA);

    // Datos para enviar notificación de comentario
    $data['id'] = $id;
    $data['comentario'] = $comentario;
    $data['correos'] = $mails;
    $data['nombre'] = $this->session->nombre;

    $this->correos_pr->comentarioPR($data);

    // Guarda el comentario en la base de datos
    $insert = [
        'pr' => $id,
        'usuario' => $this->session->id,
        'comentario' => $comentario
    ];
    $this->Modelo->agregar_comentarioPR($insert);
}

```



```

// Envía comentario a correos marcados con etiquetas (si hay)
if (count($correos) > 0) {
    $datos['id'] = $id;
    $datos['comentario'] = $comentario;
    $datos['correos'] = $correos;
    $this->correos_pr->comentarioPR($datos);
}

redirect(base_url('compras/ver_pr/' . $id));
}

// Cambia estatus de PR con comentario incluido y notificación por correo
function ajax_setEstatusMsjPR() {
    $this->load->model('compras_model');

    $id = $this->input->post('id');
    $estatus = $this->input->post('estatus');
    $comentario_original = $this->input->post('comentario');
    $comentario = "<b><font color='red'>$estatus:</font></b> " . $comentario_original;
    $tags = $this->input->post('txtTags');
    $correos = explode(",", $tags);

    // Actualiza estatus en base de datos
    $query = "UPDATE prs SET estatus = '$estatus' WHERE id = '$id'";
    $data['pr'] = $id;
    $data['user'] = $this->session->id;
    $data['estatus'] = $estatus;
    $this->compras_model->estatusPR($data);

    $res = $this->Modelo->update($query);
    if ($res) {
        // Registra comentario con el estatus como prefijo
        $data = [
            'pr' => $id,
            'usuario' => $this->session->id,
            'comentario' => $comentario,
        ];
        $this->Modelo->agregar_comentarioPR($data);
    }
}

```

```

$pr = $this->Modelo->getPR($id);

$datos['id'] = $id;
$datos['fecha'] = date('d/m/Y h:i A');
$datos['usuario'] = $pr->User;
$datos['prioridad'] = $pr->prioridad;
$datos['unidad'] = $pr->unidad;
$datos['cantidad'] = $pr->cantidad;
$datos['descripcion'] = $pr->descripcion;
$datos['atributos'] = $pr->atributos;
$datos['comentarios'] = $pr->comentarios;
$datos['estatus'] = $estatus;

// Enviar correo solo si fue rechazado
if ($estatus == "RECHAZADO") {
    $datos['correos'] = [$pr->correo];
    $this->correos_pr->rechazoPR($datos);
}

// Correos personalizados vía etiquetas
if (count($correos) > 0) {
    $datos2['id'] = $id;
    $datos2['comentario'] = $comentario;
    $datos2['correos'] = $correos;
    $this->correos_pr->comentarioPR($datos2);
}

echo $comentario;
} else {
    echo "";
}
}

// Retorna lista de usuarios con permiso para aprobar PRs
function ajax_getLiberadoresCompra() {
    $query = "SELECT U.id, CONCAT(U.nombre, ' ', U.paterno, ' ', U.materno) AS Name
            FROM usuarios U

```

```
INNER JOIN privilegios P ON P.usuario = U.id
WHERE U.activo = 1 AND P.aprobar_pr = 1";
```

```
$res = $this->Conexion->consultar($query);
if ($res) {
    echo json_encode($res);
}
}
```

// Retorna lista de usuarios con permiso para aprobar cotizaciones

```
function ajax_getLiberadoresCotizacion() {
    $query = "SELECT U.id, CONCAT(U.nombre, ' ', U.paterno, ' ', U.materno) AS Name
    FROM usuarios U
    INNER JOIN privilegios P ON P.usuario = U.id
    WHERE U.activo = 1 AND P.aprobar_cotizacion = 1";
```

```
$res = $this->Conexion->consultar($query);
if ($res) {
    echo json_encode($res);
}
}
```

```
function exportarQR() {
    // Recolección de filtros desde POST
    $parametro = $this->input->post('rbBusqueda');
    $texto = $this->input->post('txtBusqueda');
    $estatus = $this->input->post('opEstatus');
    $usuario = $this->input->post('cbMisQrs');
    $cbNormal = $this->input->post('cbNormal');
    $cbUrgente = $this->input->post('cbUrgente');
    $cbInfoUrgente = $this->input->post('cbInfoUrgente');
    $cbProducto = $this->input->post('cbProducto');
    $cbServicio = $this->input->post('cbServicio');
    $asignado = $this->input->post('opAsignado');
    $archivo = $this->input->post('cbArchivo');
    $fecha1 = $this->input->post('fecha1');
    $fecha2 = $this->input->post('fecha2');
    $f1 = $fecha1 . ' 00:00:00';
    $f2 = $fecha2 . ' 23:59:59';
```

```

// Consulta base
$query = "SELECT R.id, R.fecha, R.usuario, R.prioridad, R.tipo, R.subtipo, R.cantidad,
R.cantidad_aprobada, R.unidad, R.clave_unidad, R.descripcion, R.atributos, R.critico, R.destino,
R.lugar_entrega, R.comentarios, R.estatus, R.asignado, CONCAT(U.nombre, ' ', U.paterno) as
User, R.fecha_liberacion
FROM requisiciones_cotizacion R
LEFT JOIN usuarios U ON R.usuario = U.id
WHERE 1 = 1";

// Filtros aplicados
if ($estatus != 'TODO') {
    $query .= " AND R.estatus = '$estatus'";
}
if (!empty($asignado)) {
    $query .= " AND R.asignado = '$asignado'";
}
if ($usuario == 'NORMAL') {
    $idUser = $this->session->id;
    $query .= " AND R.usuario = '$idUser'";
}

// Prioridades
$query .= " AND (1 = 0";
if (!empty($cbNormal)) $query .= " OR R.prioridad = '$cbNormal'";
if (!empty($cbUrgente)) $query .= " OR R.prioridad = '$cbUrgente'";
if (!empty($cbInfoUrgente)) $query .= " OR R.prioridad = '$cbInfoUrgente'";
$query .= ")";

// Tipo de producto o servicio
$query .= " AND (1 = 0";
if (!empty($cbProducto)) $query .= " OR R.tipo = '$cbProducto'";
if (!empty($cbServicio)) $query .= " OR R.tipo = '$cbServicio'";
$query .= ")";

// Filtro por privilegios internos o de venta
if ($this->session->privilegios['crear_qr_interno'] != $this->session-
>privilegios['crear_qr_venta']) {

```

```

        if ($this->session->privilegios['editar_qr'] == "0" && $this->session->privilegios['liberar_qr']
== "0") {
            if ($this->session->privilegios['crear_qr_interno'] == "1") {
                $query .= " AND R.destino = 'CONSUMO INTERNO'";
            } else {
                $query .= " AND R.destino = 'VENTA'";
            }
        }
    }
}

```

// Búsqueda por texto

```

if (!empty($texto)) {
    if ($parametro == "folio") {
        $query .= " AND R.id = '$texto'";
    }
    if ($parametro == "usuario") {
        $query .= " AND CONCAT(U.nombre, ' ', U.paterno) LIKE '%$texto%'";
    }
    if ($parametro == "contenido") {
        $query .= " AND (R.descripcion LIKE '%$texto%'
                        OR UPPER(R.atributos->'$.marca') LIKE UPPER('%$texto%')
                        OR UPPER(R.atributos->'$.modelo') LIKE UPPER('%$texto%'))";
    }
}
}

```

// Rango de fechas

```

if (!empty($fecha1) && !empty($fecha2)) {
    $query .= " AND R.fecha BETWEEN '$f1' AND '$f2'";
}

```

// Solo QRs del último año

```

$query .= " AND R.maximo_vencimiento > (CURRENT_DATE() - INTERVAL 1 YEAR)";

```

// Filtrar si no es exportación completa

```

if ($archivo != "1") {
    $query .= " AND R.fecha > '2021-01-01 00:00:00'";
}

```

```

$query = " ORDER BY R.fecha DESC";

$result = $this->Conexion->consultar($query);

// Construcción de tabla para exportar
$salida = '<table style="border: 1px solid black; border-collapse: collapse;">
    <thead>
        <tr>
            <th style="background-color: #F3F1F1; border: 1px solid black;">QR</th>
            <th style="background-color: #F3F1F1; border: 1px solid black;">Fecha</th>
            <th style="background-color: #F3F1F1; border: 1px solid black;">Requisitor</th>
            <th style="background-color: #F3F1F1; border: 1px solid black;">Prioridad</th>
            <th style="background-color: #F3F1F1; border: 1px solid black;">Tipo</th>
            <th style="background-color: #F3F1F1; border: 1px solid black;">Subtipo</th>
            <th style="background-color: #F3F1F1; border: 1px solid black;">Cantidad</th>
            <th style="background-color: #F3F1F1; border: 1px solid black;">Estatus</th>
            <th style="background-color: #F3F1F1; border: 1px solid black;">Fecha Liberado</th>
            <th style="background-color: #F3F1F1; border: 1px solid black;">Días
Transcurridos</th>
            <th style="background-color: #F3F1F1; border: 1px solid black;">Descripción</th>
        </tr>
    </thead>
    <tbody>';

$d = 2;
foreach ($result as $row) {
    $salida .= '<tr>
        <td style="border: 1px solid black;">' . $row->id . '</td>
        <td style="border: 1px solid black;">' . $row->fecha . '</td>
        <td style="border: 1px solid black;">' . $row->User . '</td>
        <td style="border: 1px solid black;">' . $row->prioridad . '</td>
        <td style="border: 1px solid black;">' . $row->tipo . '</td>
        <td style="border: 1px solid black;">' . $row->subtipo . '</td>
        <td style="border: 1px solid black;">' . $row->cantidad . '</td>
        <td style="border: 1px solid black;">' . $row->estatus . '</td>
        <td style="border: 1px solid black;">' . $row->fecha_liberacion . '</td>
        <td style="border: 1px solid black;">=SI(ESBLANCO($I' . $d . '), "", DIAS($I' . $d . ', $B' . $d .
    '))</td>

```

```

        <td style="border: 1px solid black;">' . $row->descripcion . '</td>
    </tr>';
    $d++;
}

$salida .= '</tbody></table>';

// Preparar headers para descarga como archivo Excel
$timestamp = date('m-d-Y');
$filename = 'QR_' . $timestamp . '.xls';
header("Content-Type: application/vnd.ms-excel");
header("Content-Disposition: attachment; filename=\"$filename\"");
header("Pragma: no-cache");
header("Expires: 0");
header("Content-Transfer-Encoding: binary");

echo $salida;
}

function exportarPR()
{
    // Entrada de parámetros desde POST
    $opc = $this->input->post('rbBusqueda');
    $texto = $this->input->post('txtBusqueda');
    $estatus = $this->input->post('opEstatus');
    $fecha1 = $this->input->post('fecha1');
    $fecha2 = $this->input->post('fecha2');
    $f1 = strval($fecha1) . ' 00:00:00';
    $f2 = strval($fecha2) . ' 23:59:59';
    $stock = $this->input->post('stock');
    $archivo = $this->input->post('cbArchivo');

    // Consulta base
    $query = "SELECT
        PR.id as PR,
        PR.fecha as Fecha,
        concat(U.nombre, ' ', U.paterno) as Revisor,
        PR.prioridad as Prioridad,

```

```

PR.tipo as Tipo,
PR.subtipo as Subtipo,
PR.cantidad as Cantidad,
PR.estatus as Estatus,
ifnull((
    SELECT OCC.po
    FROM ordenes_compra_conceptos OCC
    INNER JOIN ordenes_compra OC ON OCC.po = OC.id
    WHERE OCC.pr = PR.id AND OC.estatus != 'CANCELADA'
    LIMIT 1
), 0) as PO,
concat('$ ', format(PR.importe, 2)) as MONTO,
PR.moneda as Moneda,
(
    SELECT P.entrega
    FROM qr_proveedores QP
    INNER JOIN empresas E ON E.id = QP.empresa
    INNER JOIN proveedores P ON P.empresa = E.id
    WHERE QP.id = PR.qr_proveedor
) as Lugar,
(
    SELECT E.nombre
    FROM qr_proveedores QP
    INNER JOIN empresas E ON E.id = QP.empresa
    INNER JOIN proveedores P ON P.empresa = E.id
    WHERE QP.id = PR.qr_proveedor
) as Proveedor,
PR.descripcion as Descripcion
FROM prs PR
LEFT JOIN usuarios U ON PR.usuario = U.id
WHERE 1 = 1";

```

// Filtros dinámicos

```

if ($estatus != 'TODO') {
    $query .= " AND PR.estatus = '$estatus'";
}

```

```

if (!empty($texto)) {

```



```

if ($opc == "folio") {
    $query .= " AND PR.id = '$texto'";
}
if ($opc == "usuario") {
    $query .= " HAVING Requisitor LIKE '%$texto%'";
}
if ($opc == "contenido") {
    $query .= " AND (PR.descripcion LIKE '%$texto%'
        OR UPPER(PR.atributos->'$.marca') LIKE UPPER('%$texto%')
        OR UPPER(PR.atributos->'$.modelo') LIKE UPPER('%$texto%'))";
}
}

if (!empty($fecha1) && !empty($fecha2)) {
    $query .= " AND PR.fecha BETWEEN '$fecha1' AND '$fecha2'";
}

if (!empty($stock)) {
    $query .= " AND PR.stock = 1";
}

if ($archivo != "1") {
    $query .= " AND PR.fecha > '2021-01-01 00:00:00'";
}

$query .= " ORDER BY PR.fecha DESC";

// Ejecutar consulta
$result = $this->Conexion->consultar($query);

// Construcción de tabla HTML para exportar a Excel
$salida = "";
$salida .= '<table style="border: 1px solid black; border-collapse: collapse;">
    <thead>
        <tr>
            <th style="background-color: #F3F1F1; border: 1px solid black;">PR</th>
            <th style="background-color: #F3F1F1; border: 1px solid black;">Fecha</th>
            <th style="background-color: #F3F1F1; border: 1px solid black;">Requisitor</th>

```

```

        <th style="background-color: #F3F1F1; border: 1px solid black;">Prioridad</th>
        <th style="background-color: #F3F1F1; border: 1px solid black;">Tipo</th>
        <th style="background-color: #F3F1F1; border: 1px solid black;">Subtipo</th>
        <th style="background-color: #F3F1F1; border: 1px solid black;">Cantidad</th>
        <th style="background-color: #F3F1F1; border: 1px solid black;">Estatus</th>
        <th style="background-color: #F3F1F1; border: 1px solid black;">PO</th>
        <th style="background-color: #F3F1F1; border: 1px solid black;">Monto</th>
        <th style="background-color: #F3F1F1; border: 1px solid black;">Moneda</th>
        <th style="background-color: #F3F1F1; border: 1px solid black;">Lugar de
entrega</th>
        <th style="background-color: #F3F1F1; border: 1px solid
black;">Proveedor</th>
        <th style="background-color: #F3F1F1; border: 1px solid
black;">Descripcion</th>
    </tr>
</thead>
<tbody>';

```

```

foreach ($result as $row) {

```

```

    $salida .= '

```

```

        <tr>

```

```

            <td style="border: 1px solid black;">' . $row->PR . '</td>

```

```

            <td style="border: 1px solid black;">' . $row->Fecha . '</td>

```

```

            <td style="border: 1px solid black;">' . $row->Requisitor . '</td>

```

```

            <td style="border: 1px solid black;">' . $row->Prioridad . '</td>

```

```

            <td style="border: 1px solid black;">' . $row->Tipo . '</td>

```

```

            <td style="border: 1px solid black;">' . $row->Subtipo . '</td>

```

```

            <td style="border: 1px solid black;">' . $row->Cantidad . '</td>

```

```

            <td style="border: 1px solid black;">' . $row->Estatus . '</td>

```

```

            <td style="border: 1px solid black;">' . $row->PO . '</td>

```

```

            <td style="border: 1px solid black;">' . $row->MONTO . '</td>

```

```

            <td style="border: 1px solid black;">' . $row->Moneda . '</td>

```

```

            <td style="border: 1px solid black;">' . $row->Lugar . '</td>

```

```

            <td style="border: 1px solid black;">' . $row->Proveedor . '</td>

```

```

            <td style="border: 1px solid black;">' . $row->Descripcion . '</td>

```

```

        </tr>';

```

```

    }

```

```

$salida .= '</tbody></table>';

// Preparar headers para descargar el archivo como Excel
$timestamp = date('m/d/Y', time());
$filename = 'PR_' . $timestamp . '.xls';

header("Content-Type: application/vnd.ms-excel");
header("Content-Disposition: attachment; filename=\"$filename\"");
header("Pragma: no-cache");
header("Expires: 0");
header('Content-Transfer-Encoding: binary');

echo $salida;
}

function AllPR(){
// Obtiene el ID de la PR desde POST
$id = $this->input->post('CURRENT_PR');

// Consulta toda la información de la PR específica
$query = "SELECT * from prs where id = ".$id;

// Ejecuta la consulta
$res = $this->Conexion->consultar($query, $id);

// Si hay resultado, lo devuelve como JSON
if($res) {
    echo json_encode($res);
}
}

function ajax_getNombresUsuarios(){
// Obtiene el ID de la PR desde POST
$id = $this->input->post('CURRENT_PR');

// Consulta la bitácora de estatus para esa PR y quién los realizó
$query = "SELECT concat(u.nombre, ' ', u.paterno) as user, b.fecha, b.estatus
          from bitacora_prs b

```

```

        JOIN usuarios u on b.user = u.id
        where pr = ".$ids;

// Ejecuta la consulta y devuelve JSON si hay resultados
$res = $this->Conexion->consultar($query);
if($res) {
    echo json_encode($res);
}
}

function bitacoraQR(){
    // Obtiene el ID del QR desde POST
    $ids = $this->input->post('id');

    // Consulta la bitácora de estatus para ese QR y quién los realizó
    $query = "SELECT concat(u.nombre, ' ', u.paterno) as user, b.fecha, b.estatus
        from bitacora_qrs b
        JOIN usuarios u on b.user = u.id
        where qr = ".$ids;

    // Ejecuta la consulta y devuelve JSON si hay resultados
    $res = $this->Conexion->consultar($query);
    if($res) {
        echo json_encode($res);
    }
}

public function ajax_surtirPR()
{
    // Obtiene el ID de la PR desde POST
    $pr = $this->input->post('id');

    // Establece zona horaria y obtiene fecha/hora actual
    date_default_timezone_set('America/Chihuahua');
    $date = date('Y-m-d h:i:s');

    // Prepara los datos para actualizar la PR
    $data['estatus'] = "POR RECIBIR";

```

```

$data['stock'] = 1;
$data['surtidorStock'] = $this->session->id;
$data['fechaSurtido'] = $date;

$where['id'] = intval($pr);

// Ejecuta la actualización en la tabla `prs`
$res = $this->Conexion->modificar('prs', $data, null, $where);

// Registra en bitácora el evento "SURTIDO DE STOCK"
$bitacoraPR['pr'] = intval($pr);
$bitacoraPR['user'] = $this->session->id;
$bitacoraPR['estatus'] = 'SURTIDO DE STOCK';
$this->Modelo->estatusPR($bitacoraPR);

// Registra en bitácora el nuevo estatus "POR RECIBIR"
$bitacoraPR['pr'] = intval($pr);
$bitacoraPR['user'] = $this->session->id;
$bitacoraPR['estatus'] = 'POR RECIBIR';
$this->Modelo->estatusPR($bitacoraPR);

// Devuelve 1 si la actualización fue exitosa
if ($res) {
    echo 1;
}
}

function ajax_getUsuariosQR(){
// Obtener usuarios activos del departamento de COMPRAS con privilegios de editar QR
$query = "SELECT U.id, concat(U.nombre, ' ', U.paterno) as Nombre, P.puesto as Puesto,
U.correo, PR.editar_qr
FROM usuarios U
INNER JOIN puestos P ON U.puesto = P.id
INNER JOIN privilegios PR ON PR.usuario = U.id
WHERE U.activo = 1 AND U.departamento='COMPRAS'";

$res = $this->Conexion->consultar($query);
if($res) {

```

```

        echo json_encode($res);
    }
}

function asignarUsuariosQR(){
    // Obtener datos desde POST
    $qr = $this->input->post('qr');
    $id = $this->input->post('id');

    // Consultar correo del usuario a asignar
    $mail = "SELECT correo FROM usuarios WHERE id = ".$id;
    $correo = $this->Conexion->consultar($mail);

    // Enviar correo de asignación
    foreach($correo as $elem){
        $dato['qr'] = $qr;
        $dato['correo'] = $elem->correo;
        $this->correos->asignarQR($dato);
    }

    // Guardar asignación en base de datos
    date_default_timezone_set('America/Chihuahua');
    $date = date('Y-m-d h:i:s');

    $data['asignado'] = intval($id);
    $data['asignador'] = $this->session->id;
    $data['fechaAsignacion'] = $date;
    $where['id'] = intval($qr);
    $this->Conexion->modificar('requisiciones_cotizacion', $data, null, $where);

    // Registrar bitácora
    $bitacoraQR['qr'] = intval($qr);
    $bitacoraQR['user'] = $this->session->id;
    $bitacoraQR['estatus'] = 'Asignacion por: ' . $this->session->nombre;
    $this->Modelo->estatusQR($bitacoraQR);

    // Consultar y devolver información completa de asignación

```

```

$query = "SELECT U.id, concat(U.nombre, ' ', U.paterno) as Nombre, P.puesto as Puesto,
U.correo,
        qr.id as QR, qr.asignado, fechaAsignacion, concat(A.nombre, ' ', A.paterno) as
Asignador

```

```

        FROM usuarios U
        INNER JOIN puestos P ON U.puesto = P.id
        JOIN requisiciones_cotizacion qr ON qr.asignado = U.id
        JOIN usuarios A ON A.id = qr.asignador
        WHERE qr.id = ".$qr;

```

```

$res = $this->Conexion->consultar($query);
if($res) {
    echo json_encode($res);
}
}

```

```

function buscarML(){
    // Recibe parámetros por POST
    $item = $this->input->post('item');
    $destino = $this->input->post('destino');
    $tipo = $this->input->post('tipo');

    // Consulta los items disponibles según tipo/destino
    if ($destino == 'VENTA' && $tipo == 'PRODUCTO') {
        $query = "SELECT rs.Item_id, rs.Equipo_ID, rs.serie , rs.tecnico_id, rs.fechaActEQ,
rs.Modelo, rs.Fabricante, t.Nombre
        FROM rsitems rs
        JOIN catalogo_tecnicos t ON rs.tecnico_id = t.Tecnico_Id
        WHERE item_id = ".$item." AND Equipo_id IS NULL AND fechaActEQ IS NULL";
    } else {
        $query = "SELECT rs.Item_id, rs.Equipo_ID, rs.serie , rs.tecnico_id, rs.fechaActEQ,
rs.Modelo, rs.Fabricante, t.Nombre
        FROM rsitems rs
        JOIN catalogo_tecnicos t ON rs.tecnico_id = t.Tecnico_Id
        WHERE item_id = ".$item." AND Equipo_id IS NOT NULL AND fechaActEQ IS NULL";
    }
}

```

```

// Ejecutar consulta

```

```

$res = $this->MLConexion->consultar($query);
if($res) {
    echo json_encode($res);
}
}

function agregarAtributos(){
    // Recibe atributos e ID de QR por POST
    $att = $this->input->post('ATT');
    $qr = $this->input->post('qr');

    // Inserta los atributos recibidos en la tabla temporal
    foreach($att as $elem){
        $datos['idQr'] = intval($qr);
        $datos['item'] = $elem['Item_id'];
        $datos['equipo'] = $elem['Equipo_ID'];
        $datos['serie'] = $elem['serie'];
        $datos['modelo'] = $elem['Modelo'];
        $datos['fabricante'] = $elem['Fabricante'];
        $datos['asignado'] = $elem['Nombre'];
        $this->Conexion->insertar('qr_atributos_temp', $datos, $funciones = null);
    }

    // Consultar todos los atributos agregados para ese QR
    $query = "SELECT * FROM qr_atributos_temp WHERE idQr = ".$qr;
    $res = $this->Conexion->consultar($query);
    if($res) {
        echo json_encode($res);
    }
}

function cargarItems(){
    // Recibe ID de QR por POST
    $qr = $this->input->post('qr');

    // Consulta todos los items temporales relacionados a ese QR
    $query = "SELECT * FROM qr_atributos_temp WHERE idQr = ".$qr;
    $res = $this->Conexion->consultar($query);

```



```

if($res) {
    echo json_encode($res);
}
}

function eliminarAtributos(){
// Elimina un atributo temporal del QR y devuelve los atributos restantes
$id = $this->input->post('id');
$qqr = $this->input->post('qr');
$where['id'] = $id;

$this->Conexion->eliminar('qr_atributos_temp', $where);

$query = "SELECT * FROM qr_atributos_temp WHERE idQr = ".$qqr;
$res = $this->Conexion->consultar($query);

if($res) {
    echo json_encode($res);
}
}

function asignarCompras(){
// Asigna un técnico fijo (id 34) a un item en rsitems (ML), y lo etiqueta como 'Compras' en
atributos temporales
$item = $this->input->post('item');
$qqr = $this->input->post('qr');

$data['tecnico_id'] = '34';
$where['Item_id'] = $item;
$this->MLConexion->modificar('rsitems', $data, null, $where);

$dato['asignado'] = 'Compras';
$donde['item'] = $item;
$this->Conexion->modificar('qr_atributos_temp', $dato, null, $donde);

$query = "SELECT * FROM qr_atributos_temp WHERE idQr = ".$qqr;
$res = $this->Conexion->consultar($query);

```

```

if($res) {
    echo json_encode($res);
}
}

function validarAtributos(){
    // Verifica si un item ya está registrado en un QR específico
    $item = $this->input->post('item');
    $qr = $this->input->post('qr');
    $query = "SELECT * FROM qr_atributos_temp WHERE item = ".$item." AND idQr = ".$qr;
    $res = $this->Conexion->consultar($query);

    if($res) {
        echo json_encode($res);
    }
}

function getItem(){
    // Obtiene detalles de un atributo PR específico
    $item = $this->input->post('item');
    $query = "SELECT * FROM pr_atributos WHERE id = ".$item;
    $res = $this->Conexion->consultar($query, TRUE);

    if($res) {
        echo json_encode($res);
    }
}

function ValidarItem(){
    // Verifica si un item ya está ligado a una PR activa (no cancelada)
    $item = $this->input->post('item');

    $query = "SELECT pa.item, pa.idPr, pr.estatus
    FROM pr_atributos pa
    JOIN prs pr ON pa.idPr = pr.id
    WHERE pa.item = '".$item."' AND pr.estatus != 'CANCELADO'";
    $res = $this->Conexion->consultar($query, TRUE);

```

```

if($res) {
    echo json_encode($res);
}
}

function getFileEx(){
    // Obtiene el nombre y contenido binario del archivo de ejemplo asociado a un QR
    $id = $this->input->post('qr');

    $query = "SELECT nombre_archivoEjemplo, archivoEjemplo
              FROM requisiciones_cotizacion
              WHERE id = ".$id;

    $res = $this->Modelo->consulta($query, TRUE);

    if($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

function getFileExample($qr){
    // Despliega un archivo PDF ejemplo en el navegador sin forzar descarga
    $row = $this->descargas_model->getFile($qr, 'requisiciones_cotizacion');
    $file = $row->archivoEjemplo;
    $nombre = $row->nombre_archivoEjemplo;

    header('Content-type: application/pdf');
    header('Content-Disposition: inline; filename="" . $nombre . "");
    header('Content-Transfer-Encoding: binary');
    header('Accept-Ranges: bytes');
    echo $file;
}

function ajax_subirArchivoQRClon() {
    // Clona el archivo PDF de un QR y lo asocia al QR más reciente
    $id = $this->input->post('qr');

```

```

// Obtener archivo original del QR base
$query = "SELECT nombre_archivo, archivo FROM requisiciones_cotizacion WHERE id = " . $id;
$res = $this->Conexion->consultar($query, TRUE);

$datos['archivo'] = $res->archivo;

// Obtener el último QR generado (para usar su ID como nuevo archivo)
$query = "SELECT * FROM requisiciones_cotizacion ORDER BY id DESC LIMIT 1";
$r = $this->Conexion->consultar($query, TRUE);

$datos['id'] = $r->id;
$datos['nombre_archivo'] = str_pad($datos['id'], 6, "0", STR_PAD_LEFT) . ".pdf";

if (!$this->Modelo->setQRFile($datos)) {
    trigger_error("Error al subir archivo", E_USER_ERROR);
} else {
    echo "1";
}
}

function checkFile() {
    // Verifica si el QR tiene un archivo cargado
    $id = $this->input->post('qr');
    $query = "SELECT nombre_archivo FROM requisiciones_cotizacion WHERE id = " . $id;
    $res = $this->Modelo->consulta($query, TRUE);

    if ($res->nombre_archivo) {
        echo 1;
    } else {
        echo 0;
    }
}

function uploadFileEx() {
    // Sube archivo de ejemplo (binario) a la requisición
    $datos['id'] = $this->input->post('qr');
    $datos['archivoEjemplo'] = file_get_contents($_FILES['file']['tmp_name']);

```

```

$datos['nombre_archivoEjemplo'] = $_FILES['file']['name'];

if (!$this->Modelo->archivo_ejemplo($datos)) {
    trigger_error("Error al subir archivo", E_USER_ERROR);
} else {
    echo $datos['nombre_archivoEjemplo'];
}
}

function eliminarArchivoEx() {
    // Elimina el archivo de ejemplo asociado a una requisición
    $datos['id'] = $this->input->post('qr');
    $datos['archivoejemplo'] = null;
    $datos['nombre_archivoejemplo'] = null;

    if (!$this->Modelo->archivo_ejemplo($datos)) {
        trigger_error("Error al eliminar archivo", E_USER_ERROR);
    } else {
        echo "1";
    }
}

function checkDestino() {
    // Devuelve el destino asignado a una requisición QR
    $id = $this->input->post('qr');
    $query = "SELECT destino FROM requisiciones_cotizacion WHERE id = " . $id;
    $res = $this->Conexion->consultar($query, TRUE);

    if ($res) {
        echo json_encode($res);
    } else {
        echo "1";
    }
}
}
}

```

Configuracion.php

```
<?php

defined('BASEPATH') OR exit('No direct script access allowed');

// Controlador Configuracion: Maneja vistas de configuración general
class Configuracion extends CI_Controller {

    // Constructor: Carga modelos necesarios
    function __construct() {
        parent::__construct();
        $this->load->model('conexion_model','Conexion');    // Modelo para conexión a BD
        $this->load->model('privilegios_model');              // Modelo para privilegios y
notificaciones
    }

    // Vista principal de configuración de compras
    function compras() {
        $this->load->view('header');                          // Carga el encabezado
        $this->load->view('configuracion/compra_opciones');   // Carga opciones de compras
    }

    // Vista principal de configuración de servicios
    function servicios() {
        $this->load->view('header');                          // Carga el encabezado
        $this->load->view('configuracion/servicios_opciones'); // Carga opciones de servicios
    }

    // Vista principal de configuración de requerimientos
    function requerimientos() {
        $this->load->view('header');                          // Carga el encabezado
        $this->load->view('configuracion/requerimientos_opciones'); // Carga opciones de
requerimientos
    }

    // ===== SERVICIOS =====
```

```

// Vista de claves y precios de servicios
function claves_precio(){
    $this->load->view('header');           // Carga el encabezado
    $this->load->view('configuracion/servicios/claves_precio');// Vista de claves y precios
}

// Vista de magnitudes de servicios
function magnitudes(){
    $this->load->view('header');           // Carga el encabezado
    $this->load->view('configuracion/servicios/magnitudes');// Vista de magnitudes
}

// Vista para configuración de correo en servicios
function correo(){
    $this->load->view('header');           // Carga el encabezado
    $this->load->view('configuracion/servicios/correo');// Vista de correo
}

// ===== COMPRAS =====

// Vista para dirección de envío
function shipping_address() {
    $this->load->view('header');           // Carga el encabezado
    $this->load->view('configuracion/compras/shipping_address');// Dirección de envío
}

// Vista para dirección de facturación
function billing_address() {
    $this->load->view('header');           // Carga el encabezado
    $this->load->view('configuracion/compras/billing_address');// Dirección de facturación
}

// Vista para opciones de pago
function pagos() {
    $this->load->view('header');           // Carga el encabezado
    $this->load->view('configuracion/compras/pagos');// Vista de pagos
}

```

```

// ===== NOTIFICACIONES =====

// Vista de notificaciones con datos desde modelo de privilegios
function notificaiones() {
    $datos['noti'] = $this->privilegios_model->getNotificaciones(); // Obtiene notificaciones
    $this->load->view('header'); // Carga el encabezado
    $this->load->view('configuracion/notificaciones', $datos); // Vista con notificaciones
}
}

// Obtiene claves de precio desde la base de datos (por ID si se proporciona)
function ajax_getClavesPrecio() {
    $id = 0;
    if(isset($_POST['id'])) {
        $id = $this->input->post('id'); // Recupera ID si está presente en POST
    }

    $query = "SELECT * from claves_precio";
    $row = $id > 0;

    // Si se especificó un ID, se agrega condición WHERE
    if($row) {
        $query .= " where id = $id";
    }

    $res = $this->Conexion->consultar($query, $row);

    // Devuelve resultado en JSON o "0" si falla
    if($res) {
        echo json_encode($res);
    } else {
        echo "0";
    }
}

// Actualiza valores de bajo y alto en la tabla claves_precio
function ajax_setClavesPrecio() {
    $id = $this->input->post('id'); // ID del registro a actualizar

```



```

$bajo = $this->input->post('bajo'); // Valor bajo
$alto = $this->input->post('alto'); // Valor alto

$datos['bajo'] = $bajo;
$datos['alto'] = $alto;
$where['id'] = $id;

$this->Conexion->modificar('claves_precio', $datos, null, $where); // Realiza la actualización
echo "1"; // Éxito
}

// Obtiene registros de magnitudes desde la base de datos (opcionalmente por ID)
function ajax_getMagnitudes() {
    $id = 0;
    if(isset($_POST['id'])) {
        $id = $this->input->post('id'); // Recupera ID si está presente en POST
    }

    $query = "SELECT * from magnitudes";
    $row = $id > 0;

    // Si se especificó un ID, se agrega condición WHERE
    if($row) {
        $query .= " where id = $id";
    }

    $res = $this->Conexion->consultar($query, $row);

    // Devuelve resultado en JSON o "0" si falla
    if($res) {
        echo json_encode($res);
    } else {
        echo "0";
    }
}

// Obtiene textos de correo activos, con nombre del usuario asociado (opcionalmente por ID)
function ajax_getTexto_correo() {

```

```

$id = 0;
if(isset($_POST['id'])) {
    $id = $this->input->post('id'); // Recupera ID si está presente en POST
}

// Consulta que une texto_correo con usuarios y filtra solo activos
$query = "SELECT tc.*, concat(u.nombre, ' ', u.paterno) as us
        FROM texto_correo tc
        JOIN usuarios u on u.id = tc.id_us
        WHERE tc.activo = 1";

$row = $id > 0;

// Si se especificó un ID, se agrega condición adicional
if($row) {
    $query .= " and tc.id = $id";
}

$res = $this->Conexion->consultar($query, $row);

// Devuelve resultado en JSON o "0" si falla
if($res) {
    echo json_encode($res);
} else {
    echo "0";
}
}

// Inserta o actualiza una magnitud
function ajax_setMagnitudes() {
    $id = 0;
    if(isset($_POST['id'])) {
        $id = $this->input->post('id'); // ID del registro (si aplica)
    }

    $magnitud = $this->input->post('magnitud'); // Nombre de la magnitud
    $prefijo = $this->input->post('prefijo'); // Prefijo asociado

    $datos['magnitud'] = $magnitud;

```

```

$datos['prefijo'] = $prefijo;

$res = FALSE;

// Si el ID es 0, se inserta nuevo registro
if($id == 0) {
    $res = $this->Conexion->insertar('magnitudes', $datos) > 0;
} else {
    // Si ya existe, se actualiza el registro
    $where['id'] = $id;
    $res = $this->Conexion->modificar('magnitudes', $datos, null, $where) >= 0;
}

if($res) {
    echo "1"; // Éxito
}
}

// Elimina una magnitud por ID
function ajax_deleteMagnitudes() {
    $id = $this->input->post('id'); // ID del registro a eliminar
    $where['id'] = $id;

    // Elimina y devuelve "1" si tiene éxito
    if($this->Conexion->eliminar('magnitudes', $where)) {
        echo "1";
    }
}

// Verifica si un prefijo ya existe en la tabla magnitudes
function ajax_prefijoExists() {
    $prefijo = $this->input->post('prefijo'); // Prefijo a verificar

    $query = "SELECT count(M.prefijo) as Qty
    FROM magnitudes M
    WHERE 1 = 1 AND prefijo = '$prefijo'";

    $res = $this->Conexion->consultar($query, TRUE);

```

```

// Devuelve cantidad de coincidencias o vacío si falla
if($res) {
    echo json_encode($res);
} else {
    echo "";
}
}

// Obtiene direcciones de envío (shipping_address), por ID si se especifica
function ajax_getShippingAddresses() {
    $id = 0;
    if(isset($_POST['id'])) {
        $id = $_this->input->post('id'); // ID del registro si aplica
    }

    $query = "SELECT * from shipping_address";
    $row = $id > 0;

    // Si hay ID, se agrega condición WHERE
    if($row) {
        $query .= " where id = $id";
    }

    $res = $_this->Conexion->consultar($query, $row);

    // Devuelve resultados en JSON o "0" si no hay resultados
    if($res) {
        echo json_encode($res);
    } else {
        echo "0";
    }
}

// Inserta o actualiza una dirección de envío (shipping_address)
function ajax_setShippingAddresses() {
    $id = 0;
    if(isset($_POST['id'])) {

```

```

    $id = $this->input->post('id'); // ID si se va a actualizar
}

$nombre = $this->input->post('nombre');    // Nombre del destinatario
$direccion = $this->input->post('direccion'); // Dirección de envío
$pais = $this->input->post('pais');        // País

$datos['nombre'] = $nombre;
$datos['direccion'] = $direccion;
$datos['pais'] = $pais;

$res = FALSE;

// Inserta nuevo registro si ID es 0
if($id == 0) {
    $datos['default'] = '0'; // No es default por defecto
    $res = $this->Conexion->insertar('shipping_address', $datos) > 0;
} else {
    // Actualiza dirección existente
    $where['id'] = $id;
    $res = $this->Conexion->modificar('shipping_address', $datos, null, $where) >= 0;
}

if($res) {
    echo "1"; // Éxito
}
}

// Elimina una dirección de envío por ID
function ajax_deleteShippingAddresses() {
    $id = $this->input->post('id'); // ID de la dirección a eliminar
    $where['id'] = $id;

    // Elimina el registro y responde "1" si tuvo éxito
    if($this->Conexion->eliminar('shipping_address', $where)) {
        echo "1";
    }
}
}

```

```

// Establece una dirección como predeterminada para un país específico
function ajax_setDefaultShippingAddresses() {
    $where['pais'] = $this->input->post('pais'); // País al que pertenece la dirección
    $datos['default'] = '0';

    // Primero quita el estatus "default" de todas las direcciones del país
    $this->Conexion->modificar('shipping_address', $datos, null, $where);

    // Luego asigna como default la dirección indicada por ID
    $where['id'] = $this->input->post('id');
    $datos['default'] = '1';
    $this->Conexion->modificar('shipping_address', $datos, null, $where);

    echo "1"; // Confirmación de éxito
}

// Obtiene direcciones de facturación (billing_address), por ID si se especifica
function ajax_getBillingAddresses() {
    $id = 0;
    if(isset($_POST['id'])) {
        $id = $this->input->post('id'); // ID si se desea un registro específico
    }

    $query = "SELECT * from billing_address";
    $row = $id > 0;

    // Si se especificó ID, agrega condición WHERE
    if($row) {
        $query .= " where id = $id";
    }

    $res = $this->Conexion->consultar($query, $row);

    // Devuelve resultado en JSON o "0" si no hay datos
    if($res) {
        echo json_encode($res);
    } else {

```

```

        echo "0";
    }
}

// Inserta o actualiza una dirección de facturación (billing_address)
function ajax_setBillingAddresses() {
    $id = 0;
    if(isset($_POST['id'])) {
        $id = $this->input->post('id'); // ID si es una actualización
    }

    $nombre = $this->input->post('nombre'); // Nombre del cliente
    $direccion = $this->input->post('direccion'); // Dirección fiscal

    $datos['nombre'] = $nombre;
    $datos['direccion'] = $direccion;

    $res = FALSE;

    // Si no hay ID, se inserta nuevo registro
    if($id == 0) {
        $datos['default'] = '0'; // No es default por defecto
        $res = $this->Conexion->insertar('billing_address', $datos) > 0;
    } else {
        // Actualiza registro existente
        $where['id'] = $id;
        $res = $this->Conexion->modificar('billing_address', $datos, null, $where) >= 0;
    }

    if($res) {
        echo "1"; // Éxito
    }
}

// Elimina una dirección de facturación por ID
function ajax_deleteBillingAddresses() {
    $id = $this->input->post('id'); // ID del registro a eliminar
    $where['id'] = $id;

```

```

// Ejecuta la eliminación y devuelve "1" si tiene éxito
if($this->Conexion->eliminar('billing_address', $where)) {
    echo "1";
}
}

// Establece una dirección de facturación como predeterminada
function ajax_setDefaultBillingAddresses() {
    $where['id'] = $this->input->post('id'); // ID de la dirección que será default

    $datos['default'] = '0';

    // Pone todas las direcciones como no predeterminadas
    $this->Conexion->modificar('billing_address', $datos, null, null);

    // Establece como predeterminada la dirección especificada
    $datos['default'] = '1';
    $this->Conexion->modificar('billing_address', $datos, null, $where);

    echo "1"; // Éxito
}

// Obtiene métodos de pago (empresa_metodos_pago), por ID si se proporciona
function ajax_getPagos() {
    $id = 0;
    if(isset($_POST['id'])) {
        $id = $this->input->post('id'); // ID del método de pago a obtener
    }

    $query = "SELECT * from empresa_metodos_pago";
    $row = $id > 0;

    // Aplica filtro si se especificó un ID
    if($row) {
        $query .= " where id = $id";
    }
}

```



```

$res = $this->Conexion->consultar($query, $row);

// Devuelve los datos en JSON o "0" si no hay resultados
if($res) {
    echo json_encode($res);
} else {
    echo "0";
}
}

// Inserta o actualiza un método de pago
function ajax_setPagos() {
    $metodo = json_decode($this->input->post('metodo_pago')); // Decodifica objeto desde
JSON
    $metodo->fecha_vencimiento = date("Y-m-d", strtotime($metodo->fecha_vencimiento)); //
Formatea fecha

    $res = FALSE;

    // Inserta nuevo método si el ID es 0
    if($metodo->id == 0) {
        $res = $this->Conexion->insertar('empresa_metodos_pago', $metodo) > 0;
    } else {
        // Actualiza método existente
        $where['id'] = $metodo->id;
        $res = $this->Conexion->modificar('empresa_metodos_pago', $metodo, null, $where) >= 0;
    }

    if($res) {
        echo "1"; // Éxito
    }
}

// Elimina un método de pago por ID
function ajax_deletePagos() {
    $id = $this->input->post('id'); // ID del método a eliminar
    $where['id'] = $id;

```

```

if($this->Conexion->eliminar('empresa_metodos_pago', $where)) {
    echo "1"; // Éxito
}
}

////////// REQUERIMIENTOS //////////

// Vista del evaluador predeterminado de requerimientos
function evaluador_requerimiento() {
    $this->load->view('header'); // Carga el header
    $this->load->view('configuracion/requerimientos/evaluador'); // Vista del evaluador
}

// Obtiene el evaluador predeterminado desde la configuración
function ajax_getEvaluador() {
    $query = "SELECT evaluador_default FROM requerimientos_config WHERE id = 1";
    $res = $this->Conexion->consultar($query, TRUE);
    echo json_encode($res); // Devuelve resultado en formato JSON
}

// Establece un nuevo evaluador predeterminado
function ajax_setEvaluador() {
    $datos['evaluador_default'] = $this->input->post('id'); // ID del nuevo evaluador

    // Actualiza el campo evaluador_default donde id = 1
    $res = $this->Conexion->modificar('requerimientos_config', $datos, null, array('id' => 1)) >= 0;

    if($res) {
        echo "1"; // Éxito
    }
}

// Guarda o actualiza preferencias de notificaciones del usuario
function guardarNot() {
    $query = "SELECT * FROM notificaciones WHERE idus = " . $this->session->id;
    $res = $this->Conexion->consultar($query);

    if (empty($res)) {

```

```

// Si no existen preferencias, se insertan
$not = array(
    'idUs' => $this->session->id,
    'qr' => $this->input->post('qr'),
    'tickets' => $this->input->post('tickets'),
    'pr' => $this->input->post('pr'),
    'po' => $this->input->post('po'),
    'cotizaciones' => $this->input->post('cot'),
    'facturas' => $this->input->post('fact'),
    'agenda' => $this->input->post('agenda'),
    'tool' => $this->input->post('tool'),
);
$this->privilegios_model->guardarNot($not);
} else {
    // Si ya existen, se actualizan
    $not = array(
        'qr' => $this->input->post('qr'),
        'tickets' => $this->input->post('tickets'),
        'pr' => $this->input->post('pr'),
        'po' => $this->input->post('po'),
        'cotizaciones' => $this->input->post('cot'),
        'facturas' => $this->input->post('fact'),
        'agenda' => $this->input->post('agenda'),
        'tool' => $this->input->post('tool'),
    );
    $this->privilegios_model->updateguardarNot($not);
}

// Redirige a la vista de notificaciones
redirect(base_url('configuracion/notificaciones'));
}

// Inserta o actualiza un texto de correo
function ajax_setTexto_correo() {
    $id = 0;
    if($this->input->post('id')) {
        $id = $this->input->post('id'); // ID del texto a modificar
    }
}

```

```

$datos['texto'] = $this->input->post('texto');    // Contenido del texto
$datos['activo'] = $this->input->post('activo');    // Estado (activo/inactivo)
$datos['id_us'] = $this->session->id;            // ID del usuario que lo modifica

$res = FALSE;

if($id == 0) {
    // Inserta nuevo texto
    $res = $this->Conexion->insertar('texto_correo', $datos) > 0;
} else {
    // Actualiza texto existente
    $where['id'] = $id;
    $res = $this->Conexion->modificar('texto_correo', $datos, null, $where) >= 0;
}

if($res) {
    echo "1"; // Éxito
}
}

// "Elimina" un texto de correo marcándolo como inactivo
function ajax_deleteTexto() {
    $id = $this->input->post('id');    // ID del texto a desactivar
    $where['id'] = $id;
    $datos['activo'] = 0;            // Lo marca como inactivo

    if($this->Conexion->modificar('texto_correo', $datos, null, $where)) {
        echo "1"; // Éxito
    }
}

```

Correo.php

```

<?php
defined('BASEPATH') OR exit('No direct script access allowed');

```

```
// Controlador para pruebas de envío de correo
class Correo extends CI_Controller {

    // Constructor vacío
    function __construct() {
        parent::__construct();
    }

    // Prueba de envío de correo usando librería 'correos'
    function index() {
        $this->load->library('correos');          // Carga librería personalizada
        $datos['titulo'] = 'Mantenimiento de Rutina'; // Título del ticket
        $datos['id'] = 'AT000023';                // ID del ticket
        $datos['prefijo'] = 'AT';                  // Prefijo del ticket
        $datos['fecha'] = '17-Ene-2019';           // Fecha de creación
        $datos['usuario'] = 'ALEJANDRO ORTIZ';      // Usuario responsable
        $this->correos->creacionTicket($datos);     // Envía correo de creación de ticket
    }

    // Prueba de múltiples configuraciones SMTP y envío con MailJet
    function prueba() {
        // Configuración para Gmail (no usada)
        $configGOOGLE = Array(
            'smtp_host' => 'smtp.gmail.com',
            'smtp_port' => '465',
            'smtp_user' => 'aleksrocknlove@gmail.com',
            'smtp_pass' => 'Aleksedr14',
            'mailtype' => 'html',
            'newline' => "\r\n",
            'crlf' => "\r\n",
            'protocol' => 'smtp',
        );

        // Configuración para GoDaddy/masmetrologia (no usada)
        $configMAS = Array(
            'smtp_host' => 'smtpout.secureserver.net',
            'smtp_port' => '80',
            'smtp_user' => 'tickets@masmetrologia.mx',
        );
    }
}
```

```

'smtp_pass' => 'Soporte2018@',
'mailtype' => 'html',
'newline' => '\r\n',
'crlf' => '\r\n',
'protocol' => 'smtp',
);

// Configuración para MailJet (usada en esta prueba)
$configMJET = Array(
    'smtp_host' => 'in-v3.mailjet.com',
    'smtp_port' => '587',
    'smtp_user' => '1f55d1e5c5da7c2c10ee96e0a5d166af',
    'smtp_pass' => '0c68495c162a80a883412b1106045cb3',
    'mailtype' => 'html',
    'newline' => '\r\n',
    'crlf' => '\r\n',
    'protocol' => 'smtp',
);

$this->load->library('email', $configMJET); // Usa configuración de MailJet

$this->email->from('aortiz@masmetrologia.com', 'PRUEBA MAIL-JET'); // Remitente
$this->email->to('alejandro_ortiz426@hotmail.com'); // Destinatario

$logo = base_url('template/images/logo.png');

// Mensaje con logotipo (comentado)
$mensaje = <<<EOD
<a href='#'><img width='400' src='$logo'><br></a>
<h1><font face="Arial">SIGA-MAS</font></h1>
EOD;

$this->email->subject('SIGA MAILJET');
// $this->email->message($mensaje); // Envío de mensaje con HTML
$this->email->message("PRUEBA MAIL-JET"); // Mensaje de prueba (texto simple)
$this->email->send();

echo $this->email->print_debugger(); // Muestra resultado del envío

```

```

}

// Envío de correo con formato de ticket de servicio
public function ticket($datos) {
    $logo = base_url('template/images/logo.png'); // Ruta del logotipo
    $titulo = $datos['titulo']; // Título del ticket

    // Cuerpo del correo en HTML
    $mensaje = <<<EOD
        <img width='500' src='$logo'>
        <h1>Ticket de Servicio</h1>
        <h3>Titulo: $titulo</h3>
    EOD;

    $this->load->library('email');

    $this->email->from('alejandro_ortiz426@hotmail.com', 'Aleks Ortiz'); // Remitente
    $this->email->to('aortiz@masmetrologia.com'); // Destinatario

    $this->email->subject('EXlaaaaaTO'); // Asunto del correo
    $this->email->message($mensaje); // Cuerpo HTML

    $this->email->send(); // Enviar

    echo $this->email->print_debugger(); // Muestra resultado del envío
}
}

```

Cotizaciones.php

```

<?php

defined('BASEPATH') OR exit('No direct script access allowed');

class Cotizaciones extends CI_Controller {
    public $idsub=null;

    // Constructor de la clase

```

```

function __construct() {
    parent::__construct();
    $this->load->library('correos_cotizaciones');
    $this->load->model('privilegios_model');
}

// Vista principal del catálogo de cotizaciones
function index($estatus = 'TODO', $user = "") {
    $us = null;
    $data['estatus'] = strtoupper($estatus); // Convierte el estatus a mayúsculas
    $data['user'] = strtoupper($user); // Convierte el nombre de usuario a mayúsculas
    $this->load->model('inicio_model');
    $this->load->view('header');
    $this->load->view('cotizaciones/catalogo', $data);
}

// Carga el formulario para crear una nueva cotización
function crear_cotizacion() {
    $usd = $this->aos_funciones->getUSD(); // Obtiene tipos de cambio USD
    $data["id"] = 0;
    $data['USD'] = $usd[0];
    $data['USD_ACT'] = $usd[1];
    $data['COPY'] = 0;

    if (isset($_POST["id"])) {
        $id = $this->input->post('id');
        $rev = $this->input->post('rev');
        $data['COPY'] = $id . '-' . $rev; // Marca como copia de una cotización existente
    }

    $this->load->view('header');
    $this->load->view('cotizaciones/generar', $data);
}

// Carga una cotización específica en modo visualización/edición
function ver_cotizacion($id = 0) {
    $this->load->model('usuarios_model');
    $data['sub'] = $this->usuarios_model->userCots(); // Obtiene cotizaciones del usuario

```



```

$data['COPY'] = 0;
$usd = $this->aos_funciones->getUSD();
$data['USD'] = $usd[0];
$data['USD_ACT'] = $usd[1];

if (isset($_POST["id"])) {
    $id = $this->input->post('id'); // Prioriza el ID enviado por POST
} else if ($id == 0) {
    redirect(base_url('inicio')); // Redirige si no hay ID válido
}

$data["id"] = $id;

$this->load->view('header');
$this->load->view('cotizaciones/generar', $data);
}

// Muestra el dashboard de cotizaciones
function dashboard() {
    $this->load->model('usuarios_model');
    $datos['sub'] = $this->usuarios_model->userCots(); // Carga cotizaciones del usuario
    $this->load->view('header');
    $this->load->view('cotizaciones/dashboard', $datos);
}

function cotizacion_pdf($param, $modo = 'I') {
    // Separa el parámetro compuesto (id-rev)
    $param = explode('-', $param);
    $id = $param[0];
    $rev = $param[1];

    // Construcción del query SQL para obtener los datos de la cotización y conceptos
    $query = "SELECT
        (SELECT ifnull(max(CC2.revision), 0)
        FROM cotizaciones_conceptos CC2
        WHERE CC2.cotizacion = $id) as UltRev,
        C.fecha, C.moneda, C.tipo, C.impuesto_factor, C.impuesto_nombre,
        C.aprobador, C.estatus, E.razon_social, E.calle, E.numero,

```

```

E.numero_interior, E.colonia, E.ciudad, E.estado, E.pais, E.rfc,
EC.nombre, EC.telefono, EC.correo,
(if(E.credito_cliente = 1, concat(E.credito_cliente_plazo, ' Días'), 'Contado')) as Credito,
concat(R.nombre, ' ', R.paterno) as Resp, R.correo as RespCorreo,
CC.*, C.planta,
ifnull(EP.nombre, 'N/A') as PlantaNombre,
ifnull(EP.calle, 'N/A') as PlantaCalle,
ifnull(EP.colonia, 'N/A') as PlantaColonia,
ifnull(EP.ciudad, 'N/A') as PlantaCiudad,
ifnull(EP.estado, 'N/A') as PlantaEstado
FROM cotizaciones_conceptos CC
INNER JOIN cotizaciones C ON C.id = CC.cotizacion
LEFT JOIN empresa_plantas EP ON EP.id = C.planta
INNER JOIN empresas E ON E.id = C.empresa
INNER JOIN empresas_contactos EC ON EC.id = C.contactos->'${0}'
INNER JOIN usuarios R ON R.id = C.responsable
WHERE CC.cotizacion = $id AND CC.revision = $rev";

```

```
// Ejecuta la consulta
```

```
$res = $this->Conexion->consultar($query);
```

```
// Inicialización de variables utilizadas en el PDF
```

```
$conceptos = [];
```

```
$APROB = -1;
```

```
$ESTATUS = "";
```

```
$SERVICIOS = new stdClass;
```

```
$SUBTOTAL = 0;
```

```
$IMPUESTO = 0;
```

```
$TOTAL = 0;
```

```
foreach ($res as $i => $elem) {
```

```
    // Determina si la revisión es obsoleta
```

```
    $OBS = $elem->UltRev != $rev ? " OBSOLETA" : "";
```

```
    // Captura información general de la cotización
```

```
    $APROB = $elem->aprobador;
```

```
    $ESTATUS = $elem->estatus;
```

```
    $COT = 'COT-' . str_pad($elem->cotizacion, 6, "0", STR_PAD_LEFT) . " Rev: " . $rev . $OBS;
```

```

$FECHA = date_format(date_create($selem->fecha), 'd/m/Y h:i A');
$CLIENTE = $selem->razon_social;
$DOMICILIO = $selem->calle . ' ' . $selem->numero . ' ' . $selem->numero_interior;
$COLONIA = $selem->colonia;
$RFC = $selem->rfc;

// Datos de contacto
$CONTACTO = $selem->nombre;
$TELEFONO = $selem->telefono;
$CORREO = $selem->correo;
$UBICACION = $selem->ciudad . ', ' . $selem->estado . ', ' . $selem->pais;

// Responsable de la cotización
$RESPONSABLE = $selem->Resp;
$RESPONSABLE_CORREO = $selem->RespCorreo;

// Planta asociada
$ID_PLANTA = $selem->planta;
$NOMBRE_PLANTA = $selem->PlantaNombre;
$CALLE_PLANTA = $selem->PlantaCalle;
$COLONIA_PLANTA = $selem->PlantaColonia;
$CIUDAD_PLANTA = $selem->PlantaCiudad;
$ESTADO_PLANTA = $selem->PlantaEstado;

// Construcción del objeto concepto
$concepto = new stdClass;
$concepto->cantidad = $selem->cantidad;
$concepto->atributos = json_decode($selem->atributos);
$concepto->descripcion = $selem->descripcion;
$concepto->servicios = json_decode($selem->servicios);

// Acumula servicios únicos
foreach ($concepto->servicios as $key => $value) {
    $cod = $value[1];
    if ($cod != "N/A") {
        $SERVICIOS->$cod = $value[2];
    }
}

```

```

// Campos adicionales del concepto
$concepto->comentarios = $selem->comentarios;
$concepto->tiempo_entrega = $selem->tiempo_entrega;
$concepto->sitio = $selem->sitio;
$concepto->precio_unitario = $selem->precio_unitario;
$concepto->importe = floatval($selem->precio_unitario) * floatval($selem->cantidad);

// Agrega el concepto al arreglo
array_push($conceptos, $concepto);

// Cálculos de totales
$IMPUESTO_NOMBRE = "IVA / TAX [" . ($selem->impuesto_factor) * 100 . "%]";
$MONEDA = $selem->moneda;
$CREDITO = $selem->Credito;

$SUBTOTAL += $concepto->importe;
$IMPUESTO += $concepto->importe * ($selem->impuesto_factor);
$TOTAL += $concepto->importe * ($selem->impuesto_factor + 1);

// Inicializa notas y margen por tipo de servicio
$NOTAS = "TERMINOS Y CONDICIONES:";

switch ($selem->tipo) {
    case 'CALIBRACION':
        $Margin = 195;
        $NOTAS .= "
        . \"\n1.- El servicio se programa con la orden de compra correspondiente. Si su orden de
        compra incluye más de un equipo se requiere que los mismos se calibren dentro de un periodo
        de 30 días máximo.\"
        . \"\n2.- La Cotización es válida por 120 días a partir de la fecha de elaboración.\"
        . \"\n3.- El servicio de calibración descrito se ofrece usando los métodos y
        procedimientos internos del laboratorio. Si existen requisitos específicos del cliente, se iniciará
        nuevamente el proceso de cotización.\"
        . \"\n4.- El costo del servicio es aplicable aun cuando su equipo no pase la calibración o
        no responda al proceso de ajuste. En ese caso, se entregará un reporte detallado.\"
        . \"\n5.- No hay garantía de que el equipo mantendrá las tolerancias especificadas
        durante el intervalo de calibración.\"

```

- . "\n6.- Identificación y frecuencia de calibración son asignadas por el cliente."
- . "\n7.- El tiempo de entrega depende de la programación y empieza tras la aprobación y recepción física del instrumento."
- . "\n8.- Si el cliente no proporciona regla de decisión, nuestros certificados no consideran la incertidumbre salvo que se indique lo contrario."
- . "\nESTE SERVICIO CONSTA DE:"
- . "\n1.- Revisión y limpieza externa del equipo."
- . "\n2.- Calibración del equipo.";
- break;

case 'ESTUDIO DIMENSIONAL':

\$Margin = 215;

\$NOTAS .= "

- . "\n1.- Se requiere orden de compra para iniciar el servicio. Una vez recibida, se confirma la fecha de entrega."
- . "\n2.- Generar la orden de compra con base en esta cotización y sus políticas internas."
- . "\n3.- Cotización válida por 120 días desde la fecha de elaboración."
- . "\n4.- Se requieren planos legibles."
- . "\n5.- Puede ser necesario seccionar piezas; proveer cantidad suficiente.";

break;

case 'RENTA':

\$Margin = 215;

\$NOTAS .= "

- . "\n1.- Se requiere orden de compra para iniciar el servicio."
- . "\n2.- Generar la orden de compra conforme a esta cotización y políticas internas."
- . "\n3.- Cotización válida por 30 días desde la fecha de elaboración."
- . "\nESTE SERVICIO CONSTA DE:"
- . "\n1.- Entrega del equipo en planta."
- . "\n2.- Instrucción sobre el uso correcto del equipo."
- . "\n3.- Instalación en sitio (solo básculas y contadores).";

break;

case 'REPARACION':

\$Margin = 215;

\$NOTAS .= "

- . "\n1.- Se requiere orden de compra para iniciar el servicio."

```
. "\n2.- Generar la orden de compra conforme a esta cotización y políticas internas."
. "\n3.- Cotización válida por 15 días desde la fecha de elaboración.";
break;
```

```
case 'VENTA':
```

```
    $Margin = 215;
    $NOTAS .= "
. "\n1.- Generar la orden de compra conforme a esta cotización y políticas internas."
. "\n2.- Cotización válida por 30 días desde la fecha de elaboración."
. "\n3.- Pedido no cancelable."
. "\n4.- Disponibilidad sujeta a existencias si no hay orden de compra."
. "\n5.- Se requiere orden de compra por escrito para entrega."
. "\n6.- Al recibir la orden, se confirma la fecha de entrega.";
break;
```

```
case 'SOPORTE':
```

```
    $Margin = 215;
    $NOTAS .= "
. "\n1.- Se requiere orden de compra para iniciar el servicio."
. "\n2.- Generar la orden de compra conforme a esta cotización y políticas internas."
. "\n3.- Cotización válida por 30 días desde la fecha de elaboración.";
break;
```

```
case 'CALIBRACION EXTERNA':
```

```
    $Margin = 205;
    $NOTAS .= "
. "\n1.- El servicio se programa con orden de compra y equipo disponible para
Metrología Aplicada."
. "\n2.- Cotización válida por 30 días desde la fecha de elaboración."
. "\n3.- Si el cliente solicita otro proveedor, se iniciará nuevo proceso de cotización."
. "\n4.- Costo aplicable incluso si el equipo no pasa calibración."
. "\n5.- No hay garantía de estabilidad en tolerancias por factores externos."
. "\n6.- Tiempos de entrega pueden cambiar, se notificará con anticipación."
. "\n7.- El tiempo empieza a correr desde la aprobación y recepción del equipo."
. "\nESTE SERVICIO CONSTA DE:"
. "\n1.- Revisión y limpieza externa del equipo."
. "\n2.- Calibración del equipo.";
break;
```

```

case 'MAPEO':
    $Margin = 215;
    $NOTAS .= "
        . \"\n1.- El servicio se programa con orden de compra.\"
        . \"\n2.- El equipo debe estar disponible en planta para el servicio.\"
        . \"\n3.- La cotización cubre el tiempo estimado. Excesos se cotizan por separado.\"
        . \"\n4.- Cambios en requerimientos se cotizan y requieren autorización.\"
        . \"\n5.- Cotización válida por 60 días desde su elaboración.\";
    break;

case 'LISTA PRECIOS':
    $Margin = 210;
    $NOTAS .= "
        . \"\n1.- Se requiere orden de compra para programar el servicio.\"
        . \"\n2.- Calibración según procedimientos internos. Requerimientos especiales implican
nueva cotización.\"
        . \"\n3.- Costo aplicable aunque el equipo no pase calibración.\"
        . \"\n4.- No se garantiza estabilidad del equipo debido a factores externos.\"
        . \"\n5.- Vigencia establecida en contrato o acuerdo correspondiente.\"
        . \"\nESTE SERVICIO CONSTA DE:\"
        . \"\n1.- Revisión y limpieza externa del equipo.\"
        . \"\n2.- Calibración del equipo.\";
    break;
}
}

// Oculta errores para evitar que interfieran con la salida del PDF
ini_set('display_errors', 0);

// Carga la librería personalizada para generar PDF
$this->load->library('pdfview');

// Selección de plantilla según estatus de la cotización
if ($ESTATUS == "CANCELADA") {
    $pdf = new pdfview_CANCEL(PDF_PAGE_ORIENTATION, PDF_UNIT, PDF_PAGE_FORMAT,
true, 'UTF-8', false);
} else {

```

```

    if ($APROB > 0) {
        $pdf = new pdfview(PDF_PAGE_ORIENTATION, PDF_UNIT, PDF_PAGE_FORMAT, true,
'UTF-8', false);
    } else {
        $pdf = new pdfview_NA(PDF_PAGE_ORIENTATION, PDF_UNIT, PDF_PAGE_FORMAT, true,
'UTF-8', false);
    }
}

```

```

// Configuración general del PDF
$pdf->SetCreator(PDF_CREATOR);
$pdf->SetAuthor('AleksOrtiz');
$pdf->SetTitle('Masmetrologia');
$pdf->SetSubject('Formato Cotización');

// Texto de cabecera personalizado
$spc = "          ";
$head = "Metrologia Aplicada y Servicios, S. de R.L. de C.V. $spc Cotización / $COT";
$txt = "Av. Ramón Rayón #1520 Int-9                               Fecha: " . $FECHA;
$txt .= "\nCol. Rio Bravo                                       Ejecutivo: " .
$RESPONSABLE;
$txt .= "\nCd. Juárez, Chih. C.P. 32550                           Correo: " .
$RESPONSABLE_CORREO;
$txt .= "\nRFC: MAS080825EE7";

```

```

$pdf->SetHeaderData(PDF_HEADER_LOGO_ORIGINAL, '40', $head, $txt);

```

```

// Fuentes y márgenes
$pdf->setHeaderFont(Array(PDF_FONT_NAME_MAIN, "", 9));
$pdf->setFooterFont(Array(PDF_FONT_NAME_DATA, "", PDF_FONT_SIZE_DATA));
$pdf->SetDefaultMonospacedFont(PDF_FONT_MONOSPACED);
$pdf->SetMargins(8, PDF_MARGIN_TOP, 8); // Márgenes modificados manualmente
$pdf->SetHeaderMargin(PDF_MARGIN_HEADER);
$pdf->SetFooterMargin(PDF_MARGIN_FOOTER);
$pdf->SetAutoPageBreak(TRUE, PDF_MARGIN_BOTTOM);
$pdf->setImageScale(PDF_IMAGE_SCALE_RATIO);

```

```

// Carga idioma si existe

```



```

if (@file_exists(dirname(__FILE__) . '/lang/eng.php')) {
    require_once(dirname(__FILE__) . '/lang/eng.php');
    $pdf->setLanguageArray($l);
}

// Fuente por defecto y nueva página
$pdf->SetFont('times', "", 8);
$pdf->AddPage();

// Construcción de la tabla de cabecera con datos del cliente
$tbl = <<<EOD
    <table border="0">
        <tr>
            <td>
                <b>Cliente/Client:</b><br>
                $CLIENTE<br><br>
                <b>Dirección / Address:</b><br>
                $DOMICILIO<br>
                $COLONIA<br>
                $UBICACION<br>
                RFC: $RFC<br>
            </td>
            <td>
EOD;

// Si existe planta asociada, incluir sus datos
if ($ID_PLANTA > 0) {
    $tbl .= <<<EOD
        <b>Planta/Plant:</b><br>
        $NOMBRE_PLANTA<br>
        $CALLE_PLANTA<br>
        $COLONIA_PLANTA<br>
        $CIUDAD_PLANTA $ESTADO_PLANTA<br><br>
EOD;
}

// Sección de contacto
$tbl .= <<<EOD

```

```

        <b>Contacto / Contact:</b><br>
        $CONTACTO<br>
        $TELEFONO<br>
        $CORREO<br>
    </td>
</tr>
</table>
EOD;

// Imprime la tabla de cabecera
$pdf->writeHTML($tbl, false, false, false, false, "");

// Ancho de columnas para tabla de conceptos
$w = array(8, 125, 12, 24, 24);

// Encabezado de tabla de conceptos
$pdf->SetFont('helvetica', 'B', 9);
$pdf->SetTextColor(255); // Blanco
$pdf->SetTextColor(0); // Negro

// Nombres de columnas
$pdf->Cell($w[0], 6, "#", 1, 0, 'C');
$pdf->Cell($w[1], 6, "Descripción", 1, 0, 'C');
$pdf->Cell($w[2], 6, "Cant.", 1, 0, 'C');
$pdf->Cell($w[3], 6, "Precio Unit.", 1, 0, 'C');
$pdf->Cell($w[4], 6, "Importe", 1, 1, 'C');

// Fuente base para los conceptos
$pdf->SetFont("", "", 7);

// Inicializa índice de conceptos
$i = 0;

// Altura por renglón base para celdas
$RenSpace = 2.8;

foreach ($conceptos as $indice => $concepto) {
    $pdf->SetFont("", "", 7);

```

```

// Si el concepto tiene atributo "otro" (CABEZAL o SEPARADOR)
if (array_key_exists("otro", $concepto->atributos)) {
    // Espacio en blanco para separación visual
    for ($j = 0; $j < 5; $j++) {
        $pdf->writeHTMLCell($w[$j], 0, "", "", "LR", ($j == 4), 0, true, 'J', false);
    }

    $startX = $pdf->GetX();
    $startY = $pdf->GetY();
    $cellcount = [];

    if ($concepto->atributos->otro == 'CABEZAL') {
        $pdf->SetFont("", 'B', 8);
        $pdf->MultiCell($w[0], "", "", 'LR', 'C', 0, 0);
        $pdf->MultiCell($w[1], "", $concepto->descripcion, 'LR', 'C', 0, 0);
        $pdf->MultiCell($w[2], "", "", 'LR', 'C', 0, 0);
        $pdf->MultiCell($w[3], "", "", 'LR', 'C', 0, 0);
        $pdf->MultiCell($w[4], "", "", 'LR', 'C', 0, 1);

        $h = (max($cellcount)) * $RenSpace;
        $pdf->SetXY($startX, $startY);
        for ($j = 0; $j < 5; $j++) {
            $pdf->writeHTMLCell($w[$j], $h, "", "", "LR", ($j == 4), 0, true, 'J', false);
        }
        $pdf->SetFont("", "", 7);
    } elseif ($concepto->atributos->otro == 'SEPARADOR') {
        $pdf->MultiCell($w[0], "", '==', 'LR', 'C', 0, 0);
        $pdf->MultiCell($w[1], "", str_repeat('=', 84), 'LR', 'C', 0, 0);
        $pdf->MultiCell($w[2], "", '====', 'LR', 'C', 0, 0);
        $pdf->MultiCell($w[3], "", str_repeat('=', 12), 'LR', 'C', 0, 0);
        $pdf->MultiCell($w[4], "", str_repeat('=', 12), 'LR', 'C', 0, 1);

        $h = (max($cellcount)) * $RenSpace;
        $pdf->SetXY($startX, $startY);
        for ($j = 0; $j < 5; $j++) {
            $pdf->writeHTMLCell($w[$j], $h, "", "", "LR", ($j == 4), 0, true, 'J', false);
        }
    }
}

```

```

    }
} else {
    // Concepto estándar
    $i++;

    // Espacio inicial
    for ($j = 0; $j < 5; $j++) {
        $pdf->writeHTMLCell($w[$j], 0, "", "", "LR", ($j == 4), 0, true, 'J', false);
    }

    // Sitio y tiempo de entrega
    $dicSitios = ['OS' => 'En Planta', 'LAB' => 'LAB', 'EXT' => 'Prov. Externo'];
    $sitio = $concepto->sitio != "N/A" ? "Donde: " . $dicSitios[$concepto->sitio] : "";
    $tEntrega = $concepto->tiempo_entrega > 0
        ? "T. Entrega: {$concepto->tiempo_entrega} Día" . ($concepto->tiempo_entrega > 1 ?
"s" : "") . $sitio
        : "T. Entrega: N/A" . $sitio;

    // Si tiene servicios: mostrarlos con descripción extendida
    if (count($concepto->servicios) > 0) {
        $codigos = "";
        foreach ($concepto->servicios as $value) {
            $codigos .= ($value[1] == "N/A" ? $value[2] : $value[1]) . ", ";
        }

        $startX = $pdf->GetX();
        $startY = $pdf->GetY();
        $cellcount = [];
        $cellcount[] = $pdf->MultiCell($w[0], "", $indice + 1, 0, 'C', 0, 0);
        $cellcount[] = $pdf->MultiCell($w[1], "", "Servicios: $codigos $tEntrega", 0, 'L', 0, 0);
        $cellcount[] = $pdf->MultiCell($w[2], "", number_format($concepto->cantidad, 0), 0,
'C', 0, 0);
        $cellcount[] = $pdf->MultiCell($w[3], "", "$" . number_format($concepto-
>precio_unitario, 2), 0, 'R', 0, 0);
        $cellcount[] = $pdf->MultiCell($w[4], "", "$" . number_format($concepto->importe, 2),
0, 'R', 0, 1);

        $h = max($cellcount) * $RenSpace;

```

```

$pdf->SetXY($startX, $startY);
for ($j = 0; $j < 5; $j++) {
    $pdf->writeHTMLCell($w[$j], $h, "", "", "LR", ($j == 4), 0, true, 'J', false);
}
}

// Datos del equipo desde los atributos
$att = "";
foreach ([ 'ID', 'Marca', 'Modelo', 'Serie' ] as $key) {
    if (isset($concepto->atributos->$key)) {
        $att .= "$key: {$concepto->atributos->$key}, ";
    }
}

foreach ($concepto->atributos as $key => $value) {
    if (!in_array($key, [ 'ID', 'Marca', 'Modelo', 'Serie' ])) {
        $att .= "$key: $value, ";
    }
}

$att = rtrim($att, ', ');

$startX = $pdf->GetX();
$startY = $pdf->GetY();
$cellcount = [];
$cellcount[] = $pdf->MultiCell($w[0], "", (count($concepto->servicios) > 0 ? "" : $indice +
1), 0, 'C', 0, 0);
$cellcount[] = $pdf->MultiCell($w[1], "", $concepto->descripcion . (count($concepto-
>servicios) > 0 ? " $att" : " $tEntrega"), 0, 'L', 0, 0);
$cellcount[] = $pdf->MultiCell($w[2], "", (count($concepto->servicios) > 0 ? "" :
$concepto->cantidad), 0, 'C', 0, 0);
$cellcount[] = $pdf->MultiCell($w[3], "", (count($concepto->servicios) > 0 ? "" : "$" .
number_format($concepto->precio_unitario, 2)), 0, 'R', 0, 0);
$cellcount[] = $pdf->MultiCell($w[4], "", (count($concepto->servicios) > 0 ? "" : "$" .
number_format($concepto->importe, 2)), 0, 'R', 0, 1);

$h = max($cellcount) * $RenSpace;
$pdf->SetXY($startX, $startY);

```

```

for ($j = 0; $j < 5; $j++) {
    $pdf->writeHTMLCell($w[$j], $h, "", "", "", "LR", ($j == 4), 0, true, 'J', false);
}

// Comentarios del concepto
if (!empty($concepto->comentarios)) {
    $startX = $pdf->GetX();
    $startY = $pdf->GetY();
    $cellcount = [];
    $cellcount[] = $pdf->MultiCell($w[0], "", "", 0, 'C', 0, 0);
    $cellcount[] = $pdf->MultiCell($w[1], "", "*** " . $concepto->comentarios, 0, 'L', 0, 0);
    for ($j = 2; $j < 5; $j++) {
        $cellcount[] = $pdf->MultiCell($w[$j], "", "", 0, 'R', 0, 0);
    }

    $h = max($cellcount) * $RenSpace;
    $pdf->SetXY($startX, $startY);
    for ($j = 0; $j < 5; $j++) {
        $pdf->writeHTMLCell($w[$j], $h, "", "", "", "LR", ($j == 4), 0, true, 'J', false);
    }
}

// Salto de página si se rebasa el límite vertical
if ($pdf->GetY() > 260) {
    for ($j = 0; $j < 5; $j++) {
        $pdf->writeHTMLCell($w[$j], "", "", "", "", "LRB", ($j == 4), 0, true, 'J', false);
    }

    $pdf->AddPage();
    $pdf->SetFont('times', "", 8);
    $pdf->writeHTML($tbl, false, false, false, false, "");

    if ($i < count($conceptos)) {
        $pdf->SetFont('helvetica', 'B', 9);
        $pdf->SetTextColor(0);
        $pdf->Cell($w[0], 6, "#", 1, 0, 'C');
        $pdf->Cell($w[1], 6, "Descripción", 1, 0, 'C');
        $pdf->Cell($w[2], 6, "Cant.", 1, 0, 'C');
    }
}

```

```

$pdf->Cell($w[3], 6, "Precio Unit.", 1, 0, 'C');
$pdf->Cell($w[4], 6, "Importe", 1, 1, 'C');

for ($j = 0; $j < 5; $j++) {
    $pdf->writeHTMLCell($w[$j], " ", " ", " ", "LRT", ($j == 4), 0, true, 'J', false);
}
}
}
}
}
}

```

```

// DESCRIPCIÓN DE CÓDIGOS DE SERVICIO (solo si existen servicios únicos)
if (count((array)$SERVICIOS) > 0) {

```

```

    // Línea separadora visual
    $pdf->MultiCell($w[0], " '===' 'LR', 'C', 0, 0);
    $pdf->MultiCell($w[1], " str_repeat('=', 84), 'LR', 'C', 0, 0);
    $pdf->MultiCell($w[2], " '=====' 'LR', 'C', 0, 0);
    $pdf->MultiCell($w[3], " str_repeat('=', 12), 'LR', 'C', 0, 0);
    $pdf->MultiCell($w[4], " str_repeat('=', 12), 'LR', 'C', 0, 1);

```

```

// Encabezado de sección
$pdf->SetFont(" 'B', 7);
$pdf->MultiCell($w[0], " 'LR', 'C', 0, 0);
$pdf->MultiCell($w[1], " 'Descripción de Servicios:', 0, 'L', 0, 0);
$pdf->MultiCell($w[2], " 'LR', 'R', 0, 0);
$pdf->MultiCell($w[3], " 'LR', 'R', 0, 0);
$pdf->MultiCell($w[4], " 'LR', 'R', 0, 1);

```

```

// Cuerpo de descripción de servicios
$pdf->SetFont(" ", 7);
foreach ($SERVICIOS as $codigo => $servicio) {
    $startX = $pdf->GetX();
    $startY = $pdf->GetY();
    $cellcount = [];

```

```

    // Calcula número de líneas necesarias por celda
    $cellcount[] = $pdf->MultiCell($w[0], " ", 0, 'C', 0, 0);
    $cellcount[] = $pdf->MultiCell($w[1], " '$codigo: $servicio", 0, 'L', 0, 0);

```

```

$cellcount[] = $pdf->MultiCell($w[2], " ", 0, 'R', 0, 0);
$cellcount[] = $pdf->MultiCell($w[3], " ", 0, 'R', 0, 0);
$cellcount[] = $pdf->MultiCell($w[4], " ", 0, 'R', 0, 0);

// Regresa cursor y escribe celdas con altura unificada
$pdf->SetXY($startX, $startY);
$h = (max($cellcount) + 1) * $RenSpace;
$pdf->MultiCell($w[0], $h, " ", 'LR', 'C', 0, 0);
$pdf->MultiCell($w[1], $h, " ", 'LR', 'L', 0, 0);
$pdf->MultiCell($w[2], $h, " ", 'LR', 'R', 0, 0);
$pdf->MultiCell($w[3], $h, " ", 'LR', 'R', 0, 0);
$pdf->MultiCell($w[4], $h, " ", 'LR', 'R', 0, 1);
}
}

// Rellena hasta el margen para separar los totales
$startY = $pdf->GetY();
for ($startY; $startY < $Margin; $startY += 6) {
    $pdf->MultiCell($w[0], 6, '----', 'LR', 'C', 0, 0);
    $pdf->MultiCell($w[1], 6, str_repeat('-', 120), 'LR', 'C', 0, 0);
    $pdf->MultiCell($w[2], 6, '-----', 'LR', 'C', 0, 0);
    $pdf->MultiCell($w[3], 6, str_repeat('-', 19), 'LR', 'C', 0, 0);
    $pdf->MultiCell($w[4], 6, str_repeat('-', 19), 'LR', 'C', 0, 0);
    $pdf->Ln();
}

// Notas de servicio
$pdf->MultiCell($w[0] + $w[1] + $w[2], 6, $NOTAS, 'T', 'M', 0, 0);

// Subtotal
$pdf->SetFont(" ", 8);
$pdf->MultiCell($w[3], 4, 'Sub-Total', 'T', 'C', 0, 0);
$pdf->MultiCell($w[4], 4, "$" . number_format($SUBTOTAL, 2), 1, 'R', 0, 0);
$pdf->Ln();

// Impuesto
$pdf->MultiCell($w[0] + $w[1] + $w[2], 6, " ", 0, 'L', 0, 0);
$pdf->MultiCell($w[3], 4, $IMPUESTO_NOMBRE, 0, 'C', 0, 0);

```



```

$pdf->MultiCell($w[4], 4, "$" . number_format($IMPUESTO, 2), 1, 'R', 0, 0);
$pdf->Ln();

// Total
$pdf->MultiCell($w[0] + $w[1] + $w[2], 6, "", 0, 'M', 0, 0);
$pdf->MultiCell($w[3], 4, 'Total', 0, 'C', 0, 0);
$pdf->MultiCell($w[4], 4, "$" . number_format($TOTAL, 2), 1, 'R', 0, 0);
$pdf->Ln();

// Moneda
$pdf->MultiCell($w[0] + $w[1] + $w[2], 6, "", 0, 'M', 0, 0);
$pdf->MultiCell($w[3] + $w[4], 6, '* Moneda / Currency: ' . $MONEDA, 0, 'C', 0, 0);
$pdf->Ln();

// Espacios de separación
$pdf->MultiCell(180, 6, "", 0, 'M', 0, 0);
$pdf->Ln();
$pdf->MultiCell(180, 6, "", 0, 'M', 0, 0);
$pdf->Ln();
$pdf->Ln();

// Salida del PDF: como string ('S') o mostrar en navegador ('I')
if ($modo === 'S') {
    return $pdf->Output("", 'S'); // Devuelve como string (por ejemplo, para adjuntar a correo)
} else {
    $cot = str_pad($id, 6, '0', STR_PAD_LEFT) . '_Rev_' . $rev;
    $pdf->Output("COT-$cot.pdf", 'I'); // Muestra en el navegador
}
}

function ajax_setCotizacion() {
    $id_cotizacion = null;
    $id_cotizacion_nueva = null;

    // Decodifica los objetos JSON recibidos por POST
    $cotizacion = json_decode($this->input->post("cotizacion"));
    $conceptos = json_decode($this->input->post("conceptos"));

```

```

// Si se copia desde una cotización existente, guarda el ID de origen
if (isset($cotizacion->copiar_desde)) {
    $id_cotizacion = $cotizacion->copiar_desde;
}

// Si es una nueva cotización
if ($cotizacion->id == 0) {
    $cotizacion->usuario = $this->session->id;

    // Inserta la nueva cotización con timestamp automático
    $cotizacion->id = $this->Conexion->insertar(
        'cotizaciones',
        $cotizacion,
        array('fecha' => 'CURRENT_TIMESTAMP()')
    );
    $id_cotizacion_nueva = $cotizacion->id;

    // Verifica si la empresa es un prospecto para marcarlo
    $res = $this->Conexion->consultar(
        'SELECT prospecto FROM empresas WHERE id = ' . $cotizacion->empresa,
        TRUE
    );

    if ($res->prospecto == '1') {
        $prospectoCot['prospectoEmp'] = '1';
        $this->Conexion->modificar(
            'cotizaciones',
            $prospectoCot,
            null,
            array('id' => $cotizacion->id)
        );
    }

    } else {
        // MODIFICACIÓN DE COTIZACIÓN
        $func = null;

        // Si se aprueba, se registra aprobador y fecha

```

```

if ($cotizacion->estatus == 'AUTORIZADA') {
    $cotizacion->aprobador = $this->session->id;
    $func['fecha_aprobacion'] = 'CURRENT_TIMESTAMP()';
}

// Si se confirma, se registra la fecha de confirmación
if ($cotizacion->estatus == 'CONFIRMADA') {
    $func['fecha_confirmacion'] = 'CURRENT_TIMESTAMP()';
}

// Si es aprobado total, actualiza prospecto de la empresa y cancela seguimientos
if ($cotizacion->estatus == 'APROBADO TOTAL') {
    $res = $this->Conexion->consultar(
        'SELECT prospecto FROM empresas WHERE id = ' . $cotizacion->empresa,
        TRUE
    );

    if ($res->prospecto == '1') {
        $empresa['prospecto'] = '0';
        $this->Conexion->modificar(
            'empresas',
            $empresa,
            null,
            array('id' => $cotizacion->empresa)
        );
    }

    $seg = $this->Conexion->consultar(
        "SELECT ca.*, u.correo
        FROM cot_acciones ca
        JOIN usuarios u ON u.id = ca.usuario
        WHERE estatus = 'PENDIENTE' AND idCot = " . $cotizacion->id
    );

    if ($seg) {
        foreach ($seg as $value) {
            $seguimiento['estatus'] = 'CANCELADA';
        }
    }
}

```

```
$this->Conexion->modificar('cot_acciones', $seguimiento, null, array('id' => $value->id));
```

```
$datos['id'] = $value->idCot;  
$datos['fecha_limite'] = $value->fecha_limite;  
$datos['accion'] = $value->accion;  
$datos['correos'] = $value->correo;  
$datos['correo_cancelar'] = $this->session->correo;
```

```
$this->correos_cotizaciones->cancelarSeguimiento($datos);
```

```
    }  
  }  
}
```

```
// Actualiza los datos de la cotización
```

```
$this->Conexion->modificar(  
    'cotizaciones',  
    $cotizacion,  
    $func,  
    array('id' => $cotizacion->id)  
);
```

```
// Si la cotización fue cancelada, cancela seguimientos relacionados
```

```
if ($cotizacion->estatus == 'CANCELADA') {  
    $seg = $this->Conexion->consultar(  
        "SELECT ca.*, u.correo  
        FROM cot_acciones ca  
        JOIN usuarios u ON u.id = ca.usuario  
        WHERE estatus = 'PENDIENTE' AND idCot = " . $cotizacion->id  
    );
```

```
    if ($seg) {  
        foreach ($seg as $value) {  
            $seguimiento['estatus'] = 'CANCELADA';  
            $this->Conexion->modificar('cot_acciones', $seguimiento, null, array('id' => $value->id));  
        }  
    }
```

```
$datos['id'] = $value->idCot;
```

```

        $datos['fecha_limite'] = $value->fecha_limite;
        $datos['accion'] = $value->accion;
        $datos['correos'] = $value->correo;
        $datos['correo_cancelar'] = $this->session->correo;

        $this->correos_cotizaciones->cancelarSeguimiento($datos);
    }
}
}

// ENVÍO DE CORREOS

// Si se envió un comentario desde el formulario
if (isset($_POST['comentarios']) && $_POST['comentarios']) {
    $datos['comentarios'] = $this->input->post('comentarios');
}

// Si está pendiente de autorización, notifica al autorizador
if ($cotizacion->estatus == 'PENDIENTE AUTORIZACION') {
    $res = $this->Conexion->consultar(
        "SELECT
            E.nombre AS NombreCliente,
            EC.nombre AS NombreContacto,
            CONCAT(R.nombre, ' ', R.paterno) AS Responsable,
            A.correo
        FROM cotizaciones C
        INNER JOIN usuarios R ON R.id = C.responsable
        INNER JOIN usuarios A ON R.autorizador_cotizacion = A.id
        INNER JOIN empresas E ON E.id = C.empresa
        INNER JOIN empresas_contactos EC ON EC.id = C.contactos->'${0}'
        WHERE C.id = $cotizacion->id",
        TRUE
    );

    $datos['id'] = $cotizacion->id;
    $datos['correos'] = $res->correo;
    $datos['nombreCliente'] = $res->NombreCliente;
    $datos['nombreContacto'] = $res->NombreContacto;
    $datos['nombreResponsable'] = $res->Responsable;

```

```

        if (isset($cotizacion->enviar_autorizar) && $cotizacion->enviar_autorizar == 1) {
            // Lógica adicional si se requiere enviar autorización explícitamente (actualmente
vacía)
        }

        // Envío de solicitud de aprobación
        $this->correos_cotizaciones->solicitarAprobacion($datos);
    }

    // Si fue rechazada, notificar al responsable
    if ($cotizacion->estatus == 'RECHAZADA') {
        $res = $this->Conexion->consultar(
            "SELECT
                E.nombre AS NombreCliente,
                EC.nombre AS NombreContacto,
                CONCAT(R.nombre, ' ', R.paterno) AS Responsable,
                R.correo
            FROM cotizaciones C
            INNER JOIN usuarios R ON R.id = C.responsable
            INNER JOIN empresas E ON E.id = C.empresa
            INNER JOIN empresas_contactos EC ON EC.id = C.contactos->'${0}'
            WHERE C.id = $cotizacion->id",
            TRUE
        );

        $datos['id'] = $cotizacion->id;
        $datos['correos'] = $res->correo;
        $datos['nombreCliente'] = $res->NombreCliente;
        $datos['nombreContacto'] = $res->NombreContacto;
        $datos['nombreResponsable'] = $res->Responsable;

        $this->correos_cotizaciones->rechazoCotizacion($datos);
    }

    // Si fue autorizada, notificar también al responsable
    if ($cotizacion->estatus == 'AUTORIZADA') {
        $res = $this->Conexion->consultar(

```

```

"SELECT
    E.nombre AS NombreCliente,
    EC.nombre AS NombreContacto,
    CONCAT(R.nombre, ' ', R.paterno) AS Responsable,
    R.correo,
    correo_cc
FROM cotizaciones C
INNER JOIN usuarios R ON R.id = C.responsable
INNER JOIN empresas E ON E.id = C.empresa
INNER JOIN empresas_contactos EC ON EC.id = C.contactos->'${0}'
WHERE C.id = $cotizacion->id",
TRUE
);

$datos['id'] = $cotizacion->id;
$datos['correos'] = $res->correo;
$datos['nombreCliente'] = $res->NombreCliente;
$datos['nombreContacto'] = $res->NombreContacto;
$datos['nombreResponsable'] = $res->Responsable;

$this->correos_cotizaciones->aprobacionCotizacion($datos);
}
}

// Procesamiento de los conceptos asociados a la cotización
foreach ($conceptos as $key => $elem) {
    $elem->cotizacion = $cotizacion->id;

    // Inserta nuevo concepto
    if ($elem->id == 0) {
        $this->Conexion->insertar('cotizaciones_conceptos', $elem);
    }

    // Elimina concepto si el ID es negativo (marcado para eliminar)
    if ($elem->id < 0) {
        $this->Conexion->eliminar('cotizaciones_conceptos', array('id' => (intval($elem->id) * -
1)));
    }
}

```

```

// Actualiza o re-inserta concepto si ya existía
else if ($selem->id != 0) {
    $cot = $this->Conexion->consultar(
        "SELECT cotizacion FROM cotizaciones_conceptos WHERE cotizacion = '$selem-
>cotizacion'",
        TRUE
    );

    if ($cot) {
        $id_cotizacion = $cot;
        $this->Conexion->modificar('cotizaciones_conceptos', $selem, null, array('id' =>
$selem->id));
    } else {
        unset($selem->id); // Elimina el ID para forzar una inserción nueva
        $this->Conexion->insertar('cotizaciones_conceptos', $selem);
    }
}

// Si no hay estatus (es decir, la cotización aún no ha sido enviada formalmente)
if (!isset($cotizacion->estatus)) {
    if ($id_cotizacion) {
        // Comentario en la nueva cotización indicando de cuál se copió
        $comentarios = array(
            'cotizacion' => $id_cotizacion_nueva,
            'usuario' => $this->session->id,
            'comentario' => 'Esta cotización es copia de: #' . $id_cotizacion,
        );
        $this->Conexion->insertar('cotizaciones_comentarios', $comentarios, array('fecha' =>
'CURRENT_TIMESTAMP()));

        // Comentario en la cotización original indicando que fue copiada
        $comentarios2 = array(
            'cotizacion' => $id_cotizacion,
            'usuario' => $this->session->id,
            'comentario' => 'Esta cotización fue copiada para la cotización: #' .
$id_cotizacion_nueva,

```



```

    );
    $this->Conexion->insertar('cotizaciones_comentarios', $comentarios2, array('fecha' =>
'CURRENT_TIMESTAMP()));
    }
}

```

```

// Si se envió un comentario general en POST, agregarlo como comentario nuevo
if (isset($_POST['comentarios']) && $_POST['comentarios']) {
    $comentario = new stdClass;
    $comentario->cotizacion = $cotizacion->id;
    $comentario->usuario = $this->session->id;
    $comentario->comentario = $this->input->post('comentarios');

    $this->Conexion->insertar('cotizaciones_comentarios', $comentario, array('fecha' =>
'CURRENT_TIMESTAMP()));
}

```

```

// Devuelve el ID de la cotización como respuesta (por ejemplo, para JS)
echo $cotizacion->id;
}

```

```

function ajax_getCotizaciones() {
// Limpia la caché de la vista de catálogo
$this->output->delete_cache('cotizaciones/catalogo');

```

```

// Obtiene los filtros enviados por POST
$id = $this->input->post('id');
$texto = $this->input->post('texto');
$parametro = $this->input->post('parametro');
$cliente = $this->input->post('cliente');
$estatus = $this->input->post('estatus');
$tipo = $this->input->post('tipo');
$cerradas = $this->input->post('cerradas');
$fecha1 = $this->input->post('fecha1');
$fecha2 = $this->input->post('fecha2');

```

```

// Formateo de fechas para rango de búsqueda

```

```

$fecha1 = strval($fecha1) . ' 00:00:00';
$fecha2 = strval($fecha2) . ' 23:59:59';

// Filtro de fecha por defecto (último año)
$Limitante_Fecha = " AND (C.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR))";

// Bandera para agrupar resultados
$group = TRUE;

// Query base
$query = "SELECT C.*, IFNULL(MAX(CC.revision), 0) AS UltRev, E.nombre AS Cliente,
EC.nombre AS Contacto, CONCAT(R.nombre, ' ', R.paterno) AS Responsable, E.prospecto FROM
cotizaciones C LEFT JOIN empresas E ON E.id = C.empresa LEFT JOIN empresas_contactos EC ON
EC.id = C.contactos->'$[0]' LEFT JOIN usuarios R On R.id = C.responsable LEFT JOIN
cotizaciones_conceptos CC ON C.id = CC.cotizacion WHERE 1 = 1
";

// Si se busca por ID específico
if ($id) {
    $query .= " AND C.id = '$id'";
} else {
    // Filtro por cliente seleccionado
    if (!empty($cliente) && $cliente != 0) {
        $query .= " AND C.empresa = '$cliente'";
    }

    // Filtro por tipo de cotización
    if (!empty($tipo) && $tipo != "TODOS") {
        $query .= " AND C.tipo = '$tipo'";
    }

    // Filtro por estatus
    if (!empty($estatus) && $estatus != "TODOS") {
        $query .= " AND C.estatus = '$estatus'";
    } else {
        // Si no se muestran cerradas
        if ($cerradas == "0") {

```

```

        $query .= " AND (C.estatus != 'CERRADO TOTAL' AND C.estatus != 'CERRADO PARCIAL'
AND C.estatus != 'CANCELADA')";
    }
}

// Filtro por texto (folio, ID, marca, etc.)
if (!empty($texto)) {
    if ($parametro == "folio") {
        $query .= " AND C.id = '$texto'";
    }
    if ($parametro == "id") {
        $query .= " AND UPPER(CC.atributos->'$.ID') LIKE '%" . strtoupper($texto) . "%'";
    }
    if ($parametro == "marca") {
        $query .= " AND UPPER(CC.atributos->'$.Marca') LIKE '%" . strtoupper($texto) . "%'";
    }
    if ($parametro == "serie") {
        $query .= " AND UPPER(CC.atributos->'$.Serie') LIKE '%" . strtoupper($texto) . "%'";
    }
    if ($parametro == "modelo") {
        $query .= " AND UPPER(CC.atributos->'$.Modelo') LIKE '%" . strtoupper($texto) . "%'";
    }
    if ($parametro == "contenido") {
        $query .= " AND CC.descripcion LIKE '%$texto%'";
    }
}

// Si el filtro es por responsable, requiere agrupamiento especial
if ($parametro == "responsable") {
    $query .= $Limitante_Fecha;
    $group = FALSE;
    $query .= " GROUP BY C.id";
    $query .= " HAVING Responsable LIKE '%$texto%' ";
}
}
}

// Filtro adicional por rango de fechas (si ambos campos están presentes)
if (!empty($fecha1) && !empty($fecha2)) {

```

```

        $query .= " AND C.fecha BETWEEN '$f1' AND '$f2'";
    }

    // Aplica agrupamiento si está habilitado
    if ($group) {
        $query .= $Limitante_Fecha;
        $query .= " GROUP BY C.id";
    }

    // Orden descendente por ID
    $query .= " ORDER BY C.id DESC";

    // Ejecuta la consulta
    $res = $this->Conexion->consultar($query, $id);

    // Devuelve los resultados en formato JSON
    if ($res) {
        echo json_encode($res);
    }
}

function ajax_getClientesCotizaciones() {
    $texto = $this->input->post('texto');

    // Consulta que devuelve clientes con número de cotizaciones
    $query = "SELECT E.id, E.nombre, COUNT(C.id) AS NumCot FROM cotizaciones C INNER JOIN
empresas E ON E.id = C.empresa
";

    // Filtro por nombre de cliente si se proporcionó texto
    if ($texto) {
        $query .= " WHERE E.nombre LIKE '%$texto%'";
    }

    $query .= " GROUP BY E.id";
    $res = $this->Conexion->consultar($query);

    if ($res) {

```

```

        echo json_encode($res);
    }
}

function ajax_getCotizacionConceptos() {
    $coti = $this->input->post('cotizacion');
    $rev = $this->input->post('revision');

    // Consulta conceptos de una cotización (opcionalmente por revisión)
    $query = "SELECT CC.* FROM cotizaciones_conceptos CC WHERE cotizacion = $coti";

    // Si se envió la revisión como parámetro, agregarla al filtro
    if (isset($_POST["revision"])) {
        $query .= " AND revision = $rev";
    }

    $res = $this->Conexion->consultar($query);

    if ($res) {
        echo json_encode($res);
    }
}

function ajax_setRevision() {
    $cotizacion = json_decode($this->input->post("cotizacion"));
    $conceptos = json_decode($this->input->post("conceptos"));
    $id = $cotizacion->id;

    // Elimina el ID de la cotización (solo se usará como referencia, no se modificará)
    unset($cotizacion->id);

    // Obtiene el siguiente número de revisión
    $rv = $this->Conexion->consultar(
        "SELECT (MAX(CC.revision) + 1) AS Rev
        FROM cotizaciones_conceptos CC
        WHERE CC.cotizacion = $id",
        TRUE
    );
}

```

```

// Inserta todos los conceptos con la nueva revisión
foreach ($conceptos as $key => $elem) {
    $elem->cotizacion = $id;
    $elem->revision = $rv->Rev;
    $this->Conexion->insertar('cotizaciones_conceptos', $elem);
}

// Actualiza la cabecera de la cotización (excepto el ID)
$this->Conexion->modificar('cotizaciones', $cotizacion, null, array('id' => $id));

echo "1";
}

function ajax_setComentarios() {
    $comentario = json_decode($this->input->post('comentario'));

    // Asigna al comentario el usuario actual y la fecha actual
    $comentario->usuario = $this->session->id;
    $funciones = array('fecha' => 'CURRENT_TIMESTAMP()');

    // Inserta el comentario
    $res = $this->Conexion->insertar('cotizaciones_comentarios', $comentario, $funciones);

    if ($res > 0) {
        echo "1";
    }
}

function ajax_getComentarios() {
    $id = $this->input->post('id');

    // Consulta los comentarios de una cotización y el nombre del usuario que los hizo
    $query = "SELECT C.*, CONCAT(U.nombre, ' ', U.paterno) AS User FROM
cotizaciones_comentarios C INNER JOIN usuarios U ON U.id = C.usuario WHERE 1 = 1
";

    // Filtra por ID de cotización si se proporciona

```

```

if ($id) {
    $query .= " AND C.cotizacion = '$id'";
}

$res = $this->Conexion->consultar($query);

if ($res) {
    echo json_encode($res);
}
}

function ajax_getSAData() {
    // Permite acceso desde cualquier origen (CORS)
    header("Access-Control-Allow-Origin: *");

    // Obtiene el ID del equipo desde POST
    $id_equipo = $this->input->post('id_equipo');

    // Ejecuta un archivo externo .exe pasando el ID como parámetro
    $res = shell_exec("C:/xampp/htdocs/sa_reader/sa_reader.exe \"$id_equipo\"");
    echo $res;
}

function ajax_getClientes() {
    $id = $this->input->post('id');
    $nombre = $this->input->post('nombre');

    // Consulta empresas que sean clientes y tengan moneda de cotización
    $query = "SELECT E.* FROM empresas E WHERE E.cliente = 1 AND
JSON_LENGTH(E.moneda_cotizacion) > 0
";

    // Filtro por ID si se proporciona
    if ($id) {
        $query .= " AND E.id = '$id'";
    } else {
        // Filtro por nombre parcial si se proporciona
        if ($nombre) {

```

```

        $query .= " AND E.nombre LIKE '%$nombre%';
    }
}

$res = $this->Conexion->consultar($query, $id);

if ($res) {
    echo json_encode($res);
}
}

function ajax_getPlanta() {
    $id = $this->input->post('id');

    // Consulta la planta de la empresa por ID
    $query = "SELECT * FROM empresa_plantas WHERE id = $id";

    $res = $this->Conexion->consultar($query, TRUE);

    if ($res) {
        echo json_encode($res);
    }
}

function ajax_getbuscarAutores() {
    $id = $this->input->post('id');

    // Consulta usuarios activos con privilegio para administrar cotizaciones
    $query = "SELECT U.id, CONCAT(U.nombre, ' ', U.paterno) AS Nombre, P.puesto AS Puesto,
    U.correo FROM usuarios U INNER JOIN puestos P ON U.puesto = P.id INNER JOIN privilegios PR
    ON PR.usuario = U.id WHERE U.activo = 1 AND PR.administrar_cotizaciones = 1";

    // Filtro por ID si se proporciona
    if ($id) {
        $query .= " AND U.id = '$id'";
    }

    $res = $this->Conexion->consultar($query, $id);

```



```

    if ($res) {
        echo json_encode($res);
    }
}

function ajax_getContactos() {
    $id = $this->input->post('id');
    $empresa = $this->input->post('id_cliente');

    // Consulta contactos activos y cotizables
    $query = "SELECT * FROM empresas_contactos WHERE activo = 1 AND cotizable = 1";

    // Filtro por ID si se proporciona
    if ($id) {
        $query .= " AND id = '$id'";
    } else {
        // Filtro por empresa si se proporciona
        if ($empresa) {
            $query .= " AND empresa = '$empresa'";
        }
    }

    $res = $this->Conexion->consultar($query, $id);

    if ($res) {
        echo json_encode($res);
    }
}

function ajax_getNombresUsuarios() {
    $ids = $this->input->post('ids');

    // Convierte el array en sintaxis SQL válida para IN
    $ids = str_replace('[', '(', $ids);
    $ids = str_replace(']', ')', $ids);

    // Consulta nombres de los usuarios dados sus IDs

```

```

$query = "SELECT id, CONCAT(nombre, ' ', paterno) AS User FROM usuarios WHERE id IN
$id";

$res = $this->Conexion->consultar($query);

if ($res) {
    echo json_encode($res);
}
}

function ajax_getServicio() {
    $codigo = $this->input->post('codigo');

    // Consulta servicio por código y su precio
    $query = "SELECT S.id, S.codigo, S.sitio, S.descripcion AS DescripcionServicio, CP.alto_a AS
Precio FROM servicios S INNER JOIN claves_precio CP ON S.clave_precio = CP.id WHERE S.codigo
= '$codigo'";

    $res = $this->Conexion->consultar($query);

    if ($res) {
        echo json_encode($res);
    }
}

function ajax_getServicioMarMod() {
    $fabricante = $this->input->post('fabricante');
    $modelo = $this->input->post('modelo');

    // Consulta requerimiento catalogado por marca y modelo
    $query = "SELECT R.id, R.descripcion, R.servicio FROM requerimientos R WHERE R.catalogado
= '1' AND UPPER(TRIM(R.fabricante)) = '$fabricante' AND UPPER(TRIM(R.modelo)) = '$modelo'
LIMIT 1";

    $res = $this->Conexion->consultar($query, TRUE);

    if ($res) {
        echo json_encode($res);
    }
}

```

```

    }
}

function ajax_enviarCorreo() {
    // Carga la librería encargada del envío de correos con archivos adjuntos
    $this->load->library('correos_archivos');

    // Decodifica la cotización recibida por POST
    $cotizacion = json_decode($this->input->post("cotizacion"));

    $id_cot = $cotizacion->id;
    $param = $cotizacion->id . "-" . $cotizacion->UltRev;

    // Genera el PDF de la cotización
    $pdf_content = $this->cotizacion_pdf($param, 'S');

    // Consulta los correos del contacto, responsable, creador y copia
    $res = $this->Conexion->consultar("SELECT ec.correo AS contacto, r.correo AS responsable,
u.correo AS creador, c.correo_cc FROM cotizaciones c JOIN empresas_contactos ec ON
JSON_CONTAINS(c.contactos, CAST(ec.id AS JSON), '$') JOIN usuarios r ON c.responsable = r.id
JOIN usuarios u ON c.usuario = u.id WHERE c.id = $id_cot",TRUE );

    // Consulta el cuerpo del correo activo
    $body = $this->Conexion->consultar("SELECT * FROM texto_correo WHERE activo = 1",
TRUE);

    // Obtiene la firma del usuario en sesión
    $firma = $this->Conexion->consultar("SELECT foto_firma FROM usuarios WHERE id = " . $this-
>session->id, TRUE);

    // Guarda la firma en un archivo temporal
    $nombre_archivo = 'firma_' . $this->session->id . '.png';
    $ruta_temporal = sys_get_temp_dir() . '/' . $nombre_archivo;
    file_put_contents($ruta_temporal, $firma->foto_firma);

    // Construye los datos del correo
    $datos['asunto'] = "Cotización Masmetrologia – #" . $id_cot;
    $datos['pdf'] = $pdf_content;

```

```

$datos['pdf_nombre'] = "Cotización - #" . $param . ".pdf";
$datos['body'] = $body->texto;
$datos['firma'] = $ruta_temporal;
$datos['firma_nombre'] = $nombre_archivo;
$datos['contacto'] = $res->contacto;
$datos['responsable'] = $res->responsable;
$datos['creador'] = $res->creador;
$datos['correo_cc'] = $res->correo_cc;

// Envía el correo y actualiza estatus si es AUTORIZADA
if ($this->correos_archivos->Solicitar_Autorización($datos)) {
    if ($datos['estatus'] == "AUTORIZADA") {
        $this->Conexion->modificar(
            'cotizaciones',
            array(
                'estatus' => 'ENVIADA',
                'bitacora_estatus' => $cotizacion->bitacora_estatus
            ),
            null,
            array('id' => $datos['id'])
        );
    }
    echo "1";
}

// Carga la vista del calendario de cotizaciones con los datos del subusuario actual
function calendario() {
    $this->load->model('usuarios_model');
    $datos['sub'] = $this->usuarios_model->userCots();

    $this->load->view('header');
    $this->load->view('cotizaciones/calendario', $datos);
}

// Obtiene las acciones programadas para mostrarlas en el calendario
function ajax_getAcciones_calendar() {
    $idsub = $this->input->get('idSub');

```

```

$query = "SELECT A.*, concat('Responsable: ',U.nombre, ' ', U.paterno,'\r\n','Cotizacion: ',
A.idCot ) as title, A.fecha_limite as start, A.fecha_limite + interval 1 hour as end,
concat(U.nombre, ' ', U.paterno) as User,";

$query .= " if(estatus = 'CANCELADA', 'gray', if(estatus = 'PENDIENTE' and
current_timestamp() > A.fecha_limite, 'red', if(estatus = 'PENDIENTE' and current_timestamp()
<= A.fecha_limite, '#f0ad4e', if(estatus = 'REALIZADA' and A.fecha_realizada > A.fecha_limite,
'#76A874', if(estatus = 'REALIZADA' and A.fecha_realizada <= A.fecha_limite, 'green', '')))) as
color";

$query .= " from cot_acciones A inner join usuarios U on U.id = A.usuario";

if (!empty($idsub)) {
    $query .= " where U.id=" . $idsub;
}

$res = $this->Conexion->consultar($query);

if ($res) {
    echo json_encode($res);
}
}

// Registra una nueva acción de seguimiento para una cotización y envía un evento de
calendario por correo
function ajax_setAccion() {
    $this->load->library('correos_archivos');

    $accion = json_decode($this->input->post('data'));
    $rev = $this->input->post('rev');
    $responsable = $this->input->post('responsable');
    $enviar_contacto = $this->input->post('enviar_contacto');

    $idUser = null;
    $contacto = null;

    $param = $accion->idCot . "-" . $rev;
    $pdf_content = $this->cotizacion_pdf($param, 'S');

```

```

if (empty($responsable)) {
    $accion->usuario = $this->session->id;
    $idUser = $this->session->id;
} else {
    $accion->usuario = $responsable;
    $idUser = $responsable;
}

$accion->estatus = "PENDIENTE";
$func['fecha_creacion'] = 'CURRENT_TIMESTAMP()';

$id = $this->Conexion->insertar('cot_acciones', $accion, $func);

$query = "SELECT concat(nombre , ", paterno) as name, correo from usuarios where id = " .
$idUser;
$res = $this->Conexion->consultar($query, TRUE);

if ($enviar_contacto == 1) {
    $contacto = $this->Conexion->consultar(
        "SELECT ec.correo
        FROM cotizaciones c
        JOIN empresas_contactos ec
        ON JSON_CONTAINS(c.contactos, CAST(ec.id AS JSON), '$')
        WHERE c.id = $accion->idCot",
        TRUE
    );
}

// Generar contenido para evento de calendario
$fecha = strtotime($accion->fecha_limite);
$name = "Cotización : " . $accion->idCot;
$start = date('Ymd', $fecha) . 'T' . date('His', $fecha);
$end = date('Ymd', $fecha) . 'T' . date('His', $fecha);
$description = $accion->accion;

$ical_content = "BEGIN:VCALENDAR
VERSION:2.0
PRODID:-//LearnPHP.co//NONSGML {$res->name}//EN

```

```

METHOD:REQUEST
BEGIN:VEVENT
ORGANIZER;CN=" . $this->session->nombre . ":tickets@masmetrologia.com
UID:" . date('Ymd') . 'T' . date('His') . rand() . "-learnphp.co
DTSTAMP:" . date('Ymd') . 'T' . date('His') . "
DTSTART:{$start}
DTEND:{$end}
SUMMARY:{$name}
DESCRIPTION:{$description}
END:VEVENT
END:VCALENDAR";

```

```

// Datos del correo
$datos['nombre'] = "Seguimiento-" . $accion->idCot . ".ics";
$datos['cuerpo'] = "Esto es un evento de seguimiento para la cotización - " . $accion->idCot;
$datos['cal'] = $ical_content;
$datos['correo'] = $res->correo;
$datos['accion'] = $description;
$datos['contacto'] = $contacto->correo;
$datos['pdf'] = $pdf_content;
$datos['pdf_nombre'] = "Cotización-" . $param . ".pdf";
$datos['correoResponsable'] = $this->session->correo;

$this->correos_archivos->evento_cotizaciones($datos);

if ($id > 0) {
    echo $id;
}
}

```

// Obtiene las acciones asociadas a una cotización específica

```

function ajax_getAcciones() {
    $po = $this->input->post('idCot');

    $res = $this->Conexion->consultar(
        "SELECT A.*, concat(U.nombre, ' ', U.paterno) as User
        FROM cot_acciones A
        INNER JOIN usuarios U ON U.id = A.usuario

```

```

        WHERE A.idCot != 0 AND A.idCot = $po"
    );

    if ($res) {
        echo json_encode($res);
    }
}

// Marca una acción como realizada, registrando la fecha actual
function ajax_setAccionRealizada() {
    $accion = json_decode($this->input->post('data'));
    $func['fecha_realizada'] = 'CURRENT_TIMESTAMP()';

    $id = $this->Conexion->modificar(
        'cot_acciones',
        $accion,
        $func,
        array('id' => $accion->id)
    );
}

// Actualiza los datos de una acción existente
function ajax_updateAccion() {
    $accion = json_decode($this->input->post('data'));

    $id = $this->Conexion->modificar('cot_acciones',$accion,null,array('id' => $accion->id)
    );
}

// Inserta un comentario asociado a una acción
function ajax_setAccionComentario() {
    $comment = json_decode($this->input->post('data'));
    $comment->usuario = $this->session->id;

    $func['fecha'] = 'CURRENT_TIMESTAMP()';

    $id = $this->Conexion->insertar('cot_acciones_comentarios', $comment, $func);
}

```



```

    if ($id > 0) {
        echo $id;
    }
}

// Obtiene los comentarios asociados a una acción específica
function ajax_getAccionComentarios() {
    $accion = $this->input->post('accion');

    $query = "SELECT C.*, concat(U.nombre, ' ', U.paterno, ' ', U.materno) as User FROM
cot_acciones_comentarios C INNER JOIN usuarios U ON U.id = C.usuario WHERE C.accion = " .
    $accion;

    $res = $this->Conexion->consultar($query);

    if ($res) {
        echo json_encode($res);
    }
}

function exportar() {
    $parametro = $this->input->post('rbBusqueda');
    $texto = $this->input->post('txtBusqueda');
    $cliente = $this->input->post('txtClientId');
    $tipoCot = $this->input->post('opTipoCotizacion');
    $estatus = $this->input->post('opEstatus');
    $cerradas = $this->input->post('cbCerradasCanceladas');
    $fecha1 = $this->input->post('fecha1');
    $fecha2 = $this->input->post('fecha2');
    $f1 = strval($fecha1) . ' 00:00:00';
    $f2 = strval($fecha2) . ' 23:59:59';
    $query2 = "";

    if (empty($cerradas)) {
        $close = " and (C.estatus != 'CERRADO TOTAL' and C.estatus != 'CERRADO PARCIAL' and
C.estatus != 'CANCELADA')";
    } else {
        $close = "";
    }
}

```

```
}
```

```
$sep = "',';
```

```
$coma = "";
```

```
$url = base_url('cotizaciones/ver_cotizacion/');
```

```
$con = $coma . '=HYPERLINK(' . $url . " . $coma;
```

```
$query = "SELECT C.id as Cotizacion, C.fecha as Fecha, C.tipo as Servicio, C.estatus as Estatus,
EC.nombre as Contacto, EC.telefono as Telefono, EC.celular as Celular, EC.correo as Correo,
E.nombre as Cliente, concat(R.nombre, ' ', R.paterno) as Responsable, (SELECT fecha_creacion
from cot_acciones WHERE idCot= C.id ORDER BY id DESC limit 1) as Fecha_Seguimiento,
(SELECT accion from cot_acciones WHERE idCot= C.id ORDER BY id DESC limit 1) as
Accion_Seguimiento, C.prospectoEmp from cotizaciones C left join empresas E on E.id =
C.empresa left join empresas_contactos EC on EC.id = C.contactos->'$[0]' left join usuarios R on
R.id = C.responsable inner join cotizaciones_conceptos CC on C.id = CC.cotizacion where 1 = 1 " .
$con . "and (C.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) ";
```

```
if (!empty($cliente) && $cliente != 0) {
    $query .= " and C.empresa = '$cliente'";
}
```

```
if (!empty($tipoCot) && $tipoCot != "TODOS") {
    $query .= " and C.tipo = '$tipoCot'";
}
```

```
if (!empty($estatus) && $estatus != 'TODOS') {
    $query .= " and C.estatus = '$estatus'";
}
```

```
if (!empty($texto)) {
    if ($parametro == "folio") {
        $query .= " and C.id = '$texto'";
    }
    if ($parametro == "id") {
        $query .= " and UPPER(CC.atributos->'$.ID') like '%" . strtoupper($texto) . "%'";
    }
    if ($parametro == "marca") {
        $query .= " and UPPER(CC.atributos->'$.Marca') like '%" . strtoupper($texto) . "%'";
    }
    if ($parametro == "serie") {
```

```

        $query .= " and UPPER(CC.atributos->'$.Serie') like '%" . strtoupper($texto) . "%'";
    }
    if ($parametro == "modelo") {
        $query .= " and UPPER(CC.atributos->'$.Modelo') like '%" . strtoupper($texto) . "%'";
    }
    if ($parametro == "contenido") {
        $query .= " and CC.descripcion like '%$texto%'";
    }
    if ($parametro == "responsable") {
        $query2 = " having Responsable like '%$texto%' ";
    }
}

if (!empty($fecha1) && !empty($fecha2)) {
    $query .= " and C.fecha BETWEEN '" . $f1 . "' AND '" . $f2 . "'";
}

$query = ' group by C.id ' . $query2 . ' order by C.fecha desc';

$result = $this->Conexion->consultar($query);

$salida = "";
$salida .= '<table style="border: 1px solid black; border-collapse: collapse;">
    <thead>
        <tr>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Cotizacion</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Fecha</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Servicio</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Estatus</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Contacto</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Telefono</th>

```

```

        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Celular</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Correo</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Cliente</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Prospecto</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Responsable</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Fecha Seguimiento</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Accion Seguimiento</th>
    </tr>
</thead>
<tbody>';

```

```

foreach ($result as $row) {
    $prospecto = ($row->prospectoEmp == 1) ? 'Si' : 'No';
    $salida .= '
        <tr>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">' .
$ row->Cotizacion . '</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">' .
$ row->Fecha . '</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">' .
$ row->Servicio . '</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">' .
$ row->Estatus . '</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">' .
$ row->Contacto . '</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">' .
$ row->Telefono . '</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">' .
$ row->Celular . '</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">' .
$ row->Correo . '</td>

```

```

        <td style="color: $444; border: 1px solid black; border-collapse: collapse">' .
$row->Cliente . '</td>
        <td style="color: $444; border: 1px solid black; border-collapse: collapse">' .
$prospecto . '</td>
        <td style="color: $444; border: 1px solid black; border-collapse: collapse">' .
$row->Responsable . '</td>
        <td style="color: $444; border: 1px solid black; border-collapse: collapse">' .
$row->Fecha_Seguimiento . '</td>
        <td style="color: $444; border: 1px solid black; border-collapse: collapse">' . $row-
>Accion_Seguimiento . '</td>
    </tr>';
}

```

```
$salida .= '</tbody></table>';
```

```

$timestamp = date('m/d/Y', time());
$filename = 'Cotizaciones' . $timestamp . '.xls';

```

```

header("Content-Type: application/vnd.ms-excel");
header("Content-Disposition: attachment; filename=\"$filename\"");
header("Pragma: no-cache");
header("Expires: 0");
header('Content-Transfer-Encoding: binary');
echo $salida;
}

```

```

function ajax_getDashboard(){
    $sub = $this->input->post('idSub');

```

```

    $query = "SELECT C.*, ifnull(max(CC.revision), 0) as UltRev, E.nombre as Cliente, EC.nombre as
Contacto, concat(R.nombre, ' ', R.paterno) as Responsable from cotizaciones C left join
empresas E on E.id = C.empresa left join empresas_contactos EC on EC.id = C.contactos left join
usuarios R on R.id = C.responsable inner join cotizaciones_conceptos CC on C.id = CC.cotizacion
where 1 = 1 and (C.estatus != 'CERRADO TOTAL' and C.estatus != 'CERRADO PARCIAL' and
C.estatus != 'CANCELADA') and (C.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) ";

```

```

if ($this->session->privilegios['cotDashboard']) {
    $query .= " ";

```

```

} elseif ($this->session->privilegios['generar_cotizaciones']) {
    $query .= " and C.usuario = ".$this->session->id;
}

if ($sub != 'TODO') {
    $query .= " and R.nombre like '%" . $sub . "%'";
}

$query .= " group by C.id order by C.fecha desc";

$res = $this->Conexion->consultar($query);
$res = count($res);

if ($res) {
    echo json_encode($res);
}
}

function ajax_getDashboardCreadas(){
    $sub = $this->input->post('idSub');

    // Consulta para obtener cotizaciones con estatus "CREADA" del último año
    $query = 'SELECT C.*, ifnull(max(CC.revision), 0) as UltRev, E.nombre as Cliente, EC.nombre as
Contacto, concat(R.nombre, " ", R.paterno) as Responsable from cotizaciones C left join
empresas E on E.id = C.empresa left join empresas_contactos EC on EC.id = C.contactos->"$[0]"
left join usuarios R on R.id = C.responsable inner join cotizaciones_conceptos CC on C.id =
CC.cotizacion where 1 = 1 and C.estatus = "CREADA" and (C.fecha > (CURRENT_DATE() -
INTERVAL 1 YEAR)) ' ;

    // Filtro según privilegios de sesión
    if ($this->session->privilegios['cotDashboard']) {
        $query .= " ";
    } elseif ($this->session->privilegios['generar_cotizaciones']) {
        $query .= " and C.usuario = ".$this->session->id;
    }

    // Filtro por responsable si se indica un subusuario específico
    if ($sub != 'TODO') {

```

```

        $query .= " and R.nombre like '%" . $sub . "%'";
    }

    $query .= ' group by C.id order by C.fecha desc';

    $res = $this->Conexion->consultar($query);

    if ($res) {
        echo json_encode($res);
    }
}

function ajax_getDashboardRechazadas(){
    $sub = $this->input->post('idSub');

    // Consulta para obtener cotizaciones RECHAZADAS del último año
    $query = 'SELECT C.*, ifnull(max(CC.revision), 0) as UltRev, E.nombre as Cliente, EC.nombre as
    Contacto, concat(R.nombre, " ", R.paterno) as Responsable from cotizaciones C left join
    empresas E on E.id = C.empresa left join empresas_contactos EC on EC.id = C.contactos->"$[0]"
    left join usuarios R on R.id = C.responsable inner join cotizaciones_conceptos CC on C.id =
    CC.cotizacion where 1 = 1 and C.estatus = "RECHAZADA" and (C.fecha > (CURRENT_DATE() -
    INTERVAL 1 YEAR)) ';

    // Filtro según privilegios del usuario en sesión
    if ($this->session->privilegios['cotDashboard']) {
        $query .= " ";
    } elseif ($this->session->privilegios['generar_cotizaciones']) {
        $query .= " and C.usuario = " . $this->session->id;
    }
}

// Filtro por subusuario si aplica
if ($sub != 'TODOS') {
    $query .= " and R.nombre like '%" . $sub . "%'";
}

// Agrupación y orden de resultados
$query .= ' group by C.id order by C.fecha desc';

```

```

// Depuración: se muestra la consulta y se detiene la ejecución
echo $query; die();

$res = $this->Conexion->consultar($query);

if ($res) {
    echo json_encode($res);
}
}

function ajax_getDashboardPendientes(){
    $sub = $this->input->post('idSub');

    // Consulta para obtener cotizaciones en estado "PENDIENTE AUTORIZACION" del último año
    $query = 'SELECT C.*, ifnull(max(CC.revision), 0) as UltRev, E.nombre as Cliente, EC.nombre as
    Contacto, concat(R.nombre, " ", R.paterno) as Responsable from cotizaciones C left join
    empresas E on E.id = C.empresa left join empresas_contactos EC on EC.id = C.contactos->"$[0]"
    left join usuarios R on R.id = C.responsable inner join cotizaciones_conceptos CC on C.id =
    CC.cotizacion where 1 = 1 and C.estatus = "PENDIENTE AUTORIZACION" and (C.fecha >
    (CURRENT_DATE() - INTERVAL 1 YEAR)) ';

    // Filtro según los privilegios del usuario en sesión
    if ($this->session->privilegios['cotDashboard']) {
        $query .= " ";
    } elseif ($this->session->privilegios['generar_cotizaciones']) {
        $query .= " and C.usuario = " . $this->session->id;
    }
}

// Filtro adicional si se seleccionó un subusuario específico
if ($sub != 'TODOS') {
    $query .= " and R.nombre like '%" . $sub . "%'";
}

// Agrupación por cotización y orden descendente por fecha
$query .= ' group by C.id order by C.fecha desc';

// Ejecución de la consulta
$res = $this->Conexion->consultar($query);

```



```

// Si hay resultados, se regresan en formato JSON
if ($res) {
    echo json_encode($res);
}
}

function ajax_getDashboardRevision(){
    $sub = $this->input->post('idSub');

    // Consulta para obtener cotizaciones en estado "EN REVISION" del último año
    $query = 'SELECT C.*, ifnull(max(CC.revision), 0) as UltRev, E.nombre as Cliente, EC.nombre as
    Contacto, concat(R.nombre, " ", R.paterno) as Responsable from cotizaciones C left join
    empresas E on E.id = C.empresa left join empresas_contactos EC on EC.id = C.contactos->"$[0]"
    left join usuarios R on R.id = C.responsable inner join cotizaciones_conceptos CC on C.id =
    CC.cotizacion where 1 = 1 and C.estatus = "EN REVISION" and (C.fecha > (CURRENT_DATE() -
    INTERVAL 1 YEAR)) ';

    // Filtro según privilegios del usuario en sesión
    if ($this->session->privilegios['cotDashboard']) {
        $query .= " ";
    } elseif ($this->session->privilegios['generar_cotizaciones']) {
        $query .= " and C.usuario = " . $this->session->id;
    }

    // Filtro por subusuario si se especifica uno
    if ($sub != 'TODOS') {
        $query .= " and R.nombre like '%" . $sub . "%'";
    }

    // Agrupamiento y ordenamiento
    $query .= ' group by C.id order by C.fecha desc';

    // Ejecución de la consulta
    $res = $this->Conexion->consultar($query);

    // Devolver resultados en formato JSON
    if ($res) {

```

```

        echo json_encode($res);
    }
}

function ajax_getDashboardAutorizadas(){
    $sub = $this->input->post('idSub');

    // Consulta para obtener cotizaciones con estatus "AUTORIZADA" del último año
    $query = 'SELECT C.*, ifnull(max(CC.revision), 0) as UltRev, E.nombre as Cliente, EC.nombre as
    Contacto, concat(R.nombre, " ", R.paterno) as Responsable from cotizaciones C left join
    empresas E on E.id = C.empresa left join empresas_contactos EC on EC.id = C.contactos->"$[0]"
    left join usuarios R on R.id = C.responsable inner join cotizaciones_conceptos CC on C.id =
    CC.cotizacion where 1 = 1 and C.estatus = "AUTORIZADA" and (C.fecha > (CURRENT_DATE() -
    INTERVAL 1 YEAR)) ';

    // Filtro según privilegios del usuario en sesión
    if ($this->session->privilegios['cotDashboard']) {
        $query .= " ";
    } elseif ($this->session->privilegios['generar_cotizaciones']) {
        $query .= " and C.usuario = " . $this->session->id;
    }

    // Filtro por subusuario si se especifica uno
    if ($sub != 'TODOS') {
        $query .= " and R.nombre like '%" . $sub . "%'";
    }

    // Agrupamiento y ordenamiento
    $query .= ' group by C.id order by C.fecha desc';

    // Ejecución de la consulta
    $res = $this->Conexion->consultar($query);

    // Devolver resultados en formato JSON si hay datos
    if ($res) {
        echo json_encode($res);
    }
}

```

```

function ajax_getDashboardConfirmadas(){
    $sub = $this->input->post('idSub');

    // Consulta principal para cotizaciones con estatus "CONFIRMADA"
    $query = 'SELECT C.*, ifnull(max(CC.revision), 0) as UltRev, E.nombre as Cliente, EC.nombre as
    Contacto, concat(R.nombre, " ", R.paterno) as Responsable from cotizaciones C left join
    empresas E on E.id = C.empresa left join empresas_contactos EC on EC.id = C.contactos->"$[0]"
    left join usuarios R on R.id = C.responsable inner join cotizaciones_conceptos CC on C.id =
    CC.cotizacion where 1 = 1 and C.estatus = "CONFIRMADA" and (C.fecha > (CURRENT_DATE() -
    INTERVAL 1 YEAR)) ';

    // Filtro por privilegios del usuario
    if ($this->session->privilegios['cotDashboard']) {
        $query .= " ";
    } elseif ($this->session->privilegios['generar_cotizaciones']) {
        $query .= " and C.usuario = " . $this->session->id;
    }

    // Filtro por subusuario si se indicó alguno
    if ($sub != 'TODOS') {
        $query .= " and R.nombre like '%" . $sub . "%'";
    }

    // Agrupamiento y orden por fecha descendente
    $query .= ' group by C.id order by C.fecha desc';

    // Ejecutar consulta y retornar resultado si existe
    $res = $this->Conexion->consultar($query);

    if ($res) {
        echo json_encode($res);
    }
}

function ajax_getDashboardAprobacion(){
    $sub = $this->input->post('idSub');

```

```
// Consulta para obtener cotizaciones con estatus "EN APROBACION" del último año
$query = 'SELECT C.*, ifnull(max(CC.revision), 0) as UltRev, E.nombre as Cliente, EC.nombre as
Contacto, concat(R.nombre, " ", R.paterno) as Responsable from cotizaciones C left join
empresas E on E.id = C.empresa left join empresas_contactos EC on EC.id = C.contactos->"$[0]"
left join usuarios R on R.id = C.responsable inner join cotizaciones_conceptos CC on C.id =
CC.cotizacion where 1 = 1 and C.estatus = "EN APROBACION" and (C.fecha > (CURRENT_DATE() -
INTERVAL 1 YEAR)) ';
```

```
// Filtro según privilegios del usuario
if ($this->session->privilegios['cotDashboard']) {
    $query .= " ";
} elseif ($this->session->privilegios['generar_cotizaciones']) {
    $query .= " and C.usuario = " . $this->session->id;
}
}
```

```
// Filtro por subusuario si corresponde
if ($sub != 'TODO') {
    $query .= " and R.nombre like '%" . $sub . "%'";
}
}
```

```
// Agrupamiento y orden final
$query .= ' group by C.id order by C.fecha desc';
```

```
// Ejecutar consulta y retornar resultado
$res = $this->Conexion->consultar($query);
```

```
if ($res) {
    echo json_encode($res);
}
}
```

```
function ajax_getDashboardApParcial(){
    $sub = $this->input->post('idSub');
```

```
// Consulta para obtener cotizaciones con estatus "APROBADO PARCIAL" del último año
$query = 'SELECT C.*, ifnull(max(CC.revision), 0) as UltRev, E.nombre as Cliente, EC.nombre as
Contacto, concat(R.nombre, " ", R.paterno) as Responsable from cotizaciones C left join
empresas E on E.id = C.empresa left join empresas_contactos EC on EC.id = C.contactos->"$[0]"
```

```
left join usuarios R on R.id = C.responsable inner join cotizaciones_conceptos CC on C.id =
CC.cotizacion where 1 = 1 and C.estatus = "APROBADO PARCIAL" and (C.fecha >
(CURRENT_DATE() - INTERVAL 1 YEAR)) ';
```

```
// Filtro según privilegios del usuario
if ($this->session->privilegios['cotDashboard']) {
    $query .= " ";
} elseif ($this->session->privilegios['generar_cotizaciones']) {
    $query .= " and C.usuario = " . $this->session->id;
}
}
```

```
// Filtro por subusuario si corresponde
if ($sub != 'TODOS') {
    $query .= " and R.nombre like '%" . $sub . "%'";
}
}
```

```
// Agrupamiento y orden final
$query .= ' group by C.id order by C.fecha desc';
```

```
// Ejecutar consulta y retornar resultado
$res = $this->Conexion->consultar($query);
```

```
if ($res) {
    echo json_encode($res);
}
}
```

```
function ajax_getDashboardApTotal(){
    $sub = $this->input->post('idSub');
```

```
// Consulta base: obtiene cotizaciones con estatus "APROBADO TOTAL" del último año
$query = 'SELECT C.*, ifnull(max(CC.revision), 0) as UltRev, E.nombre as Cliente, EC.nombre as
Contacto, concat(R.nombre, " ", R.paterno) as Responsable from cotizaciones C left join
empresas E on E.id = C.empresa left join empresas_contactos EC on EC.id = C.contactos->"$[0]"
left join usuarios R on R.id = C.responsable inner join cotizaciones_conceptos CC on C.id =
CC.cotizacion where 1 = 1 and C.estatus = "APROBADO TOTAL" and (C.fecha > (CURRENT_DATE()
- INTERVAL 1 YEAR)) ';
```

```

// Filtro por privilegios del usuario
if ($this->session->privilegios['cotDashboard']) {
    $query .= " ";
} elseif ($this->session->privilegios['generar_cotizaciones']) {
    $query .= " and C.usuario = " . $this->session->id;
}

// Filtro por subusuario si se especifica
if ($sub != 'TODO') {
    $query .= " and R.nombre like '%" . $sub . "%'";
}

// Agrupar por ID de cotización y ordenar por fecha descendente
$query .= ' group by C.id order by C.fecha desc';

// Ejecutar la consulta
$res = $this->Conexion->consultar($query);

// Si hay resultados, devolverlos como JSON
if ($res) {
    echo json_encode($res);
}
}

function excel()
{
    // Consulta todos los conceptos de la cotización con ID 16747
    $query = "SELECT CC.* from cotizaciones_conceptos CC where cotizacion = 16747";
    $result = $this->Conexion->consultar($query);

    $salida = "";

    // Encabezado de la tabla con estilos en línea
    $salida .= '<table style="border: 1px solid black; border-collapse: collapse;">
        <thead>
            <tr>
                <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Cantidad</th>

```

```

        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Concepto</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Servicio</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Comentarios</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Servicio a Realizarse</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">T. Entrega </th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Precio Unitario</th>
    </tr>
</thead>
<tbody>';
$d = 2;

```

```

// Cuerpo de la tabla con los datos obtenidos
foreach ($result as $row) {
    $salida .= '
        <tr>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">' .
$row->cantidad . '</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">' .
$row->descripcion . '</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">' .
$row->servicios . '</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">' .
$row->comentarios . '</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">' .
$row->sitio . '</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">' .
$row->tiempo_entrega . '</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">' .
$row->precio_unitario . '</td>
        </tr>';
    $d = $d + 1;
}

```

```

$salida .= '</tbody>
          </table>';

// Encabezados para forzar la descarga como archivo Excel
$timestamp = date('m/d/Y', time());
$filename = 'QR_' . $timestamp . '.xls';
header("Content-Type: application/vnd.ms-excel");
header("Content-Disposition: attachment; filename=\"$filename\"");
header("Pragma: no-cache");
header("Expires: 0");
header('Content-Transfer-Encoding: binary');

// Salida final del archivo
echo $salida;
}

function buscarQrs()
{
    // Obtiene el valor ingresado para búsqueda del QR
    $qr = $this->input->post('txtBuscarQr');

    // Consulta el ID y estatus de la requisición que coincide con el QR
    $query = "SELECT id, estatus FROM requisiciones_cotizacion where id = " . $qr;
    $res = $this->Conexion->consultar($query);

    // Devuelve los resultados en formato JSON si existen
    if ($res)
    {
        echo json_encode($res);
    }
}

function ajax_setQr()
{
    // Recibe los datos enviados por POST
    $qr = $this->input->post('qr');
    $id_cotizacion = $this->input->post('id_cotizacion');

```



```

// Crea el arreglo de datos a actualizar
$data = array('id_cotizacion' => $id_cotizacion);

// Actualiza la tabla requisiciones_cotizacion con el ID de cotización
$res = $this->Conexion->modificar('requisiciones_cotizacion', $data, null, array('id' => $qr));

// Devuelve 1 si la operación fue exitosa
if ($res)
{
    echo 1;
}
}

function ajax_getQrs()
{
    // Recibe el ID de cotización desde POST
    $id_cotizacion = $this->input->post('id_cotizacion');

    // Consulta los QRs asociados a la cotización
    $query = "SELECT id FROM requisiciones_cotizacion where id_cotizacion = " . $id_cotizacion;
    $res = $this->Conexion->consultar($query);

    // Devuelve los IDs en formato JSON si existen resultados
    if ($res)
    {
        echo json_encode($res);
    }
}

function ajax_getReporteCotizaciones()
{
    $requisitor = $this->input->post('asignado');
    $us=null;
    $usF=null;
    if ($requisitor != 'TODO') {
        $us = ' and C1.usuario =' . $requisitor;
        $usF = ' and C.usuario =' . $requisitor;
    }
}

```

```

}
$query="SELECT
COUNT(DISTINCT C.id) AS Total,
(SELECT COUNT(DISTINCT C1.id)
FROM cotizaciones C1
WHERE C1.estatus = 'CREADA'".$us."
AND C1.estatus NOT IN ('CERRADO TOTAL', 'CERRADO PARCIAL', 'CANCELADA')
AND C1.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) AS TotalCreadas,
(SELECT MIN(C1.fecha)
FROM cotizaciones C1
WHERE C1.estatus = 'CREADA'".$us."
AND C1.estatus NOT IN ('CERRADO TOTAL', 'CERRADO PARCIAL', 'CANCELADA')
AND C1.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) AS UltFechaCreadas,
(SELECT COUNT(DISTINCT C1.id)
FROM cotizaciones C1
WHERE C1.estatus = 'RECHAZADA'".$us."
AND C1.estatus NOT IN ('CERRADO TOTAL', 'CERRADO PARCIAL', 'CANCELADA')
AND C1.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) AS TotalRechazadas,
(SELECT MIN(C1.fecha)
FROM cotizaciones C1
WHERE C1.estatus = 'RECHAZADA'".$us."
AND C1.estatus NOT IN ('CERRADO TOTAL', 'CERRADO PARCIAL', 'CANCELADA')
AND C1.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) AS UltFechaRechazadas,
(SELECT COUNT(DISTINCT C1.id)
FROM cotizaciones C1
WHERE C1.estatus = 'PENDIENTE AUTORIZACION'".$us."
AND C1.estatus NOT IN ('CERRADO TOTAL', 'CERRADO PARCIAL', 'CANCELADA')
AND C1.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) AS TotalPendienteAutorizacion,
(SELECT MIN(C1.fecha)
FROM cotizaciones C1
WHERE C1.estatus = 'PENDIENTE AUTORIZACION'".$us."
AND C1.estatus NOT IN ('CERRADO TOTAL', 'CERRADO PARCIAL', 'CANCELADA')
AND C1.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) AS UltFechaPendienteAutorizacion,
(SELECT COUNT(DISTINCT C1.id)
FROM cotizaciones C1
WHERE C1.estatus = 'EN REVISION'".$us."
AND C1.estatus NOT IN ('CERRADO TOTAL', 'CERRADO PARCIAL', 'CANCELADA')

```

```

    AND C1.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) AS TotalEnRevision,
(SELECT MIN(C1.fecha)
FROM cotizaciones C1
WHERE C1.estatus = 'EN REVISION'".$us."
    AND C1.estatus NOT IN ('CERRADO TOTAL', 'CERRADO PARCIAL', 'CANCELADA')
    AND C1.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) AS UltFechaEnRevision,
(SELECT COUNT(DISTINCT C1.id)
FROM cotizaciones C1
WHERE C1.estatus = 'AUTORIZADA'".$us."
    AND C1.estatus NOT IN ('CERRADO TOTAL', 'CERRADO PARCIAL', 'CANCELADA')
    AND C1.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) AS TotalAutorizadas,
(SELECT MIN(C1.fecha)
FROM cotizaciones C1
WHERE C1.estatus = 'AUTORIZADA'".$us."
    AND C1.estatus NOT IN ('CERRADO TOTAL', 'CERRADO PARCIAL', 'CANCELADA')
    AND C1.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) AS UltFechaAutorizadas,
(SELECT COUNT(DISTINCT C1.id)
FROM cotizaciones C1
WHERE C1.estatus = 'CONFIRMADA'".$us."
    AND C1.estatus NOT IN ('CERRADO TOTAL', 'CERRADO PARCIAL', 'CANCELADA')
    AND C1.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) AS TotalConfirmada,
(SELECT MIN(C1.fecha)
FROM cotizaciones C1
WHERE C1.estatus = 'CONFIRMADA'".$us."
    AND C1.estatus NOT IN ('CERRADO TOTAL', 'CERRADO PARCIAL', 'CANCELADA')
    AND C1.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) AS UltFechaConfirmada,
(SELECT COUNT(DISTINCT C1.id)
FROM cotizaciones C1
WHERE C1.estatus = 'EN APROBACION'".$us."
    AND C1.estatus NOT IN ('CERRADO TOTAL', 'CERRADO PARCIAL', 'CANCELADA')
    AND C1.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) AS TotalEnAprobacion,
(SELECT MIN(C1.fecha)
FROM cotizaciones C1
WHERE C1.estatus = 'EN APROBACION'".$us."
    AND C1.estatus NOT IN ('CERRADO TOTAL', 'CERRADO PARCIAL', 'CANCELADA')
    AND C1.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) AS UltFechaEnAprobacion,
(SELECT COUNT(DISTINCT C1.id)
FROM cotizaciones C1

```

```

WHERE C1.estatus = 'APROBADO TOTAL'".$us."
  AND C1.estatus NOT IN ('CERRADO TOTAL', 'CERRADO PARCIAL', 'CANCELADA')
  AND C1.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) AS TotalAprobadoTotal,
(SELECT MIN(C1.fecha)
FROM cotizaciones C1
WHERE C1.estatus = 'APROBADO TOTAL'".$us."
  AND C1.estatus NOT IN ('CERRADO TOTAL', 'CERRADO PARCIAL', 'CANCELADA')
  AND C1.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) AS UltFechaAprobadoTotal,
(SELECT COUNT(DISTINCT C1.id)
FROM cotizaciones C1
WHERE C1.estatus = 'APROBADO PARCIAL'".$us."
  AND C1.estatus NOT IN ('CERRADO TOTAL', 'CERRADO PARCIAL', 'CANCELADA')
  AND C1.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) AS TotalAprobadoParcial,
(SELECT MIN(C1.fecha)
FROM cotizaciones C1
WHERE C1.estatus = 'APROBADO PARCIAL'".$us."
  AND C1.estatus NOT IN ('CERRADO TOTAL', 'CERRADO PARCIAL', 'CANCELADA')
  AND C1.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) AS UltFechaAprobadoParcial
FROM cotizaciones C
WHERE C.estatus NOT IN ('CERRADO TOTAL', 'CERRADO PARCIAL', 'CANCELADA')
  AND C.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)".$usF;
$res = $this->Conexion->consultar($query, TRUE);
//echo $query;die
    echo json_encode($res);
}
}

```

Descargas.php

```
<?php
```

```
defined('BASEPATH') OR exit('No direct script access allowed');
```

```
class Descargas extends CI_Controller {
```

```

function __construct() {
    parent::__construct();
    $this->load->model('descargas_model', 'Modelo'); // Modelo principal para descargas

```

```

        $this->load->helper('download'); // Helper para forzar descargas
        $this->load->model('MLConexion_model', 'MLConexion'); // Modelo para conexión con
WO_Master
    }

    public function index() {
        // Sin implementación; puede utilizarse como landing para el controlador si se requiere
    }

    // Descarga archivos adjuntos a tickets de autos
    function tickets_AT($id) {
        $row = $this->Modelo->getFile($id, 'tickets_autos_archivos');
        $file = $row->archivo;
        $nombre = $row->nombre;
        force_download($nombre, $file);
    }

    // Descarga archivos adjuntos a tickets de edificio
    function tickets_ED($id) {
        $row = $this->Modelo->getFile($id, 'tickets_edificio_archivos');
        $file = $row->archivo;
        $nombre = $row->nombre;
        force_download($nombre, $file);
    }

    // Descarga archivos adjuntos a tickets de sistemas
    function tickets_IT($id) {
        $row = $this->Modelo->getFile($id, 'tickets_sistemas_archivos');
        $file = $row->archivo;
        $nombre = $row->nombre;
        force_download($nombre, $file);
    }

    // Descarga archivos adjuntos a órdenes de trabajo (WO_Master)
    function ordenes_trabajo($id) {
        $row = $this->MLConexion->consultar("SELECT * from WO_Master where
WorkOrder_ID=".$id, true);
        $file = $row->archivo;

```

```

        $nombre = $row->nombre_archivo;
        force_download($nombre, $file);
    }
}

```

Documentacion.php

```
<?php
```

```
defined('BASEPATH') OR exit('No direct script access allowed');
```

```
class Documentacion extends CI_Controller {
```

```

    function __construct() {
        parent::__construct();
        $this->load->model('conexion_model', 'Conexion'); // Carga el modelo de conexión general
    }

```

```
// Muestra la vista del catálogo de manuales
```

```

function manuales() {
    $this->load->view('header');
    $this->load->view('documentacion/catalogo');
}

```

```
// Muestra la vista del mapa del sitio
```

```

function mapa_sitio() {
    $this->load->view('header');
    $this->load->view('documentacion/mapa_sitio');
}
}

```

Empresas.php

```
<?php
```

```
defined('BASEPATH') OR exit('No direct script access allowed');
```

```
class Empresas extends CI_Controller {
```

```

function __construct() {
    parent::__construct();
    $this->load->model('empresas_model', 'Modelo');
    $this->db->db_debug = FALSE; // Desactiva los errores detallados de la base de datos
}

// Carga la vista principal del catálogo de empresas
function index() {
    // $this->output->enable_profiler(TRUE); // Línea comentada para depuración
    // $datos['empresas'] = $this->Modelo->getEmpresas(); // Comentado, no se usa en la vista
    actual
    $this->load->view('header');
    $this->load->view('empresas/catalogo');
}

// Muestra el formulario para dar de alta una nueva empresa
function alta() {
    $datos['países'] = $this->Modelo->listadopaises(); // Lista de países para el formulario
    $this->load->view('header');
    $this->load->view('empresas/alta', $datos);
}

// Muestra los detalles de una empresa específica
function ver($id) {
    $datos['empresa'] = $this->Modelo->getEmpresa($id); // Datos generales de la empresa
    $datos['proveedor'] = $this->Modelo->getProveedor($id); // Información si es proveedor
    $datos['contactos'] = $this->Modelo->getContactos($id); // Contactos asociados
    $datos['archivos'] = $this->Modelo->getArchivos($id); // Archivos relacionados
    $datos['requisitos'] = $this->Modelo->getRequisitos_empresa($id); // Requisitos
    pendientes
    $datos['países'] = $this->Modelo->listadopaises(); // Nuevamente países
    $datos['documento'] = $this->Modelo->documento(); // Documentos de referencia

    $this->load->view('header');
    $this->load->view('empresas/ver', $datos);
}

```

```

function registrar() {
$ACIERTOS = array();
$ERRORES = array();

// Recolección y limpieza de datos del formulario
$data = array(
    'nombre' => trim($this->input->post('nombre')),
    'nombre_corto' => trim($this->input->post('nombre_corto')),
    'razon_social' => trim($this->input->post('razon_social')),
    'giro' => trim($this->input->post('giro')),
    'clasificacion' => trim($this->input->post('clasificacion')),
    'rfc' => trim($this->input->post('rfc')),
    'calle' => trim($this->input->post('calle')),
    'numero' => trim($this->input->post('numero')),
    'numero_interior' => trim($this->input->post('numero_interior')),
    'colonia' => trim($this->input->post('colonia')),
    'calles_aux' => trim($this->input->post('calles_aux')),
    'pais' => trim($this->input->post('pais')),
    'estado' => trim($this->input->post('estado')),
    'ciudad' => trim($this->input->post('ciudad')),
    'cp' => trim($this->input->post('cp')),
    'foto' => 'default.png', // Imagen por defecto
    'ubicacion' => "",
    'cliente' => $this->input->post('cliente') != NULL ? '1' : '0',
    'proveedor' => $this->input->post('proveedor') != NULL ? '1' : '0',
    'credito_cliente' => '0',
    'credito_cliente_plazo' => '0',
    'documentos_facturacion' => '[]',
    'codigo_impresion' => "",
    'comentarios' => "",
    'moneda_cotizacion' => '[]',
    'iva_cotizacion' => '[]',
    'notas_cotizacion' => "",
    'contacto_cotizacion' => '[]',
    'requisitos_logisticos' => "",
    'requisitos_documento' => "",
    'factura_ejemplo' => "",
    'dejar_factura' => 0,

```



```

        'prospecto' => $this->input->post('prospecto') != NULL ? '1' : '0',
    );

    // Intento de inserción de nueva empresa
    if ($this->Modelo->crear_empresa($data)) {
        $acierto = array('titulo' => 'Agregar Empresa', 'detalle' => 'Se ha agregado Empresa con
Éxito');
        array_push($ACIERTOS, $acierto);

        // Recupera el ID recién insertado
        $res = $this->Conexion->consultar('SELECT MAX(id) as id FROM empresas', TRUE);

        // Registra en bitácora
        $data = array(
            'id_empresa' => $res->id,
            'user' => $this->session->id,
            'estatus' => 'ALTA',
        );
        $this->Modelo->bitacoraEmpresas($data);
    } else {
        // En caso de error al insertar
        $error = array('titulo' => 'ERROR', 'detalle' => 'Error al agregar Empresa');
        array_push($ERRORES, $error);
    }

    // Se guardan mensajes en sesión
    $this->session->aciertos = $ACIERTOS;
    $this->session->errores = $ERRORES;

    // Redirige al catálogo de empresas
    redirect(base_url('empresas'));
}

function editar() {
    $ACIERTOS = array();
    $ERRORES = array();

```

```

$id = $this->input->post('id');

// Recolección y limpieza de datos del formulario
$data = array(
    'id' => trim($id),
    'nombre' => trim($this->input->post('nombre')),
    'giro' => trim($this->input->post('giro')),
    'razon_social' => trim($this->input->post('razon_social')),
    'rfc' => trim($this->input->post('rfc')),
    'calle' => trim($this->input->post('calle')),
    'numero' => trim($this->input->post('numero')),
    'numero_interior' => trim($this->input->post('numero_interior')),
    'colonia' => trim($this->input->post('colonia')),
    'calles_aux' => trim($this->input->post('calles_aux')),
    'cp' => trim($this->input->post('cp')),
    'pais' => trim($this->input->post('pais')),
    'estado' => trim($this->input->post('estado')),
    'ciudad' => trim($this->input->post('ciudad')),
    'ubicacion' => "",
    'cliente' => $this->input->post('cliente') != NULL ? '1' : '0',
    'proveedor' => $this->input->post('proveedor') != NULL ? '1' : '0',
    'prospecto' => $this->input->post('prospecto') != NULL ? '1' : '0',
);

// Actualiza la empresa
if ($this->Modelo->update($data)) {
    $acierto = array('titulo' => 'Editar Empresa', 'detalle' => 'Se ha editado Empresa con Éxito');
    array_push($ACIERTOS, $acierto);

    // Registro en bitácora
    $data = array(
        'id_empresa' => trim($id),
        'user' => $this->session->id,
        'estatus' => 'Editar Empresa',
    );
    $this->Modelo->bitacoraEmpresas($data);
} else {
    // Error al actualizar

```

```

        $error = array('titulo' => 'ERROR', 'detalle' => 'Error al editar Empresa');
        array_push($ERRORES, $error);
    }

    // Almacena los mensajes en la sesión
    $this->session->aciertos = $ACIERTOS;
    $this->session->errores = $ERRORES;

    // Redirige a la vista de la empresa
    redirect(base_url('empresas/ver/' . $id));
}

function paises() {
    // Carga la vista del catálogo de países
    $this->load->view('header');
    $this->load->view('empresas/catalogo_paises');
}

function ajax_getEmpresas() {
    $texto = trim($this->input->post('texto'));
    $parametro = $this->input->post('parametro');
    $cliente = $this->input->post('cliente');
    $proveedor = $this->input->post('proveedor');

    // Consulta base: une empresas con proveedores
    $query = "SELECT E.* FROM empresas E JOIN proveedores P ON E.id = P.empresa WHERE 1 = 1";

    // Filtro según tipo (cliente o proveedor)
    if ($cliente != $proveedor) {
        if ($cliente == "1") {
            $query .= " AND E.cliente = '1'";
        } else {
            $query .= " AND E.proveedor = '1'";
        }
    }

    // Filtros de búsqueda por parámetro

```

```

if (!empty($texto)) {
    if ($parametro == "nombre") {
        $query .= " AND (E.nombre LIKE '%$texto%' OR E.razon_social LIKE '%$texto%')";
    } else if ($parametro == "id") {
        $query .= " AND E.id = '$texto'";
    } else if ($parametro == "tag") {
        $query .= " AND P.tags LIKE '%$texto%'";
    }
}

// Ejecuta la consulta y retorna el resultado
$res = $this->Conexion->consultar($query);
if ($res) {
    echo json_encode($res);
} else {
    echo "";
}
}

```

```

function ajax_getContactos() {
    // Obtiene el ID de la empresa desde el POST
    $id = $this->input->post('id');
    $where['empresa'] = $id;

    // Consulta los contactos asociados a la empresa
    $res = $this->Conexion->get('empresas_contactos', $where);
    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

```

```

function ajax_getContacto() {
    // Obtiene el ID del contacto desde el POST
    $id = $this->input->post('id');

    // Consulta la información del contacto junto con el nombre de la planta si existe

```

```

$res = $this->Conexion->consultar(
    "SELECT EC.*, IFNULL(Pl.nombre, 'NO DEFINIDO') AS Planta
    FROM empresas_contactos EC
    LEFT JOIN empresa_plantas Pl ON Pl.id = EC.planta
    WHERE EC.id = $id",
    TRUE
);
if ($res) {
    echo json_encode($res);
}
}

function ajax_getArchivos() {
    // Obtiene el ID de la empresa y texto de búsqueda desde el POST
    $empresa = $this->input->post('empresa');
    $texto = $this->input->post('texto');

    // Consulta base para obtener los archivos de la empresa con información del usuario y tipo
    de documento
    $query = "SELECT A.*, CONCAT(U.nombre, ' ', U.paterno) AS User, D.tipo, D.clave
    FROM empresas_archivos A
    INNER JOIN usuarios U ON U.id = A.usuario
    LEFT JOIN documentosEmpresas D ON D.id = A.id_documento
    WHERE A.empresa = $empresa";

    // Filtro por texto si se proporciona
    if ($texto) {
        $query .= " AND (A.nombre LIKE '%$texto%' OR A.comentarios LIKE '%$texto%')";
    }

    // Ejecuta la consulta y responde con los resultados
    $res = $this->Conexion->consultar($query);
    if ($res) {
        echo json_encode($res);
    }
}

function ajax_getFacturaEjemplo() {

```

```

// Obtiene el ID de la empresa desde POST
$empresa = $this->input->post('empresa');

// Consulta el nombre del archivo de factura de ejemplo
$query = "SELECT factura_ejemplo FROM empresas WHERE id = $empresa";
$res = $this->Conexion->consultar($query, TRUE);

// Devuelve el nombre del archivo como respuesta
echo $res->factura_ejemplo;
}

function ajax_setFacturaEjemplo() {
    // Obtiene datos desde POST
    $id = $this->input->post('empresa');
    $file = $this->input->post('file');

    // Si se recibió un archivo válido
    if ($file != "undefined") {
        $config['upload_path'] = 'data/empresas/ejemplo_facturas/';
        $config['allowed_types'] = 'pdf';
        $config['encrypt_name'] = TRUE;

        $this->load->library('upload', $config);

        // Intenta subir el archivo
        if ($this->upload->do_upload('file')) {
            $data['factura_ejemplo'] = $this->upload->data('file_name');

            // Guarda el nombre del archivo en la base de datos
            $this->Conexion->modificar('empresas', $data, null, array('id' => $id));
        }
    }
}

function ajax_deleteFacturaEjemplo() {
    // Obtiene ID de la empresa
    $id = $this->input->post('empresa');

```

```

// Consulta el nombre del archivo actual
$res = $this->Conexion->consultar("SELECT factura_ejemplo FROM empresas WHERE id =
$id", TRUE);

// Elimina el archivo del servidor
unlink('data/empresas/ejemplo_facturas/' . $res->factura_ejemplo);

// Limpia el campo en la base de datos
$data['factura_ejemplo'] = "";
$this->Conexion->modificar('empresas', $data, null, array('id' => $id));
}

```

```

function editar_otros_datos() {
// Inicializa arreglos para mensajes de éxito y error
$ACIERTOS = array();
$ERRORES = array();

// Obtiene el ID de la empresa desde POST
$id = $this->input->post('id');

// Recoge los datos del formulario
$data = array(
'id' => $id,
'horario_facturas' => trim($this->input->post('horario_facturas')),
'ultimo_dia_facturas' => trim($this->input->post('ultimo_dia_facturas')),
'requisitos_logisticos' => trim($this->input->post('requisitos_logisticos')),
'requisitos_documento' => trim($this->input->post('requisitos_documento')),
'comentarios' => trim($this->input->post('comentarios')),
'dejar_factura' => $this->input->post('dejar_factura') == '1' ? '1' : '0',
);

// Intenta guardar los cambios
if ($this->Modelo->update($data)) {
array_push($ACIERTOS, array('titulo' => 'Editar Empresa', 'detalle' => 'Se ha editado
Empresa con Éxito'));
} else {
array_push($ERRORES, array('titulo' => 'ERROR', 'detalle' => 'Error al editar Empresa'));
}
}

```

```

// Almacena mensajes y dirige
$this->session->aciertos = $ACIERTOS;
$this->session->errores = $ERRORES;
redirect(base_url('empresas/ver/' . $id));
}

```

// CONTACTOS

```

function agregarContacto() {
    // Recoge datos del formulario
    $empresa = $this->input->post('empresa');
    $data = array(
        'empresa' => trim($empresa),
        'nombre' => trim($this->input->post('nombre')),
        'telefono' => trim($this->input->post('telefono')),
        'ext' => trim($this->input->post('ext')),
        'celular' => trim($this->input->post('celular')),
        'celular2' => trim($this->input->post('celular2')),
        'correo' => trim($this->input->post('correo')),
        'puesto' => trim($this->input->post('puesto')),
        'red_social' => trim($this->input->post('red_social')),
        'activo' => $this->input->post('activo'),
        'cotizable' => $this->input->post('cotizable'),
        'planta' => $this->input->post('planta'),
    );

    // Inserta el nuevo contacto
    $res = $this->Modelo->insertContacto($data);

    // Registra en bitácora
    $r = $this->Conexion->consultar('SELECT * FROM empresas_contactos ORDER BY id DESC
LIMIT 1', TRUE);
    $bitacora = array(
        'id_contacto' => $r->id,
        'id_empresa' => $r->empresa,
        'user' => $this->session->id,
        'estatus' => 'ALTA',
    );
}

```



```

);
$this->Modelo->bitacoraContactosEmpresas($bitacora);

// Devuelve el nuevo contacto en formato JSON
if ($res) {
    $res = json_encode($this->Modelo->getContacto($res));
} else {
    $res = "";
}

echo $res;
}

// Edita los datos de un contacto existente
function editarContacto() {
    $id = $this->input->post('id');

    // Prepara los datos recibidos del formulario
    $data = array(
        'id' => trim($id),
        'nombre' => trim($this->input->post('nombre')),
        'telefono' => trim($this->input->post('telefono')),
        'ext' => trim($this->input->post('ext')),
        'celular' => trim($this->input->post('celular')),
        'celular2' => trim($this->input->post('celular2')),
        'correo' => trim($this->input->post('correo')),
        'puesto' => trim($this->input->post('puesto')),
        'red_social' => trim($this->input->post('red_social')),
        'activo' => $this->input->post('activo'),
        'cotizable' => $this->input->post('cotizable'),
        'planta' => $this->input->post('planta'),
    );

    // Consulta la empresa del contacto para registrar en bitácora
    $r = $this->Conexion->consultar('SELECT * FROM empresas_contactos WHERE id = '.trim($id),
    TRUE);
    $dataBit = array(
        'id_contacto' => trim($id),

```

```

        'id_empresa' => $r->empresa,
        'user' => $this->session->id,
        'estatus' => 'Editar Contacto',
    );
    $this->Modelo->bitacoraContactosEmpresas($dataBit);

    // Actualiza el contacto y responde con el objeto actualizado
    if ($this->Modelo->updateContacto($data)) {
        $res = json_encode($this->Modelo->getContacto($id));
    } else {
        $res = "";
    }

    echo $res;
}

// Retorna un contacto en formato JSON
function getContacto_json(){
    $contact = $this->Modelo->getContacto($this->input->post('id'));
    if($contact){
        echo json_encode($contact);
    } else {
        echo "";
    }
}

// Elimina un contacto y retorna "1" si fue exitoso, o cadena vacía si falló
function deleteContacto_json(){
    if($this->Modelo->deleteContacto($this->input->post('id'))){
        echo "1";
    } else {
        echo ""; // jQuery interpreta como FALSE
    }
}

// Sube o reemplaza la foto de una empresa
function subir_foto() {
    $id = $this->input->post('id_empresa');    // ID de la empresa

```

```

$foto = $this->input->post('fotoActual');    // Nombre del archivo actual
$file = $this->input->post('iptFoto');        // Archivo recibido

if ($file != "undefined") {
    // Configuración para la carga del archivo
    $config['upload_path'] = 'data/empresas/fotos/';
    $config['allowed_types'] = 'gif|jpg|png';
    $config['encrypt_name'] = TRUE;
    $this->load->library('upload', $config);

    // Si la carga del archivo fue exitosa
    if ($this->upload->do_upload('iptFoto')) {
        $data['id'] = $id;
        $data['foto'] = $this->upload->data('file_name');

        // Actualiza la foto en la base de datos
        if ($this->Modelo->update($data)) {
            // Elimina la foto anterior si no es la por defecto
            if ($foto != "default.png") {
                unlink('data/empresas/fotos/' . $foto);
            }
            echo $data['foto'];
        }
    } else {
        echo "";
    }
} else {
    // Si no se sube un archivo, se asigna la imagen por defecto
    $data['id'] = $id;
    $data['foto'] = 'default.png';

    if ($this->Modelo->update($data)) {
        if ($foto != "default.png") {
            unlink('data/empresas/fotos/' . $foto);
        }
        echo $data['foto'];
    }
}
}

```

```

}

// Sube un archivo relacionado a una empresa y lo registra en la base de datos
function subir_archivo() {
    $id = $this->input->post('id');           // ID de la empresa
    $comentarios = $this->input->post('comentarios'); // Comentarios del archivo
    $documento = $this->input->post('documento'); // Tipo de documento

    // Crea el directorio de la empresa si no existe
    if (!is_dir('data/empresas/archivos/' . $id)) {
        mkdir('data/empresas/archivos/' . $id, 0777, TRUE);
    }

    // Configuración de subida
    $config['upload_path'] = 'data/empresas/archivos/' . $id;
    $config['allowed_types'] = '*';
    $this->load->library('upload', $config);

    // Si la carga del archivo fue exitosa
    if ($this->upload->do_upload('userfile')) {
        $data['empresa'] = $id;
        $data['usuario'] = $this->session->id;
        $data['nombre'] = $this->upload->data('file_name');
        $data['comentarios'] = $comentarios;
        $data['id_documento'] = $documento;

        $idFile = $this->Modelo->insertArchivo($data);

        if ($idFile) {
            $arreglo['id'] = $idFile;
            $arreglo['nombre'] = $data['nombre'];
            $arreglo['fecha'] = date('d/m/Y h:i A');
            $arreglo['icono'] = $this->aos_funciones->file_image($data['nombre']);
            $arreglo['error'] = '';
            echo json_encode($arreglo);
        }
    } else {
        $arreglo['id'] = '0';
    }
}

```

```

    $arreglo['nombre'] = 'ERROR';
    $arreglo['fecha'] = 'ERROR';
    // Se intenta generar un ícono aunque no haya nombre (puede generar warning si no hay
archivo)
    $arreglo['icono'] = $this->aos_funciones->file_image($this->upload->data('file_name') ??
");
    $arreglo['error'] = $this->upload->display_errors();
    echo json_encode($arreglo);
}
}

// Elimina un archivo físico y su registro en la base de datos
function deleteArchivo_json() {
    $id_file = $this->input->post('id');
    $id_empresa = $this->input->post('id_empresa');
    $nombre = $this->input->post('nombre_archivo');

    if ($this->Modelo->deleteArchivo($id_file)) {
        unlink('data/empresas/archivos/' . $id_empresa . '/' . $nombre);
        echo "1";
    } else {
        echo ""; // jQuery interpreta como FALSE (no 0)
    }
}

// Edita los metadatos de un archivo (comentarios y tipo de documento)
function editArchivo_json() {
    $data['id'] = $this->input->post('id');
    $data['comentarios'] = trim($this->input->post('comentarios'));
    $data['id_documento'] = $this->input->post('documento');

    if ($this->Modelo->updateArchivo($data)) {
        echo $data['comentarios'];
    } else {
        echo ""; // jQuery interpreta como FALSE (no 0)
    }
}
}

```

////////// REQUISITOS DE FACTURACIÓN //////////

// Carga la vista del catálogo de requisitos

```
function requisitos() {  
    $data['requisitos'] = $this->Modelo->getRequisitos("");  
    $this->load->view('header');  
    $this->load->view('empresas/catalogo_requisitos', $data);  
}
```

// Retorna los requisitos de facturación filtrados por tipo

```
function getRequisitos_json() {  
    $tipo = $this->input->post('tipo');  
    $requisitos = $this->Modelo->getRequisitos($tipo);  
  
    if ($requisitos) {  
        echo json_encode($requisitos->result());  
    } else {  
        echo "";  
    }  
}
```

// Asocia un requisito existente a una empresa

```
function setRequisitos_json() {  
    $data['empresa'] = $this->input->post('id_empresa');  
    $data['requisito'] = $this->input->post('requisito');  
    $data['tipo'] = $this->input->post('tipo');  
    $data['detalles'] = $this->input->post('detalles');  
  
    $id = $this->Modelo->setRequisitos($data);  
  
    if ($id) {  
        $res['id'] = $id;  
        $res['requisito'] = $data['requisito'];  
        $res['detalles'] = $data['detalles'];  
        echo json_encode($res);  
    } else {  
        echo "";  
    }  
}
```

```

}

// Elimina un requisito vinculado a una empresa
function deleteRequisito_json() {
    if ($this->Modelo->deleteRequisito($this->input->post('id'))) {
        echo "1";
    } else {
        echo ""; // jQuery interpreta como FALSE (no 0)
    }
}

// Agrega un nuevo requisito genérico al catálogo
function agregarRequisito_ajax() {
    $data = array(
        'requisito' => strtoupper(trim($this->input->post('requisito'))),
        'tipo' => strtoupper(trim($this->input->post('tipo'))),
        'detalle' => $this->input->post('detalle'),
    );

    $res = $this->Modelo->insertRequisito($data);

    if ($res) {
        $data['id'] = $res;
        echo json_encode($data);
    } else {
        echo "";
    }
}

// Edita un requisito del catálogo general
function editarRequisito_ajax() {
    $data = array(
        'id' => strtoupper(trim($this->input->post('id'))),
        'requisito' => strtoupper(trim($this->input->post('requisito'))),
        'tipo' => strtoupper(trim($this->input->post('tipo'))),
        'detalle' => $this->input->post('detalle'),
    );
}

```

```

$res = $this->Modelo->updateRequisito($data);

if ($res) {
    echo json_encode($data);
} else {
    echo "";
}
}

// Elimina un requisito del catálogo general
function eliminarRequisito_ajax() {
    $data = array(
        'id' => trim($this->input->post('id')),
    );

    $res = $this->Modelo->deleteRequisitoCatalogo($data);

    if ($res) {
        echo "1";
    } else {
        echo "";
    }
}

// Actualiza la ubicación geográfica de la empresa (latitud, longitud y nivel de zoom)
function editarUbicacion_ajax() {
    $data = array(
        'id' => $this->input->post('id'),
        'lat' => $this->input->post('lat'),
        'lng' => $this->input->post('lng'),
        'zoom' => $this->input->post('zoom'),
    );

    $res = $this->Modelo->update($data);

    if ($res) {
        echo "1";
    } else {

```



```

        echo "";
    }
}

// Actualiza el listado de documentos y código de impresión de una empresa
function ajax_setListadoDocumentos(){
    $where['id'] = $this->input->post('id');
    $data['documentos_facturacion'] = $this->input->post('documentos');
    $data['codigo_impresion'] = strtoupper($this->input->post('codigo'));

    $this->Conexion->modificar('empresas', $data, null, $where);
    echo "1";
}

// Registra o actualiza los datos del proveedor asociados a la empresa
function ajax_setProveedor(){
    $data = array(
        'empresa' => $this->input->post('id_empresa'),
        'aprobado' => $this->input->post('aprobado'),
        'clasificacion_proveedor' => $this->input->post('clasificacion_proveedor'),
        'tipo' => $this->input->post('tipo'),
        'credito' => $this->input->post('credito'),
        'monto_credito' => $this->input->post('monto_credito'),
        'moneda_credito' => $this->input->post('moneda_credito'),
        'terminos_pago' => $this->input->post('terminos_pago'),
        'tags' => "," . $this->input->post('tags') . ",",
        'formas_pago' => $this->input->post('formas_pago'),
        'formas_compra' => $this->input->post('formas_compra'),
        'entrega' => $this->input->post('entrega'),
        'pasos_cotizacion' => $this->input->post('pasos_cotizacion'),
        'pasos_compra' => $this->input->post('pasos_compra'),
        'rma_requerido' => $this->input->post('rma_requerido'),
        'valResico' => $this->input->post('valResico'),
        'persona' => $this->input->post('persona'),
    );

    $res = $this->Modelo->setProveedor($data);

```

```

// Registrar en bitácora la modificación del proveedor
$data = array(
    'id_empresa' => $this->input->post('id_empresa'),
    'user' => $this->session->id,
    'estatus' => 'Editar Proveedor',
);
$this->Modelo->bitacoraEmpresas($data);

if ($res) {
    echo "1";
} else {
    echo "";
}
}

// Devuelve clientes cuyo nombre coincida parcialmente con el texto ingresado
function ajax_getClientes(){
    $texto = $this->input->post('texto');
    $query = "SELECT E.* FROM empresas E WHERE E.nombre LIKE '%" . $texto . "%'";

    $res = $this->Modelo->consulta($query);
    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

// Devuelve proveedores cuyo nombre o tags coincidan parcialmente con el texto ingresado
function ajax_getProveedores(){
    $texto = $this->input->post('texto');
    $query = "SELECT E.* FROM empresas E INNER JOIN proveedores P ON E.id = P.empresa
    WHERE E.proveedor = 1 AND (P.tags LIKE '%" . $texto . "%' OR E.nombre LIKE '%" . $texto .
    "%')";

    $res = $this->Modelo->consulta($query);
    if ($res) {
        echo json_encode($res);
    }
}

```

```

    } else {
        echo "";
    }
}

// Devuelve los datos del proveedor a partir de su ID
function ajax_getProveedor(){
    $id = $this->input->post('id');
    $query = "SELECT P.*, E.nombre FROM proveedores P INNER JOIN empresas E ON E.id =
P.empresa WHERE P.empresa = " . $id . " ";

    $res = $this->Modelo->consulta($query, TRUE);
    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

// Devuelve la lista de ciudades registradas en empresas (sin duplicados)
function ajax_getCiudades(){
    $query = "SELECT DISTINCT ciudad FROM empresas";

    $res = $this->Modelo->consulta($query);
    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

// Guarda la información de cotización (moneda, IVA, etc.) de una empresa
function ajax_setInfoCotizaciones(){
    $datos = json_decode($this->input->post('datos'));
    $datos->moneda_cotizacion = json_encode($datos->moneda_cotizacion);
    $datos->iva_cotizacion = json_encode($datos->iva_cotizacion);

    $this->Conexion->modificar('empresas', $datos, null, array('id' => $datos->id));
}

```

```

    echo "1";
}

// Obtiene los contactos cotizables de una empresa específica
function ajax_getContactosCotizacion(){
    $empresa = $this->input->post('empresa');
    $res = $this->Conexion->consultar("SELECT EC.*, IFNULL(Pl.nombre, 'NO DEFINIDO') as Planta
FROM empresas_contactos EC LEFT JOIN empresa_plantas Pl ON Pl.id = EC.planta WHERE
EC.empresa = '$empresa' AND EC.cotizable = 1");
    echo json_encode($res);
}

// Crea o edita una planta asociada a la empresa
function ajax_setPlanta(){
    $planta = json_decode($this->input->post('planta'));
    if ($planta->id) {
        $this->Conexion->modificar('empresa_plantas', $planta, null, array('id' => $planta->id));
    } else {
        $this->Conexion->insertar('empresa_plantas', $planta);
    }
}

// Obtiene las plantas asociadas a una empresa
function ajax_getPlantas(){
    $empresa = $this->input->post('empresa');
    $query = "SELECT * FROM empresa_plantas WHERE empresa = $empresa";

    $res = $this->Conexion->consultar($query);
    if ($res) {
        echo json_encode($res);
    }
}

// Elimina una planta por su ID
function ajax_deletePlanta(){
    $id = $this->input->post('id');
    $this->Conexion->eliminar('empresa_plantas', array('id' => $id));
}

```

```

// Devuelve el listado de estados del país seleccionado en formato <option>
function estados() {
    $pais = $this->input->post('paisid');

    if ($pais) {
        $sedo = $this->Modelo->getestados($pais);
        foreach ($sedo as $fila) {
            echo '<option value="'. $fila->estadonombre ."'>'. $fila->estadonombre .'</option>';
        }
    } else {
        echo '<option value="0">Estados</option>';
    }
}

// Obtiene la bitácora de cambios de una empresa específica
function ajax_getBitacoraEmpresas(){
    $ids = $this->input->post('id');
    $query = "SELECT CONCAT(u.nombre, ' ', u.paterno) as user, b.fecha, b.estatus FROM
bitacora_empresas b JOIN usuarios u ON b.user = u.id WHERE id_empresa = ". $ids;

    $res = $this->Conexion->consultar($query);
    if ($res) {
        echo json_encode($res);
    }
}

// Obtiene la bitácora de cambios de contactos de empresa
function ajax_getBitacoraEmpresasContactos(){
    $ids = $this->input->post('id');
    $query = "SELECT CONCAT(u.nombre, ' ', u.paterno) as user, b.fecha, b.estatus FROM
bitacora_contactos_empresas b JOIN usuarios u ON b.user = u.id WHERE id_contacto = ". $ids;

    $res = $this->Conexion->consultar($query);
    if ($res) {
        echo json_encode($res);
    }
}

```

// Devuelve lista de países según texto buscado, y si se desea solo los activos

```
function ajax_paises(){
    $texto = trim($this->input->post('texto'));
    $activo = $this->input->post('activo');
    $all = null;

    if ($activo == 0) {
        $all = " AND activo = 1";
    }

    $query = "SELECT * FROM pais WHERE 1 = 1 " . $all;
    if (!empty($texto)) {
        $query .= " AND paisnombre LIKE '%" . $texto . "%'";
    }

    $res = $this->Conexion->consultar($query);
    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}
```

// Activa un país estableciendo el campo 'activo' en 1

```
function ajax_Activarpaises()
{
    $id = $this->input->post('id');
    $data['activo'] = '1';
    $this->Conexion->modificar('pais', $data, null, array('id' => $id));
    echo "1";
}
```

// Desactiva un país estableciendo el campo 'activo' en 0

```
function ajax_Desactivarpaises()
{
    $id = $this->input->post('id');
    $data['activo'] = '0';
```

```

$this->Conexion->modificar('pais', $data, null, array('id' => $id));
echo "1";
}

// Obtiene todos los estados pertenecientes a un país específico
function ajax_Estados()
{
    $id = $this->input->post('id');
    $query = "SELECT * FROM estado WHERE paisid = " . $id;

    $res = $this->Conexion->consultar($query);
    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

// Asigna un estado como predeterminado (defecto = 1) dentro de un país
function ajax_AsignarEstados()
{
    $id = $this->input->post('id');
    $idp = $this->input->post('idp');

    // Desactiva el estado predeterminado anterior del país
    $dato['defecto'] = '0';
    $this->Conexion->modificar('estado', $dato, null, array('paisid' => $idp));

    // Activa el nuevo estado predeterminado
    $data['defecto'] = '1';
    $this->Conexion->modificar('estado', $data, null, array('id' => $id));

    // Retorna el nuevo estado marcado como predeterminado
    $query = "SELECT * FROM estado WHERE defecto = 1 AND id = " . $id;
    $res = $this->Conexion->consultar($query, TRUE);
    if ($res) {
        echo json_encode($res);
    } else {

```

```

        echo "";
    }
}

// Retorna el estado marcado como defecto (predeterminado) para un país dado
function ajax_setEstados()
{
    $id = $this->input->post('id');

    $query = "SELECT * FROM estado WHERE defecto = 1 AND paisid = " . $id;
    $res = $this->Conexion->consultar($query, TRUE);

    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

function excel_empresas(){
    $texto = $this->input->post('texto');
    $texto = trim($texto);
    $parametro = $this->input->post('rbBusqueda');
    $cliente = $this->input->post('cliente');
    $proveedor = $this->input->post('proveedor');
    $c=0;
    $p=0;
    $fecha1 = $this->input->post('fecha1');
    $fecha2 = $this->input->post('fecha2');
    $f1=strval($fecha1).' 00:00:00';
    $f2=strval($fecha2).' 23:59:59' ;
    if ($cliente == 'cliente') {
        $c=1;
    }
    if ($proveedor == 'proveedor') {
        $p=1;
    }
}

```



```
$query = "SELECT c.id as cot, c.fecha,concat(uc.nombre, ' ', uc.paterno) usercot, c.tipo,P.entrega,
c.estatus as estatus_cot, e.calle, e.numero, e.estado, e.pais, e.nombre, e.razon_social, e.ciudad,
e.cliente, e.proveedor, (SELECT estatus from bitacora_empresas WHERE id_empresa = e.id AND
estatus = 'ALTA') as estatus, (SELECT fecha from bitacora_empresas WHERE id_empresa = e.id
AND estatus = 'ALTA') as fecha_alta, (SELECT concat(u.nombre, ' ', u.paterno) from
bitacora_empresas be join usuarios u on u.id=be.user WHERE id_empresa = e.id AND estatus =
'ALTA') as usuario_alta FROM cotizaciones c JOIN empresas e on e.id=c.empresa JOIN
proveedores P on e.id=P.empresa join usuarios uc on uc.id=c.responsable WHERE 1=1";
```

```
if($c != $p)
{
    if($cliente == "cliente")
    {
        $query .= " and e.cliente = '1'";
    }
    else if ($proveedor == "proveedor")
    {
        $query .= " and e.proveedor = '1'";
    }
}

if(!empty($texto))
{
    if($parametro == "nombre")
    {
        $query .= " and (e.nombre like '%$texto%' or e.razon_social like '%$texto%')";
    }
    else if($parametro == "id")
    {
        $query .= " and e.id = '$texto'";
    }
    else if($parametro == "tag")
    {
        $query .= " and P.tags like '%$texto%'";
    }
}

if (!empty($fecha1) && !empty($fecha2)) {
    $query .= " HAVING fecha_alta BETWEEN '". $f1.'" AND '". $f2.'" ";
}
```

```

}
$query .= " ORDER BY e.nombre ASC";

$result= $this->Conexion->consultar($query);

$salida="";

$salida .= '<table style="border: 1px solid black; border-collapse: collapse;">
    <thead>
        <tr>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Nombre</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Razon Social</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Calle</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Cidudad</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Estado</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Pais</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Cliente</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Proveedor</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Entrega</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Fecha Alta</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Usuario Alta</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Cotizacion</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Fecha</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Estatus</th>

```

```

                <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Responsable</th>
                <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Tipo</th>
            </tr>
        </thead>
        <tbody>';
foreach($result as $row){
    $ECLIENTE="";
    $EPROVEEDOR="";
    if ($row->cliente == 1) {
        $ECLIENTE="CLIENTE";
    }
    if ($row->proveedor == 1) {
        $EPROVEEDOR="PROVEEDOR";
    }

    $salida .='

```

```

        <tr>
            <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'. $row->nombre.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'. $row->razon_social.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'. $row->calle." ". $row->numero.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'. $row->ciudad.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'. $row->estado.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'. $row->pais.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'. $ECLIENTE.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'. $EPROVEEDOR.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'. $row->entrega.'</td>

```

```

        <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'. $row->fecha_alta.'</td>
        <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'. $row->usuario_alta.'</td>
        <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'. $row->cot.'</td>
        <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'. $row->fecha.'</td>
        <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'. $row->estatus_cot.'</td>
        <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'. $row->usercot.'</td>
        <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'. $row->tipo.'</td>
    </tr>';
}

```

```

$salida .= '</tbody>
</table>';

```

```

$timestamp = date('m/d/Y', time());

```

```

$filename='Empresas_'. $timestamp.'.xls';
header("Content-Type: application/vnd.ms-excel");
header("Content-Disposition: attachment; filename=\"$filename\"");
header("Pragma: no-cache");
header("Expires: 0");
header('Content-Transfer-Encoding: binary');
echo $salida;

```

```

}

```

```

}

```

[Equipos.php](#)

<?php

```
defined('BASEPATH') OR exit('No direct script access allowed');
```

```
class Equipos extends CI_Controller {
```

```
function __construct() {  
    parent::__construct();  
    $this->load->model('tickets_IT_model','Modelo');  
    $this->load->model('privilegios_model');  
}
```

```
// Vista principal del catálogo de equipos TI
```

```
public function ti() {  
    $datos['usuarios'] = $this->privilegios_model->listadoJefes();  
    $this->load->view('header');  
    $this->load->view('equipos/ti/catalogo', $datos);  
}
```

```
// Muestra historial de un equipo específico
```

```
function historial($idequipo){  
    $datosequipo = $this->Modelo->getEquipo($idequipo); // No usado en esta vista, podrías  
revisar si es necesario  
    $datos['equipo'] = $this->Modelo->historial($idequipo);  
    $this->load->view('header');  
    $this->load->view('equipos/ti/historial', $datos);  
}
```

```
// Vista general de revisiones de equipos
```

```
function revisiones() {  
    $datos['equipo'] = $this->Modelo->Equipos();  
    $this->load->view('header');  
    $this->load->view('equipos/ti/revisiones', $datos);  
}
```

```
// Muestra la vista para realizar mantenimiento a un equipo específico
```

```
function mantenimiento($idE) {  
    $datos['equipo'] = $this->Modelo->getEquipos($idE);  
    $this->load->view('header');  
    $this->load->view('equipos/ti/mantenimiento', $datos);  
}
```

```
}
```

```
// Muestra el historial de mantenimiento de un equipo
```

```
function historialMantto($idE) {  
    $datos['equipo'] = $this->Modelo->getEquipos($idE);  
    $datos['manto'] = $this->Modelo->historialMantto($idE);  
    $this->load->view('header');  
    $this->load->view('equipos/ti/historialMantto', $datos);  
}
```

```
// Vista de hallazgos para un mantenimiento específico
```

```
public function hallazgos($iM) {  
    $id = $datos['manto'] = $this->Modelo->Mantto($iM);  
    $a = $id->idEquipo;  
    $datos['equipo'] = $this->Modelo->getEquipos($a);  
    $datos['fotos'] = $this->Modelo->fotosMantto($iM);  
    $this->load->view('header');  
    $this->load->view('equipos/ti/hallazgos', $datos);  
}
```

```
function ajax_setEquiposTI(){  
    $equipo = json_decode($this->input->post('equipo'));  
    $file = $this->input->post('iptFoto');
```

```
// Manejo de imagen anterior y subida de nueva imagen
```

```
if($equipo->foto != "default.png") {  
    unlink('data/equipos/ti/fotos/' . $equipo->foto);  
} else if($file == "undefined") {  
    $equipo->foto = 'default.png';  
} else {  
    $config['upload_path'] = 'data/equipos/ti/fotos/';  
    $config['allowed_types'] = 'gif|jpg|png';  
    $config['encrypt_name'] = TRUE;  
    $this->load->library('upload', $config);  
  
    if ($this->upload->do_upload('iptFoto')) {  
        $equipo->foto = $this->upload->data('file_name');  
    }  
}
```

```

}

// Tiempos de alta/asignación automáticos
$funciones = array('fecha_alta' => 'CURRENT_TIMESTAMP()', 'fecha_asignacion' =>
'CURRENT_TIMESTAMP());

if($equipo->id == 0) {
    // Inserta nuevo equipo
    $res = $this->Conexion->insertar('equipos_it', $equipo, $funciones);
} else {
    // Guarda histórico de asignación
    $query = 'INSERT INTO `equiposHis` (`idEqH`, `idequipo`, `idus`, `fecha`) VALUES (NULL,
'. $equipo->id.', '. $equipo->asignado.', CURRENT_TIMESTAMP);';
    $this->Conexion->comando($query);

    // Actualiza equipo existente
    $this->Conexion->modificar('equipos_it', $equipo, $funciones, array('id' => $equipo->id));
}
}

function ajax_getEquiposTI(){
    $texto = $this->input->post('texto');
    $tipo = $this->input->post('tipo');
    $inactivo = $this->input->post('inactivo');
    $asignado = $this->input->post('asignado');

    // Consulta base con unión para obtener el nombre del usuario asignado
    $query = "SELECT E.*, ifnull(concat(U.nombre, ' ', U.paterno, ' ', U.materno), 'N/A') as
Asignado from equipos_it E left join usuarios U on U.id = E.asignado ";

    // Si se recibe un ID específico, se filtra directamente por él
    if(isset($_POST['id'])){
        $id = $this->input->post('id');
        $query .= " where E.id = $id";
    }
    // Equipos inactivos por tipo
    else if ($inactivo == 1 && !empty($tipo)) {

```

```

$query = "SELECT E.*, ifnull(concat(U.nombre, ' ', U.paterno, ' ', U.materno), 'N/A') as
Asignado from equipos_it E left join usuarios U on U.id = E.asignado where E.activo = 0 and
E.tipo like ' ".$tipo."";
    if(isset($_POST['id'])){
        $id = $this->input->post('id');
        $query .= " and E.id = $id";
    }
}
// Equipos inactivos por texto (buscando por ID)
else if ($inactivo == 1 && !empty($texto)) {
    $query = "SELECT E.*, ifnull(concat(U.nombre, ' ', U.paterno, ' ', U.materno), 'N/A') as
Asignado from equipos_it E left join usuarios U on U.id = E.asignado where E.activo = 0 and E.id
= ' ".$texto."";
    if(isset($_POST['id'])){
        $id = $this->input->post('id');
        $query .= " and E.id = $id";
    }
}
// Todos los equipos inactivos
else if ($inactivo == 1) {
    $query = "SELECT E.*, ifnull(concat(U.nombre, ' ', U.paterno, ' ', U.materno), 'N/A') as
Asignado from equipos_it E left join usuarios U on U.id = E.asignado where E.activo = 0";
    if(isset($_POST['id'])){
        $id = $this->input->post('id');
        $query .= " and E.id = $id";
    }
}
// Búsqueda por campo JSON: No_Inventario_Interno o Serie (en caso de celulares)
else if(!empty($texto)){
    $campo = '$.No_Inventario_Interno';
    if ($tipo == 'Celular') {
        $campo = '$.Serie';
    }
    $query = "SELECT E.*, ifnull(concat(U.nombre, ' ', U.paterno, ' ', U.materno), 'N/A') as
Asignado from equipos_it E left join usuarios U on U.id = E.asignado where E.activo=1
and JSON_EXTRACT(campos, ' ".$campo." ) = ' ".$texto."";
    if(isset($_POST['id'])){
        $id = $this->input->post('id');

```



```

        $query .= " and E.id = $id";
    }
}
// Filtro por tipo
else if(!empty($tipo)){
    $query = "SELECT E.*, ifnull(concat(U.nombre, ' ', U.paterno, ' ', U.materno), 'N/A') as
Asignado from equipos_it E left join usuarios U on U.id = E.asignado where E.activo=1 and E.tipo
like '$tipo.'";
    if(isset($_POST['id'])){
        $id = $_this->input->post('id');
        $query .= " and E.id = $id";
    }
}
// Filtro por usuario asignado
else if(!empty($asignado)){
    $query = "SELECT E.*, ifnull(concat(U.nombre, ' ', U.paterno, ' ', U.materno), 'N/A') as
Asignado from equipos_it E left join usuarios U on U.id = E.asignado where E.activo=1 and
E.asignado = '$asignado.'";
    if(isset($_POST['id'])){
        $id = $_this->input->post('id');
        $query .= " and E.id = $id";
    }
}
// Por defecto, mostrar todos los equipos activos
else{
    $query .= " where E.activo=1";
}

// Ejecutar la consulta y retornar resultados en formato JSON
$res = $_this->Conexion->consultar($query, isset($_POST['id']));
echo json_encode($res);
}

function getEquiposTI(){
    $tipo = $_this->input->post('tipo');
    $texto = $_this->input->post('texto');

    // Consulta base con nombre del asignado y fecha de última revisión (Ultrev)

```

```
$query = "SELECT E.*, ifnull(concat(U.nombre, ' ', U.paterno, ' ', U.materno), 'N/A') as
Asignado, ifnull((SELECT max(fecha) from manttoEquipos where idEquipo=E.id), '') as Ultrev
FROM equipos_it E LEFT JOIN usuarios U ON U.id = E.asignado where E.activo = 1";
```

```
// Si se filtra por tipo, se construye la consulta según ese parámetro
if(!empty($tipo)){
    $query = "SELECT E.*, ifnull(concat(U.nombre, ' ', U.paterno, ' ', U.materno), 'N/A') as
Asignado, ifnull((SELECT max(fecha) from manttoEquipos where idEquipo=E.id), '') as Ultrev
from equipos_it E left join usuarios U on U.id = E.asignado where E.activo = 1 and E.tipo like
'".$tipo."'";
}
```

```
// Se filtra por equipos cuya última revisión fue hace al menos 2 meses
$query .= " HAVING `Ultrev` < last_day(now()) + interval 1 day - interval 2 month";
```

```
if(!empty($texto)){
    $campo = '$.No_Inventario_Interno';
    if ($tipo == 'Celular') {
        $campo = '$.Serie';
    }
    $query .= " and JSON_EXTRACT(campos, '".$campo "') = '".$texto."'";
}
```

```
$res = $this->Conexion->consultar($query);
if($res){
    echo json_encode($res);
} else {
    echo "";
}
}
```

```
function equipos(){
    $texto = $this->input->post('texto');
    $texto = trim($texto);
    $inactivo = $this->input->post('inactivo');
```

```
// Consulta base
$query = "SELECT * from equipos_it";
```

```

// Filtrado por estado de actividad
if($activo!="1"){
    $query.="where activo=1";
} elseif($activo=="1"){
    $query.="where activo=0";
}

$res = $this->Conexion->consultar($query);
if($res)
{
    echo json_encode($res);
}
else{
    echo "";
}
}

function registrarMantto(){

$equipo = $this->input->post('equipo');
$id = null;
$fototmp = $_FILES['foto'];
$fSize = intval(implode(".", $fototmp['size']));
$countfoto = count($fototmp['name']);

// Datos de mantenimiento recopilados del formulario
$datos = array(
    'idEquipo' => $this->input->post('equipo'),
    'usMantto' => $this->session->id,
    'case' => $this->input->post('case'),
    'vidTem' => $this->input->post('vidTem'),
    'usb' => $this->input->post('usb'),
    'manosL' => $this->input->post('ml'),
    'bateria' => $this->input->post('bateria'),
    'datos' => $this->input->post('datos'),
    'comentarios' => $this->input->post('comentarios'),
    'disco' => $this->input->post('disco'),

```

```

'cpu' => $this->input->post('cpu'),
'tecMouse' => $this->input->post('tecMouse'),
'abanicos' => $this->input->post('abanicos'),
);

$this->Modelo->registrarMantto($datos);
echo var_dump($datos);

// Obtener el ID del mantenimiento recién registrado
$query = "SELECT MAX(idME) as id FROM manttoEquipos WHERE idEquipo= '". $equipo. "'";
$res = $this->Conexion->consultar($query);
foreach($res as $elem){
    $id = $elem->id;
}

// Guardar fotos si se cargaron
if($fSize != 0){
    for ($i = 0; $i < $countfoto; $i++) {
        $fotofile = file_get_contents($fototmp['tmp_name'][$i]);
        $data = array(
            'idMantto' => $id,
            'fotos' => $fotofile,
        );
        $id_inserted = $this->Modelo->registrar($data);
    }
}

redirect(base_url('equipos/revisiones'));
}

function generador() {
    // Consulta registros del generador junto con el nombre del usuario asociado
    $datos['data'] = $this->Conexion->consultar("SELECT g.*, concat(u.nombre, ' ', u.paterno) as
name FROM generador g join usuarios u on u.id=g.usuario ORDER BY fecha DESC");
    $this->load->view('header');
    $this->load->view('equipos/ti/generador', $datos);
}

```

```

function registrar_generador() {
    // Se obtiene la imagen del formulario como archivo binario
    $foto = file_get_contents($_FILES['foto']['tmp_name']);

    // Datos recopilados del formulario de registro
    $data = array(
        'fecha_inicio' => $this->input->post('fecha_inicio'),
        'fecha_final' => $this->input->post('fecha_final'),
        'duracion' => $this->input->post('duracion'),
        'foto' => $foto,
        'porcentaje' => $this->input->post('porcentaje') . "% capacidad",
        'amperaje' => $this->input->post('amperaje'),
        'usuario' => $this->session->id,
        'consumo' => $this->input->post('consumo') . " kW",
        'motor' => $this->input->post('motor'),
        'arranques' => $this->input->post('arranques'),
    );

    $this->Conexion->insertar('generador', $data);
    redirect(base_url('equipos/generador'));
}

function uploadFoto() {
    // Actualiza la foto para un registro de generador específico
    $id = $this->input->post('id');
    $datos['foto'] = file_get_contents($_FILES['file']['tmp_name']);
    $this->Conexion->modificar('generador', $datos, null, array('id' => $id));
}
}

```

Facturas.php

```

<?php
require_once ('data/Merger/PDFMerger.php');
use PDFMerger\PDFMerger;

defined('BASEPATH') OR exit('No direct script access allowed');

```

```

class Facturas extends CI_Controller {

    function __construct() {
        parent::__construct();
        $this->load->library('correos_facturacion'); // Carga librería para envío de correos de
facturación
        $this->load->model('MLConexion_model', 'MLConexion'); // Modelo para conexión con
base de datos relacionada
    }

    function index(){
        $this->load->view('header');
        $this->load->view('facturas/catalogo_facturas'); // Vista principal del catálogo de facturas
    }

    function solicitudes(){
        $this->load->view('header');
        $this->load->view('facturas/catalogo_solicitudes'); // Vista del catálogo de solicitudes de
factura
    }

    function solicitud_factura(){
        $data["id"] = 0; // Para nueva solicitud, id = 0
        $this->load->view('header');
        $this->load->view('facturas/solicitud_facturas', $data); // Vista para crear solicitud
    }

    function editar_solicitud(){
        if (isset($_POST["id"])) {
            $id = $this->input->post('id'); // Recupera ID desde POST
        } else if ($id == 0) {
            redirect(base_url('inicio')); // Redirige si no hay ID válido
        }

        $data["id"] = $id;
        $data["editar"] = true; // Modo edición
        $this->load->view('header');
        $this->load->view('facturas/solicitud_facturas', $data); // Vista para editar solicitud
    }
}

```

```

}

function ver_solicitud($id = 0){
    if (isset($_POST["id"])) {
        $id = $this->input->post('id'); // Puede llegar también por POST
    } else if ($id == 0) {
        redirect(base_url('inicio')); // Redirección si no hay ID
    }

    $data["id"] = $id;
    $this->load->view('header');
    $this->load->view('facturas/solicitud_facturas', $data); // Vista para ver detalles de la
solicitud
}

function documentos_globales() {
    // Carga la vista de documentos globales de facturación
    $this->load->view('header');
    $this->load->view('facturas/documentos_globales');
}

function logistica() {
    // Carga la vista de logística dentro del módulo de facturación
    $this->load->view('header');
    $this->load->view('facturas/logistica');
}

function programacion_recorrido() {
    // Carga la vista de programación de recorrido de facturación
    $this->load->view('header');
    $this->load->view('facturas/programacion_recorrido');
}

function ajax_setSolicitud() {
    $solicitud = json_decode($this->input->post('solicitud'));
    $other = json_decode($this->input->post('other'));
    $rs_items = json_decode($this->input->post('rs_items'));

```

```

// Archivos adjuntos (orden de compra y remisión)
if (isset($_FILES['f_O'])) {
    $solicitud->f_orden_compra = file_get_contents($_FILES['f_O']['tmp_name']);
}
if (isset($_FILES['f_R'])) {
    $solicitud->f_remision = file_get_contents($_FILES['f_R']['tmp_name']);
}

$funciones = array('fecha' => 'CURRENT_TIMESTAMP()');
$res = false;

// Insertar o actualizar solicitud
if ($solicitud->id == 0) {
    $solicitud->usuario = $this->session->id;
    $res = $this->Conexion->insertar('solicitudes_facturas', $solicitud, $funciones);
    $solicitud->id = $res;
} else {
    $res = $this->Conexion->modificar('solicitudes_facturas', $solicitud, null, array('id' =>
    $solicitud->id)) >= 0;
}

// Manejo de items relacionados
$this->load->model('MLConexion_model', 'MLConexion');
foreach ($rs_items as $item) {
    $borrar = $item->BORRAR;
    unset($item->BORRAR);

    if ($item->id == 0) {
        $rsitems = $this->MLConexion->consultar(
            "SELECT rs.folio_id, rs.item_id, rs.Fec_CalibracionMT, s.DescripcionDeServicio,
rs.HojadeDatos_Id
            FROM rsitems rs
            JOIN catalogo_servicios s ON s.servicio_id = item_servicio_id
            WHERE rs.item_id = " . $item->item_id,
            true
        );
    }

    $item->servicio = $rsitems->DescripcionDeServicio;

```



```

$item->HojadeDatos_Id = $rsitems->HojadeDatos_Id;
$item->Fec_CalibracionMT = $rsitems->Fec_CalibracionMT;
$item->id_factura = $solicitud->id;

$this->Conexion->insertar('rsitems_facturas', $item);
$this->MLConexion->modificar('rsitems', array('Solicitud_ID' => $item->id_factura), null,
array('item_id' => $item->item_id));
} else {
    if ($borrar) {
        $this->Conexion->eliminar('rsitems_facturas', array('id' => $item->id));
        $this->MLConexion->modificar('rsitems', array('Solicitud_ID' => 0), null, array('item_id'
=> $item->item_id));
    } else {
        $this->Conexion->modificar('rsitems_facturas', $item, null, array('id' => $item->id));
    }
}
}

// Preparar datos para enviar correo de notificación
$solicitud->User = $other->User;
$solicitud->Client = $other->Client;
$solicitud->Contact = $other->Contact;

$correos = [];
$correos_a = $this->Conexion->consultar("SELECT U.correo FROM privilegios P INNER JOIN
usuarios U ON P.usuario = U.id WHERE P.responder_facturas = 1");
foreach ($correos_a as $value) {
    array_push($correos, $value->correo);
}
$solicitud->correos = array_merge(array($this->session->correo), $correos);

// Enviar correo
$this->correos_facturacion->solicitud($solicitud);

if ($res) {
    echo "1";
}
}

```

```

function ajax_getSolicitudes(){
    $id = $this->input->post('id');
    $aceptadas = $this->input->post('aceptadas');
    $cliente = $this->input->post('cliente');
    $ejecutivo = $this->input->post('ejecutivo');
    $texto = $this->input->post('texto');
    $parametro = $this->input->post('parametro');

    // Consulta base con joins a empresas y usuarios
    $query = "SELECT F.id, F.monto_factura, F.fecha, F.usuario, F.ejecutivo, F.cliente, F.contacto,
F.reporte_servicio, F.orden_compra, F.forma_pago, F.pagada, F.urgente, F.conceptos, F.notas,
F.estatus, F.documentos_requeridos, F.serie, F.folio, F.codigo_impresion, F.bitacora_estatus,
E.nombre as Cliente, concat(U.nombre, ' ', U.paterno) as User from solicitudes_facturas F inner
join empresas E on E.id = F.cliente inner join usuarios U on U.id = F.ejecutivo where 1 = 1";

    // Filtro por ID específico
    if($id)
    {
        $query .= " and F.id = '$id'";
    }
    else
    {
        // Búsqueda por texto en campo específico
        if($texto)
        {
            $query .= " and F.$parametro = '$texto'";
        }

        // Excluir solicitudes aceptadas o canceladas si $aceptadas es 0
        if($aceptadas == 0)
        {
            $query .= " and (F.estatus != 'ACEPTADO' && F.estatus != 'CANCELADO')";
        }

        // Filtro por cliente
        if($cliente && $cliente != 0)
        {

```

```

        $query .= " and F.cliente = '$cliente'";
    }

    // Filtro por ejecutivo
    if($sejecutivo && $sejecutivo != 0)
    {
        $query .= " and F.ejecutivo = '$sejecutivo'";
    }
}

// Filtro por estatus (si se manda por POST)
if(isset($_POST['estatus']))
{
    $estatus = $this->input->post('estatus');
    $query .= " and F.estatus = '$estatus'";
}

// Filtro por logistica (si se manda por POST)
if(isset($_POST['logistica']))
{
    $logistica = $this->input->post('logistica');
    $query .= " and F.logistica = '$logistica'";
}

$query .= " order by F.fecha desc";

// Ejecutar y devolver resultados
$res = $this->Conexion->consultar($query, $id);
if($res)
{
    echo json_encode($res);
}
}

function ajax_editSolicitud(){

    $solicitud = json_decode($this->input->post('solicitud'));
    $other = json_decode($this->input->post('other'));

```

```

// Procesar archivo de acuse (PDF)
if(isset($_FILES['f_A']))
{
    $solicitud->f_acuse = file_get_contents($_FILES['f_A']['tmp_name']);
}

// Procesar archivo de factura (PDF)
if(isset($_FILES['f_F']))
{
    $solicitud->f_factura = file_get_contents($_FILES['f_F']['tmp_name']);
    $solicitud->name_factura = $this->input->post('f_F_name');
}

// Procesar archivo XML
if(isset($_FILES['f_X']))
{
    $solicitud->f_xml = file_get_contents($_FILES['f_X']['tmp_name']);
    $solicitud->name_xml = $this->input->post('f_X_name');
}

$comentario = $this->input->post('comentario');

// Actualizar solicitud en la base de datos
$res = $this->Conexion->modificar('solicitudes_facturas', $solicitud, null, array('id' =>
$solicitud->id));

// Si la solicitud fue aceptada, actualiza la factura en los items relacionados
if($solicitud->estatus_factura == "ACEPTADO")
{
    $this->load->model('MLConexion_model', 'MLConexion');
    $this->MLConexion->comando("UPDATE rsitems set Factura = ifnull(Factura, $solicitud-
>folio) where Solicitud_ID = $solicitud->id;");
}

if($res > 0)
{
    // Si se agregó comentario, registrar en tabla de comentarios

```

```

        if(isset($_POST['comentario']) && !empty($comentario))
        {
            $this->Conexion->insertar('solicitudes_facturas_comentarios', array('solicitud' =>
            $solicitud->id, 'usuario' => $this->session->id, 'comentario' => $comentario), array('fecha' =>
            'CURRENT_TIMESTAMP()'));
            $solicitud->comentario = $comentario;
        }

        // Obtener correos de los usuarios con privilegio para responder facturas
        $correos = [];
        $correos_a = $this->Conexion->consultar("SELECT U.correo from privilegios P inner join
        usuarios U on P.usuario = U.id where P.responder_facturas = 1");
        foreach ($correos_a as $key => $value) {
            array_push($correos, $value->correo);
        }

        // Preparar información adicional para el correo
        $solicitud->correos = array_merge(array($this->session->correo), $correos);
        $solicitud->User = $other->User;
        $solicitud->Client = $other->Client;
        $solicitud->Contact = $other->Contact;

        $mail['id'] = $solicitud->id;
        $mail['estatus_factura'] = $solicitud->estatus_factura;
        $mail['comentario'] = $solicitud->comentario;
        $mail['User'] = $other->User;
        $mail['Client'] = $other->Client;
        $mail['Contact'] = $other->Contact;
        $mail['correos'] = array_merge(array($this->session->correo), $correos);

        // Enviar notificación por correo
        $this->correos_facturacion->editar_solicitud($mail);

        echo "1";
    }
    else
    {
        echo "";
    }

```

```

    }
}

function ajax_getReporteServicios() {
    $this->load->model('MLConexion_model', 'MLConexion');

    $texto = $this->input->post('texto'); // Texto a buscar
    $rs = $this->input->post('rs'); // Folio del reporte de servicio

    // Consulta de items de un reporte de servicio con descripción extendida
    $query = "SELECT item_id, folio_id, descripcion, concat(descripcion, if(isnull(Fabricante), '',
concat(' ', Fabricante)), if(isnull(Modelo), '', concat(' ', Modelo)), if(isnull(Serie), '', concat(' Serie:
', Serie)), if(isnull(Equipo_ID), '', concat(' ID: ', Equipo_ID))) as CadenaDescripcion, Solicitud_ID,
Monto from rsitems where 1=1 ";
    $query .= " and folio_id = '$rs'";

    if($texto) {
        $query .= " having CadenaDescripcion like '%$texto%'";
    }
    $res = $this->MLConexion->Consultar($query);
    if($res) {
        echo json_encode($res);
    }
}

function ajax_getRSItems() {
    $id_factura = $this->input->post('id_factura'); // ID de la solicitud de factura
    $res = $this->Conexion->Consultar("SELECT * from rsitems_facturas where id_factura =
$id_factura");
    if($res) {
        echo json_encode($res);
    }
}

function ajax_setComentarios() {
    $comentario = json_decode($this->input->post('comentario')); // Decodificar comentario
    recibido por POST
    $comentario->usuario = $this->session->id; // Agregar ID del usuario actual

```

```

    $funciones = array('fecha' => 'CURRENT_TIMESTAMP()'); // Establecer fecha actual
    automáticamente

    // Insertar el comentario en la base de datos
    $res = $this->Conexion->insertar('solicitudes_facturas_comentarios', $comentario,
    $funciones);
    if($res > 0) {
        echo "1";
    } else {
        echo "";
    }
}

function ajax_getComentarios() {
    $id = $this->input->post('id'); // ID de la solicitud de factura

    // Consulta de comentarios asociados a una solicitud
    $query = "SELECT C.*, concat(U.nombre, ' ', U.paterno) as User from
    solicitudes_facturas_comentarios C inner join usuarios U on U.id = C.usuario where 1 = 1";

    if($id) {
        $query .= " and C.solicitud = '$id'";
    }

    $res = $this->Conexion->consultar($query);
    if($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

function ajax_getRequisitores() {
    $id = $this->input->post('id'); // ID opcional para filtrar un requisito específico

    // Consulta a usuarios activos con privilegio para solicitar facturas

```

```
$query = "SELECT U.id, concat(U.nombre, ' ', U.paterno) as Nombre, P.puesto as Puesto from
usuarios U inner join puestos P on U.puesto = P.id inner join privilegios PR on PR.usuario = U.id
where U.activo = 1 and PR.solicitar_facturas = 1";
```

```
if($id) {
    $query .= " and U.id = '$id'";
}

$res = $this->Conexion->consultar($query, $id);
if($res) {
    echo json_encode($res);
} else {
    echo "";
}
}
```

```
function ajax_getVFPData() {
    $modelo = $this->input->post('modelo'); // Obtener modelo por POST

    // Ejecutar archivo externo de Visual FoxPro Reader con el modelo como argumento
    $res = shell_exec("C:/xampp/htdocs/MASMetrologia/vfp_reader/vfp_reader.exe
\"$modelo\");

    echo $res;
}
```

```
function archivo_impresion() {
    // Inicializa el objeto para combinar PDFs
    $pdf = new PDFMerger;

    // Limpieza de búfer y configuración de errores
    ob_start();
    error_reporting(E_ALL & ~E_NOTICE);
    ini_set('display_errors', 0);
    ini_set('log_errors', 1);
    ob_end_clean();
```

```
$id = $this->input->post('id'); // ID de la solicitud
```



```

$codigo = $this->input->post('codigo'); // Códigos de documentos a incluir

// Inicializa nuevamente el objeto PDFMerger por seguridad
$pdf = new PDFMerger();

// Consulta los datos de la solicitud
$q = "SELECT SF.* from solicitudes_facturas SF where SF.id = $id";
$res = $this->Conexion->consultar($q, TRUE);

// Recorre el código de documentos y agrega los archivos PDF correspondientes
for ($i = 0; $i < strlen($codigo); $i++) {
    switch (strtoupper($codigo[$i])) {
        case 'F': $campo = 'f_factura'; break;
        case 'R': $campo = 'f_remision'; break;
        case 'O': $campo = 'f_orden_compra'; break;
        case 'A': $campo = 'f_acuse'; break;
        case 'P': $campo = 'OPINION'; break;
        case 'S': $campo = 'EMISI ON'; break;
        default: $campo = null; break;
    }

    if ($campo != null) {
        if (substr($campo, 0, 2) == "f_") {
            // Documentos adjuntos en la base de datos (binarios)
            $file = $res->$campo;
            $fichero = sys_get_temp_dir() . '/' . $campo . '.pdf';
            file_put_contents($fichero, $file);
            $pdf->addPDF($fichero, 'all');
        } else {
            // Documentos globales predefinidos
            $fichero = "data/empresas/documentos_globales/" . $campo . "_000001.pdf";
            $pdf->addPDF($fichero, 'all');
        }
    }
}

ob_end_clean();
$pdf->merge('browser'); // Genera y muestra el archivo combinado en el navegador

```

```

}

function ajax_getClientes() {
    // Consulta clientes activos marcados como tales
    $query = "SELECT C.id, C.nombre, C.razon_social, C.foto, C.opinion_positiva, C.emision_sua
from empresas C where C.cliente = 1";

    $res = $this->Conexion->consultar($query);
    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

function ajax_getClientesSolicitudes() {
    $texto = $this->input->post('texto'); // Texto para filtrar por nombre de empresa

    // Consulta empresas con el número de solicitudes que tienen asociadas
    $query = "SELECT E.id, E.nombre, count(S.id) as NumSol from solicitudes_facturas S inner join
empresas E on E.id = S.cliente";

    if ($texto) {
        $query .= " where E.nombre like '%$texto%'";
    }

    $query .= " group by E.id;";

    $res = $this->Conexion->consultar($query);
    if ($res) {
        echo json_encode($res);
    }
}

function ajax_getEjecutivosSolicitudes() {
    $texto = $this->input->post('texto'); // Texto para filtrar por nombre del ejecutivo

    // Consulta ejecutivos con el número de solicitudes que tienen asociadas

```

```
$query = "SELECT U.id, concat(U.nombre, ' ', U.paterno) as Ejecutivo, count(S.id) as NumSol  
from solicitudes_facturas S inner join usuarios U on U.id = S.ejecutivo";
```

```
if ($texto) {  
    $query .= " where concat(U.nombre, ' ', U.paterno) like '%$texto%'";  
}
```

```
$query .= " group by U.id;";
```

```
$res = $this->Conexion->consultar($query);  
if ($res) {  
    echo json_encode($res);  
}  
}
```

```
function ajax_getDocumentosGlobales() {  
    // Consulta los documentos globales (opinión positiva y emisión SUA)  
    $query = "SELECT id, opinion_positiva, emision_sua from documentos_globales where id = 1";  
  
    $res = $this->Conexion->consultar($query);  
    if ($res) {  
        echo json_encode($res);  
    } else {  
        echo "";  
    }  
}
```

```
function ajax_filesExists($id) {  
    $this->load->helper('file'); // Carga helper para manejo de archivos  
  
    // Asegura que el ID tenga longitud de 6 caracteres, rellenando con ceros a la izquierda  
    $id = str_pad($id, 6, "0", STR_PAD_LEFT);  
  
    // Verifica si existen los archivos PDF requeridos (ACUSE y EMISION)  
    $acuse = read_file(base_url("data/empresas/documentos_facturacion/ACUSE_" . $id . ".pdf"))  
? "1" : "0";  
    $emision = read_file(base_url("data/empresas/documentos_facturacion/EMISION_" . $id .  
".pdf")) ? "1" : "0";
```

```

// Retorna el resultado como arreglo JSON (1 si existe, 0 si no)
echo json_encode(array($acuse, $emision));
}

function ajax_readXML() {
// Crea un nuevo objeto DOMDocument para cargar y procesar el archivo XML
$dom = new DomDocument;
$dom->preserveWhiteSpace = FALSE; // Elimina espacios en blanco al procesar el XML
$dom->loadXML(file_get_contents($_FILES['f_X']['tmp_name'])); // Carga el contenido del
archivo XML

$comp = $dom->getElementsByTagName('Comprobante'); // Obtiene el nodo
<Comprobante>
$ext = 1; // Bandera para determinar si el folio ya existe
$data = array(); // Arreglo para almacenar los atributos encontrados

// Recorre los atributos del nodo <Comprobante>
foreach ($comp[0]->attributes as $elem) {
// Si el atributo es Serie, Folio o SubTotal, se agrega al arreglo $data
if($elem->name == "Serie" | $elem->name == "Folio" | $elem->name == "SubTotal") {
    $e = array($elem->name => $elem->value);
    array_push($data, $e);
}

// Si el atributo es Folio, se verifica si ya existe en la base de datos
if($elem->name == "Folio") {
    $folio = $elem->value;
    $res = $this->Conexion->consultar("SELECT count(*) as existe FROM solicitudes_facturas
where folio = '$folio'", TRUE);
    $ext = $res->existe;
}
}

// Si el folio no existe, se devuelve el contenido extraído del XML
if($ext == 0) {
    echo json_encode($data);
} else {

```

```

        echo "0"; // Si el folio ya existe, se devuelve "0"
    }
}

function ajax_setDocumentoFacturacion() {
    $file = $this->input->post('file');          // Nombre del archivo enviado
    $documento = $this->input->post('documento'); // Tipo de documento (EMISION u
OPINION)
    $id = $this->input->post('empresa');          // ID de la empresa
    $id = str_pad($id, 6, "0", STR_PAD_LEFT);    // Rellena el ID con ceros a la izquierda (mínimo
6 caracteres)

    // Verifica que se haya enviado un archivo válido
    if($file != "undefined") {
        // Configuración para la carga del archivo
        $config['upload_path'] = 'data/empresas/documentos_facturacion/';
        $config['allowed_types'] = 'pdf';
        $config['overwrite'] = TRUE;
        $config['file_name'] = $documento . '_' . $id; // Nombre del archivo con prefijo del tipo de
documento e ID

        $this->load->library('upload', $config);

        // Si la carga del archivo fue exitosa
        if ($this->upload->do_upload('file')) {
            $where['id'] = $id;

            // Determina el campo correspondiente en la tabla según el tipo de documento
            switch($documento) {
                case "EMISION":
                    $campo = "emision_sua";
                    break;
                case "OPINION":
                    $campo = "opinion_positiva";
                    break;
            }

            // Guarda el nombre del archivo en el campo correspondiente de la base de datos

```

```

        $data[$campo] = $this->upload->data('file_name');
        $this->Conexion->modificar('empresas', $data, null, $where);

        echo "1";
    }
}
}

function ajax_deleteDocumentoFacturacion(){
    $documento = $this->input->post('documento');           // Tipo de documento:
    EMISION u OPINION
    $id = $this->input->post('empresa');                     // ID de la empresa
    $id = str_pad($id, 6, "0", STR_PAD_LEFT);               // Rellena el ID con ceros a la
    izquierda hasta tener 6 caracteres

    // Elimina el archivo físico del servidor
    unlink('data/empresas/documentos_facturacion/' . $documento . '_' . $id . '.pdf');

    $where['id'] = $id;

    // Determina el campo a limpiar en la base de datos según el tipo de documento
    switch($documento)
    {
        case "EMISION":
            $campo = "emision_sua";
            break;

        case "OPINION":
            $campo = "opinion_positiva";
            break;
    }

    // Limpia el campo correspondiente en la base de datos
    $data[$campo] = "";
    $this->Conexion->modificar('empresas', $data, null, $where);
}

function ajax_setDocumentoGlobal(){

```

```

$file = $this->input->post('file'); // Archivo enviado desde el formulario
$documento = $this->input->post('documento'); // Tipo de documento:
EMISION u OPINION
$id = $this->input->post('empresa'); // ID de la empresa
$id = str_pad($id, 6, "0", STR_PAD_LEFT); // Rellena con ceros a la izquierda
hasta 6 dígitos

if($file != "undefined") // Verifica que haya un archivo definido
{
    $config['upload_path'] = 'data/empresas/documentos_globales/'; // Ruta de
almacenamiento
    $config['allowed_types'] = 'pdf'; // Solo permite archivos PDF
    $config['overwrite'] = TRUE; // Sobrescribe si ya existe
    $config['file_name'] = $documento . '_' . $id; // Nombre del archivo final
    $this->load->library('upload', $config); // Carga la librería de subida

    if ($this->upload->do_upload('file')) // Si se sube el archivo correctamente
    {
        $where['id'] = $id;

        // Determina qué campo actualizar en la tabla según el tipo de documento
        switch($documento)
        {
            case "EMISION":
                $campo = "emision_sua";
                break;

            case "OPINION":
                $campo = "opinion_positiva";
                break;
        }

        // Actualiza la base de datos con el nombre del archivo cargado
        $data[$campo] = $this->upload->data('file_name');
        $this->Conexion->modificar('documentos_globales', $data, null, $where);
        echo "1";
    }
}

```

```

}

function ajax_deleteDocumentoGlobal(){
    $documento = $this->input->post('documento');           // Tipo de documento:
    EMISION u OPINION
    $id = $this->input->post('empresa');                     // ID de la empresa
    $id = str_pad($id, 6, "0", STR_PAD_LEFT);               // Rellena con ceros a la
    izquierda hasta 6 dígitos

    // Elimina el archivo PDF correspondiente en la carpeta de documentos globales
    unlink('data/empresas/documentos_globales/' . $documento . '_' . $id . '.pdf');
    $where['id'] = $id;

    // Determina qué campo de la base de datos se debe limpiar según el documento
    switch($documento)
    {
        case "EMISION":
            $campo = "emision_sua";
            break;

        case "OPINION":
            $campo = "opinion_positiva";
            break;
    }
    // Limpia el campo del documento en la base de datos
    $data[$campo] = "";
    $this->Conexion->modificar('documentos_globales', $data, null, $where);
}

function ajax_enviarCorreo(){
    $id = $this->input->post('id');                          // ID de la solicitud
    $body = $this->input->post('body');                      // Cuerpo del correo
    $subject = $this->input->post('subject');                // Asunto del correo
    $para = $this->input->post('para');                      // Destinatario(s)
    $cc = $this->input->post('cc');                          // Copia(s)

    $campos = json_decode($this->input->post('campos'));     // Campos a adjuntar
    (archivos)

```



```

    $archivos = [];
    $res = $this->Conexion->consultar("SELECT SF.* from solicitudes_facturas SF where SF.id =
$id", TRUE); // Consulta la solicitud

    foreach ($campos as $value) {
        if(substr($value, 0, 2) == "f_") // Archivos directamente en la tabla
(PDF o XML)
        {
            $file = $res->$value;
            $fichero = sys_get_temp_dir(). '/' . $value . ($value == "f_xml" ? '.xml' : '.pdf');
            file_put_contents($fichero, $file);
        }
        else // Archivos globales por nombre fijo
        {
            switch ($value) {
                case 'opinion_positiva':
                    $value = 'OPINION';
                    break;

                case 'emision_sua':
                    $value = 'EMISION';
                    break;
            }
            $fichero = "data/empresas/documentos_globales/" . $value . "_000001.pdf";
        }

        array_push($archivos, $fichero); // Agrega ruta del archivo al array de
adjuntos
    }

    // Arma el paquete de datos para enviar por correo
    $datos['id'] = $id;
    $datos['para'] = $para;
    $datos['cc'] = $cc;
    $datos['subject'] = $subject;
    $datos['body'] = $body;
    $datos['campos'] = $campos;

```

```

$datos['archivos'] = $archivos;
$this->correos_facturacion->enviarCorreo($datos);
}

```

//////////////////////////////// FACTURAS //////////////////////////////////

```

function ajax_getFacturas(){
    $id = $this->input->post('id');

    // Consulta de facturas con detalles de cliente, usuario y número de recorridos/envíos
    $query = "SELECT F.id, F.fecha, F.usuario, F.cliente, F.contacto, F.reporte_servicio,
F.orden_compra, F.forma_pago, F.pagada, F.conceptos, F.notas, F.estatus_factura,
F.documentos_requeridos, F.serie, F.folio, F.codigo_impresion, (SELECT count(id) from
recorrido_conceptos where id_concepto = F.id and tipo = 'FACTURA') as Recorridos, (SELECT
count(id) from envios_factura where factura = F.id) as Envios, E.nombre as Cliente,
concat(U.nombre, ' ', U.paterno) as User, U.correo, ifnull(EC.correo, 'N/A') as CorreoContacto
from solicitudes_facturas F inner join empresas E on E.id = F.cliente inner join usuarios U on U.id
= F.usuario left join empresas_contactos EC on EC.id = F.contacto";

    $query .= " where F.folio > 0 and F.estatus = 'ACEPTADO'"; // Solo facturas aceptadas con
folio

    if($id)
    {
        $query .= " and F.id = '$id'";
    }
    if(isset($_POST['estatus']))
    {
        $estatus = $this->input->post('estatus');
        $query .= " and F.estatus = '$estatus'";
    }
    $res = $this->Conexion->consultar($query, $id);
    if($res)
    {
        echo json_encode($res);
    }
}

```

//////////////////////////////// LOGISTICA //////////////////////////////////

```

function ajax_getMensajeros(){
    // Obtiene usuarios activos con privilegio de mensajero
    $query = "SELECT U.id, U.nombre, U.paterno, U.materno, U.no_empleado, U.puesto,
U.correo, U.ultima_sesion, U.departamento, U.activo, U.jefe_directo, U.autorizador_compras,
U.autorizador_compras_venta, concat(U.nombre, ' ', U.paterno) as User, concat(U.nombre, ' ',
U.paterno, ' ', U.materno) as CompleteName from usuarios U inner join privilegios P on
P.usuario = U.id where U.activo = '1'";
    $query .= " and P.mensajero = '1'";
    $query .= " order by User";

    $res = $this->Conexion->consultar($query);
    if($res)
    {
        echo json_encode($res);
    }
}

```

```

function ajax_setRecorrido(){
    // Obtener datos desde POST
    $mensajero = $this->input->post('mensajero');
    $fecha = $this->input->post('fecha');
    $recorrido = json_decode($this->input->post('recorrido'));

    // Insertar encabezado del recorrido
    $data['mensajero'] = $mensajero;
    $data['fecha_recorrido'] = $fecha;
    $recorrido_id = $this->Conexion->insertar('recorridos', $data);

    // Insertar detalles del recorrido y actualizar estatus de factura
    foreach ($recorrido as $value) {
        $data2['recorrido'] = $recorrido_id;
        $data2['factura'] = $value[1];
        $data2['accion'] = $value[0];
        $data2['estatus'] = "EN RECORRIDO";

        $this->Conexion->insertar('recorrido_conceptos', $data2);
    }
}

```

```

        $this->Conexion->modificar('solicitudes_facturas', array('estatus' => $data2['estatus']),
        null, array('id' => $data2['factura']));
    }
}

function ajax_getRecorridos(){
    $pendientes = $this->input->post('pendientes');
    $factura = $this->input->post('factura');

    // Consulta de recorridos con detalles adicionales como comentarios, pendientes y nombre
    del cliente/mensajero
    $query = "SELECT RF.*, R.mensajero, R.fecha_recorrido, (SELECT count(RC.id) from
    recorrido_comentarios RC where RC.recorrido_factura = RF.id) as Comentarios, (SELECT
    count(RF2.id) from recorrido_facturas RF2 where RF2.recorrido = RF.recorrido and RF2.estatus =
    'EN RECORRIDO') as Pendientes, (SELECT E.nombre from solicitudes_facturas F inner join
    empresas E on E.id = F.cliente where F.id = RF.factura) as Cliente, ifnull(concat(M.nombre, ' ',
    M.paterno), 'N/A') as Mensajero from recorrido_facturas RF inner join recorridos R on R.id =
    RF.recorrido left join usuarios M on M.id = R.mensajero where 1 = 1";

    // Filtrar por factura si se proporciona
    if(isset($_POST['factura']))
    {
        $query .= " and RF.factura = $factura";
    }

    // Mostrar solo recorridos con pendientes si se solicita
    if($pendientes == "1")
    {
        $query .= " having Pendientes > 0";
    }

    $query .= " order by R.fecha_recorrido, R.id, RF.id asc";

    $res = $this->Conexion->consultar($query);

    echo json_encode($res);
}

```

```

function ajax_updateRecorrido(){
    // Decodificar datos recibidos por POST
    $recorrido = json_decode($this->input->post('recorrido'));
    $recolecta = $this->input->post('recolecta');
    $comentario = $this->input->post('comentario');

    // Actualizar registro de recorrido_facturas
    $this->Conexion->modificar('recorrido_facturas', $recorrido, null, array('id' => $recorrido-
>id));

    // Determinar estatus a aplicar en la tabla solicitudes_facturas
    if(substr($recorrido->estatus, 0, 2) == "NO")
    {
        $sestat = "PENDIENTE " . $recorrido->accion;
        $this->Conexion->modificar('solicitudes_facturas', array('estatus' => $sestat), null,
array('id' => $recorrido->factura));
    }
    else
    {
        $sestat = $recolecta == "1" ? "PENDIENTE RECOLECTA" : "CERRADO";
        $this->Conexion->modificar('solicitudes_facturas', array('estatus' => $sestat), null,
array('id' => $recorrido->factura));
    }

    // Si se envió comentario, registrar con color y estatus
    if($comentario)
    {
        $color = substr($recorrido->estatus, 0, 2) == "NO" ? "red" : "green";
        $comentario = '<font color=' . $color . '><b>'. $recorrido->estatus . ':</b></font> ' .
$comentario;

        $data_com['recorrido_factura'] = $recorrido->id;
        $data_com['usuario'] = $this->session->id;
        $data_com['comentario'] = $comentario;
        $func_com['fecha'] = "CURRENT_TIMESTAMP()";
        $this->Conexion->insertar('recorrido_comentarios', $data_com, $func_com);
    }
}

```

```

function ajax_getComentariosRecorrido(){
    // Obtener el ID de la factura de recorrido desde POST
    $id = $this->input->post('id');
    // Construir consulta para obtener los comentarios junto con el nombre del usuario
    $query = "SELECT C.*, concat(U.nombre, ' ', U.paterno) as User from recorrido_comentarios
C inner join usuarios U on U.id = C.usuario where 1 = 1";

    // Filtro por ID
    if($id)
    {
        $query .= " and C.recorrido_factura = '$id'";
    }
    $query .= " order by C.fecha";

    $res = $this->Conexion->consultar($query);
    if($res)
    {
        echo json_encode($res);
    }
}

function ver_POD($id){
    // Consulta para obtener datos de la solicitud de factura
    $query = "SELECT sf.id, sf.folio,sf.serie, sf.reporte_servicio, sf.fecha, sf.orden_compra,
concat(u.nombre, ' ', u.paterno) as responsable, e.razon_social, concat(e.calle, ' ',e.numero, ' CP
',e.cp) as direccion,e.ciudad, e.estado, e.rfc,e.colonia, ec.nombre as contacto, ec.correo FROM
solicitudes_facturas sf join usuarios u on sf.ejecutivo = u.id JOIN empresas e on sf.cliente = e.id
JOIN empresas_contactos ec on sf.contacto = ec.id WHERE sf.id = $id";
    $factura = $this->Conexion->consultar($query, TRUE);

    // Consulta para obtener los items relacionados a la solicitud
    $query2 = "SELECT rs.descripcion, rs.Equipo_ID, rs.Fec_CalibracionMT,
s.DescripcionDeServicio, concat(rs.descripcion,if(isnull(rs.Fabricante), '', concat(' ',
rs.Fabricante)), if(isnull(rs.Modelo), '', concat(' ', rs.Modelo))), if(isnull(rs.Serie), '', concat(' Serie:
', rs.Serie)) ) as CadenaDescripcion from rsitems rs JOIN catalogo_servicios s on s.servicio_id =
rs.item_servicio_id WHERE rs.Solicitud_ID = ".$id." and rs.Factura = ".$factura->folio."";
    $rs = $this->MLConexion->consultar($query2);

```

```

// Contar total de equipos
$total=0;
foreach($rs as $row){
    $total++;
}
ini_set('display_errors', 0);
$this->load->library('pdfview');

$pdf = new TCPDF(PDF_PAGE_ORIENTATION, PDF_UNIT, PDF_PAGE_FORMAT, true, 'UTF-8',
false);

$f=$factura->serie . "-" . $factura->folio;

$pdf->SetCreator(PDF_CREATOR);
$pdf->SetAuthor('AleksOrtiz');
$pdf->SetTitle('Masmetrologia');
$pdf->SetSubject('Formato Cotización');

// Encabezado del PDF

$spc = "      ";
$head = "$spc      Prueba de Entrega      $spc Folio: $id / Factura: $f";
$txt = "      Proof of delivery      Responsable: " . $factura-
>responsable;
$txt .= "\n      Fec. de elaboración:: " . $factura-
>fecha;
$pdf->SetHeaderData(PDF_HEADER_LOGO_ORIGINAL, '40', $head, $txt);

// Configuración general de fuentes y márgenes
$pdf->setHeaderFont(Array(PDF_FONT_NAME_MAIN, "", 10));
$pdf->setFooterFont(Array(PDF_FONT_NAME_DATA, "", PDF_FONT_SIZE_DATA));
$pdf->SetDefaultMonospacedFont(PDF_FONT_MONOSPACED);
$pdf->SetMargins(8, PDF_MARGIN_TOP, 8);
$pdf->SetHeaderMargin(PDF_MARGIN_HEADER);
$pdf->SetFooterMargin(PDF_MARGIN_FOOTER);
$pdf->SetAutoPageBreak(TRUE, PDF_MARGIN_BOTTOM);

```

```

$pdf->setImageScale(PDF_IMAGE_SCALE_RATIO);

// Soporte para idioma inglés
if (@file_exists(dirname(__FILE__).'/lang/eng.php')) {
    require_once(dirname(__FILE__).'/lang/eng.php');
    $pdf->setLanguageArray($l);
}

$pdf->SetFont('helvetica', '', 8);

$pdf->AddPage();
$pdf->SetFillColor(255, 255, 255);
$pdf->SetFont('helvetica', '', 10);

// Tabla con los datos del cliente
$tbl = <<<EOD
<br>
<table border="0">
    <tr>
        <td>
            <b>Cliente/Customer:</b><br>
            $factura->razon_social<br>
            $factura->direccion<br>
            $factura->colonia<br>
            $factura->ciudad, $factura->estado<br>
            RFC: $factura->rfc<br>

        </td>
        <td>

EOD;

$tbl .= <<<EOD
<b>Orden de Compra: </b>
$factura->orden_compra<br>
<b>Contacto / Contact: </b><br>
$factura->contacto<br>
$factura->correo<br>
<br>

```



```

        Total de Equipo(s): $total<br>
    </td>
</tr>
</table>
EOD;

```

```

$pdf->writeHTML($tbl, false, false, false, false, "");
$w = array(8, 125, 12, 24, 24);

```

```

// Configuración de la tabla de servicios
$pdf->SetFont('helvetica', 'B', 9);
$pdf->SetTextColor(255);
$pdf->Ln();$pdf->Ln();
$w = array(20, 20, 125, 20);

```

```

$pdf->SetTextColor(0);
$pdf->SetFont('helvetica', '', 10);
    $pdf->Ln();
$tabla_items="";

```

```

// Construcción HTML de la tabla con los equipos
$tabla_items .= '<table style=" ">
    <thead>
        <tr>
            <th style="border-bottom: 1px solid #000; text-align: center; font-weight:
bold; width: 15%;">Servicio</th>
            <th style="border-bottom: 1px solid #000; text-align: center; font-weight:
bold; width: 15%;">Equipo ID</th>
            <th style="border-bottom: 1px solid #000; text-align: center; font-weight:
bold; width: 55%;">Descripcion</th>
            <th style="border-bottom: 1px solid #000; text-align: center; font-weight:
bold; width: 15%;">Realizado</th>
        </tr>
    </thead>
    <tbody>';
foreach($rs as $row){
    $date=date_create($row->Fec_CalibracionMT);
    $tabla_items .='

```

```

        <tr>
            <td style="border-bottom: 1px solid #000; text-align: center; width: 15%; tr:nth-
child:background: #F8F8F8;">'. $row->DescripcionDeServicio .'</td>
            <td style="border-bottom: 1px solid #000; text-align: center; width: 15%; tr:nth-
child:background: #F8F8F8;">'. $row->Equipo_ID.'</td>
            <td style="border-bottom: 1px solid #000; text-align:center ; width: 55%; tr:nth-
child:background: #F8F8F8;">'. $row->CadenaDescripcion.'</td>
            <td style="border-bottom: 1px solid #000; text-align: center;width: 15%; tr:nth-
child:background: #F8F8F8;">'.date_format($date,'d/M/Y').'</td>
        </tr>';
    }
    $tabla_items .= '</tbody>
</table>
<br>
<br>
<br>
<br>
<br>
<br>';

// Renderizar la tabla de items
$pdf->writeHTML($tabla_items, true, false, false, false, "");
$pdf->Ln();
$pdf->Ln();
$pdf->SetFont('helvetica', '', 10);
$pdf->MultiCell(150, 8,"Recibí el servicio a los equipos arriba listados", 0, 'L', 1, 0, "", true,
0, false, true, 0);
$pdf->SetFont('helvetica', 'B', 10);
$pdf->writeHTML("_____", true, false, false, false, "");
$pdf->MultiCell(150, 8,"Firma de recibido", 0, 'L', 1, 0, "", true, 0, false, true, 0);
$pdf->Ln();

// Nombre del archivo generado
$name = "POD ".$id. " - PO ".$factura->orden_compra.'.pdf';

// Salida del archivo PDF al navegador
$pdf->Ln();
$pdf->Output(sys_get_temp_dir().'/'.$name, 'I');

```

```
}
```

```
function ajax_enviarCorreoPOD(){
```

```
    // Se obtienen datos del formulario (POST)
```

```
    $id = $this->input->post('id');
```

```
    $body = $this->input->post('body');
```

```
    $subject = $this->input->post('subject');
```

```
    $para = $this->input->post('para');
```

```
    $cc = $this->input->post('cc');
```

```
    $certificados=[];
```

```
    $pdfname=null;
```

```
    $xmlname =null;
```

```
    $campos = json_decode($this->input->post('campos'));
```

```
    $archivos = [];
```

```
    $name_files = [];
```

```
    // Se consulta la solicitud de factura correspondiente al ID
```

```
    $res = $this->Conexion->consultar("SELECT SF.* from solicitudes_facturas SF where SF.id =  
$id", TRUE);
```

```
    $pdfname=$res->name_factura;
```

```
    $xmlname=$res->name_xml;
```

```
    // Se preparan los archivos adjuntos
```

```
    foreach ($campos as $value) {
```

```
        if(substr($value, 0, 2 ) == "f_")
```

```
        {
```

```
            $file = $res->$value;
```

```
            $fichero = sys_get_temp_dir(). '/' . $value . ($value == "f_xml" ? '.xml' : '.pdf');
```

```
            $name =$value == "f_xml" ? $res->name_xml : $res->name_factura;
```

```
            file_put_contents($fichero, $file);
```

```
        }
```

```
    else
```

```
    {
```

```
        switch ($value) {
```

```

        case 'opinion_positiva':
            $value = 'OPINION';
            break;

        case 'emision_sua':
            $value = 'EMISION';
            break;
    }
    $fichero = "data/empresas/documentos_globales/" . $value . "_000001.pdf";
}

array_push($archivos, $fichero);

}

$query = "SELECT sf.id, sf.folio,sf.serie, sf.reporte_servicio, sf.fecha, sf.orden_compra,
concat(u.nombre, ' ', u.paterno) as responsable, e.razon_social, concat(e.calle, ' ',e.numero, ' CP
',e.cp) as direccion,e.ciudad, e.estado, e.rfc,e.colonia, ec.nombre as contacto, ec.correo FROM
solicitudes_facturas sf join usuarios u on sf.ejecutivo = u.id JOIN empresas e on sf.cliente = e.id
JOIN empresas_contactos ec on sf.contacto = ec.id WHERE sf.id = $id";

$factura = $this->Conexion->consultar($query, TRUE);

// Consulta para obtener los equipos relacionados a la factura
$query2 = "SELECT rs.descripcion, rs.Equipo_ID, rs.documento_id, rs.Fec_CalibracionMT,
s.DescripcionDeServicio, concat(rs.descripcion,if(isnull(rs.Fabricante), '', concat(' ',
rs.Fabricante)), if(isnull(rs.Modelo), '', concat(' ', rs.Modelo)), if(isnull(rs.Serie), '', concat(' Serie:
', rs.Serie)) ) as CadenaDescripcion from rsitems rs JOIN catalogo_servicios s on s.servicio_id =
rs.item_servicio_id WHERE rs.Solicitud_ID = ".$id." and rs.Factura = ".$factura->folio."";
$rs = $this->MLConexion->consultar($query2);
$total=0;
foreach($rs as $row){
    $total++;
}

// Se recopilan IDs de documentos para certificados
foreach ($rs as $key) {

```

```

    array_push($certificados, $key->documento_id);
}

ini_set('display_errors', 0);
$this->load->library('pdfview');

// Generación del archivo PDF tipo POD
$pdf = new TCPDF(PDF_PAGE_ORIENTATION, PDF_UNIT, PDF_PAGE_FORMAT, true, 'UTF-8',
false);

$f=$factura->serie . "-" . $factura->folio;
$pdf->SetCreator(PDF_CREATOR);
$pdf->SetAuthor('AleksOrtiz');
$pdf->SetTitle('Masmetrologia');
$pdf->SetSubject('Formato Cotización');
$spc = "      ";
$head = "$spc      Prueba de Entrega      $spc Folio: $id / Factura: $f";
$txt = "      Proof of delivery      Responsable: " . $factura-
>responsable;
    $txt .= "\n      Fec. de elaboración:: " . $factura-
>fecha;

$pdf->SetHeaderData(PDF_HEADER_LOGO_ORIGINAL, '40', $head, $txt);

$pdf->setHeaderFont(Array(PDF_FONT_NAME_MAIN, "", 10));
$pdf->setFooterFont(Array(PDF_FONT_NAME_DATA, "", PDF_FONT_SIZE_DATA));
$pdf->SetDefaultMonospacedFont(PDF_FONT_MONOSPACED);
$pdf->SetMargins(8, PDF_MARGIN_TOP, 8);
$pdf->SetHeaderMargin(PDF_MARGIN_HEADER);
$pdf->SetFooterMargin(PDF_MARGIN_FOOTER);
$pdf->SetAutoPageBreak(TRUE, PDF_MARGIN_BOTTOM);
$pdf->setImageScale(PDF_IMAGE_SCALE_RATIO);

if (@file_exists(dirname(__FILE__).'/lang/eng.php')) {
    require_once(dirname(__FILE__).'/lang/eng.php');
    $pdf->setLanguageArray($l);
}

```

```

}

$pdf->SetFont('helvetica', "", 8);

$pdf->AddPage();
$pdf->SetFillColor(255, 255, 255);
$pdf->SetFont('helvetica', "", 10);

// Tabla con los datos del cliente
$tbl = <<<EOD
<br>
<table border="0">
  <tr>
    <td>
      <b>Cliente/Client:</b><br>
      $factura->razon_social<br>
      $factura->direccion<br>
      $factura->colonia<br>
      $factura->ciudad, $factura->estado<br>
      $factura->rfc<br>

    </td>
    <td>

EOD;

$tbl .= <<<EOD
<b>Orden de Compra: </b>
$factura->orden_compra<br>
<b>Contacto / Contact: </b><br>
$factura->contacto<br>
$factura->correo<br>
<br>
Total de Equipo(s): $total<br>
</td>
</tr>
</table>
EOD;

```

```

$pdf->writeHTML($tbl, false, false, false, false, "");
$w = array(8, 125, 12, 24, 24);

$pdf->SetFont('helvetica', 'B', 9);
$pdf->SetTextColor(255);
$pdf->Ln();$pdf->Ln();
$w = array(20, 20, 125, 20);

$pdf->SetTextColor(0);
$pdf->SetFont('helvetica', '', 10);
    $pdf->Ln();

// Tabla de servicios/equipos
$tabla_items="";
    $tabla_items .= '<table style=" ">
        <thead>
            <tr>
                <th style="border-bottom: 1px solid #000; text-align: center; font-weight:
bold; width: 15%;">Servicio</th>
                <th style="border-bottom: 1px solid #000; text-align: center; font-weight:
bold; width: 15%;">Equipo ID</th>
                <th style="border-bottom: 1px solid #000; text-align: center; font-weight:
bold; width: 55%;">Descripcion</th>
                <th style="border-bottom: 1px solid #000; text-align: center; font-weight:
bold; width: 15%;">Realizado</th>
            </tr>
        </thead>
        <tbody>';
    foreach($rs as $row){
        $date=date_create($row->Fec_CalibracionMT);
        $tabla_items .= '
            <tr>
                <td style="border-bottom: 1px solid #000; text-align: center; width: 15%; tr:nth-
child:background: #F8F8F8;">'.$row->DescripcionDeServicio .'</td>
                <td style="border-bottom: 1px solid #000; text-align: center; width: 15%; tr:nth-
child:background: #F8F8F8;">'.$row->Equipo_ID.'</td>
                <td style="border-bottom: 1px solid #000; text-align:center ; width: 55%; tr:nth-
child:background: #F8F8F8;">'.$row->CadenaDescripcion.'</td>

```

```

        <td style="border-bottom: 1px solid #000; text-align: center; width: 15%; text-align: center; background-color: #F8F8F8;">'.date_format($date,'d/M/Y').'</td>
    </tr>';
}
$tabla_items .= '</tbody>
</table>
<br>
<br>
<br>
<br>
<br>
<br>
<br>';
$pdf->writeHTML($tabla_items, true, false, false, false, "");
$pdf->Ln();
$pdf->Ln();
$pdf->SetFont('helvetica', "", 10);
$pdf->MultiCell(150, 8, "Recibí el servicio a los equipos arriba listados", 0, 'L', 1, 0, "", true,
0, false, true, 0);
$pdf->SetFont('helvetica', 'B', 10);
$pdf->writeHTML("_____", true, false, false, false, "");
$pdf->MultiCell(150, 8, "Firma de recibido", 0, 'L', 1, 0, "", true, 0, false, true, 0);
$pdf->Ln();
$name = "POD ".$id." - PO ".$factura->orden_compra.'.pdf';

$pdf->Ln();
$pdf->Output(sys_get_temp_dir().'/'.$name, 'F');

$datos['pod']=sys_get_temp_dir().'/'.$name;

$datos['id'] = $id;
$datos['para'] = $para;
$datos['cc'] = $cc;
$datos['subject'] = "POD ".$id." - PO ".$factura->orden_compra;
$datos['body'] = $body;
$datos['campos'] = $campos;
$datos['archivos'] = $archivos;
$datos['certificados'] = $certificados;
$datos['pdfname']=$pdfname;

```



```

        $datos['xmlname']=$xmlname;
        $datos['po']=$factura->orden_compra;
        $this->correos_facturacion->archivos_facturacion($datos);

    }

    public function validar_archivos(){
        $id=$this->input->post('ID');
        $factura = $this->Conexion->consultar("SELECT id, folio from solicitudes_facturas where
id= ".$id, true);

        $query = " select item_id, documento_id,Fec_CalibracionMT from rsitems where factura =
".$factura->folio." and Solicitud_ID";
        $rs = $this->MLConexion->consultar($query);

        if($rs)
        {
            echo json_encode($rs);
        }
        else{
            echo "";
        }
    }
}

```

Inicio.php

```

<?php

defined('BASEPATH') OR exit('No direct script access allowed');

class Inicio extends CI_Controller {

    public function index() {
        if(isset($this->session->id)) {
            $id = $this->session->id;

            // Carga de modelos necesarios

```

```

$this->load->model('tickets_model');
$this->load->model('agenda_model');
$this->load->model('tool_model');
$this->load->model('compras_model');
$this->load->model('privilegios_model');

// Se obtienen tickets pendientes del usuario
$datos['tickets'] = $this->tickets_model->getMis_tickets_pendientes($this->session->id);

// Se obtienen juntas agendadas
$datos['agenda'] = $this->agenda_model->getjuntas();

// Pedidos pendientes del módulo "tool"
$datos['tool'] = $this->tool_model->pedidosPendientes();

// Requisiciones QR personales del usuario
$datos['compras'] = $this->compras_model->misQRS();

// Notificaciones del sistema para el usuario
$datos['noti'] = $this->privilegios_model->getNotificaciones();

// Consulta de requisiciones rechazadas del usuario
$queryQR = "SELECT QR.id, QR.fecha, QR.subtipo, QR.cantidad, QR.descripcion,
QR.estatus from requisiciones_cotizacion QR where QR.usuario = $id";
$queryQR .= " and QR.estatus = 'RECHAZADO'";

// Consulta de PRs por recibir del usuario
$queryPR = "SELECT PR.id, PR.fecha, PR.usuario, PR.prioridad, PR.tipo, PR.subtipo,
PR.cantidad, PR.unidad, PR.clave_unidad, PR.descripcion, PR.atributos, PR.critico, PR.destino,
PR.lugar_entrega, PR.comentarios, PR.estatus, concat(U.nombre, ' ', U.paterno) as User";
$queryPR .= " from prs PR left join usuarios U on PR.usuario = U.id where 1 = 1 and
PR.usuario = $id and PR.estatus = 'POR RECIBIR' order by PR.fecha desc";

// Consulta de facturas con ciertos estatus (comentada/deshabilitada)
$queryFact = "SELECT F.id, F.fecha, F.usuario, F.cliente, F.contacto, F.reporte_servicio,
F.orden_compra, F.forma_pago, F.pagada, F.conceptos, F.notas, F.estatus, F.estatus_factura,
F.documentos_requeridos, F.serie, F.folio, F.codigo_impresion, (SELECT count(id) from
recorrido_facturas where factura = F.id) as Recorridos, E.nombre as Cliente, concat(U.nombre, '

```

```
, U.paterno) as User from solicitudes_facturas F inner join empresas E on E.id = F.cliente inner join usuarios U on U.id = F.usuario";
```

```
$queryFact .= " where (F.folio > 0 and F.estatus = 'ACEPTADO' and F.estatus_factura != 'RETORNADA AUTORIZADA') or (F.estatus = 'RECHAZADO')";
```

```
// Resultados de consultas QR y PR
```

```
$datos['qrs'] = $this->Conexion->consultar($queryQR, FALSE, FALSE);
```

```
$datos['prs'] = $this->Conexion->consultar($queryPR, FALSE, FALSE);
```

```
// Desactivado: resultados de facturas
```

```
$datos['facturas'] = "";
```

```
}
```

```
// Carga de vistas principales
```

```
$this->load->view('header');
```

```
$this->load->view('inicio', $datos);
```

```
}
```

```
function primera_sesion() {
```

```
// Muestra vista para capturar datos en primera sesión
```

```
$this->load->view('inicio/capturar_datos');
```

```
}
```

```
function confirmar_datos() {
```

```
// Modelo personalizado para actualizar datos de usuario
```

```
$this->load->model('inicio_model','Modelo');
```

```
$ACIERTOS = array();
```

```
$ERRORES = array();
```

```
// Datos recibidos del formulario
```

```
$data['correo'] = trim($this->input->post('correo'));
```

```
$data['password'] = sha1($this->input->post('password'));
```

```
$data['activo'] = "1";
```

```
// Validación y actualización de datos
```

```
if($this->Modelo->confirmarDatos($this->session->id, $data)) {
```

```
$this->session->correo = $data['correo'];
```

```
$this->session->password = $data['password'];
```

```

        $acierto = array(
            'titulo' => 'Datos Actualizados',
            'detalle' => 'Recuerda que puedes iniciar sesión con tu Numero de Empleado o Correo'
        );
        array_push($ACIERTOS, $acierto);
    } else {
        $error = array(
            'titulo' => 'ERROR',
            'detalle' => 'Error al actualizar Datos'
        );
        array_push($ERRORES, $error);
    }

    // Almacenar mensajes de éxito o error en sesión
    $this->session->aciertos = $ACIERTOS;
    $this->session->errores = $ERRORES;

    // Redirección al inicio
    redirect(base_url('inicio'));
}

}

```

Login.php

```

<?php

defined('BASEPATH') OR exit('No direct script access allowed');

class Login extends CI_Controller {

    function __construct() {
        parent::__construct();
        $this->load->model('usuarios_model');
        $this->load->library('correos_tickets');
    }

    function index() {

```

```

date_default_timezone_set('America/Chihuahua');
$data['url_actual'] = base_url('inicio');

// Si existe una URL previa en sesión, se conserva para redireccionar luego
if(isset($this->session->url_actual)) {
    $data['url_actual'] = $this->session->url_actual;
}

// Destruye la sesión existente
$this->session->sess_destroy();

// Carga vista de login
$this->load->view('login', $data);
}

function autenticar() {
    $url_actual = $this->input->post('url_actual');
    $usuario = $this->input->post('user');
    $pass = $this->input->post('pass');

    // Intenta autenticar al usuario con los datos proporcionados
    $res = $this->usuarios_model->autenticar($usuario, $pass);

    if ($res) {
        // Datos del usuario autenticado
        $row = $res->row();

        // Guardar datos importantes en la sesión
        $this->session->nombre = $row->User;
        $this->session->id = $row->id;
        $this->session->no_empleado = $row->no_empleado;
        $this->session->password = $row->password;
        $this->session->vencimiento_password = $row->vencimiento_password;
        $this->session->password_correo = $row->password_correo;
        $this->session->correo = $row->correo;
        $this->session->puesto = $row->puesto;
        $this->session->activo = $row->activo;
        $this->session->foto = $row->foto;
    }
}

```

```

$this->session->chats = '{ "chatbox0" : "CHAT"}';
$this->session->departamento = $row->departamento;

// Se asignan privilegios del usuario
$this->session->privilegios = $this->usuarios_model->getPrivilegios($this->session->id);

// Se actualiza última sesión
$this->usuarios_model->ultimaSesion($row->id);

// Se valida si hay tickets abiertos o en curso del usuario
$us = $this->Conexion->consultar("SELECT max(ultima_sesion) as us from usuarios",
true);
$fecha = date("y-m-d", strtotime($us->us));
$today = date("y-m-d", strtotime("today"));

$query = "SELECT 'IT' as tipo, T.id, T.usuario, concat(U.nombre,' ',U.paterno) as User,
U.correo, T.estatus from tickets_sistemas T inner join usuarios U on U.id = T.usuario where
T.estatus = 'ABIERTO' and usuario = $row->id"
. " union "
. "SELECT 'IT' as tipo, T.id, T.usuario, concat(U.nombre,' ',U.paterno) as User, U.correo,
T.estatus from tickets_sistemas T inner join usuarios U on U.id = T.usuario where T.estatus = 'EN
CURSO' and usuario = $row->id"
. " union "
. "SELECT 'AT', T.id, T.usuario, concat(U.nombre,' ',U.paterno) as User, U.correo,
T.estatus from tickets_autos T inner join usuarios U on U.id = T.usuario where T.estatus =
'ABIERTO' and usuario = $row->id"
. " union "
. "SELECT 'AT', T.id, T.usuario, concat(U.nombre,' ',U.paterno) as User, U.correo,
T.estatus from tickets_autos T inner join usuarios U on U.id = T.usuario where T.estatus = 'EN
CURSO' and usuario = $row->id"
. " union "
. "SELECT 'ED', T.id, T.usuario, concat(U.nombre,' ',U.paterno) as User, U.correo,
T.estatus from tickets_edificio T inner join usuarios U on U.id = T.usuario where T.estatus =
'ABIERTO' and usuario = $row->id"
. " union "
. "SELECT 'ED', T.id, T.usuario, concat(U.nombre,' ',U.paterno) as User, U.correo,
T.estatus from tickets_edificio T inner join usuarios U on U.id = T.usuario where T.estatus = 'EN
CURSO' and usuario = $row->id";

```

```

    $arr = array();
    $res = $this->Conexion->consultar($query);
    if($res) {
        foreach ($res as $key => $value) {
            if(!isset($arr[$value->usuario])) {
                $arr[$value->usuario]['nombre'] = $value->User;
                $arr[$value->usuario]['correo'] = $value->correo;
                $arr[$value->usuario]['tickets'] = array();
            }
            array_push($arr[$value->usuario]['tickets'], $value->tipo . str_pad($value->id, 6, "0",
STR_PAD_LEFT));
        }
    }

    // Posible envío de correo (actualmente desactivado)
    // $this->correos_tickets->ticketsPendientes($arr);

    // Redirige a la URL original
    redirect($url_actual);

} else {
    // Autenticación fallida, se guarda mensaje de error en sesión
    $ERRORES = array();
    $error = array('titulo' => 'ERROR', 'detalle' => 'Usuario y/o Contraseña incorrectas');
    array_push($ERRORES, $error);
    $this->session->errores = $ERRORES;

    // Se conserva URL actual para reintentar
    $this->session->url_actual = $url_actual;

    // Redirige de nuevo a login
    redirect(base_url('login'));
}
}

```

```

public function cerrar_sesion() {
    // Destruye la sesión y redirige a login

```

```

        $this->session->sess_destroy();
        redirect(base_url('login'));
    }

}

```

Logistica.php

```

<?php

defined('BASEPATH') OR exit('No direct script access allowed');

class Logistica extends CI_Controller {

    function __construct() {
        parent::__construct();

        // Carga la librería personalizada para envío de correos de logística
        $this->load->library('correos_logistica');

        // Carga el modelo de conexión a base de datos para logística
        $this->load->model('MLConexion_model', 'MLConexion');
    }

    function index() {
        // Vista principal de opciones de logística
        $this->load->view('header');
        $this->load->view('logistica/logistica_opciones');
    }

    function documentacion() {
        // Vista para el módulo de documentación logística
        $this->load->view('header');
        $this->load->view('logistica/documentacion');
    }

    function equipos() {
        // Vista del módulo de equipos logísticos
    }
}

```



```

$this->load->view('header');
$this->load->view('logistica/equipos');
}

function programacion_recorridos() {
    // Vista para programar recorridos logísticos
    $this->load->view('header');
    $this->load->view('logistica/programacion_recorridos');
}

function recorridos() {
    // Vista para ver y gestionar recorridos
    $this->load->view('header');
    $this->load->view('logistica/recorridos');
}

function dashboard() {
    // Vista del panel de control logístico
    $this->load->view('header');
    $this->load->view('logistica/dashboard');
}

function firmar() {
    // Muestra formulario o vista para firmar documentos/logística
    $data['data'] = $this->input->post();
    $this->load->view('logistica/firmar', $data);
}

function formato_requisitos() {
    // Decodifica lista de IDs de empresas enviada vía POST
    $empresas = json_decode($this->input->post('empresas'));
    $data = [];

    // Consulta datos de cada empresa y los agrega a un arreglo
    foreach ($empresas as $key => $value) {
        $res = $this->Conexion->consultar("SELECT * from empresas where id = $value", TRUE);

        $empresa = new stdClass;
    }
}

```

```

    $empresa->nombre = $res->nombre;
    $empresa->requisitos_logisticos = $res->requisitos_logisticos;
    $empresa->requisitos_documento = $res->requisitos_documento;
    array_push($data, $empresa);
}

// Se ocultan errores PHP
ini_set('display_errors', 0);

// Carga librería para generar PDF
$this->load->library('pdfview');
$pdf = new TCPDF(PDF_PAGE_ORIENTATION, PDF_UNIT, PDF_PAGE_FORMAT, true, 'UTF-8',
false);

// Configuración del documento PDF
$pdf->SetCreator(PDF_CREATOR);
$pdf->SetAuthor('AleksOrtiz');
$pdf->SetTitle('Masmetrologia');
$pdf->SetSubject('Formato Requisitos');
$pdf->SetHeaderData(PDF_HEADER_LOGO_ORIGINAL, '40', 'Requisitos de
Empresa');
$pdf->setHeaderFont(Array(PDF_FONT_NAME_MAIN, '', 10));
$pdf->setFooterFont(Array(PDF_FONT_NAME_DATA, '', PDF_FONT_SIZE_DATA));
$pdf->SetDefaultMonospacedFont(PDF_FONT_MONOSPACED);
$pdf->SetMargins(PDF_MARGIN_LEFT, PDF_MARGIN_TOP, PDF_MARGIN_RIGHT);
$pdf->SetHeaderMargin(PDF_MARGIN_HEADER);
$pdf->SetFooterMargin(PDF_MARGIN_FOOTER);
$pdf->SetAutoPageBreak(TRUE, PDF_MARGIN_BOTTOM);
$pdf->setImageScale(PDF_IMAGE_SCALE_RATIO);
$pdf->SetFont('times', 'B', 15);
$pdf->AddPage();
$pdf->SetFillColor(255, 255, 255);

// Escribe la información de cada empresa en el PDF
foreach ($data as $key => $empresa) {
    $pdf->SetFont('times', 'B', 16);
    $pdf->Write(0, $empresa->nombre, '', 0, 'L', true, 0, false, false, 0);
}

```

```

$pdf->SetFont('times', 'B', 10);
$pdf->Write(0, 'Requisitos Logísticos:', '', 0, 'L', true, 0, false, false, 0);
$pdf->SetFont('times', '', 10);
$pdf->Write(0, $empresa->requisitos_logisticos, '', 0, 'L', true, 0, false, false, 0);
$pdf->Ln();

$pdf->SetFont('times', 'B', 10);
$pdf->Write(0, 'Requisitos de Documento:', '', 0, 'L', true, 0, false, false, 0);
$pdf->SetFont('times', '', 10);
$pdf->Write(0, $empresa->requisitos_documento, '', 0, 'L', true, 0, false, false, 0);
$pdf->Ln();
$pdf->Ln();
}

// Muestra el PDF en pantalla
$pdf->Output('Requisitos.pdf', 'I');
}

//////////////////// A J A X //////////////////////////////////////
function ajax_getSolicitudes(){

// Consulta base: solicitudes de factura con empresa, usuario y conteo de recorridos
$query = "SELECT F.id, F.fecha, F.fecha_retorno, F.usuario, F.cliente, F.contacto,
F.reporte_servicio, F.orden_compra, F.forma_pago, F.pagada, F.conceptos, F.notas,
F.estatus_factura, F.documentos_requeridos, F.serie, F.folio, F.codigo_impresion, E.nombre as
Cliente, concat(U.nombre, ' ', U.paterno) as User, ";
$query .= "(SELECT count(id) from recorrido_conceptos where id_concepto = F.id and tipo =
'FACTURA') as Recorridos, (SELECT count(id) from recorrido_conceptos where id_concepto = F.id
and tipo = 'FACTURA' and cerrado = 1) as RecorridosCerrados from solicitudes_facturas F inner
join empresas E on E.id = F.cliente inner join usuarios U on U.id = F.usuario where 1 = 1";

if(isset($_POST['estatus_factura']))
{
    $estatus = $this->input->post('estatus_factura');

// Filtra solicitudes por estatus de entrega si corresponde
if($estatus == "ENTREGA")
{

```

```

        $query .= " and (F.estatus_factura = 'RECIBIDO EN LOGISTICA' or F.estatus_factura = 'NO
ENTREGADA' or F.estatus_factura = 'ENTREGA RECHAZADA') having Recorridos =
RecorridosCerrados";
    }
    // Filtra solicitudes por estatus de recolección si corresponde
    else if($estatus == "RECOLECTA")
    {
        $query .= " and (F.estatus_factura = 'DEJADA CON CLIENTE' or F.estatus_factura = 'NO
RECOLECTADA' or F.estatus_factura = 'RECOLECTA RECHAZADA') having Recorridos =
RecorridosCerrados";
        $query .= " order by F.fecha_retorno asc";
    }
    // Filtra por otro estatus específico
    else
    {
        $query .= " and F.estatus_factura = '$estatus'";
    }
}

// Ejecuta la consulta y devuelve resultados en JSON
$res = $this->Conexion->consultar($query);
if($res)
{
    echo json_encode($res);
}
}

//////////////////////////////// LOGISTICA //////////////////////////////////
function ajax_getMLEquipos() {
    // Consulta equipos en proceso de entrega desde la vista v_items_entregando
    $res = $this->MLConexion->consultar("SELECT *, ifnull(Item,'') as Equipo_ID,
ifnull(fabricante,'') as Fabricante, ifnull(modelo,'') as Modelo, ifnull(serie,'') as Serie from
v_items_entregando");
    if($res) {
        echo json_encode($res);
    }
}
}

```

```

function ajax_getMensajeros() {
    // Consulta usuarios activos, potencialmente filtrando solo mensajes
    $query = "SELECT U.id, U.nombre, U.paterno, U.materno, U.no_empleado, U.puesto,
U.correo, U.ultima_sesion, U.departamento, U.activo, U.jefe_directo, U.autorizador_compras,
U.autorizador_compras_venta, concat(U.nombre, ' ', U.paterno) as User, concat(U.nombre, ' ',
U.paterno, ' ', U.materno) as CompleteName from usuarios U inner join privilegios P on
P.usuario = U.id where U.activo = '1'";

    if(isset($_POST['mensajeros']) && $_POST['mensajeros'] == "1") {
        $query .= " and P.mensajero = '1'";
    }

    $query .= " order by User";

    $res = $this->Conexion->consultar($query);
    if($res) {
        echo json_encode($res);
    }
}

function ajax_setRecorrido() {
    // Obtiene datos del formulario
    $mensajero = $this->input->post('mensajero');
    $fecha = $this->input->post('fecha');
    $recorrido = json_decode($this->input->post('recorrido'));

    // Crea nuevo recorrido principal
    $data['mensajero'] = $mensajero;
    $data['fecha_recorrido'] = $fecha;
    $data['estatus'] = 'ASIGNADO A MENSAJERO';
    $recorrido_id = $this->Conexion->insertar('recorridos', $data);

    // Inserta cada concepto dentro del recorrido
    foreach ($recorrido as $value) {
        $data2['recorrido'] = $recorrido_id;
        $data2['tipo'] = $value[0];
        $data2['id_concepto'] = $value[1];
        $data2['rs'] = $value[2];
    }
}

```

```

$data2['cliente'] = $value[3];
$data2['descripcion'] = $value[4];
$data2['accion'] = $value[5];
$data2['estatus'] = "ASIGNADO A MENSAJERO";
$data2['discrepancia'] = "0";
$data2['cerrado'] = "0";
$data2['reporte'] = "0";

$this->Conexion->insertar('recorrido_conceptos', $data2);

// Actualiza estatus en la tabla solicitudes_facturas (si corresponde)
$this->Conexion->modificar('solicitudes_facturas', array('estatus_factura' =>
$data2['estatus']), null, array('id' => $data2['factura']));
}
}

function ajax_getRecorridos() {
$pendientes = $this->input->post('pendientes');
$factura = $this->input->post('factura');

// Consulta de recorridos asociados a facturas con datos de usuario, cliente y estado
$query = "SELECT RF.*, SF.folio, SF.cliente, R.mensajero, R.fecha_recorrido, R.estatus as
Estatus_recorrido, ifnull(RR.id,'N/A') as Reporte, E.nombre as Cliente,";
$query .= "(SELECT count(RF2.id) from recorrido_facturas RF2 where RF2.recorrido =
RF.recorrido and (RF2.estatus = 'ASIGNADO A MENSAJERO' or RF2.estatus = 'EN RECORRIDO'))
as Pendientes,";
$query .= "ifnull(concat(M.nombre, ' ', M.paterno), 'N/A') as Mensajero";
$query .= " from recorrido_facturas RF";
$query .= " inner join recorridos R on R.id = RF.recorrido";
$query .= " left join recorrido_reporte RR on RR.id = RF.reporte";
$query .= " inner join solicitudes_facturas SF on SF.id = RF.factura";
$query .= " inner join empresas E on E.id = SF.cliente";
$query .= " left join usuarios M on M.id = R.mensajero where 1 = 1";

// Filtro por factura específica
if(isset($_POST['factura'])) {
    $query .= " and RF.factura = $factura";
}
}

```

```

// Filtro por recorridos pendientes
if($pendientes == "1") {
    $query .= " and (R.estatus = 'ACEPTADO' or R.estatus = 'ASIGNADO A MENSAJERO')";
}

$query .= " order by R.fecha_recorrido, R.id, SF.cliente, RF.accion, SF.folio, RF.id asc";

$res = $this->Conexion->consultar($query);

// Devuelve resultados como JSON
echo json_encode($res);
}

function ajax_getEnvios() {
    $factura = $this->input->post('factura');

    // Consulta de envíos asociados a una factura específica
    $query = "SELECT EF.*, concat(U.nombre, ' ', U.paterno) as User from envios_factura EF inner
join usuarios U on U.id = EF.usuario where EF.factura = $factura";

    $res = $this->Conexion->consultar($query);

    if($res) {
        echo json_encode($res);
    }
}

function ajax_setEstatusEnvioCorreo() {
    // Actualiza el estatus de un envío en la tabla envios_factura
    $where['id'] = $this->input->post('id_envio');
    $data['estatus'] = $this->input->post('estatus');

    $this->Conexion->modificar('envios_factura', $data, null, $where);
}

function ajax_setEnviosComentarios() {
    // Inserta un comentario asociado a un envío

```

```

$data['envio'] = $this->input->post('id_envio');
$data['comentario'] = $this->input->post('comentario');
$data['usuario'] = $this->session->id;
$func['fecha'] = 'CURRENT_TIMESTAMP()';

$this->Conexion->insertar('envios_factura_comentarios', $data, $func);
}

function ajax_getEnviosComentarios() {
    // Consulta comentarios relacionados a un envío
    $id_envio = $this->input->post('id_envio');

    $query = "SELECT EFC.*, concat(U.nombre, ' ', U.paterno) as User from
    envios_factura_comentarios EFC inner join usuarios U on U.id = EFC.usuario where EFC.envio =
    $id_envio";

    $res = $this->Conexion->consultar($query);

    if($res) {
        echo json_encode($res);
    }
}

function ajax_getFacturasRecorrido() {
    // Consulta facturas que forman parte de un recorrido específico
    $id = $this->input->post('id');

    $query = "SELECT RF.*, SF.folio from recorrido_facturas RF inner join solicitudes_facturas SF
    on SF.id = RF.factura where RF.recorrido = $id";

    $res = $this->Conexion->consultar($query);

    echo json_encode($res);
}

function ajax_aceptarRecorrido() {
    // Cambia estatus del recorrido a "ACEPTADO"
    $id = $this->input->post('recorrido');

```



```

$data["estatus"] = "ACEPTADO";

$res = $this->Conexion->modificar('recorridos', $data, null, array('id' => $id));

// Actualiza estatus de recorrido_facturas y solicitudes_facturas a "EN RECORRIDO"
$this->Conexion->comando("UPDATE solicitudes_facturas SF, recorrido_facturas RF set
SF.estatus_factura = 'EN RECORRIDO', RF.estatus = 'EN RECORRIDO' where SF.id = RF.factura and
RF.recorrido = $id");

if($res > 0) {
    echo "1";
}
}

function ajax_rechazarRecorrido() {
    // Cambia estatus del recorrido a "RECHAZADO POR MENSAJERO"
    $id = $this->input->post('recorrido');
    $acc = $this->input->post('accion');
    $data["estatus"] = "RECHAZADO POR MENSAJERO";

    $res = $this->Conexion->modificar('recorridos', $data, null, array('id' => $id));

    // Actualiza estatus de recorrido_facturas y solicitudes_facturas a "{ACCION} RECHAZADA"
    $this->Conexion->comando("UPDATE solicitudes_facturas SF, recorrido_facturas RF set
SF.estatus_factura = '$acc RECHAZADA', RF.estatus = 'RECHAZADO POR MENSAJERO' where SF.id
= RF.factura and RF.recorrido = $id");

    if($res > 0) {
        echo "1";
    }
}

function ajax_pendienteCierreRecorrido() {
    // Marca recorrido como "PENDIENTE CIERRE"
    $id = $this->input->post('recorrido');
    $data["estatus"] = "PENDIENTE CIERRE";

    $res = $this->Conexion->modificar('recorridos', $data, null, array('id' => $id));

```

```

if($res > 0) {
    echo "1";
}
}

function ajax_getRecorridosPendienteCierre() {
    // Obtiene recorridos que están pendientes por cerrar
    $res = $this->Conexion->consultar('SELECT R.*, CONCAT(M.nombre, " ", M.paterno) as Mensajero from recorridos R inner join usuarios M on M.id = R.mensajero where R.estatus = "PENDIENTE CIERRE"');
    if($res) {
        echo json_encode($res);
    }
}

function guardarFirma() {
    //////////// F I R M A J P G ////////////

    // Se obtiene la firma en base64 desde POST
    $file = $this->input->post('firma');

    // Decodifica la imagen base64 a binario
    $image = base64_decode(str_replace('data:image/png;base64,', '', $file));

    // Define el nombre del archivo y la ruta de guardado
    $image_name = 'HOLA';
    $filename = $image_name . '.' . 'jpg';
    $path = "data/logistica/firmas/" . $filename;

    // Guarda la imagen en disco
    file_put_contents($path, $image);

    // Retorna el contenido recibido
    echo $file;
    return;

    if($file != "undefined")

```

```

{
    $config['upload_path'] = 'data/logistica/firmas/';
    $config['allowed_types'] = 'gif|jpg|png';
    $config['encrypt_name'] = TRUE;
    $this->load->library('upload', $config);

    if ($this->upload->do_upload('iptFoto'))
    {
        $data['id'] = $id;
        $data['foto'] = $this->upload->data('file_name');
        if($this->Modelo->update($data)){
            if($foto != "default.png"){
                unlink('data/empresas/fotos/' . $foto );
            }
            echo $data['foto'];
        }
    } else {
        echo "";
    }
} else {
    $data['id'] = $id;
    $data['foto'] = 'default.png';
    if($this->Modelo->update($data)){
        if($foto != "default.png"){
            unlink('data/empresas/fotos/' . $foto );
        }
        echo $data['foto'];
    }
}
}
////////////////////

```

```

function ajax_updateRecorrido() {
    $recorrido = json_decode($this->input->post('recorrido'));
    $facturas = json_decode($this->input->post('facturas'));

```

```

    $recolecta = $this->input->post('recolecta');
    $comentario = $this->input->post('comentario');

```

```

$fecha = $this->input->post('fecha');

// Si la acción fue completada y se requiere recolección, cambia estatus y fecha
if($recorrido->estatus == "ENTREGADA") {
    if($recolecta == "1" && $recorrido->accion == "ENTREGA") {
        $recorrido->estatus = "DEJADA CON CLIENTE";
        $data["fecha_retorno"] = $fecha;
    } else {
        $recorrido->estatus = "RETORNADA AUTORIZADA";
    }
}

// Se genera reporte del recorrido
$recorrido_reporte['recorrido'] = $recorrido->id;
$recorrido_reporte['cliente'] = $recorrido->cliente;
$recorrido_reporte['contacto'] = $recorrido->contacto;
$recorrido_reporte['accion'] = $recorrido->accion;
$recorrido_reporte['resultado'] = $recorrido->estatus;
$recorrido_reporte['firma'] = isset($_POST['firma']) ? 1 : 0;

$RR = $this->Conexion->insertar('recorrido_reporte', $recorrido_reporte, array('fecha' =>
'CURRENT_TIMESTAMP()));

// Se registran facturas del recorrido y se actualiza su estatus
$folios = "";
foreach ($facturas as $value) {
    $folios .= $value->folio . ", ";
    $recorrido_facturas['recorrido'] = $recorrido->id;
    $recorrido_facturas['factura'] = $value->factura;
    $recorrido_facturas['accion'] = $recorrido->accion;
    $recorrido_facturas['estatus'] = $recorrido->estatus;
    $recorrido_facturas['reporte'] = $RR;

    if($value->nueva == "1") {
        $this->Conexion->insertar('recorrido_facturas', $recorrido_facturas);
    } else {
        $this->Conexion->modificar('recorrido_facturas', $recorrido_facturas, null, array('id' =>
$value->id));
    }
}

```

```

    }

    $data['estatus_factura'] = $re corrido->estatus;
    $this->Conexion->modificar('solicitudes_facturas', $data, null, array('id' => $value-
>factura));
    }

    // Si hay comentario, se registra con color según el estatus
    if($comentario) {
        $color = substr($re corrido->estatus, 0, 2) == "NO" ? "red" : "green";
        $comentario = '<font color=' . $color . '><b>' . $re corrido->estatus . ':</b></font>' .
$comentario;

        $data_com['reporte'] = $RR;
        $data_com['usuario'] = $this->session->id;
        $data_com['comentario'] = $comentario;
        $func_com['fecha'] = "CURRENT_TIMESTAMP()";
        $this->Conexion->insertar('re corrido_comentarios', $data_com, $func_com);
    }

    // Si se capturó firma, se guarda como imagen y se envía correo si aplica
    if(isset($_POST['firma'])) {
        $file = $this->input->post('firma');
        $image = base64_decode(str_replace('data:image/png;base64,', '', $file));
        $image_name = $RR . '.jpg';
        $path = "data/logistica/firmas/" . $image_name;
        file_put_contents($path, $image);

        // Enviar correo de acuse si se proporcionó correo de contacto
        if($re corrido->CorreoContacto) {
            $data = new stdClass;
            $data->facturas = rtrim($folios, ', ');
            $data->contacto = $re corrido->NombreContacto;
            $data->correo = $re corrido->CorreoContacto;
            $data->comentarios = $comentario;
            $data->firma = $image_name;
            $data->fecha_retorno = $fecha;
            $this->correos_logistica->acuse($data);
        }
    }

```

```

    }
}
function ajax_getComentariosRecorrido() {
    $id = $this->input->post('id');

    // Consulta comentarios del recorrido con nombre del usuario
    $query = "SELECT C.*, concat(U.nombre, ' ', U.paterno) as User from recorrido_comentarios C
inner join usuarios U on U.id = C.usuario where 1 = 1";

    if($id) {
        $query .= " and C.reporte = '$id'";
    }

    $query .= " order by C.fecha";

    $res = $this->Conexion->consultar($query);
    if($res) {
        echo json_encode($res);
    }
}

function ajax_getReporte() {
    $idReporte = $this->input->post('id');

    // Consulta detalles del reporte de recorrido con información de factura, cliente, contacto y
    // requisito
    $query = "SELECT RF.id, RR.fecha, RF.discrepancia, RR.accion, RR.resultado, SF.folio,
RF.estatus, E.nombre as Cliente, ifnull(EC.nombre, 'N/A') as Contacto, RR.firma,
concat(U.nombre, ' ', U.paterno) as Requisitor FROM recorrido_facturas RF";
    $query .= " inner join solicitudes_facturas SF on SF.id = RF.factura";
    $query .= " inner join usuarios U on U.id = SF.usuario";
    $query .= " inner join recorrido_reporte RR on RR.id = RF.reporte";
    $query .= " inner join empresas E on E.id = RR.cliente";
    $query .= " left join empresas_contactos EC on EC.id = RR.contacto";
    $query .= " where RF.reporte = $idReporte";

    $res = $this->Conexion->Consultar($query);

```

```

if($res) {
    echo json_encode($res);
}
}

function ajax_aceptarCierreRecorrido() {
    // Marca recorrido como cerrado y actualiza sus conceptos
    $recorrido = json_decode($this->input->post('recorrido'));

    $this->Conexion->modificar('recorridos', $recorrido, null, array('id' => $recorrido->id));
    $this->Conexion->modificar('recorrido_facturas', array('cerrado' => 1), null, array('recorrido'
=> $recorrido->id));
}

function ajax_rechazarCierreRecorrido() {
    // Marca recorrido como cerrado pero identifica discrepancias en facturas específicas
    $recorrido = json_decode($this->input->post('recorrido'));
    $facturas = json_decode($this->input->post('facturas'));

    $this->Conexion->modificar('recorridos', $recorrido, null, array('id' => $recorrido->id));
    $this->Conexion->modificar('recorrido_facturas', array('cerrado' => 1), null, array('recorrido'
=> $recorrido->id));

    foreach ($facturas as $id_factura) {
        $this->Conexion->comando("UPDATE recorrido_facturas RF, solicitudes_facturas SF set
RF.discrepancia = 1, RF.estatus = concat('NO ', RF.accion, 'DA'), SF.estatus_factura = concat('NO ',
RF.accion, 'DA') where SF.id = RF.factura and RF.id = $id_factura");
    }
}

function ajax_getEmpresasRecorrido() {
    // Consulta empresas involucradas en un recorrido
    $recorrido = $this->input->post('recorrido');
    $query = "SELECT E.id, E.nombre from empresas E inner join solicitudes_facturas SF on
SF.cliente = E.id inner join recorrido_facturas RF on RF.factura = SF.id where RF.recorrido =
$recorrido order by E.nombre";

    $res = $this->Conexion->consultar($query);
}

```

```

    echo json_encode($res);
}

function ajax_getDashboard() {
    // Consulta resumen de estatus y fechas mínimas por categoría para el dashboard logístico
    $query = 'SELECT count(*) as Total, (SELECT count(*) FROM solicitudes_facturas where
    estatus_factura = "ENVIADA LOGISTICA") as D1, (SELECT min(fecha) FROM solicitudes_facturas
    where estatus_factura = "ENVIADA LOGISTICA") as ult1,';
    $query .= ' (SELECT count(*) FROM solicitudes_facturas where estatus_factura = "RECIBIDO
    EN LOGISTICA") as D2, (SELECT min(fecha) FROM solicitudes_facturas where estatus_factura =
    "RECIBIDO EN LOGISTICA") as ult2,';
    $query .= ' (SELECT count(*) FROM solicitudes_facturas where estatus_factura = "RECHAZADO
    EN LOGISTICA") as D3, (SELECT min(fecha) FROM solicitudes_facturas where estatus_factura =
    "RECHAZADO EN LOGISTICA") as ult3,';
    $query .= ' (SELECT count(*) FROM solicitudes_facturas where estatus_factura = "ASIGNADO
    A MENSAJERO") as D4, (SELECT min(fecha) FROM solicitudes_facturas where estatus_factura =
    "ASIGNADO A MENSAJERO") as ult4,';
    $query .= ' (SELECT count(*) FROM solicitudes_facturas where estatus_factura = "ENTREGA
    RECHAZADA") as D5, (SELECT min(fecha) FROM solicitudes_facturas where estatus_factura =
    "ENTREGA RECHAZADA") as ult5,';
    $query .= ' (SELECT count(*) FROM solicitudes_facturas where estatus_factura = "RECOLECTA
    RECHAZADA") as D6, (SELECT min(fecha) FROM solicitudes_facturas where estatus_factura =
    "RECOLECTA RECHAZADA") as ult6,';
    $query .= ' (SELECT count(*) FROM solicitudes_facturas where estatus_factura = "EN
    RECORRIDO") as D7, (SELECT min(fecha) FROM solicitudes_facturas where estatus_factura =
    "EN RECORRIDO") as ult7,';
    $query .= ' (SELECT count(*) FROM solicitudes_facturas where estatus_factura = "DEJADA
    CON CLIENTE") as D8, (SELECT min(fecha) FROM solicitudes_facturas where estatus_factura =
    "DEJADA CON CLIENTE") as ult8,';
    $query .= ' (SELECT count(*) FROM solicitudes_facturas where estatus_factura = "NO
    ENTREGADA") as D9, (SELECT min(fecha) FROM solicitudes_facturas where estatus_factura =
    "NO ENTREGADA") as ult9,';
    $query .= ' (SELECT count(*) FROM solicitudes_facturas where estatus_factura = "NO
    RECOLECTADA") as D10, (SELECT min(fecha) FROM solicitudes_facturas where estatus_factura =
    "NO RECOLECTADA") as ult10';
    $query .= ' FROM solicitudes_facturas;';

    $res = $this->Conexion->consultar($query, TRUE);
}

```



```

    echo json_encode($res);
}

function ajax_setSolicitud() {
    $solicitud = json_decode($this->input->post('solicitud'));
    $other = json_decode($this->input->post('other'));

    // Adjunta archivos binarios si existen
    if(isset($_FILES['f_O'])) {
        $solicitud->f_orden_compra = file_get_contents($_FILES['f_O']['tmp_name']);
    }
    if(isset($_FILES['f_R'])) {
        $solicitud->f_remision = file_get_contents($_FILES['f_R']['tmp_name']);
    }

    $funciones = array('fecha' => 'CURRENT_TIMESTAMP()');
    $res = false;

    // Inserta nueva solicitud o actualiza una existente
    if($solicitud->id == 0) {
        $res = $this->Conexion->insertar('solicitudes_facturas', $solicitud, $funciones);
        $solicitud->id = $res;
    } else {
        $res = $this->Conexion->modificar('solicitudes_facturas', $solicitud, null, array('id' =>
        $solicitud->id)) >= 0;
    }

    // Si se cargó archivo "other", se notifica vía correo
    if(isset($_FILES['other'])) {
        $solicitud->User = $other->User;
        $solicitud->Client = $other->Client;
        $solicitud->Contact = $other->Contact;

        $correos = [];
        $correos_a = $this->Conexion->consultar("SELECT U.correo from privilegios P inner join
        usuarios U on P.usuario = U.id where P.responder_facturas = 1");

        foreach ($correos_a as $key => $value) {

```

```

        array_push($correos, $value->correo);
    }

    $solicitud->correos = array_merge(array($this->session->correo), $correos);

    // Envía correo de notificación
    $this->correos_facturacion->solicitud($solicitud);
}

if($res) {
    echo "1";
}
}

function ajax_editSolicitud() {
    $solicitud = json_decode($this->input->post('solicitud'));
    $other = json_decode($this->input->post('other'));

    // Se cargan archivos si se enviaron en el formulario
    if(isset($_FILES['f_A'])) {
        $solicitud->f_acuse = file_get_contents($_FILES['f_A']['tmp_name']);
    }
    if(isset($_FILES['f_F'])) {
        $solicitud->f_factura = file_get_contents($_FILES['f_F']['tmp_name']);
    }
    if(isset($_FILES['f_X'])) {
        $solicitud->f_xml = file_get_contents($_FILES['f_X']['tmp_name']);
    }

    $comentario = $this->input->post('comentario');

    // Actualiza la solicitud
    $res = $this->Conexion->modificar('solicitudes_facturas', $solicitud, null, array('id' =>
    $solicitud->id));
    if($res > 0) {
        // Si se envió comentario, lo guarda
        if(isset($_POST['comentario']) && !empty($comentario)) {

```

```

        $this->Conexion->insertar('solicitudes_facturas_comentarios', array('solicitud' =>
        $solicitud->id, 'usuario' => $this->session->id, 'comentario' => $comentario), array('fecha' =>
        'CURRENT_TIMESTAMP()));

```

```

        $solicitud->comentario = $comentario;
    }

```

```

    // Prepara correos de notificación
    $correos = [];
    $correos_a = $this->Conexion->consultar("SELECT U.correo from privilegios P inner join
    usuarios U on P.usuario = U.id where P.responder_facturas = 1");
    foreach ($correos_a as $key => $value) {
        array_push($correos, $value->correo);
    }

```

```

    $solicitud->correos = array_merge(array($this->session->correo), $correos);
    $solicitud->User = $other->User;
    $solicitud->Client = $other->Client;
    $solicitud->Contact = $other->Contact;

```

```

    // Envía correo de modificación de solicitud
    $this->correos_facturacion->editar_solicitud($solicitud);
    echo "1";
} else {
    echo "";
}
}

```

```

function ajax_setComentarios() {
    $comentario = json_decode($this->input->post('comentario'));

```

```

    // Asigna el usuario de sesión al comentario y registra la fecha actual
    $comentario->usuario = $this->session->id;
    $funciones = array('fecha' => 'CURRENT_TIMESTAMP());

```

```

    // Inserta el comentario en la base de datos
    $res = $this->Conexion->insertar('solicitudes_facturas_comentarios', $comentario,
    $funciones);
    if($res > 0) {

```

```

        echo "1";
    }
}

function ajax_getComentarios() {
    $id = $this->input->post('id');

    // Consulta comentarios asociados a una solicitud específica
    $query = "SELECT C.*, concat(U.nombre, ' ', U.paterno) as User from
solicitudes_facturas_comentarios C inner join usuarios U on U.id = C.usuario where 1 = 1";

    if($id) {
        $query .= " and C.solicitud = '$id'";
    }

    $res = $this->Conexion->consultar($query);
    if($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

function ajax_getRequisitores() {
    $id = $this->input->post('id');

    // Consulta de usuarios con privilegio de solicitar facturas
    $query = "SELECT U.id, concat(U.nombre, ' ', U.paterno) as Nombre, P.puesto as Puesto from
usuarios U inner join puestos P on U.puesto = P.id inner join privilegios PR on PR.usuario = U.id
where U.activo = 1 and PR.solicitar_facturas = 1";

    if($id) {
        $query .= " and U.id = '$id'";
    }

    $res = $this->Conexion->consultar($query, $id);
    if($res) {
        echo json_encode($res);
    }
}

```

```

    } else {
        echo "";
    }
}

function ajax_getVFPData() {
    // Ejecuta lector de datos VFP externo pasando el modelo como argumento
    $modelo = $this->input->post('modelo');
    $res = shell_exec("C:/xampp/htdocs/MASMetrologia/vfp_reader/vfp_reader.exe
\"$modelo\"");
    echo $res;
}

function archivo_impresion() {
    // Desactiva errores en pantalla y carga librería de combinación de PDFs
    ini_set('display_errors', 0);
    $this->load->library('pdfmerge');

    $id = $this->input->post('id');
    $codigo = $this->input->post('codigo');

    $pdf = new PDFMerger();

    // Consulta solicitud de factura por ID
    $res = $this->Conexion->consultar("SELECT SF.* from solicitudes_facturas SF where SF.id =
$id", TRUE);

    // Itera cada letra del código recibido para determinar qué documentos incluir
    for ($i = 0; $i < strlen($codigo); $i++) {
        switch (strtoupper($codigo[$i])) {
            case 'F':
                $campo = 'f_factura';
                break;
            case 'R':
                $campo = 'f_remision';
                break;
            case 'O':
                $campo = 'f_orden_compra';

```

```

        break;
    case 'A':
        $campo = 'f_acuse';
        break;
    case 'P':
        $campo = 'OPINION';
        break;
    case 'S':
        $campo = 'EMISION';
        break;
    default:
        $campo = null;
        break;
}

if($campo != null) {
    if(substr($campo, 0, 2 ) == "f_") {
        // Documentos almacenados como BLOB en BD (factura, remisión, etc.)
        $file = $res->$campo;
        $fichero = sys_get_temp_dir() . '/' . $campo . '.pdf';
        file_put_contents($fichero, $file);
        $pdf->addPDF($fichero, 'all');
    } else {
        // Documentos externos por código fijo (OPINION, EMISION)
        $fichero = "data/empresas/documentos_globales/" . $campo . "_000001.pdf";
        $pdf->addPDF($fichero, 'all');
    }
}

}

// Combina y muestra los PDF en el navegador
$pdf->merge('browser');
}

function ajax_getClientes() {
    // Consulta clientes registrados como empresas con rol de cliente
    $query = "SELECT C.id, C.nombre, C.razon_social, C.foto, C.opinion_positiva, C.emision_sua
from empresas C where C.cliente = 1";

```

```

$res = $this->Conexion->consultar($query);
if($res) {
    echo json_encode($res);
} else {
    echo "";
}
}

function ajax_getDocumentosGlobales() {
    // Consulta los documentos globales configurados para la empresa
    $query = "SELECT id, opinion_positiva, emision_sua from documentos_globales where id = 1";

    $res = $this->Conexion->consultar($query);
    if($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

function ajax_filesExists($id) {
    $this->load->helper('file');

    // Rellenar el ID con ceros a la izquierda (ej. 1 => 000001)
    $id = str_pad($id, 6, "0", STR_PAD_LEFT);

    // Verifica si existen los archivos PDF de ACUSE y EMISIÓN para la empresa
    $acuse = read_file(base_url("data/empresas/documentos_facturacion/ACUSE_" . $id . ".pdf"))
? "1" : "0";
    $emision = read_file(base_url("data/empresas/documentos_facturacion/EMISION_" . $id .
".pdf")) ? "1" : "0";

    echo json_encode(array($acuse, $emision));
}

function ajax_readXML() {
    $dom = new DomDocument;

```

```

$dom->preserveWhiteSpace = FALSE;

// Cargar el XML desde archivo temporal subido
$dom->loadXML(file_get_contents($_FILES['f_X']['tmp_name']));

$comp = $dom->getElementsByTagName('Comprobante');
$data = array();

// Obtener atributos Serie y Folio del nodo <Comprobante>
foreach ($comp[0]->attributes as $elem) {
    if($elem->name == "Serie" | $elem->name == "Folio") {
        $e = array($elem->name => $elem->value);
        array_push($data, $e);
    }
}

echo json_encode($data);
}

function ajax_setDocumentoFacturacion() {
    $file = $this->input->post('file');
    $documento = $this->input->post('documento');
    $id = $this->input->post('empresa');

    // Formatear ID a 6 dígitos
    $id = str_pad($id, 6, "0", STR_PAD_LEFT);

    // Verificar que se haya subido un archivo
    if($file != "undefined") {
        $config['upload_path'] = 'data/empresas/documentos_facturacion/';
        $config['allowed_types'] = 'pdf';
        $config['overwrite'] = TRUE;
        $config['file_name'] = $documento . '_' . $id;

        $this->load->library('upload', $config);

        if ($this->upload->do_upload('file')) {
            $where['id'] = $id;

```



```

// Determinar campo a actualizar según tipo de documento
switch($documento) {
    case "EMISION":
        $campo = "emision_sua";
        break;
    case "OPINION":
        $campo = "opinion_positiva";
        break;
}

$data[$campo] = $this->upload->data('file_name');
$this->Conexion->modificar('empresas', $data, null, $where);
echo "1";
}
}
}

function ajax_deleteDocumentoFacturacion() {
    $documento = $this->input->post('documento');
    $id = $this->input->post('empresa');

    // Formatear ID a 6 dígitos (rellenando con ceros a la izquierda)
    $id = str_pad($id, 6, "0", STR_PAD_LEFT);

    // Eliminar archivo físico del documento
    unlink('data/empresas/documentos_facturacion/' . $documento . '_' . $id . '.pdf');

    // Actualizar campo correspondiente en base de datos
    $where['id'] = $id;
    switch($documento) {
        case "EMISION":
            $campo = "emision_sua";
            break;
        case "OPINION":
            $campo = "opinion_positiva";
            break;
    }
}

```

```

$data[$campo] = "";
$this->Conexion->modificar('empresas', $data, null, $where);
}

function ajax_setDocumentoGlobal() {
    $file = $this->input->post('file');
    $documento = $this->input->post('documento');
    $id = $this->input->post('empresa');

    // Formatear ID a 6 dígitos (rellenando con ceros a la izquierda)
    $id = str_pad($id, 6, "0", STR_PAD_LEFT);

    // Validar que se haya recibido un archivo
    if($file != "undefined") {
        $config['upload_path'] = 'data/empresas/documentos_globales/';
        $config['allowed_types'] = 'pdf';
        $config['overwrite'] = TRUE;
        $config['file_name'] = $documento . '_' . $id;

        $this->load->library('upload', $config);

        if ($this->upload->do_upload('file')) {
            $where['id'] = $id;

            // Determinar campo correspondiente a actualizar
            switch($documento) {
                case "EMISION":
                    $campo = "emision_sua";
                    break;
                case "OPINION":
                    $campo = "opinion_positiva";
                    break;
            }

            $data[$campo] = $this->upload->data('file_name');
            $this->Conexion->modificar('documentos_globales', $data, null, $where);
            echo "1";
        }
    }
}

```

```

    }
}

function ajax_deleteDocumentoGlobal() {
    $documento = $this->input->post('documento');
    $id = $this->input->post('empresa');

    // Formatear ID a 6 dígitos con ceros a la izquierda
    $id = str_pad($id, 6, "0", STR_PAD_LEFT);

    // Eliminar archivo físico
    unlink('data/empresas/documentos_globales/' . $documento . '_' . $id . '.pdf');

    // Determinar campo a limpiar
    $where['id'] = $id;
    switch($documento) {
        case "EMISION":
            $campo = "emision_sua";
            break;
        case "OPINION":
            $campo = "opinion_positiva";
            break;
    }

    $data[$campo] = "";
    $this->Conexion->modificar('documentos_globales', $data, null, $where);
}

function ajax_enviarCorreo() {
    $id = $this->input->post('id');
    $body = $this->input->post('body');
    $para = $this->input->post('para');
    $cc = $this->input->post('cc');

    // Archivos a enviar
    $campos = ['f_xml', 'f_factura'];
    $archivos = [];

```

```
$res = $this->Conexion->consultar("SELECT SF.* from solicitudes_facturas SF where SF.id = $id", TRUE);
```

```
foreach ($campos as $value) {  
    if(substr($value, 0, 2) == "f_") {  
        $file = $res->$value;  
        $fichero = sys_get_temp_dir(). '/' . $value . ($value == "f_xml" ? '.xml' : '.pdf');  
        file_put_contents($fichero, $file);  
    } else {  
        switch ($value) {  
            case 'opinion_positiva':  
                $value = 'OPINION';  
                break;  
            case 'emision_sua':  
                $value = 'EMISION';  
                break;  
        }  
        $fichero = "data/empresas/documentos_globales/" . $value . "_000001.pdf";  
    }  
  
    array_push($archivos, $fichero);  
}
```

```
// Armar datos y enviar
```

```
$datos['id'] = $id;  
$datos['para'] = $para;  
$datos['cc'] = $cc;  
$datos['body'] = $body;  
$datos['campos'] = $campos;  
$datos['archivos'] = $archivos;
```

```
$this->correos_logistica->enviarCorreoFactura($datos);  
}
```

```
function ajax_enviarCorreoLogistica() {  
    $id = $this->input->post('id');  
    $destinatario = $this->input->post('destinatario');
```

```

$mensaje = $this->input->post('mensaje');

$data['factura'] = $id;
$data['destino'] = $destinatario;
$data['usuario'] = $this->session->id;
$data['estatus'] = "ENVIADA";
$data['mensaje'] = $mensaje;

$func['fecha'] = 'CURRENT_TIMESTAMP()';
$recorrido_id = $this->Conexion->insertar('envios_factura', $data, $func);
}

```

//////////////////////////////// F A C T U R A S //////////////////////////////////

```

function ajax_getFacturas() {
    $id = $this->input->post('id');

    $query = "SELECT F.id, F.fecha, F.usuario, F.cliente, F.contacto, F.reporte_servicio,
F.orden_compra, F.forma_pago, F.pagada, F.conceptos, F.notas, F.estatus_factura,
F.documentos_requeridos, F.serie, F.folio, F.codigo_impresion,";
    $query .= " (SELECT count(id) from recorrido_conceptos where factura = F.id) as Recorridos,";
    $query .= " E.nombre as Cliente, concat(U.nombre, ' ', U.paterno) as User";
    $query .= " from solicitudes_facturas F";
    $query .= " inner join empresas E on E.id = F.cliente";
    $query .= " inner join usuarios U on U.id = F.usuario";
    $query .= " where F.folio > 0 and F.estatus = 'ACEPTADO'";

    if ($id) {
        $query .= " and F.id = '$id'";
    }

    if (isset($_POST['estatus'])) {
        $estatus = $this->input->post('estatus');
        $query .= " and F.estatus = '$estatus'";
    }

    if (isset($_POST['estatus_factura'])) {
        $estatus_factura = $this->input->post('estatus_factura');
    }
}

```

```

        $query .= " and F.estatus_factura = '$estatus_factura'";
    }

    if (isset($_POST['cliente'])) {
        $cliente = $this->input->post('cliente');
        $query .= " and F.cliente = '$cliente'";
    }

    $res = $this->Conexion->consultar($query, $id);

    if ($res) {
        echo json_encode($res);
    }
}
}

```

Ordenes_compra.php

```
<?php
```

```
defined('BASEPATH') OR exit('No direct script access allowed');
```

```

class Ordenes_compra extends CI_Controller {
    function __construct() {
        parent::__construct();
        $this->load->library('correos_po');
        $this->load->model('compras_model');
    }
}

```

```

function catalogo_po($estatus = 'TODO') {
    $estatus = strtoupper($estatus);
    $data['estatus'] = str_replace('_', ' ', $estatus);

    $this->load->view('header');
    $this->load->view('ordenes_compra/catalogo_po', $data);
}

```

```
function generar_po() {
```

```

$this->load->view('header');
$this->load->view('ordenes_compra/generar_po');
}

function construccion_po($idtemp) {
    $query = "SELECT OCT.*, JSON_UNQUOTE(OCT.prs) as PRS,
JSON_UNQUOTE(OCT.prs_rechazados) as PRS_R, E.nombre FROM ordenes_compra_temp OCT
INNER JOIN empresas E ON OCT.proveedor = E.id WHERE OCT.idtemp = '$idtemp'";

    $res = $this->Conexion->consultar($query, TRUE);
    if ($res) {
        $data['id_prov'] = $res->proveedor;
        $data['moneda'] = $res->moneda;
        $data['tipo'] = $res->tipo;
        $data['prs'] = $res->PRS;
        $data['prs_rechazadas'] = $res->PRS_R;
        $data['idtemp'] = $idtemp;

        $this->load->view('header');
        $this->load->view('ordenes_compra/construccion_po', $data);
    }
}

function editar_po($id){
    $data['id'] = $id;

    // Carga la vista de edición para la orden de compra con el ID proporcionado
    $this->load->view('header');
    $this->load->view('ordenes_compra/editar_po', $data);
}

function modificar_po($id){
    $data['id'] = $id;

    // Obtiene el último estatus registrado de la orden de compra en la bitácora
    $query = "SELECT estatus from bitacora_po where po='$id' ORDER BY idBitPO DESC LIMIT 1";
    $res = $this->Conexion->consultar($query, TRUE);

```

```

// Si no está en edición, lo cambia a 'EDICION' y registra el cambio en la bitácora
if($res->estatus != 'EDICION'){
    $estatus = 'EDICION';
    $datos['id'] = $id;
    $datos['estatus'] = $estatus;
    $this->compras_model->updateEstatusEditar($datos); // Actualiza el estatus en la orden
    $dato['po'] = $id;
    $dato['user'] = $this->session->id;
    $dato['estatus'] = $estatus;
    $this->compras_model->estatusPO($dato); // Registra el cambio en la bitácora
}

// Carga la vista de edición específica para modificar una orden existente
$this->load->view('header');
$this->load->view('ordenes_compra/editarPO', $data);
}

function ver_po($id){
    $data['id'] = $id;

    // Obtiene los comentarios generales relacionados con la PO
    $data['comentarios'] = $this->compras_model->verPo_comentarios($id);

    // Obtiene el historial de rastreo o seguimiento de la PO
    $data['rastreo'] = $this->compras_model->verPo_rastreo($id);

    // Obtiene comentarios que contienen imágenes adjuntas
    $data['comentarios_fotos'] = $this->compras_model->verPo_comentarios_fotos($id);

    // Obtiene imágenes relacionadas con el rastreo de la PO
    $data['rastreo_fotos'] = $this->compras_model->verPo_rastreo_fotos($id);

    // Carga las vistas necesarias para visualizar la orden de compra
    $this->load->view('header');
    $this->load->view('ordenes_compra/ver_po', $data);
}

```



```

function po_pdf($id){
    $query="SELECT PO.id, PO.fecha, PO.shipping_address, PO.billing_address, PO.conceptos,
    PO.subtotal, PO.descuento, PO.impuesto, PO.impuesto_nombre, PO.total, PO.retencion,
    PO.moneda, PO.rma, PO.fecha_aprobacion, concat(U.nombre, ' ', U.paterno) as User, U.correo
    as UserMail, concat(UA.nombre, ' ', UA.paterno) as UserA, E.razon_social, E.calle, E.numero,
    E.numero_interior, E.colonia, E.ciudad, E.estado, E.pais, E.rfc, EC.nombre, EC.telefono, EC.correo
    FROM ordenes_compra PO inner join empresas E on PO.proveedor = E.id inner join usuarios U
    on U.id = PO.usuario inner join usuarios UA on UA.id = PO.aprobador inner join
    empresas_contactos EC on PO.contacto = EC.id where PO.id='$id'";

    // Consulta toda la información de la orden de compra, proveedor, aprobador y contacto

    $res = $this->Conexion->consultar($query, TRUE);

    $conceptos = json_decode($res->conceptos);
    // Decodifica los conceptos incluidos en la orden (formato JSON)

    $PO = 'PO-' . str_pad($res->id, 6, "0", STR_PAD_LEFT);
    $FECHA = date_format(date_create($res->fecha), 'd/m/Y h:i A');
    // Formatea la fecha de creación de la orden

    $PROVEEDOR = $res->razon_social;
    $RMA = $res->rma;
    $DOMICILIO = $res->calle . ' ' . $res->numero . ' ' . $res->numero_interior;
    $COLONIA = $res->colonia;
    $RFC = $res->rfc;

    $REQUISITOR = $res->User;
    $REQUISITOR_CORREO = $res->UserMail;

    $APROBADOR = $res->UserA;
    $FECHA_APROBACION = date_format(date_create($res->fecha_aprobacion), 'd/m/Y h:i A');
    // d/m/Y h:i A

    $CONTACTO = $res->nombre;
    $TELEFONO = $res->telefono;
    $CORREO = $res->correo;
    $UBICACION = $res->ciudad . ', ' . $res->estado . ', ' . $res->pais;

```

```

$BILLING_A = $res->billing_address;
$SHIPPING_A = $res->shipping_address;

$SUBTOTAL = $res->subtotal;
$DESCUENTO = $res->descuento;
$IMPUESTO = $res->impuesto;
$IMPUESTO_NOMBRE = $res->impuesto_nombre;
$TOTAL = $res->total;
$MONEDA = $res->moneda;
$RETENCION = $res->retencion;

ini_set('display_errors', 0);
$this->load->library('pdfview');

$pdf = new TCPDF(PDF_PAGE_ORIENTATION, PDF_UNIT, PDF_PAGE_FORMAT, true, 'UTF-8',
false);

$pdf->SetCreator(PDF_CREATOR);
$pdf->SetAuthor('AleksOrtiz');
$pdf->SetTitle('Masmetrologia');
$pdf->SetSubject('Formato PO');

$pdf->SetHeaderData(PDF_HEADER_LOGO_ORIGINAL, '40', '
Orden de Compra / ' . $PO, "
Comprador: " . $REQUISITOR . " \n
$REQUISITOR_CORREO);
Fecha: " . $FECHA . " \n
Correo: " .

$pdf->setHeaderFont(Array(PDF_FONT_NAME_MAIN, "", 10));
$pdf->setFooterFont(Array(PDF_FONT_NAME_DATA, "", PDF_FONT_SIZE_DATA));
$pdf->SetDefaultMonospacedFont(PDF_FONT_MONOSPACED);
$pdf->SetMargins(PDF_MARGIN_LEFT, PDF_MARGIN_TOP, PDF_MARGIN_RIGHT);
$pdf->SetHeaderMargin(PDF_MARGIN_HEADER);
$pdf->SetFooterMargin(PDF_MARGIN_FOOTER);
$pdf->SetAutoPageBreak(TRUE, PDF_MARGIN_BOTTOM);
$pdf->setImageScale(PDF_IMAGE_SCALE_RATIO);
if (@file_exists(dirname(__FILE__).'/lang/eng.php')) {
    require_once(dirname(__FILE__).'/lang/eng.php');
    $pdf->setLanguageArray($l);

```

```

}
$pdf->SetFont('times', 'B', 15);
$pdf->AddPage();

$pdf->SetFillColor(255, 255, 255);

$pdf->SetFont('times', 'B', 12);
$pdf->MultiCell(90, 0, "Proveedor/Supplier:", 0, 'L', 1, 0, "", true, 0, false, true, 0);
$pdf->MultiCell(90, 0, $RMA ? "RMA:" : "", 0, 'L', 1, 1, "", true, 0, false, true, 0);
$pdf->SetFont('times', "", 10);
$pdf->MultiCell(90, 6, $PROVEEDOR, 0, 'L', 1, 0, "", true, 0, false, true, 0);
$pdf->MultiCell(90, 0, $RMA ? $RMA : "", 0, 'L', 1, 1, "", true, 0, false, true, 0);

$pdf->SetFont('times', 'B', 12);
$pdf->MultiCell(90, 0, "Dirección / Address:", 0, 'L', 1, 0, "", true, 0, false, true, 0);
$pdf->MultiCell(90, 0, "Contacto / Contact:", 0, 'L', 1, 1, "", true, 0, false, true, 0);
$pdf->SetFont('times', "", 10);

$pdf->MultiCell(90, 0, $DOMICILIO, 0, 'L', 1, 0, "", true, 0, false, true, 0);
$pdf->MultiCell(90, 0, $CONTACTO, 0, 'L', 1, 1, "", true, 0, false, true, 0);
$pdf->MultiCell(90, 0, $COLONIA, 0, 'L', 1, 0, "", true, 0, false, true, 0);
$pdf->MultiCell(90, 0, $TELEFONO, 0, 'L', 1, 1, "", true, 0, false, true, 0);
$pdf->MultiCell(90, 0, $UBICACION, 0, 'L', 1, 0, "", true, 0, false, true, 0);
$pdf->MultiCell(90, 0, $CORREO, 0, 'L', 1, 1, "", true, 0, false, true, 0);

$pdf->MultiCell(90, 6, $RFC, 0, 'L', 1, 1, "", true, 0, false, true, 0);

$pdf->SetFont('times', 'B', 12);
$pdf->MultiCell(90, 0, "Facturar a / Billing Address:", 0, 'L', 1, 0, "", true, 0, false, true, 0);
$pdf->MultiCell(90, 6, "Enviar a / Shipping Address:", 0, 'L', 1, 1, "", true, 0, false, true, 0);
$pdf->SetFont('times', "", 10);

$pdf->MultiCell(90, 20, $BILLING_A, 0, 'L', 1, 0, "", true, 0, false, true, 0);
$pdf->MultiCell(90, 20, $SHIPPING_A, 0, 'L', 1, 1, "", true, 0, false, true, 0);

$pdf->MultiCell(180, 0, "", 0, 'L', 1, 1, "", true, 0, false, true, 0);

```

```

$pdf->SetFont('times', 'B', 12);
$pdf->SetFillColor(1, 59, 117);
$pdf->SetTextColor(255);

$pdf->SetTextColor(0);
$pdf->Cell(15, 6, "Cant.", 1, 0, 'C', 0);
$pdf->Cell(115, 6, "Descripción", 1, 0, 'C', 0);
$pdf->Cell(25, 6, "Precio Unit.", 1, 0, 'C', 0);
$pdf->Cell(25, 6, "Importe", 1, 0, 'C', 0);
$pdf->Ln();

$w = array(15, 115, 25, 25);
$pdf->SetFont("", "", 10);
$pdf->MultiCell($w[0], 3, "", 'LR', 'C', 0, 0);
$pdf->MultiCell($w[1], 3, "", 'LR', 'L', 0, 0);
$pdf->MultiCell($w[2], 3, "", 'LR', 'R', 0, 0);
$pdf->MultiCell($w[3], 3, "", 'LR', 'R', 0, 0);
$pdf->Ln();
$i = 0;

// Itera por cada concepto para mostrar cantidad, descripción, precio unitario e importe
foreach($conceptos as $indice => $concepto) {
    $cellcount = array();
    $startX = $pdf->GetX();
    $startY = $pdf->GetY();

    $pu = $concepto[2] / $concepto[0];
    $cellcount[] = $pdf->MultiCell($w[0], 6, $concepto[0], 0, 'C', 0, 0);
    if (empty($concepto[3])) {
        $cellcount[] = $pdf->MultiCell($w[1], 6, $concepto[1], 0, 'L', 0, 0);
    } else {
        $cellcount[] = $pdf->MultiCell($w[1], 6, $concepto[1]."\n Iva Retenido: ".$concepto[3]." - Retencion: ".$concepto[4].number_format(floatval($concepto[4]), 2), 0, 'L', 0, 0);
    }

    $cellcount[] = $pdf->MultiCell($w[2], 6, "$" . number_format($pu, 2), 0, 'R', 0, 0);
}

```

```
$cellcount[] = $pdf->MultiCell($w[3], 6, "$" . number_format($concepto[2], 2), 0, 'R', 0, 0); // VER AQUI
```

```
$pdf->SetXY($startX, $startY);
```

```
$maxnocells = max($cellcount);
```

```
$pdf->MultiCell($w[0], $maxnocells * 6, "", 'LR', 'C', 0, 0);
```

```
$pdf->MultiCell($w[1], $maxnocells * 6, "", 'LR', 'L', 0, 0);
```

```
$pdf->MultiCell($w[2], $maxnocells * 6, "", 'LR', 'R', 0, 0);
```

```
$pdf->MultiCell($w[3], $maxnocells * 6, "", 'LR', 'R', 0, 0);
```

```
$pdf->Ln();
```

```
// Si el concepto incluye IVA retenido o retención, agrega línea adicional con detalles
```

```
if(isset($concepto[3]) && !empty($concepto[3]))
```

```
{
```

```
    $cellcount = array();
```

```
    $startX = $pdf->GetX();
```

```
    $startY = $pdf->GetY();
```

```
    $cellcount[] = $pdf->MultiCell($w[0], 6, "", 0, 'C', 0, 0);
```

```
    $pdf->SetXY($startX, $startY);
```

```
    $maxnocells = max($cellcount);
```

```
    $pdf->MultiCell($w[0], $maxnocells * 6, "", 'LR', 'C', 0, 0);
```

```
    $pdf->MultiCell($w[1], $maxnocells * 6, "", 'LR', 'L', 0, 0);
```

```
    $pdf->MultiCell($w[2], $maxnocells * 6, "", 'LR', 'R', 0, 0);
```

```
    $pdf->MultiCell($w[3], $maxnocells * 6, "", 'LR', 'R', 0, 0);
```

```
    $pdf->Ln();
```

```
}
```

```
$i++;
```

```
if($startY > 212)
```

```
// Si el contenido se acerca al final de la hoja, crea una nueva página
```

```
{
```

```

    $pdf->MultiCell(180, 6, '', 'T', 'C', 0, 0);
    $pdf->AddPage();
}
}

for ($startY; $startY < 200; $startY = $startY + 6) {
    $pdf->MultiCell($w[0], 6, '---', 'LR', 'C', 0, 0);
    $pdf->MultiCell($w[1], 6, '-----'
-----', 'LR', 'L', 0, 0);
    $pdf->MultiCell($w[2], 6, '-----', 'LR', 'R', 0, 0);
    $pdf->MultiCell($w[3], 6, '-----', 'LR', 'R', 0, 0);
    $pdf->Ln();
}

// Muestra el resumen de importes: subtotal, descuento, impuestos, retención y total
$pdf->MultiCell(130, 6, '* Moneda / Currency: ' . $MONEDA, 'T', 'M', 0, 0);
$pdf->MultiCell($w[2], 6, 'Sub-Total', 'T', 'C', 0, 0, "", "", TRUE, 0, FALSE, TRUE, 0, 'M');
$pdf->MultiCell($w[3], 6, "$" . number_format($SUBTOTAL, 2), 1, 'R', 0, 0, "", "", TRUE, 0,
FALSE, TRUE, 0, 'M');
$pdf->Ln();

$pdf->MultiCell(130, 6, '* If you have any questions regarding this Purchase Order please
contact the buyer.', 0, 'M', 0, 0);
$pdf->MultiCell($w[2], 6, 'Descuento', 0, 'C', 0, 0, "", "", TRUE, 0, FALSE, TRUE, 0, 'M');
$pdf->MultiCell($w[3], 6, "$" . number_format($DESCUENTO, 2), 1, 'R', 0, 0, "", "", TRUE, 0,
FALSE, TRUE, 0, 'M');
$pdf->Ln();

$pdf->MultiCell(130, 6, '* Any changes or exceptions must be authorized by the Purchasing
Department.', 0, 'M', 0, 0);
$pdf->MultiCell($w[2], 6, "IVA / TAX", 0, 'C', 0, 0, "", "", TRUE, 0, FALSE, TRUE, 0, 'M');
$pdf->MultiCell($w[3], 6, "$" . number_format($IMPUESTO, 2), 1, 'R', 0, 0, "", "", TRUE, 0,
FALSE, TRUE, 0, 'M');
$pdf->Ln();

$pdf->MultiCell(130, 6, '* Elaborate one invoice for each Purchase Order.', 0, 'M', 0, 0);
$pdf->MultiCell($w[2], 6, 'Retencion', 0, 'C', 0, 0, "", "", TRUE, 0, FALSE, TRUE, 0, 'M');

```

```

$pdf->MultiCell($w[3], 6, "$" . number_format($RETENCION, 2), 1, 'R', 0, 0, "", "", TRUE, 0, FALSE, TRUE, 0, 'M');
$pdf->Ln();

```

```

$pdf->MultiCell(130, 6, "* Favor de contactar al Comprador si usted tiene alguna duda acerca de esta Orden de Compra.", 0, 'M', 0, 0);
$pdf->MultiCell($w[2], 6, 'Total', 0, 'C', 0, 0, "", "", TRUE, 0, FALSE, TRUE, 0, 'M');
$pdf->MultiCell($w[3], 6, "$" . number_format($TOTAL, 2), 1, 'R', 0, 0, "", "", TRUE, 0, FALSE, TRUE, 0, 'M');
$pdf->Ln();
$pdf->Ln();

```

```

$pdf->MultiCell(180, 6, '* Cualquier cambio o excepcion debera ser autorizado por el Departamento de Compras.', 0, 'M', 0, 0);
$pdf->Ln();

```

```

$pdf->MultiCell(180, 6, '* Elaborar una factura por cada orden de compra.', 0, 'M', 0, 0);
$pdf->Ln();
$pdf->Ln();

```

```

$pdf->MultiCell(180, 6, 'Aprobado por: ' . $APROBADOR . ' (' . $FECHA_APROBACION . ')', 0, 'M', 0, 0);
$pdf->Ln();

```

```

// Genera y envía el PDF al navegador
$pdf->Output($PO . '.pdf', 'I');
}

```

```

function menu_retroceder(){
// Carga la vista del menú de opciones para retroceder procesos
$this->load->view('header');
$this->load->view('ordenes_compra/retroceder_opciones');
}

```

```

function retroceder_qr(){
// Verifica si el usuario tiene privilegios para retroceder QR
if($this->session->privilegios['retroceder_qr'])
{

```

```

        // Si tiene privilegios, carga la vista correspondiente
        $this->load->view('header');
        $this->load->view('ordenes_compra/retroceder_qr');
    }
    else
    {
        // Si no tiene privilegios, redirige al inicio
        redirect(base_url('inicio'));
    }
}

function recibir_pr(){
    // Verifica si el usuario tiene privilegios para retroceder QR o PO
    if($this->session->privilegios['retroceder_qr'] || $this->session->privilegios['retroceder_po'])
    {
        // Carga la vista para recibir PR
        $this->load->view('header');
        $this->load->view('ordenes_compra/recibir_pr');
    }
    else
    {
        // Redirige al inicio si no tiene los privilegios requeridos
        redirect(base_url('inicio'));
    }
}

function retroceder_po(){
    // Verifica si el usuario tiene privilegios para retroceder PO
    if($this->session->privilegios['retroceder_po'])
    {
        // Carga la vista correspondiente
        $this->load->view('header');
        $this->load->view('ordenes_compra/retroceder_po');
    }
    else
    {
        // Si no tiene privilegios, redirige al inicio
        redirect(base_url('inicio'));
    }
}

```



```

    }
}

function historial(){
    // Carga la vista para mostrar el historial general de empresas proveedoras
    $this->load->view('header');
    $this->load->view('ordenes_compra/empresas_historial');
}

function historial_po(){
    // Recibe el ID del proveedor enviado por POST
    $data['id_proveedor'] = $this->input->post('id');

    // Carga la vista del historial de órdenes de compra del proveedor
    $this->load->view('header');
    $this->load->view('ordenes_compra/historial_po', $data);
}

function calendario_seguimiento(){
    // Obtiene la lista de usuarios del área de compras
    $data['usuario'] = $this->compras_model->usuariosCompras();

    // Carga la vista con el calendario de seguimiento
    $this->load->view('header');
    $this->load->view('ordenes_compra/calendario_seguimiento', $data);
}

function ajax_getPRs(){
    // Se obtienen parámetros enviados por POST
    $texto = $this->input->post('texto');
    $parametro = $this->input->post('parametro');
    $id_proveedor = $this->input->post('id_proveedor');
    $tipo = $this->input->post('tipo');
    $moneda = $this->input->post('moneda');

    // Consulta principal para obtener PRs aprobados relacionados con un proveedor
    $query = "SELECT PR.id, E.id as IdProv, E.nombre as Prov, PR.cantidad, PR.tipo,
    ifnull(JSON_UNQUOTE(PR.atributos->'$.modelo'),'N/A') as Modelo, PR.descripcion, PR.importe,

```

```
PR.moneda, PR.estatus from prs PR inner join qr_proveedores QR_p on PR.qr_proveedor =
QR_p.id inner join empresas E on QR_p.empresa = E.id where 1=1 ";
```

```
// Filtra por proveedor, moneda y tipo si se especifica un proveedor
```

```
if($id_proveedor > 0)
```

```
{
```

```
    $query .= " and E.id = '$id_proveedor' and PR.moneda = '$moneda' and PR.tipo = '$tipo'";
```

```
}
```

```
$query .= " and PR.estatus = 'APROBADO'";
```

```
// Filtra según el tipo de búsqueda: por folio, proveedor o contenido
```

```
if(!empty($texto))
```

```
{
```

```
    if($parametro == "folio")
```

```
    {
```

```
        $query .= " and PR.id = '$texto'";
```

```
    }
```

```
    if($parametro == "proveedor")
```

```
    {
```

```
        $query .= " having Prov like '%$texto%'";
```

```
    }
```

```
    if($parametro == "contenido")
```

```
    {
```

```
        $query .= " and (PR.descripcion like '%$texto%' or UPPER(PR.atributos->'$.marca') like
UPPER('%$texto%') or UPPER(PR.atributos->'$.modelo') like UPPER('%$texto%') )";
```

```
    }
```

```
}
```

```
$res = $this->Conexion->consultar($query);
```

```
if($res)
```

```
{
```

```
    echo json_encode($res);
```

```
}
```

```
else
```

```
{
```

```
    echo "";
```

```
}
```

```

}

function ajax_setTempPO(){
$pr_aprob = []; // Lista de PRs aprobadas
$pr_rech = []; // Lista de PRs rechazadas

// Se obtienen datos del formulario
$prs = json_decode($this->input->post('prs'));
$proveedor = $this->input->post('id_prov');
$moneda = $this->input->post('moneda');
$tipo = $this->input->post('tipo');

// Se construye la consulta con los IDs de PRs recibidos
$query = "SELECT * from prs where 1 != 1";
foreach ($prs as $elem) {
    $query .= " or id ='$elem'";
}

$res = $this->Conexion->consultar($query);
foreach ($res as $elem) {
    if($elem->estatus == "APROBADO")
    {
        // Se actualiza el estatus de la PR a "EN SELECCION"
        $data['estatus'] = "EN SELECCION";
        $where['id'] = $elem->id;
        $this->Conexion->modificar('prs', $data, null, $where);

        // Se registra el cambio en la bitácora
        $bitacora['pr']=intval($elem->id);
        $bitacora['user']=$this->session->id;
        $bitacora['estatus']="EN SELECCION";
        $this->compras_model->estatusPR($bitacora);

        array_push($pr_aprob, $elem->id); // Se añade a la lista de aprobadas
    }
    else
    {
        array_push($pr_rech, $elem->id); // Se añade a la lista de rechazadas
    }
}

```

```

    }
}

// Se genera un registro temporal para construir la PO
$tempid = uniqid();
$data2['idtemp'] = $tempid;
$data2['usuario'] = $this->session->id;
$data2['proveedor'] = $proveedor;
$data2['moneda'] = $moneda;
$data2['tipo'] = $tipo;
$data2['prs'] = json_encode($pr_aprob);
$data2['prs_rechazados'] = json_encode($pr_rech);
$funciones['fecha'] = 'CURRENT_TIMESTAMP()';

$this->Conexion->insertar('ordenes_compra_temp', $data2, $funciones);

echo $tempid; // Se devuelve el ID temporal generado
}

function ajax_getTempPO(){
    $prs = json_decode($this->input->post('prs')); // Se obtienen los IDs de PRs seleccionadas

    // Se construye la consulta para recuperar los datos de cada PR
    $query = "SELECT PR.*, ifnull(JSON_UNQUOTE(PR.atributos->'$.modelo'), '') as Modelo,
ifnull(JSON_UNQUOTE(PR.atributos->'$.marca'), '') as Marca,
ifnull(JSON_UNQUOTE(PR.atributos->'$.serie'), '') as Serie, QRP.costos, QRP.factor, P.entrega from
prs PR inner join qr_proveedores QRP on PR.qr_proveedor = QRP.id inner join proveedores P on
P.empresa = QRP.empresa where 1 != 1";
    foreach ($prs as $elem) {
        $query .= " or PR.id = '$elem'";
    }

    $res = $this->Conexion->consultar($query);

    if($res)
    {
        echo json_encode($res);
    }
}

```

```

else
{
    echo "";
}
}

function ajax_saveTempPO(){
    $idtemp = $this->input->post('idtemp'); // ID del registro temporal
    $prs_costos = $this->input->post('prs_costos'); // Costos asignados por PR

    // Se guardan los costos en la tabla temporal
    $data['prs_costos'] = $prs_costos;
    $where['idtemp'] = $idtemp;
    $this->Conexion->modificar('ordenes_compra_temp', $data, null, $where);
    echo "1";
}

function ajax_getPosConstruccion(){
    $user = $this->session->id; // ID del usuario en sesión

    // Se recuperan todas las órdenes en construcción del usuario
    $query = "SELECT CT.*, E.nombre as Proveedor FROM ordenes_compra_temp CT inner join
empresas E on E.id = CT.proveedor where CT.usuario = '$user'";
    $res = $this->Conexion->consultar($query);

    if($res)
    {
        echo json_encode($res);
    }
}

function ajax_generarPO(){
    $this->load->model('compras_model');
    ini_set('display_errors', 0);

    $moneda = $this->input->post('moneda');
    $tipo = $this->input->post('tipo');

```

```

$usd = 1;

// Si la moneda es USD, se obtiene el tipo de cambio actual
if($moneda == "USD")
{
    $usd = $this->aos_funciones->getUSD()[0];
}

// Se decodifican datos recibidos vía POST
$datos = json_decode($this->input->post('datos'), TRUE);
$idtemp = $this->input->post('idtemp');
$prs = $this->input->post('prs');
$id_prov = $this->input->post('id_prov');
$prioridad = 'NORMAL';
$entrega = $this->input->post('entrega');

// Datos base para la orden de compra
$data['usuario'] = $this->session->id;
$data['prioridad'] = $prioridad;
$data['proveedor'] = $id_prov;
$data['tipo'] = $tipo;
$data['contacto'] = 0;
$data['prs'] = $prs;
$data['moneda'] = $moneda;
$data['estatus'] = 'EN PROCESO';
$data['billing_address'] = '';
$data['shipping_address'] = '';
$data['entrega'] = $entrega;
$data['metodo_pago'] = 0;
$data['conceptos'] = '';
$data['subtotal'] = 0;
$data['descuento'] = '0';
$data['impuesto'] = '0';
$data['impuesto_nombre'] = 'Exento (0.00%)';
$data['impuesto_factor'] = '0.00';
$data['total'] = 0;
$data['recurso'] = 'PENDIENTE';
$data['rma'] = '';

```

```

$data['numero_confirmacion'] = '';
$data['tipo_cambio'] = $usd;
$data['retencion'] = 0;
$data['ivaRet'] = 0;
$data['publish'] = 1;

$functions['fecha'] = 'CURRENT_TIMESTAMP()';

// Inserta la orden de compra principal
$po_id = $this->Conexion->insertar('ordenes_compra', $data, $functions);

// Registra el estatus de la PO en la bitácora
$dato['po'] = intval($po_id);
$dato['user'] = $this->session->id;
$dato['estatus'] = 'EN PROCESO';
$this->compras_model->estatusPO($dato);

// Crea registro vacío para evidencias de la PO y elimina temporal
$this->Conexion->insertar('ordenes_compra_evidencias', array('po' => $po_id));
$this->Conexion->eliminar('ordenes_compra_temp', array('idtemp' => $idtemp));

// Se insertan los conceptos seleccionados como parte de la PO
foreach ($datos as $key => $value) {
    $data2['usuario'] = $this->session->id;
    $data2['po'] = $po_id;
    $data2['pr'] = $value[0];
    $data2['cantidad'] = $value[1];
    $data2['precio_unitario'] = $value[2];
    $data2['importe'] = $value[3];
    $data2['costos'] = json_encode($value[4]);

    $this->Conexion->insertar('ordenes_compra_conceptos', $data2, null);
    $this->Conexion->modificar('prs', array('estatus' => 'EN PO'), null, array('id' => $value[0]));

    // Registra el cambio de estatus del PR en la bitácora
    $bitacora['pr'] = intval($value[0]);
    $bitacora['user'] = $this->session->id;
    $bitacora['estatus'] = "EN PO";

```

```

        $this->compras_model->estatusPR($bitacora);
    }

    echo $po_id;
}

function ajax_agregarAPO(){
    $datos = json_decode($this->input->post('datos'), TRUE);
    $id_po = $this->input->post('id_po');
    $idtemp = $this->input->post('idtemp');

    // Verifica que la PO esté en estatus válido para agregar conceptos
    $res = $this->Conexion->consultar("SELECT estatus from ordenes_compra where id='$id_po'",
    TRUE);

    if($res->estatus == 'EN PROCESO')
    {
        foreach ($datos as $key => $value) {
            // Inserta cada concepto adicional en la PO existente
            $data2['usuario'] = $this->session->id;
            $data2['po'] = $id_po;
            $data2['pr'] = $value[0];
            $data2['cantidad'] = $value[1];
            $data2['precio_unitario'] = $value[2];
            $data2['importe'] = $value[3];
            $data2['costos'] = json_encode($value[4]);

            $this->Conexion->insertar('ordenes_compra_conceptos', $data2, null);

            // Actualiza el estatus del PR a "EN PO"
            $this->Conexion->modificar('prs', array('estatus' => 'EN PO'), null, array('id' =>
            $value[0]));
        }

        // Elimina el registro temporal ya usado
        $this->Conexion->eliminar('ordenes_compra_temp', array('idtemp' => $idtemp));

        echo "1";
    }
}

```



```

    }
    else
    {
        echo "";
    }
}

```

```

function ajax_getShippingAddress(){
    // Devuelve las direcciones de envío registradas
    $res = $this->Conexion->get('shipping_address');
    if($res)
    {
        echo json_encode($res);
    }
    else{
        echo "";
    }
}

```

```

function ajax_getBillingAddress(){
    // Obtiene todas las direcciones de facturación desde la base de datos
    $res = $this->Conexion->get('billing_address');
    if($res)
    {
        echo json_encode($res);
    }
    else{
        echo "";
    }
}

```

```

/*
    /////////////////////////////////// FUNCIONES DE PO ///////////////////////////////////
*/

```

```

function ajax_getPOs(){
    $archivo = $this->input->post('archivo');

```

```
// Consulta principal para obtener órdenes de compra junto con información de usuario,
proveedor y contacto
$query = "SELECT PO.*, concat(U.nombre, ' ', U.paterno) as User, E.nombre as Prov,
ifnull(EC.nombre,'NO DEFINIDO') as Contact from ordenes_compra PO left join usuarios U on
U.id = PO.usuario left join empresas E on E.id = PO.proveedor left join empresas_contactos EC
on EC.id = PO.contacto where 1 = 1";
```

```
// Filtro por proveedor (si se envía desde el frontend)
if(isset($_POST['proveedor'])){
    $proveedor = $_this->input->post('proveedor');
    $query .= " and PO.proveedor = '$proveedor'";
}
```

```
// Filtro por moneda
if(isset($_POST['moneda'])){
    $moneda = $_this->input->post('moneda');
    $query .= " and PO.moneda = '$moneda'";
}
```

```
// Filtro por tipo de orden
if(isset($_POST['tipo'])){
    $tipo = $_this->input->post('tipo');
    $query .= " and PO.tipo = '$tipo'";
}
```

```
// Filtro por estatus
if(isset($_POST['estatus'])){
    $estatus = $_this->input->post('estatus');
```

```
// Caso especial si el estatus es "RECIBIDA TOTAL", se incluye también "RECIBIDA"
if ($estatus == 'RECIBIDA TOTAL') {
    $estatus = $estatus."") OR (PO.estatus = 'RECIBIDA';
}
```

```
if($estatus == "TODO") {
    // Cuando se requiere todo excepto vacíos
    $query .= " and (PO.estatus != ' ')";
}
```

```

} else {
    // Estatus específico
    $query .= " and (PO.estatus = '$estatus)";

    // Filtro por proveedor (duplicado por lógica interna de algunas vistas)
    if(isset($_POST['proveedor'])){
        $proveedor = $this->input->post('proveedor');
        $query .= " and PO.proveedor = '$proveedor'";
    }

    // Filtro por prioridad (pueden ser múltiples)
    if(isset($_POST['prioridad'])){
        $prioridad = json_decode($this->input->post('prioridad'));
        if(count($prioridad) > 0) {
            $query .= " and ( 1 = 0 ";
            foreach ($prioridad as $key => $value) {
                $query .= " or PO.prioridad = '$value'";
            }
            $query .= " )";
        }
    }

    // Solo órdenes del último año
    $query .= " and (PO.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) ";
}

}

// Búsqueda general por texto, con diferentes parámetros
if(isset($_POST['texto'])){
    $texto = $this->input->post('texto');
    $parametro = $this->input->post('parametro');
    if(!empty($texto)) {
        if($parametro == "folio") {
            $query .= " and PO.id = '$texto'";
        }
        if($parametro == "usuario") {
            $query .= " having User like '%$texto%'";
        }
    }
}

```

```

        if($parametro == "proveedor") {
            $query .= " having Prov like '%$texto%'";
        }
        if($parametro == "contenido") {
            $query .= " and UPPER(conceptos) like '%".strtoupper($texto)."%";
        }
    }
}

// Solo órdenes publicadas
$query .= " and PO.publish =1 ";

// Condicional para mostrar solo órdenes recientes si no se solicita archivo completo
if ($archivo != 1) {
    $query .= " AND PO.fecha > '2022-01-01 00:00:00' ";
}

// Ordena las órdenes por ID descendente (más recientes primero)
$query .= " order by PO.id desc";

// Ejecuta la consulta y responde con JSON si hay resultados
$res = $this->Conexion->consultar($query);
if($res){
    echo json_encode($res);
}
}

function ajax_getPO(){
    $id = $this->input->post('id');
    $revision = $this->input->post('revision');

    // Construcción del subquery para obtener los conceptos de la última revisión o revisión
    // específica
    if (is_null($revision)) {
        $queryrev = "(SELECT conceptos from po_revisiones WHERE po='$id' ORDER BY id DESC
        LIMIT 1) as concepto";
    } else if (!is_null($revision)) {

```

```

    $queryrev = "(SELECT conceptos from po_revisiones WHERE po='$id' and rev='$revision'
ORDER BY id DESC LIMIT 1) as concepto";
}

```

```

// Consulta principal para obtener la orden de compra, incluyendo usuario, aprobador,
proveedor, contacto, método de pago y conceptos de revisión

```

```

$query = "SELECT PO.*, concat(U.nombre, ' ', U.paterno) as User, U.correo UserMail,
concat(UA.nombre, ' ', UA.paterno) as UserA, P.valResico, E.nombre as Prov, P.rma_requerido,
ifnull(EC.nombre,'NO DEFINIDO') as Contact, EC.puesto, EC.correo, ifnull(MP.nombre,'NO
DEFINIDO') as MetodoPago, ifnull(MP.tipo,'NO DEFINIDO') as TipoMetodoPago, ".$queryrev.",
(SELECT rev from po_revisiones WHERE po='$id' ORDER BY id DESC LIMIT 1) as UltRev from
ordenes_compra PO left join usuarios UA on UA.id = PO.aprobador inner join usuarios U on U.id
= PO.usuario inner join empresas E on E.id = PO.proveedor inner join proveedores P on
P.empresa = E.id left join empresas_contactos EC on EC.id = PO.contacto left join
empresa_metodos_pago MP on MP.id = PO.metodo_pago where 1 = 1";

```

```

// Filtro específico por ID de la orden de compra

```

```

$query .= " and PO.id = '$id'";

```

```

// Ejecuta la consulta y devuelve el resultado en formato JSON

```

```

$res = $this->Conexion->consultar($query, TRUE);

```

```

if($res){
    echo json_encode($res);
}
else {
    echo "";
}
}

```

```

function ajax_getPRsPO(){

```

```

    $po = $this->input->post('id');

```

```

// Consulta para obtener los PRs ligados a una orden de compra, con su información de
usuario, tipo, descripción, etc.

```

```

$query = "SELECT OCC.pr, concat(U.nombre, ' ', U.paterno) as User, PR.tipo, PR.subtipo,
PR.cantidad, PR.descripcion, PR.estatus, PR.recibida from ordenes_compra_conceptos OCC
inner join prs PR on PR.id = OCC.pr inner join usuarios U on U.id = PR.usuario where OCC.po =
'$po'";

```

```

$res = $this->Conexion->consultar($query);
if($res){
    echo json_encode($res);
}
else {
    echo "";
}
}

function ajax_getConceptosPO(){
    $po = $this->input->post('id');

    // Consulta para obtener todos los conceptos (líneas de detalle) de una orden de compra
    // específica
    $query = "SELECT OCC.* from ordenes_compra_conceptos OCC where OCC.po = '$po'";

    $res = $this->Conexion->consultar($query);
    if($res){
        echo json_encode($res);
    }
    else {
        echo "";
    }
}

function ajax_getConceptosPO_fromPRS(){
    $po = $this->input->post('id');
    $prs = $this->input->post('prs');
    $prs_act = $this->input->post('prs_actuales');

    // Se actualiza el campo 'prs' de la orden de compra con los PRs actuales seleccionados
    $this->Conexion->modificar('ordenes_compra', array('prs' => $prs_act), null, array('id' =>
    $po));

    $prs = json_decode($prs);

```

```

// Consulta para obtener los conceptos de la orden de compra que coinciden con los PRs
seleccionados
$query = "SELECT OCC.* from ordenes_compra_conceptos OCC where OCC.po = '$po' and ( 1
!= 1";
foreach ($prs as $value) {
    $query .= " or OCC.pr = '$value'";
}
$query .= " )";

$res = $this->Conexion->consultar($query);
if($res){
    echo json_encode($res);
}
else {
    echo "";
}
}

function ajax_solicitarAprobacionPO(){
    $this->load->library('recursos_funciones');

    $id = $this->input->post('id');
    $shipping_a = $this->input->post('shipping_a');
    $billing_a = $this->input->post('billing_a');
    $contacto = $this->input->post('contacto');
    $prioridad = 'NORMAL';
    $metodo_pago = $this->input->post('metodo_pago');
    $rma = $this->input->post('rma');
    $conceptos = $this->input->post('conceptos');
    $estatus = $this->input->post('estatus');

    $subtotal = $this->input->post('subtotal');
    $descuento = $this->input->post('descuento');
    $impuesto = $this->input->post('impuesto');
    $impuesto_nombre = $this->input->post('impuesto_nombre');
    $impuesto_factor = $this->input->post('impuesto_factor');
    $total = $this->input->post('total');

```

```

$retencion = $this->input->post('retencion');
$ivaRet = $this->input->post('ivaRet');

// Datos adicionales para notificación por correo
$nombreProveedor = $this->input->post('nombre_proveedor');
$nombreContacto = $this->input->post('nombre_contacto');

$recurso = $this->input->post('recurso');
$fecha_cobro = $this->input->post('fecha_cobro');

// Se arma el arreglo con los datos actualizados de la orden de compra
$data['shipping_address'] = $shipping_a;
$data['billing_address'] = $billing_a;
$data['contacto'] = $contacto;
$data['conceptos'] = $conceptos;
$data['metodo_pago'] = $metodo_pago;
$data['rma'] = $rma;
$data['estatus'] = $estatus;
$data['subtotal'] = $subtotal;
$data['descuento'] = $descuento;
$data['impuesto'] = $impuesto;
$data['impuesto_nombre'] = $impuesto_nombre;
$data['impuesto_factor'] = $impuesto_factor;
$data['total'] = $total;
$data['retencion'] = $retencion;
$data['ivaRet'] = $ivaRet;

// Se registra estatus de la orden en la bitácora
$bitacora['po'] = intval($id);
$bitacora['user'] = $this->session->id;
$bitacora['estatus'] = $estatus;
$this->compras_model->estatusPO($bitacora);

// Se guarda la revisión de la orden
$rev['po'] = intval($id);
$rev['rev'] = 0;
$rev['user'] = $this->session->id;
$rev['conceptos'] = $this->input->post('conceptos');

```



```

$this->compras_model->revisionPO($rev);

$func = null;
// Si el recurso ya no está pendiente, se registra la fecha de provisión
if($recurso != "PENDIENTE"){
    $func = array('fecha_provision' => 'CURRENT_TIMESTAMP()');
}

$data['recurso'] = $recurso;
$data['fecha_cobro'] = date("Y-m-d", strtotime($fecha_cobro));
$data['publish'] = 1;

$where['id'] = $id;
// Actualiza la orden de compra con los nuevos datos
$this->Conexion->modificar('ordenes_compra', $data, $func, $where);

// Envía notificación si el método de pago activa alertas
$this->recursos_funciones->NotificacionMinimo($metodo_pago);

// Prepara datos para enviar correo de creación de orden de compra
$datos['id'] = $id;
$datos['fecha'] = date('d/m/Y h:i A');
$datos['usuario'] = $this->session->nombre;
$datos['prioridad'] = $prioridad;
$datos['proveedor'] = $nombreProveedor;
$datos['contacto'] = $nombreContacto;
$datos['total'] = $total;

// Se consulta la lista de correos a notificar
$correos = [];
$correos_a = $this->Conexion->consultar("SELECT U.correo from privilegios P inner join
usuarios U on P.usuario = U.id where P.recibirNotPO = 1 and U.activo = 1");
foreach ($correos_a as $key => $value) {
    array_push($correos, $value->correo);
}

$datos['correos'] = array_merge(array($this->session->correo), $correos);
$this->correos_po->creacionPO($datos);

```

```

    echo $id;
}

function ajax_cancelarPO(){
    $id = $this->input->post('id');

    // Marca la orden de compra como cancelada y la oculta (publish = 0)
    $this->Conexion->comando("UPDATE ordenes_compra set estatus='CANCELADA', publish = 0
where id=$id");

    // Obtiene todos los PR relacionados con esa orden de compra
    $res = $this->Conexion->consultar("SELECT distinct pr from ordenes_compra_conceptos
where po = $id");

    // Prepara la consulta para restaurar el estatus de esos PR a 'APROBADO'
    $query2 = "UPDATE prs set estatus='APROBADO' where 1 != 1";
    foreach ($res as $elem) {
        $query2 .= " or id = $elem->pr";

        // Guarda el cambio de estatus en la bitácora
        $bitacora['pr'] = intval($elem->pr);
        $bitacora['user'] = $this->session->id;
        $bitacora['estatus'] = "CANCELAR PO #".$po_id;
        $this->compras_model->estatusPR($bitacora);
    }

    // Ejecuta el update en la tabla de PRs
    $this->Conexion->comando($query2);

    echo "1";
}

function ajax_cancelarTempPO(){
    $idtemp = $this->input->post('idtemp');

    // Consulta los PRs asociados a la orden temporal

```

```

    $res = $this->Conexion->consultar("SELECT prs from ordenes_compra_temp where idtemp =
'$idtemp'", TRUE);
    $PRS = json_decode($res->prs);

    // Prepara el update para regresar esos PRs a 'APROBADO'
    $query2 = "UPDATE prs set estatus='APROBADO' where 1 != 1";
    foreach ($PRS as $elem) {
        $query2 .= " or id = $elem";

        // Registra el cambio en la bitácora
        $bitacora['pr'] = intval($elem);
        $bitacora['user'] = $this->session->id;
        $bitacora['estatus'] = "CANCELAR SELECCION PO";
        $this->compras_model->estatusPR($bitacora);
    }

    // Actualiza los PRs y elimina el registro temporal
    $this->Conexion->comando($query2);
    $this->Conexion->eliminar('ordenes_compra_temp', array('idtemp' => $idtemp));

    echo "1";
}

function ajax_getMetodos(){
    // Obtiene todos los métodos de pago disponibles para las empresas
    $res = $this->Conexion->get('empresa_metodos_pago', null);
    if($res){
        echo json_encode($res);
    }
}

function agregarComentarioPO() {
    $id = $this->input->post('id');
    $comentario = $this->input->post('comentario');
    $tags = $this->input->post('txtTags');
    $correos = explode(",", $tags); // Convierte la lista de correos separados por coma en arreglo

    // Prepara el comentario a insertar en la tabla

```

```

$data = array(
    'po' => $id,
    'usuario' => $this->session->id,
    'comentario' => $comentario,
);

// Fecha automática de registro
$funciones['fecha'] = 'current_timestamp()';

// Inserta el comentario en la base de datos
$this->Conexion->insertar('po_comentarios', $data, $funciones);

if(count($correos) > 0)
{
    $datos['id'] = $id;
    $datos['comentario'] = $comentario;
    $datos['correos'] = $correos;

    // Posible envío de notificación por correo (actualmente comentado)
    //$this->correos_pr->comentarioPR($datos); VER AQUI
}

redirect(base_url('ordenes_compra/ver_po/' . $id));
}

function ajax_setEstatusMsjPO(){
    $PO = json_decode($this->input->post('PO'));
    $id = $this->input->post('id');
    $estatus = $this->input->post('estatus');

    // Construye el comentario destacando el nuevo estatus
    $comentario_original = $this->input->post('comentario');
    $comentario = "<b><font color='red'>$estatus:</font></b> " . $comentario_original;

    $tags = $this->input->post('txtTags');
    $correos = explode(",", $tags); // Convierte los correos en arreglo

    // Actualiza el estatus de la orden de compra

```

```

$query = "UPDATE ordenes_compra set estatus=" . $estatus . " where id=" . $id . " ";
$res = $this->Conexion->comando($query);

if($res){
    // Inserta el comentario relacionado al cambio de estatus
    $data = array(
        'po' => $id,
        'usuario' => $this->session->id,
        'comentario' => $comentario,
    );
    $funciones['fecha'] = 'current_timestamp()';
    $this->Conexion->insertar('po_comentarios', $data, $funciones);

    // Prepara los datos del PO para el correo
    $PO->comentario = $comentario;
    $PO->UserA = $this->session->nombre;

    // Envía correo de rechazo o estatus por medio de la librería correos_po
    $this->correos_po->rechazarPO($PO);

    echo $comentario;
}
else{
    echo "";
}
}

function ajax_poSetEstatus(){
    $this->load->model('compras_model');

    $PO = json_decode($this->input->post('PO'));
    $id = $this->input->post('id');
    $estatus = $this->input->post('estatus');
    $prs = json_decode($this->input->post('prs'));

    $prioridad = $PO->prioridad;
    $nombreProveedor = $PO->Prov;
    $nombreContacto = $PO->Contact;

```

```

$total = $PO->total;

// Si el nuevo estatus es AUTORIZADA
if($estatus == 'AUTORIZADA')
{
    $data['aprobador'] = $this->session->id;
    $funt['fecha_aprobacion'] = 'CURRENT_TIMESTAMP()';

    $stat_pr = 'PO AUTORIZADA';
    $query = "UPDATE prs set estatus='$stat_pr' where 1 != 1";

    // Bitácora de cambio de estatus de la PO
    $bitacora['po'] = intval($id);
    $bitacora['user'] = $this->session->id;
    $bitacora['estatus'] = $stat_pr;
    $this->compras_model->estatusPO($bitacora);

    // Bitácora de cada PR vinculada y actualización en la tabla
    foreach ($prs as $value) {
        $query .= " or id ='$value'";
        $bitacoraPR['pr'] = $value;
        $bitacoraPR['user'] = $this->session->id;
        $bitacoraPR['estatus'] = $stat_pr;
        $this->compras_model->estatusPR($bitacoraPR);
    }

    $this->Conexion->comando($query);

    $PO->UserA = $this->session->nombre;
    $this->correos_po->aprobarPO($PO);
}

// Si se cancela la PO
if($estatus == 'CANCELADA')
{
    $stat_pr = 'CANCELADO';
    $query = "UPDATE prs set estatus='$stat_pr' where 1 != 1";
}

```

```

$bitacora['po'] = intval($id);
$bitacora['user'] = $this->session->id;
$bitacora['estatus'] = $stat_pr;
$this->compras_model->estatusPO($bitacora);

foreach ($prs as $value) {
    $query .= " or id = '$value'";
}

$this->Conexion->comando($query);

$PO->UserA = $this->session->nombre;
$this->correos_po->cancelarPO($PO);
}

// Si la PO pasa a estatus ORDENADA
elseif($estatus == 'ORDENADA'){
    $where['id'] = $id;
    $data['estatus'] = $estatus;
    $this->Conexion->modificar('ordenes_compra', $data, $funt, $where);

    $bitacora['po'] = intval($id);
    $bitacora['user'] = $this->session->id;
    $bitacora['estatus'] = $estatus;
    $this->compras_model->estatusPO($bitacora);

    $PO->UserA = $this->session->nombre;
    $this->correos_po->ordenarPO($PO);
}

// Cualquier otro estatus (por ejemplo CERRADA)
else{
    $bitacora['po'] = intval($id);
    $bitacora['user'] = $this->session->id;
    $bitacora['estatus'] = $estatus;
    $this->compras_model->estatusPO($bitacora);

    if ($estatus == 'CERRADA') {

```

```

        $PO->UserA = $this->session->nombre;
        $this->correos_po->cerrarPO($PO);
    }
}

// Actualiza el estatus de la PO y lo marca como pendiente de pago
$where['id'] = $id;
$data['estatus'] = $estatus;
$data['estatus_pago'] = 'PENDIENTE';
$this->Conexion->modificar('ordenes_compra', $data, $funt, $where);

echo "1";
}

function ajax_subirEvidencia(){
    $campo = $this->input->post('campo');
    $id = $this->input->post('po');

    $where['po'] = $id;
    $funciones['fecha' . $campo] = 'CURRENT_TIMESTAMP()'; // Registra la fecha del archivo
    subido
    $datos['usuario' . $campo] = $this->session->id; // Guarda el usuario que subió el archivo
    $datos['archivo' . $campo] = file_get_contents($_FILES['file']['tmp_name']); // Guarda el
    contenido binario del archivo
    $datos['nombre' . $campo] = $this->input->post('nombre') . ".pdf"; // Asigna nombre al
    archivo (termina en .pdf)

    // Si la modificación de la evidencia fue exitosa
    if ($this->Conexion->modificar('ordenes_compra_evidencias', $datos, $funciones, $where) >
0)
    {
        // Consulta para verificar si ya están ambas evidencias y si el estatus es 'RECIBIDA TOTAL'
        $qry = "SELECT e.*, po.estatus FROM ordenes_compra_evidencias e join ordenes_compra
po on e.po = po.id WHERE po = " . $id;
        $res = $this->Conexion->consultar($qry, TRUE);

        // Si ambas evidencias están presentes y el estatus es 'RECIBIDA TOTAL'

```



```

        if (!is_null($res->archivo1) && !is_null($res->archivo2) && $res->estatus == 'RECIBIDA
TOTAL') {
            $this->Conexion->comando("UPDATE ordenes_compra set estatus = 'LISTA PARA CERRAR'
where id = '$id'");
            $bitacora['po'] = intval($id);
            $bitacora['user'] = $this->session->id;
            $bitacora['estatus'] = 'LISTA PARA CERRAR';
            $this->compras_model->estatusPO($bitacora);
        }
        // Alternativa: si el archivo nombre1 inicia con "comp_fact" y el estatus es 'RECIBIDA TOTAL'
        elseif (substr($res->nombre1, 0, 9) == 'comp_fact' && $res->estatus == 'RECIBIDA TOTAL') {
            $this->Conexion->comando("UPDATE ordenes_compra set estatus = 'LISTA PARA CERRAR'
where id = '$id'");
            $bitacora['po'] = intval($id);
            $bitacora['user'] = $this->session->id;
            $bitacora['estatus'] = 'LISTA PARA CERRAR';
            $this->compras_model->estatusPO($bitacora);
        }

        echo "1";
    }
    else {
        // En caso de fallo en la subida del archivo
        trigger_error("Error al subir archivo", E_USER_ERROR);
    }

    // Redirige de vuelta a la vista de la PO
    redirect(base_url('/ordenes_compra/ver_po/' . $id));
}

```

```

function ajax_eliminarEvidencia(){
    $po = $this->input->post('po'); // ID de la orden de compra
    $campo = $this->input->post('campo'); // Campo a eliminar (1 o 2)

    $where['po'] = $po;
    $datos['fecha' . $campo] = NULL; // Elimina la fecha del archivo
    $datos['usuario' . $campo] = NULL; // Elimina el ID del usuario que subió
    $datos['archivo' . $campo] = NULL; // Elimina el contenido binario del archivo
}

```

```

$datos['nombre' . $campo] = NULL; // Elimina el nombre del archivo

// Limpia los datos del archivo correspondiente en la base
$this->Conexion->modificar('ordenes_compra_evidencias', $datos, NULL, $where);
echo "1";
}

function ajax_getEvidencialInfo(){
    $po = $this->input->post('po');

    // Obtiene la información de evidencias 1 y 2 y sus respectivos usuarios
    $query = "SELECT E.fecha1, E.fecha2, E.nombre1, E.nombre2, E.usuario1, E.usuario2,
concat(U1.nombre, ' ', U1.paterno) as User1, concat(U2.nombre, ' ', U2.paterno) as User2 from
ordenes_compra_evidencias E left join usuarios U1 on E.usuario1 = U1.id left join usuarios U2
on U2.id = E.usuario2 where E.po = '$po'";

    $res = $this->Conexion->consultar($query, TRUE);
    echo json_encode($res); // Devuelve información para mostrarla en la vista
}

function ajax_getPRsAgregadas(){
    $id = $this->input->post('id');

    // Selecciona los PRs ya agregados a la orden de compra
    $query = "SELECT OCC.pr from ordenes_compra_conceptos OCC where OCC.po = '$id'";
    $res = $this->Conexion->consultar($query);

    $prs = array();
    foreach ($res as $row) {
        array_push($prs, $row->pr); // Almacena solo los IDs de PRs
    }

    echo json_encode($prs); // Devuelve un array simple de PRs
}

function ajax_enviarPO(){
    $body = $this->input->post('body'); // Contenido del correo a enviar
    $para = $this->input->post('para'); // Dirección de correo del proveedor

```

```

$po = $this->input->post('po'); // ID de la orden de compra

$datos['id'] = $po; // Se asigna el ID de la PO al array de datos
$datos['correo_proveedor'] = $para; // Se asigna el correo del proveedor
$datos['body'] = $body; // Se asigna el cuerpo del mensaje

// Se llama al método que envía el correo al proveedor con la PO
$this->correos_po->enviarPO_proveedor($datos);
}

function ajax_recibirPO(){
    $this->load->model('compras_model');
    $id = $this->input->post('id'); // ID de la orden de compra
    $prs = json_decode($this->input->post('prs')); // Lista de PRs asociadas
    $todos = $this->input->post('todos'); // Indicador de si se reciben todas

    // Se obtiene el correo del usuario que generó la PO
    $q="SELECT u.correo from ordenes_compra po JOIN usuarios u on u.id=po.usuario WHERE
po.id = ".$id;
    $mail=$this->Conexion->consultar($q, TRUE);

    if(boolval($todos)) // Si se reciben todos los productos de la PO
    {
        // Se marca la PO como 'RECIBIDA TOTAL'
        $this->Conexion->comando("UPDATE ordenes_compra set estatus = 'RECIBIDA TOTAL'
where id = '$id'");
        $bitacora['po']=intval($id);
        $bitacora['user']=$this->session->id;
        $bitacora['estatus']='RECIBIDA TOTAL';
        $this->compras_model->estatusPO($bitacora);

        // Se registra cada PR como 'POR RECIBIR' en su bitácora
        foreach ($prs as $value) {
            $bitacoraPR['pr']=$value;
            $bitacoraPR['user']=$this->session->id;
            $bitacoraPR['estatus']='POR RECIBIR';
            $this->compras_model->estatusPR($bitacoraPR);
        }
    }
}

```

```

}

// Se verifica si ya existen ambas evidencias cargadas
$qry="SELECT * FROM ordenes_compra_evidencias WHERE po = ".$id;
$res=$this->Conexion->consultar($qry, TRUE);

// Si ambas evidencias están presentes o el nombre1 inicia con 'comp_fact', se marca como
'LISTA PARA CERRAR'
if (!is_null($res->archivo1) && !is_null($res->archivo2)) {
    $this->Conexion->comando("UPDATE ordenes_compra set estatus = 'LISTA PARA CERRAR'
where id = '$id'");
    $bitacora['po']=intval($id);
    $bitacora['user']=$this->session->id;
    $bitacora['estatus']='LISTA PARA CERRAR';
    $this->compras_model->estatusPO($bitacora);
} elseif (substr($res->nombre1,0, 9) == 'comp_fact') {
    $this->Conexion->comando("UPDATE ordenes_compra set estatus = 'LISTA PARA CERRAR'
where id = '$id'");
    $bitacora['po']=intval($id);
    $bitacora['user']=$this->session->id;
    $bitacora['estatus']='LISTA PARA CERRAR';
    $this->compras_model->estatusPO($bitacora);
}

// Se actualiza el estatus de las PRs y se registra la fecha/hora de recepción
$query = "UPDATE prs set estatus = 'POR RECIBIR', recibido = CURRENT_TIMESTAMP() where
1 != 1";
foreach ($prs as $value) {
    $query .= " or id = '$value'";
}

// Se prepara el objeto para enviar notificación por correo
$PO->id=$id;
$PO->correo= $mail->correo;
$PO->recibidor=$this->session->nombre;
$this->correos_po->recibirPO($PO);

// Se ejecuta la actualización final de los PRs

```

```

if($this->Conexion->comando($query))
{
    echo "1";
}
}

function evidencia(){
    $po = $this->input->post('po'); // ID de la orden de compra
    $f = $this->input->post('file'); // Número de campo de evidencia (1 o 2)

    // Se obtiene el nombre y contenido binario del archivo correspondiente
    $res = $this->Conexion->consultar("SELECT nombre$f as Nombre, archivo$f as Archivo from
ordenes_compra_evidencias where po = '$po'", TRUE);
    $nombre = $res->Nombre;
    $file = $res->Archivo;

    // Encabezados para forzar descarga de archivo PDF
    header('Content-Description: File Transfer');
    header('Content-type: application/pdf');
    header('Content-Disposition: attachment; filename='.$file);
    header('Content-Transfer-Encoding: binary');
    header('Expires: 0');
    header('Cache-Control: must-revalidate');
    header('Pragma: public');

    // Se envía el archivo al navegador
    echo $file;
}

function ajax_retrocederQR(){
    $id = $this->input->post('id'); // ID de la cotización (QR)
    $comentarios = $this->input->post('comentarios'); // Comentario del usuario

    // Se restablece el estatus y campos de liberación de la cotización
    $data['estatus'] = "ABIERTO";
    $data['fecha_liberacion'] = null;
    $data['liberador'] = null;

```

```

$this->Conexion->modificar('requisiciones_cotizacion', $data, null, array('id' => $id));

// Se registra el comentario en la bitácora con marca de 'RETROCEDIDO'
$data2['qr'] = $id;
$data2['usuario'] = $this->session->id;
$data2['comentario'] = "<b><font color=blue>RETROCEDIDO:</font></b> " . $comentarios;
$this->Conexion->insertar('qr_comentarios', $data2, array('fecha' =>
'CURRENT_TIMESTAMP()));
}

function ajax_retrocederPO(){
$id = $this->input->post('id'); // ID de la orden de compra (PO)
$comentarios = $this->input->post('comentarios'); // Comentario del usuario
$prs = json_decode($this->input->post('prs')); // PRs asociadas a la PO

// Se revierte la PO a estado "EN PROCESO", eliminando aprobador y fechas
$data['estatus'] = "EN PROCESO";
$data['fecha_provision'] = null;
$data['fecha_aprobacion'] = null;
$data['aprobador'] = null;
$this->Conexion->modificar('ordenes_compra', $data, null, array('id' => $id));

// Se agrega un comentario indicando el retroceso
$data2['po'] = $id;
$data2['usuario'] = $this->session->id;
$data2['comentario'] = "<b><font color=blue>RETROCEDIDO:</font></b> " . $comentarios;
$this->Conexion->insertar('po_comentarios', $data2, array('fecha' =>
'CURRENT_TIMESTAMP()));

// Se actualiza cada PR relacionada a estatus "EN PO"
foreach ($prs as $value) {
    $data3['estatus'] = "EN PO";
    $this->Conexion->modificar('prs', $data3, null, array('id' => $value));
}
}

function ajax_getRecibirPRs(){
    // Se consultan todas las PRs pendientes de recibir, junto con el nombre del usuario

```

```
$query = "SELECT PR.id, PR.fecha, PR.usuario, PR.prioridad, PR.tipo, PR.subtipo, PR.cantidad,
PR.unidad, PR.clave_unidad, PR.descripcion, PR.atributos, PR.critico, PR.destino,
PR.lugar_entrega, PR.comentarios, PR.estatus, concat(U.nombre, ' ', U.paterno) as User";
```

```
$query .= " from prs PR left join usuarios U on PR.usuario = U.id where 1 = 1 and PR.estatus =
'POR RECIBIR' order by PR.fecha desc";
```

```
$res = $this->Conexion->consultar($query);
if($res){
    echo json_encode($res);
}
else{
    echo "";
}
}
```

```
function requisitos(){
// Carga la vista principal del catálogo de requisitos
$this->load->view('header');
$this->load->view('ordenes_compra/catalogo_requisitos');
}
```

```
function ajax_getRequisitos(){
// Consulta a todos los usuarios activos que pueden crear QR internos o de venta
$res = $this->Conexion->Consultar("SELECT U.id, concat(U.nombre, ' ', U.paterno, ' ',
U.materno) as Revisor, P.crear_qr_interno as QRI, P.crear_qr_venta as QRV from usuarios U
inner join privilegios P on P.usuario = U.id where (P.crear_qr_interno = 1 or P.crear_qr_venta =
1) and U.activo = 1");
if($res){
    echo json_encode($res);
}
}
```

```
function ajax_getAprobadores(){
// Obtiene todos los usuarios con privilegio para aprobar PRs, incluyendo cuántos requisitos
tienen a su cargo
$res = $this->Conexion->Consultar("SELECT U.id, concat(U.nombre, ' ', U.paterno, ' ',
U.materno) as Aprobador, (SELECT count(R.id) from usuarios R inner join privilegios PP on
PP.usuario = R.id where (R.autorizador_compras = U.id and PP.crear_qr_interno = 1) or
```

(R.autorizador_compras_venta = U.id and PP.crear_qr_venta = 1) and R.activo = 1) as
 RequisitosACargo from usuarios U inner join privilegios P on P.usuario = U.id where
 P.aprobar_pr = 1 and U.activo = 1");

```

    if($res){
        echo json_encode($res);
    }
}

```

```

function ajax_getRequisitosACargo(){
    // Devuelve los requisitos a cargo de un aprobador específico
    $aprobador = $this->input->post("aprobador");
    $res = $this->Conexion->Consultar("SELECT U.id, concat(U.nombre, ' ', U.paterno, ' ',
    U.materno) as Requisitor from usuarios U inner join privilegios P on P.usuario = U.id where
    (U.autorizador_compras = '$aprobador' and P.crear_qr_interno = 1) or
    (U.autorizador_compras_venta = '$aprobador' and P.crear_qr_venta = 1) and U.activo = 1");
    if($res){
        echo json_encode($res);
    }
}

```

////////// H I S T O R I A L //////////

```

function ajax_getEmpresasHistorial(){
    // Obtiene empresas con al menos una orden de compra asociada, filtrando por nombre si se
    proporciona

```

```

    $texto = $this->input->post('texto');
    $parametro = $this->input->post('parametro');
    $query = "SELECT E.id, E.foto, E.nombre, E.razon_social, E.cliente, E.proveedor, E.calle,
    E.numero, E.colonia, (SELECT count(*) from ordenes_compra where proveedor = E.id) as
    CountPO from empresas E where 1 = 1";

```

```

    if($texto){
        if($parametro == "nombre"){
            $query .= " and E.nombre like '%$texto%'";
        }
    }

```

```

    $query .= " having CountPO > 0 order by E.nombre";
    $res = $this->Conexion->consultar($query);
    if($res){

```



```

        echo json_encode($res);
    }
}

////////// ACCIONES //////////

function ajax_getAcciones(){
    // Obtiene acciones registradas para una orden de compra específica (PO)
    $po = $this->input->post('po');
    $res = $this->Conexion->consultar("SELECT A.*, concat(U.nombre, ' ', U.paterno) as User from
po_acciones A inner join usuarios U on U.id = A.usuario where A.po = $po");
    if($res){
        echo json_encode($res);
    }
}

function ajax_getAcciones_calendar(){
    // Obtiene acciones del calendario, coloreadas según su estatus y fecha de cumplimiento
    $idsub = $this->input->get('idSub');
    $query = "SELECT A.*, concat('Responsable: ',U.nombre, ' ', U.paterno,' ','PO: ', A.po) as title,
A.fecha_limite as start, A.fecha_limite + interval 1 hour as end, concat(U.nombre, ' ', U.paterno)
as User,";
    $query .= " if(estatus = 'CANCELADA', 'gray', if(estatus = 'PENDIENTE' and
current_timestamp() > A.fecha_limite, 'red', if(estatus = 'PENDIENTE' and current_timestamp()
<= A.fecha_limite, '#f0ad4e', if(estatus = 'REALIZADA' and A.fecha_realizada > A.fecha_limite,
'#76A874', if(estatus = 'REALIZADA' and A.fecha_realizada <= A.fecha_limite, 'green', '')))) as
color";
    $query .= " from po_acciones A inner join usuarios U on U.id = A.usuario";
    if (!empty($idsub)){
        $query .= " where U.id='".$idsub'";
    }
    $res = $this->Conexion->consultar($query);
    if($res){
        echo json_encode($res);
    }
}

function ajax_setAccion(){

```

```

// Carga las librerías necesarias para enviar correos
$this->load->library('correos_archivos');
$this->load->library('correos');

// Decodifica los datos enviados desde el frontend
$accion = json_decode($this->input->post('data'));

// Asigna el usuario actual y estatus por defecto
$accion->usuario = $this->session->id;
$accion->estatus = "PENDIENTE";

// Dirección de correo electrónico del responsable
$correo = $this->input->post('correo');

// Define la fecha de creación automática
$func['fecha_creacion'] = 'CURRENT_TIMESTAMP()';

// Inserta la acción en la base de datos y guarda el ID generado
$id = $this->Conexion->insertar('po_acciones', $accion, $func);

// Formateo de datos para crear el archivo .ics del evento de calendario
$fecha = strtotime($accion->fecha_limite);
$name = "PO : ".$accion->po;
$start = date('Ymd', $fecha) . 'T' . date('His', $fecha);
$end = date('Ymd', $fecha) . 'T' . date('His', $fecha);
$description = $accion->accion;

// Contenido del archivo ICS (evento de calendario)
$ical_content = "BEGIN:VCALENDAR
VERSION:2.0
PRODID:-//LearnPHP.co//NONSGML {$name}//EN
METHOD:REQUEST
BEGIN:VEVENT
ORGANIZER;CN=\".$this->session->nombre.\":tickets@masmetrologia.com
UID:\".date('Ymd').'T'.date('His').rand().\"-learnphp.co
DTSTAMP:\".date('Ymd').'T'.date('His').\"
DTSTART:{$start}
DTEND:{$end}

```

```
SUMMARY:{$name}  
DESCRIPTION:{$description}  
END:VEVENT  
END:VCALENDAR";
```

```
// Datos para enviar el correo con el archivo ICS como adjunto  
$datos['nombre'] = "Seguimiento PO-".$saccion->po.".ics";  
$datos['cuerpo'] = "Esto es un evento de seguimiento para la orden de compra - ".$saccion->po;  
$datos['cal'] = $ical_content;  
$datos['correo'] = $this->session->correo;  
$datos['accion'] = $description;  
$datos['contacto'] = $correo;  
$datos['correoResponsable'] = $this->session->correo;  
  
// Envía el evento por correo  
$this->correos_archivos->evento_cotizaciones($datos);  
  
// Retorna el ID si la acción fue registrada correctamente  
if($id > 0){  
    echo $id;  
}  
}  
  
function eventoICS($data)  
{  
    // Consulta los datos de la acción específica por su ID  
    $query = "SELECT * from po_acciones where id = ".$data;  
    $res = $this->Conexion->consultar($query, TRUE);  
  
    // Habilita la visualización de errores para depuración  
    error_reporting(E_ALL);  
    ini_set('display_errors', '1');  
  
    // Extrae y formatea la fecha para el archivo ICS  
    $fecha = strtotime($res->fecha_limite);  
    $name = "PO : ".$res->po;  
    $start = date('Ymd', $fecha) . 'T' . date('His', $fecha);
```

```

$end = date('Ymd', $fecha) . 'T' . date('His', $fecha);
$description = $res->accion;

// Genera un slug simplificado del nombre (aunque no se usa)
$slug = strtolower(str_replace(array(' ', '"', '.'), array('_', ' ', ''), $name));

// Generación del contenido del archivo ICS para el evento
echo "BEGIN:VCALENDAR\n";
echo "VERSION:2.0\n";
echo "PRODID:-//LearnPHP.co//NONSGML {$name}//EN\n";
echo "METHOD:REQUEST\n";
echo "BEGIN:VEVENT\n";
echo "ORGANIZER;CN=\".$this->session->nombre.\":tickets@masmetrologia.com\n";
echo "UID:".date('Ymd').'T'.date('His')."-".rand()."-learnphp.co\n";
echo "DTSTAMP:".date('Ymd').'T'.date('His')."\n";
echo "DTSTART:{$start}\n";
echo "DTEND:{$end}\n";
echo "SUMMARY:{$name}\n";
echo "DESCRIPTION: {$description}\n";
echo "END:VEVENT\n";
echo "END:VCALENDAR\n";
// Cabeceras para forzar la descarga como archivo de calendario
header("Content-type: text/calendar; charset=utf-8");
header("Content-Disposition: attachment; filename=Seguimiento PO - ".$res->po.".ics");
}

```

```

function ajax_updateAccion(){
// Decodifica los datos enviados desde el frontend
$accion = json_decode($this->input->post('data'));

// Actualiza la acción en la base de datos según su ID
$id = $this->Conexion->modificar('po_acciones', $accion, null, array('id' => $accion->id));
}

```

```

function ajax_setAccionRealizada(){
// Decodifica los datos recibidos

```

```

$accion = json_decode($this->input->post('data'));

// Marca la acción como realizada con fecha actual
$func['fecha_realizada'] = 'CURRENT_TIMESTAMP()';

// Actualiza en base de datos el estatus y fecha de realización
$id = $this->Conexion->modificar('po_acciones', $accion, $func, array('id' => $accion->id));
}

function ajax_setAccionComentario(){
// Recibe un comentario en formato JSON y decodifica
$comment = json_decode($this->input->post('data'));

// Asigna el ID del usuario actual desde la sesión
$comment->usuario = $this->session->id;
$func['fecha'] = 'CURRENT_TIMESTAMP()';
$id = $this->Conexion->insertar('po_acciones_comentarios', $comment, $func);

if($id > 0){
    echo $id;
}
}

function ajax_getAccionComentarios(){
// Obtiene el ID de la acción a consultar
$accion = $this->input->post('accion');
// Consulta los comentarios asociados a esa acción, incluyendo nombre del usuario
$res = $this->Conexion->consultar("SELECT C.*, concat(U.nombre, ' ', U.paterno, ' ',
U.materno) as User from po_acciones_comentarios C inner join usuarios U on U.id = C.usuario
where C.accion = $accion");
if($res)
{
    echo json_encode($res);
}
}

function ajax_setNoConfirmacion(){

```

```

// Recoge datos del número de confirmación desde POST

$data['id'] = $this->input->post('po');
$data['numero_confirmacion'] = $this->input->post('numero');

$this->Conexion->modificar('ordenes_compra', $data, null, array('id' => $data['id']));
}

function exportarPO()
{
    // Consulta para obtener las órdenes de compra del último año, con datos del requisitor,
    proveedor y contacto
    $query = 'SELECT PO.id as PO, PO.fecha as Fecha,concat(U.nombre, " ", U.paterno) as
    Requisitor, E.nombre as Proveedor, ifnull(EC.nombre,"NO DEFINIDO") as Contacto, PO.entrega
    as Entrega,PO.estatus as Estatus from ordenes_compra PO inner join usuarios U on U.id =
    PO.usuario inner join empresas E on E.id = PO.proveedor left join empresas_contactos EC on
    EC.id = PO.contacto where 1 = 1 and (PO.fecha > (CURRENT_DATE() - INTERVAL 1 YEAR)) order
    by PO.id desc';

    // Ejecuta la consulta y convierte el resultado en arreglo
    $result = $this->db->query($query)->result_array();

    // Genera un timestamp para el nombre del archivo
    $timestamp = date('m/d/Y', time());

    // Define el nombre del archivo Excel
    $filename = 'PO_' . $timestamp . '.xls';

    // Encabezados para exportación como archivo Excel
    header("Content-Type: application/vnd.ms-excel");
    header("Content-Disposition: attachment; filename=\"$filename\"");

    $isPrintHeader = false;

    // Recorre los resultados para imprimir encabezado y datos en formato tabulado
    foreach ($result as $row) {
        if (!$isPrintHeader) {
            echo implode("\t", array_keys($row)) . "\n"; // Encabezados

```

```

        $isPrintHeader = true;
    }
    echo implode("\t", array_values($row)) . "\n"; // Datos por fila
}

exit; // Finaliza la ejecución y descarga el archivo
}

function ajax_getNombresUsuarios(){
    $ids = $this->input->post('POAc'); // Se obtiene el ID de la orden de compra desde el POST
    (variable 'POAc')

    $query = "SELECT concat(u.nombre, ' ', u.paterno) as user, b.fecha, b.estatus from bitacora_po
    b JOIN usuarios u on b.user=u.id where po='". $ids; // Consulta para obtener nombre del
    usuario, fecha y estatus de la bitácora de la PO

    $res = $this->Conexion->consultar($query); // Ejecuta la consulta
    if($res){ echo json_encode($res); } // Si hay resultados, los convierte a JSON y los devuelve
}

function agregarRastreo() {
    $id = $this->input->post('id'); // ID de la orden de compra
    $tracking = $this->input->post('txtTracking'); // Número de rastreo ingresado
    $links = $this->input->post('txtLinks'); // Link(s) de seguimiento

    $data = array('po' => $id, 'usuario' => $this->session->id, 'trackNum' => $tracking, 'enlace' =>
    $links); // Datos a insertar en la tabla de rastreo
    $funciones['fecha'] = 'current_timestamp()'; // Función SQL para registrar fecha actual

    $this->Conexion->insertar('po_rastros', $data, $funciones); // Inserta el rastreo en la tabla
    correspondiente

    redirect(base_url('ordenes_compra/ver_po/' . $id)); // Redirige al detalle de la orden de
    compra
}

function ajax_generarPOTemp(){

```

```

$this->load->model('compras_model');
ini_set('display_errors', 0); // Desactiva la visualización de errores en pantalla

$moneda = $this->input->post('moneda'); // Moneda seleccionada (MXN o USD)
$tipo = $this->input->post('tipo'); // Tipo de orden (compra/servicio/etc.)
$usd = 1; // Valor por defecto del tipo de cambio

if($moneda == "USD"){ $usd = $this->aos_funciones->getUSD()[0]; } // Si es USD, se obtiene
tipo de cambio actual

$datos = json_decode($this->input->post('datos'), TRUE); // Conceptos en formato JSON
(arreglo)
$idtemp = $this->input->post('idtemp'); // ID temporal (no se usa aquí, pero puede usarse
para limpieza posterior)
$prs = $this->input->post('prs'); // PRs asociadas a la orden
$id_prov = $this->input->post('id_prov'); // ID del proveedor
$prioridad = 'NORMAL'; // Prioridad fija por defecto
$entrega = $this->input->post('entrega'); // Fecha estimada de entrega

// Datos base para crear la PO temporal
$data['usuario'] = $this->session->id;
$data['prioridad'] = $prioridad;
$data['proveedor'] = $id_prov;
$data['tipo'] = $tipo;
$data['contacto'] = 0;
$data['prs'] = $prs;
$data['moneda'] = $moneda;
$data['estatus'] = 'EN PROCESO';
$data['billing_address'] = "";
$data['shipping_address'] = "";
$data['entrega'] = $entrega;
$data['metodo_pago'] = 0;
$data['conceptos'] = [];
$data['subtotal'] = 0;
$data['descuento'] = '0';
$data['impuesto'] = '0';
$data['impuesto_nombre'] = 'Exento (0.00%)';
$data['impuesto_factor'] = '0.00';

```



```

$data['total'] = 0;
$data['recurso'] = 'PENDIENTE';
$data['rma'] = '';
$data['numero_confirmacion'] = '';
$data['tipo_cambio'] = $usd;
$data['retencion'] = 0;
$data['ivaRet'] = 0;

$functions['fecha'] = 'CURRENT_TIMESTAMP()'; // Fecha de creación actual

$po_id = $this->Conexion->insertar('po_temp', $data, $functions); // Inserta PO temporal en
base de datos

// Bitácora para la nueva PO generada
$dato['po']=intval($po_id);
$dato['user']=$this->session->id;
$dato['estatus']='EN PROCESO';
$this->compras_model->estatusPO($dato);

$this->Conexion->insertar('ordenes_compra_evidencias', array('po' => $po_id)); // Prepara
tabla de evidencias para la PO

// Recorre cada concepto recibido y lo registra en la base
foreach ($datos as $key => $value) {
    $data2['usuario'] = $this->session->id;
    $data2['po'] = $po_id;
    $data2['pr'] = $value[0]; // ID de PR
    $data2['cantidad'] = $value[1]; // Cantidad solicitada
    $data2['precio_unitario'] = $value[2]; // Precio unitario
    $data2['importe'] = $value[3]; // Total por concepto
    $data2['costos'] = json_encode($value[4]); // Costos asociados codificados en JSON

    $this->Conexion->insertar('ordenes_compra_conceptos', $data2, null); // Inserta cada
concepto en la tabla correspondiente
    $this->Conexion->modificar('prs', array('estatus' => 'EN PO'), null, array('id' => $value[0]));
// Actualiza PR a estado EN PO

    $bitacora['pr']=intval($value[0]);

```

```

    $bitacora['user']=$this->session->id;
    $bitacora['estatus']="EN PO #".$po_id;
    $this->compras_model->estatusPR($bitacora); // Registra movimiento en bitácora PR
}

echo $po_id; // Devuelve ID de la nueva PO temporal generada
}

function guardarPO(){
    $this->load->library('recursos_funciones');

    // ID de la orden de compra que se va a actualizar
    $id = $this->input->post('id');

    $est = $this->compras_model->getLastSt($id);
    $estatus = strval($est->estatus);
    $shipping_a = $this->input->post('shipping_a');
    $billing_a = $this->input->post('billing_a');
    $contacto = $this->input->post('contacto');
    $prioridad = 'NORMAL';
    $metodo_pago = $this->input->post('metodo_pago');
    $rma = $this->input->post('rma');
    $conceptos = $this->input->post('conceptos');

    $subtotal = $this->input->post('subtotal');
    $descuento = $this->input->post('descuento');
    $impuesto = $this->input->post('impuesto');
    $impuesto_nombre = $this->input->post('impuesto_nombre');
    $impuesto_factor = $this->input->post('impuesto_factor');
    $total = $this->input->post('total');

    $retencion = $this->input->post('retencion');
    $ivaRet = $this->input->post('ivaRet');
    ///////////
    $nombreProveedor = $this->input->post('nombre_proveedor');
    $nombreContacto = $this->input->post('nombre_contacto');

```

```

$recurso = $this->input->post('recurso');
$fecha_cobro = $this->input->post('fecha_cobro');

$data['shipping_address'] = $shipping_a;
$data['billing_address'] = $billing_a;
$data['contacto'] = $contacto;
$data['conceptos'] = $conceptos;
$data['metodo_pago'] = $metodo_pago;
$data['rma'] = $rma;
$data['estatus'] = strval($estatus);

$data['subtotal'] = $subtotal;
$data['descuento'] = $descuento;
$data['impuesto'] = $impuesto;
$data['impuesto_nombre'] = $impuesto_nombre;
$data['impuesto_factor'] = $impuesto_factor;
$data['total'] = $total;

$data['retencion'] = $retencion;
$data['ivaRet'] = $ivaRet;

// Registra el cambio de estatus en la bitácora de la PO
$bitacora['po']=intval($id);
$bitacora['user']=$this->session->id;
$bitacora['estatus']=strval($estatus);
$this->compras_model->estatusPO($bitacora);

// Se obtiene la última revisión de la PO para registrar una nueva
$res = $this->Conexion->consultar("SELECT rev from po_revisiones where po =
".intval($id)." ORDER by id DESC LIMIT 1");
$elem =null;
foreach($res as $elem){
    $r= $elem->rev;
}
$revision=intval($r)+1;
$rev['po']=intval($id);
$rev['rev']=$revision;
$rev['user']=$this->session->id;

```

```

$rev['conceptos']=$this->input->post('conceptos');
$this->compras_model->revisionPO($rev);

$func = null;
if($recurso != "PENDIENTE"){
    $func = array('fecha_provision' => 'CURRENT_TIMESTAMP()');
}
$data['recurso'] = $recurso;
$data['fecha_cobro'] = date("Y-m-d", strtotime($fecha_cobro));
$data['publish'] = 1;

$where['id'] = $id;
$this->Conexion->modificar('ordenes_compra', $data, $func, $where);
$this->recursos_funciones->NotificacionMinimo($metodo_pago);

// Se prepara la cadena de costos para ser guardada correctamente
$conc=$this->input->post('concep');
$concTemp=str_replace("[", "", $conc);
$concTemp=str_replace("]", "", $concTemp);
$concTemp=str_replace("'", "", $concTemp);
$concTemp=stripslashes($concTemp);
$conc ="{".$concTemp."}";

$dataC['costos'] = $conc;
$whereC['po'] = $id;
$funcC = null;

// Se guarda la información actualizada de la PO en la base de datos
$this->Conexion->modificar('ordenes_compra_conceptos', $dataC, $funcC, $whereC);

$datos['id'] = $id;
$datos['fecha'] = date('d/m/Y h:i A');
$datos['usuario'] = $this->session->nombre;
$datos['prioridad'] = $prioridad;
$datos['proveedor'] = $nombreProveedor;
$datos['contacto'] = $nombreContacto;
$datos['total'] = $total;

```

```

$correos = [];

// Se obtienen los correos de usuarios con privilegio para recibir notificaciones de PO
$correos_a = $this->Conexion->consultar("SELECT U.correo from privilegios P inner join
usuarios U on P.usuario = U.id where P.recibirNotPO = 1");
foreach ($correos_a as $key => $value) {
    array_push($correos, $value->correo);
}
$datos['correos'] = array_merge(array($this->session->correo), $correos);

// Se envía la notificación por correo de creación/modificación de la PO
$this->correos_po->creacionPO($datos);
echo $id;
}

function getArchivosPrs(){
    $prs = $this->input->post('prs');
    $pr = explode(',', $prs);

    foreach($pr as $elem){
        // Se limpia el valor de la PR eliminando corchetes
        $prF = str_replace("[", "", $elem);
        $prF = str_replace("]", "", $prF);

        // Se construye la consulta para obtener el archivo de la PR
        $query = "SELECT q.archivo, p.id from prs p JOIN qr_proveedores q on q.qr = p.qr WHERE
p.id = " . $prF;

        $res = $this->Conexion->consultar($query);
        if($res)
        {
            // Se devuelve la información en formato JSON
            echo json_encode($res);
        }
    }
}

function entregaParcial(){

```

```

$pr = $this->input->post('pr');
$qty = $this->input->post('qty');
$po = $this->input->post('po');

// Se obtiene la información actual de la PR
$query = 'select * from prs where id=' . $pr;
$res = $this->Conexion->consultar($query, TRUE);

// Se calcula la cantidad total recibida con lo nuevo
$cant = $qty + intval($res->recibida);

// Se valida que no se exceda la cantidad solicitada y que la cantidad ingresada no sea 0
if ($cant <= intval($res->cantidad) && $qty != 0) {
    // Se actualiza la PR como recibida parcialmente
    $data['recibida'] = $cant;
    $data['estatus'] = 'PARCIAL';
    $where['id'] = $pr;
    $res = $this->Conexion->modificar('prs', $data, null, $where);

    // Se marca la PO como recibida parcialmente
    $dataPo['estatus'] = 'RECIBIDA PARCIAL';
    $wherePo['id'] = $po;
    $res = $this->Conexion->modificar('ordenes_compra', $dataPo, null, $wherePo);

    // Se guarda bitácora para PR
    $bitacora['pr'] = $pr;
    $bitacora['user'] = $this->session->id;
    $bitacora['estatus'] = "PARCIAL : " . $qty;
    $this->compras_model->estatusPR($bitacora);

    // Se guarda bitácora para PO
    $bitacoraPo['po'] = $po;
    $bitacoraPo['user'] = $this->session->id;
    $bitacoraPo['estatus'] = "RECIBIDA PARCIAL";
    $this->compras_model->estatusPO($bitacoraPo);

    if ($res) {
        echo '1';
    }
}

```

```

    }
}
}
}

```

Ordebes_trabajo.php

```

<?php
require_once ('data/Merger/PDFMerger.php');
use PDFMerger\PDFMerger;

defined('BASEPATH') OR exit('No direct script access allowed');

class ordenes_trabajo extends CI_Controller {

    function __construct() {
        parent::__construct();

        // Carga bibliotecas necesarias para órdenes de trabajo
        $this->load->library('correos_facturacion');
        $this->load->model('MLConexion_model', 'MLConexion');
        $this->load->library('correos');
        $this->load->library('AOS_funciones');
        $this->load->library('ciqrbarcode');
    }

    // Vista principal del catálogo de órdenes de trabajo
    function index(){
        $this->load->view('header');
        $this->load->view('ordenes_trabajo/catalogo');
    }

    // Vista del catálogo de órdenes con listado de técnicos activos
    function catalogo_wo(){
        $data['tecnicos']= $this->MLConexion->consultar("SELECT * from catalogo_tecnicos where
activo = '-1'");
        $this->load->view('header');
        $this->load->view('ordenes_trabajo/catalogo_wo', $data);
    }
}

```

```

}

// Carga la vista para crear una nueva orden de trabajo
function crear_orden(){
    $this->load->view('header');
    $this->load->view('ordenes_trabajo/work_orders');
}

// Carga la vista para editar una solicitud de facturación
function editar_solicitud(){
    if(isset($_POST["id"])) {
        $id = $this->input->post('id');
    } else if ($id == 0){
        redirect(base_url('inicio'));
    }

    $data["id"] = $id;
    $data["editar"] = true;

    $this->load->view('header');
    $this->load->view('facturas/solicitud_facturas', $data);
}

// Visualiza el detalle de una orden de trabajo específica
function ver_wo($id = 0){
    if(isset($_POST["id"])) {
        $id = $this->input->post('id');
    } else if ($id == 0){
        redirect(base_url('inicio'));
    }

    $data["id"] = $id;

    // Consulta de detalles de la orden y descripción compuesta del equipo
    $query = "SELECT wo.*, wd.*, rs.folio_id, rs.Localizacion, ei.Status_Descripcion as
estatus_item, ew.Status_Descripcion as estatus_wo, concat(rs.descripcion,
if(isnull(rs.Fabricante), '', concat(' ', rs.Fabricante)), if(isnull(rs.Modelo), '', concat(' ', rs.Modelo)),
if(isnull(rs.Serie), '', concat(' Serie: ', rs.Serie)), if(isnull(rs.Equipo_ID), '', concat(' ID: ',

```



```

rs.Equipo_ID))) as CadenaDescripcion from WO_Master wo JOIN WO_Detail wd on
wo.WorkOrder_ID=wd.WorkOrder_ID JOIN rsitems rs ON rs.item_id=wd.Item_Id JOIN
tblStatusWO ei on ei.Status_id = wd.Item_Status JOIN tblStatusWO ew on
ew.Status_ID=wo.WorkOrder_Status where wo.WorkOrder_ID = ".$id;
    $data['wo_detail']=$this->MLConexion->consultar($query);

    // Consulta de información general de la orden y datos del cliente
    $query2 = "SELECT wo.*, wd.*, we.Status_Descripcion, rh.Empresa, rh.Direccion1,
rh.Contacto, rh.TelefonoContacto, rh.CelularContacto from WO_Master wo left JOIN WO_Detail
wd on wo.WorkOrder_ID=wd.WorkOrder_ID JOIN tblStatusWO we on
we.Status_ID=wo.WorkOrder_Status join rsheaders rh on rh.folio_id=wo.rs where
wo.WorkOrder_ID= ".$id;
    $data['wo']=$this->MLConexion->consultar($query2, TRUE);

    $this->load->view('header');
    $this->load->view('ordenes_trabajo/ver_wo', $data);
}

function ajax_setWO(){
    $wo = json_decode($this->input->post('wo'));
    $rs_items = json_decode($this->input->post('rs_items'));

    // Establece el estatus inicial de la orden de trabajo
    $wo->WorkOrder_Status = 1;
    $wo->correo_us = $this->session->correo;

    $res = false;

    // Inserta la orden de trabajo en WO_Master
    $res = $this->MLConexion->insertar('WO_Master', $wo);

    // Por cada ítem de RS asociado, inserta en WO_Detail y actualiza rsitems
    foreach ($rs_items as $key) {
        $wo_detail['WorkOrder_ID'] = $res;
        $wo_detail['Item_Id'] = $key->item_id;
        $wo_detail['Item_Status'] = 1;

        $this->MLConexion->insertar('WO_Detail', $wo_detail);
    }
}

```

```

    $this->MLConexion->comando("UPDATE rsitems SET FechaInWO =
CURRENT_TIMESTAMP(), WOrder_ID = ".$res." WHERE item_id = ".$key->item_id."");
}

// Prepara y envía correo de notificación de creación de WO
$mail['usuario'] = $this->session->nombre;
$mail['wo'] = $res;
$correo = [];
$correos_a = $this->Conexion->consultar("SELECT U.correo from privilegios P inner join
usuarios U on P.usuario = U.id where P.cerrar_wo = 1 and U.activo = 1");
foreach ($correos_a as $key => $value) {
    array_push($correo, $value->correo);
}
$mail['correos'] = array_merge(array($this->session->correo), $correo);
$this->correos->crear_wo($mail);

// Genera código QR con enlace para visualizar la orden de trabajo
$query = "SELECT * FROM WO_Master WHERE WorkOrder_ID=(SELECT MAX(WorkOrder_ID)
FROM WO_Master) and correo_us='".$this->session->correo.'";
$res = $this->MLConexion->consultar($query, TRUE);
$t = 'https://validacion.masmetrologia.mx/ordenes_trabajo/ordenes_trabajo/ver_wo/'.$res->WorkOrder_ID;
$qqr = 'WO_'.$res->WorkOrder_ID.'.png';
$file = 'data/wo/'.$qqr;
QRcode::png($t, $file, 'L', 10, 2);
$this->MLConexion->comando("UPDATE WO_Master SET qr = ".$qqr." WHERE
WorkOrder_ID=".$res->WorkOrder_ID);

// Respuesta al frontend si la creación fue exitosa
if($res) {
    echo "1";
}
}

function ajax_getSolicitudes(){
    $texto = $this->input->post('texto');
    $parametro = $this->input->post('parametro');
    $cliente = $this->input->post('cliente');

```

```

$estatus = $this->input->post('estatus');
$tipo = $this->input->post('tipo');
$cerradas = $this->input->post('cerradas');
$fecha1 = $this->input->post('fecha1');
$fecha2 = $this->input->post('fecha2');

$f1 = strval($fecha1).' 00:00:00';
$f2 = strval($fecha2).' 23:59:59';

// Consulta inicial: todas las órdenes de trabajo del usuario en sesión
$query = "select wo.WorkOrder_ID, wo.rs, wo.correo_us, wo.FCreacion, wo.FProgramado,
wo.FRetroalimentacion, wo.WorkOrder_Status, rs.`Nombre Corto` as Empresa, C.Cust_ID as
empresa, e.Status_Descripcion from WO_Master wo join tblStatusWO e on
e.Status_ID=wo.WorkOrder_Status JOIN rsheaders rs on rs.folio_id =wo.rs join catalogo_clientes
C on rs.Cust_ID = C.Cust_ID where correo_us = '". $this->session->correo.'"'";

// Filtro según si se desea ver solo abiertas o cerradas
if($cerradas == "0") {
    $query .= " and (wo.WorkOrder_Status != '4')";
} else {
    $query .= " and (wo.WorkOrder_Status = '4')";
}

// Filtro por número de folio
if(!empty($texto)) {
    $query .= " and rs.folio_id = ".$texto;
}

// Filtro por cliente
if(!empty($cliente) && $cliente != 0) {
    $query .= " and C.Cust_ID = '$cliente'";
}

// Filtro por estatus
if(!empty($estatus) && $estatus != 'TODO') {
    $query .= " and wo.WorkOrder_Status = '$estatus'";
}

```

```

// Filtro por rango de fechas programadas
if (!empty($fecha1) && !empty($fecha2)) {
    $query .= " and wo.FProgramado BETWEEN '" . $f1 . "' AND '" . $f2 . "'";
}

// Ordena los resultados por fecha programada más reciente
$query .= " order by FProgramado desc";

// Ejecuta la consulta y devuelve los resultados en formato JSON
$res = $this->MLConexion->consultar($query);
if($res) {
    echo json_encode($res);
}
}

function ajax_getWo(){
    $texto = $this->input->post('texto');
    $parametro = $this->input->post('parametro');
    $cliente = $this->input->post('cliente');
    $estatus = $this->input->post('estatus');
    $tecnico = $this->input->post('tecnico');
    $tipo = $this->input->post('tipo');
    $cerradas = $this->input->post('cerradas');
    $fecha1 = $this->input->post('fecha1');
    $fecha2 = $this->input->post('fecha2');

    $f1 = strval($fecha1).' 00:00:00';
    $f2 = strval($fecha2).' 23:59:59';

    // Consulta principal de órdenes de trabajo (WO), unida con estatus, cliente, técnico y
    encabezado RS
    $query = "select wo.WorkOrder_ID, wo.rs, wo.correo_us, wo.FCreacion, wo.FProgramado,
    wo.FRetroalimentacion, wo.WorkOrder_Status, rs.`Nombre Corto` as Empresa,
    e.Status_Descripcion, C.Cust_ID as empresa, ct.Nombre from WO_Master wo join tblStatusWO
    e on e.Status_ID=wo.WorkOrder_Status JOIN rsheaders rs on rs.folio_id =wo.rs join
    catalogo_tecnicos ct on ct.email_tecnico=wo.correo_us join catalogo_clientes C on rs.Cust_ID =
    C.Cust_ID where 1=1";

```

```

// Filtra por estado: abierta o cerrada
if($cerradas == "0") {
    $query .= " and (wo.WorkOrder_Status != '4')";
} else {
    $query .= " and (wo.WorkOrder_Status = '4')";
}

// Filtra por número de folio si se especifica
if(!empty($texto)) {
    $query .= " and rs.folio_id = ".$texto;
}

// Filtra por cliente si se especifica
if(!empty($cliente) && $cliente != 0) {
    $query .= " and C.Cust_ID = '$cliente'";
}

// Filtra por estatus de la orden si se especifica
if(!empty($estatus) && $estatus != 'TODO') {
    $query .= " and wo.WorkOrder_Status = '$estatus'";
}

// Filtra por técnico asignado si se especifica
if(!empty($tecnico) && $tecnico != 'TODO') {
    $query .= " and wo.correo_us = '$tecnico'";
}

// Filtra por fechas si ambos extremos están definidos
if (!empty($fecha1) && !empty($fecha2)) {
    $query .= " and wo.FProgramado BETWEEN '". $f1. "' AND '". $f2. "'";
}

// Ordena por fecha programada descendente
$query .= " order by FProgramado desc";

// Ejecuta consulta y responde con los resultados en formato JSON
$res = $this->MLConexion->consultar($query);
if($res) {

```

```

        echo json_encode($res);
    }
}

function realizado(){
    $id = $this->input->post('itemOk');
    $tipo_cal = $this->input->post('tipo_cal');
    $txtrealizado = $this->input->post('txtrealizado');

    // Marca el ítem como realizado (estatus 5), guarda tipo de calibración y marca como
    calibrado
    $this->MLConexion->comando("UPDATE WO_Detail set Item_Status = 5, tipo_cal =
    '". $tipo_cal.'" , calibrado = 'SI' where id = ".$id);

    // Obtiene detalles de la orden de trabajo para redireccionar
    $wo = $this->MLConexion->consultar("SELECT * from WO_Detail where id=".$id, true);

    // Inserta comentario indicando que el ítem fue realizado
    $res = $this->MLConexion->consultar("SELECT * from WO_Detail where id=".$id, true);
    $datos['id_wo'] = $res->WorkOrder_ID;
    $datos['comentario'] = "Item Realizado #" . $res->Item_Id . " = " . $txtrealizado;
    $datos['mail_us'] = $this->session->correo;
    $this->MLConexion->insertar('comentarios_wo', $datos);

    // Redirecciona a la vista de la orden de trabajo correspondiente
    redirect(base_url('ordenes_trabajo/ver_wo/' . $wo->WorkOrder_ID));
}

function rechazar_item(){
    $id = $this->input->post('item');
    $txtrechazar = $this->input->post('txtrechazar');
    $motivo = $this->input->post('motivo');

    // Marca el ítem como no realizado (estatus 6), guarda motivo, desmarca calibración y limpia
    vencimiento
    $this->MLConexion->comando("UPDATE WO_Detail set Item_Status = 6, motivo =
    '". $motivo.'" , calibrado = 'NO', vencimiento = null, tipo_cal=null where id = ".$id);

```

```

// Obtiene detalles del ítem rechazado para registrar comentario y limpieza en RS
$res = $this->MLConexion->consultar("SELECT * from WO_Detail where id=".$id, true);
$datos['id_wo'] = $res->WorkOrder_ID;
$datos['comentario'] = "Item No Realizado #".$res->Item_Id." = ".$txtrechazar;
$datos['mail_us'] = $this->session->correo;
$this->MLConexion->insertar('comentarios_wo', $datos);

// Limpia asignación del ítem en RS (se libera del WO)
$this->MLConexion->comando("UPDATE rsitems set FechaInWO = null, WOrder_ID = null
where item_id = ".$res->Item_Id);

// Redirecciona nuevamente a la orden de trabajo para revisión
redirect(base_url('ordenes_trabajo/ver_wo/'.$res->WorkOrder_ID));
}

function fecha_vencimiento(){
$id = $this->input->post('itemFecha');
$vencimiento = $this->input->post('fecha');

// Actualiza la fecha de vencimiento del ítem en la orden de trabajo
$this->MLConexion->comando("UPDATE WO_Detail set vencimiento = ".$vencimiento."
where id = ".$id);

// Consulta los datos del ítem para redirigir correctamente
$wo = $this->MLConexion->consultar("SELECT * from WO_Detail where id=".$id, true);

// Redirecciona a la vista correspondiente de la orden de trabajo
redirect(base_url('ordenes_trabajo/ver_wo/'.$wo->WorkOrder_ID));
}

function reprogramar(){
$id = $this->input->post('itemR');
$txtreprogramar = "REPROGRAMAR WO: ".$this->input->post('txtreprogramar');
$fecha = $this->input->post('txtFechaAccion');

// Reprograma la orden de trabajo con nueva fecha y cambia estatus a 2
$this->MLConexion->comando("UPDATE WO_Master set FProgramado = ".$fecha.",
WorkOrder_Status = 2 where WorkOrder_ID = ".$id);

```

```

// Inserta comentario informando la reprogramación
$datos['id_wo'] = $id;
$datos['comentario'] = $txtreprogramar;
$datos['mail_us'] = $this->session->correo;
$this->MLConexion->insertar('comentarios_wo', $datos);

// Redirecciona a la vista de la orden reprogramada
redirect(base_url('ordenes_trabajo/ver_wo/'.$id));
}

function concluir_wo()
{
    $id = $this->input->post('itemConcluir');
    $txtconcluir = $this->input->post('txtconcluir');
    $foto = file_get_contents($_FILES['foto']['tmp_name']);
    $name = $_FILES['foto']['name'];

    // Registra el comentario de conclusión de WO
    $datos['id_wo'] = $id;
    $datos['comentario'] = "CONCLUIR WO: " . $txtconcluir;
    $datos['mail_us'] = $this->session->correo;
    $this->MLConexion->insertar('comentarios_wo', $datos);

    // Actualiza el estatus de la orden de trabajo a 3 (Concluida) y guarda la foto
    $this->load->library('correos');
    $data = array(
        'WorkOrder_Status' => 3,
        'archivo' => $foto,
        'nombre_archivo' => $name,
    );
    $where['WorkOrder_ID'] = $id;
    $this->MLConexion->modificar('WO_Master', $data, null, $where);

    // Marca la fecha de conclusión en los ítems que no fueron rechazados
    $rs_items = $this->MLConexion->consultar("SELECT * from WO_Detail where
    WorkOrder_ID=" . $id);
    foreach ($rs_items as $key) {

```



```

        if ($key->Item_Status != 6) {
            $this->MLConexion->comando("UPDATE rsitems SET FechaConcluidaWO =
CURRENT_TIMESTAMP() WHERE item_id = '" . $key->Item_Id . "'");
        }
    }

    // Recupera correos del usuario actual y su jefe
    $res = $this->Conexion->consultar("SELECT u.id, u.correo, cj.correo as correo_jefe from
usuarios u JOIN usuarios cj on cj.id=u.jefe_directo WHERE u.id=" . $this->session->id, true);

    // Prepara datos del correo de notificación de conclusión
    $mail['usuario'] = $this->session->nombre;
    $mail['items'] = $this->MLConexion->consultar("SELECT wo.*, e.Status_Descripcion FROM
WO_Detail wo JOIN tblStatusWO e on e.Status_ID=wo.Item_Status where wo.WorkOrder_ID = "
. $id);
    $mail['wo'] = $id;

    // Obtiene correos de los usuarios con permiso para cerrar WO
    $correo = [];
    $correos_a = $this->Conexion->consultar("SELECT U.correo from privilegios P inner join
usuarios U on P.usuario = U.id where P.cerrar_wo = 1 and U.activo = 1");
    foreach ($correos_a as $key => $value) {
        array_push($correo, $value->correo);
    }

    // Agrega correo del usuario actual a la lista de destinatarios
    $mail['correos'] = array_merge(array($this->session->correo), $correo);

    // Envía el correo
    $this->correos->concluir_wo($mail);

    // Redirige a la vista de la orden concluida
    redirect(base_url('ordenes_trabajo/ver_wo/' . $id));
}

function cancelar_wo()
{
    $id = $this->input->post('itemCancelarWO');

```

```

$txtcancelar = $this->input->post('txtcancelar');

// Registra el comentario de cancelación de la WO
$datos['id_wo'] = $id;
$datos['comentario'] = "CANCELAR WO: " . $txtcancelar;
$datos['mail_us'] = $this->session->correo;
$this->MLConexion->insertar('comentarios_wo', $datos);

// Carga librería para enviar correos
$this->load->library('correos');

// Actualiza el estatus de la orden a 7 (cancelada)
$this->MLConexion->comando("UPDATE WO_Master set WorkOrder_Status = 7 where
WorkOrder_ID = " . $id);

// Recorre los detalles de la WO para actualizar cada ítem como cancelado y limpiar su vínculo
con rsitems
$res = $this->MLConexion->consultar("SELECT * from WO_Detail where WorkOrder_ID = " .
$id);
foreach ($res as $key) {
    $this->MLConexion->comando("UPDATE WO_Detail set Item_Status = 6 where id = " . $key-
>id);
    $this->MLConexion->comando("UPDATE rsitems set FechaInWO = null, WOrder_ID = null
where item_id = " . $key->Item_Id);
}

// Prepara información para enviar correo de notificación de cancelación
$mail['usuario'] = $this->session->nombre;
$mail['wo'] = $id;
$correo = [];
$correos_a = $this->Conexion->consultar("SELECT U.correo from privilegios P inner join
usuarios U on P.usuario = U.id where P.cerrar_wo = 1 and U.activo = 1");
foreach ($correos_a as $key => $value) {
    array_push($correo, $value->correo);
}
$mail['correos'] = array_merge(array($this->session->correo), $correo);
$this->correos->cancelar_wo($mail);

```

```

// Redirige al detalle de la WO cancelada
redirect(base_url('ordenes_trabajo/ver_wo/' . $id));
}

function cerrar_wo()
{
    $this->load->library('correos');
    $foto = null;
    $id = $this->input->post('itemCerrar');
    $txtcerrar = "CERRAR WO: " . $this->input->post('txtcerrar');
    $mail = $this->input->post('mail');

    // Registra el comentario de cierre en la bitácora de la WO
    $datos['id_wo'] = $id;
    $datos['comentario'] = "CERRAR WO: " . $txtcerrar;
    $datos['mail_us'] = $this->session->correo;
    $this->MLConexion->insertar('comentarios_wo', $datos);

    // Cambia el estatus de la WO a 'CERRADA' (estatus 4)
    $this->MLConexion->comando("UPDATE WO_Master set WorkOrder_Status = 4 where
WorkOrder_ID = " . $id);

    // Si se debe enviar correo, prepara y envía con archivo adjunto
    if ($mail == 1) {
        $rs = $this->MLConexion->consultar("SELECT rs, archivo, nombre_archivo from WO_Master
where WorkOrder_ID = " . $id, true);
        $mail = $this->MLConexion->consultar("SELECT email from rsheaders where folio_id = " .
$rs->rs, true);

        $ext = strrchr($rs->nombre_archivo, '.');
        $fichero = sys_get_temp_dir() . '/' . $rs->rs . $ext;
        $foto = file_put_contents($fichero, $rs->archivo);

        $datos['correo'] = $mail->email;
        $datos['correo_us'] = $this->session->correo;
        $datos['foto'] = $fichero;
        $datos['usuario'] = $this->session->nombre;
        $datos['wo'] = $id;
    }
}

```

```

        $this->correos->cerrar_wo($datos);
    }

    // Redirige a la vista de la WO correspondiente
    redirect(base_url('ordenes_trabajo/ver_wo/' . $id));
}

function ajax_editSolicitud(){
    $solicitud = json_decode($this->input->post('solicitud'));
    $other = json_decode($this->input->post('other'));

    // Verifica si se adjuntó el archivo de acuse y lo guarda como binario
    if(isset($_FILES['f_A'])) {
        $solicitud->f_acuse = file_get_contents($_FILES['f_A']['tmp_name']);
    }

    // Verifica si se adjuntó el archivo de factura (PDF) y guarda su contenido y nombre
    if(isset($_FILES['f_F'])) {
        $solicitud->f_factura = file_get_contents($_FILES['f_F']['tmp_name']);
        $solicitud->name_factura = $this->input->post('f_F_name');
    }

    // Verifica si se adjuntó el archivo XML y guarda su contenido y nombre
    if(isset($_FILES['f_X'])) {
        $solicitud->f_xml = file_get_contents($_FILES['f_X']['tmp_name']);
        $solicitud->name_xml = $this->input->post('f_X_name');
    }

    $comentario = $this->input->post('comentario');

    // Actualiza la solicitud de factura en la base de datos
    $res = $this->Conexion->modificar('solicitudes_facturas', $solicitud, null, array('id' =>
    $solicitud->id));

    // Si la factura fue aceptada, actualiza el campo Factura en rsitems si está nulo
    if($solicitud->estatus_factura == "ACEPTADO") {
        $this->load->model('MLConexion_model', 'MLConexion');
    }
}

```

```

    $this->MLConexion->comando("UPDATE rsitems set Factura = ifnull(Factura, $solicitud-
>folio) where Solicitud_ID = $solicitud->id;");
}

if($res > 0) {
    // Si se agregó un comentario, lo guarda en la tabla de comentarios con marca de tiempo
    if(isset($_POST['comentario']) && !empty($comentario)) {
        $this->Conexion->insertar('solicitudes_facturas_comentarios', array('solicitud' =>
        $solicitud->id, 'usuario' => $this->session->id, 'comentario' => $comentario), array('fecha' =>
        'CURRENT_TIMESTAMP()'));
        $solicitud->comentario = $comentario;
    }

    // Obtiene los correos de usuarios con permiso para responder facturas
    $correos = [];
    $correos_a = $this->Conexion->consultar("SELECT U.correo from privilegios P inner join
    usuarios U on P.usuario = U.id where P.responder_facturas = 1");
    foreach ($correos_a as $key => $value) {
        array_push($correos, $value->correo);
    }

    // Prepara los datos para el correo de notificación
    $solicitud->correos = array_merge(array($this->session->correo), $correos);
    $solicitud->User = $other->User;
    $solicitud->Client = $other->Client;
    $solicitud->Contact = $other->Contact;

    $mail['id'] = $solicitud->id;
    $mail['estatus_factura'] = $solicitud->estatus_factura;
    $mail['comentario'] = $solicitud->comentario;
    $mail['User'] = $other->User;
    $mail['Client'] = $other->Client;
    $mail['Contact'] = $other->Contact;
    $mail['correos'] = array_merge(array($this->session->correo), $correos);

    // Envía correo notificando la edición de la solicitud
    $this->correos_facturacion->editar_solicitud($mail);
    echo "1";
}

```

```

    } else {
        echo "";
    }
}

function ajax_getReporteServicios(){
    $this->load->model('MLConexion_model', 'MLConexion');

    $texto = $this->input->post('texto');
    $rs = $this->input->post('rs');

    // Consulta para obtener todos los ítems de un RS (con condiciones)
    $query = "SELECT rsitems.folio_id, rsitems.descripcion, rsn.Notas, concat(rsitems.descripcion,
if(isnull(rsitems.Fabricante), '', concat(' ', rsitems.Fabricante)), if(isnull(rsitems.Modelo), '',
concat(' ', rsitems.Modelo)), if(isnull(rsitems.Serie), '', concat(' Serie: ', rsitems.Serie)),
if(isnull(rsitems.Equipo_ID), '', concat(' ID: ', rsitems.Equipo_ID))) as CadenaDescripcion,
rsitems.item_id, rsitems.fechaCancelado, rsitems.inWork, rsitems.tecnico_id,
rsitems.fechaActEQ, rsitems.WOrder_ID, catalogo_tecnicos.email_tecnico FROM rsitems INNER
JOIN catalogo_tecnicos ON rsitems.tecnico_id = catalogo_tecnicos.Tecnico_Id join
rsheadersnotas rsn on rsn.Folio_ID=rsitems.folio_id WHERE (((rsitems.folio_id)='".$rs."') AND
((rsitems.fechaCancelado) Is Null) AND ((rsitems.inWork)=0) AND ((rsitems.fechaActEQ) Is Null)
AND ((rsitems.WOrder_ID) Is Null) AND ((catalogo_tecnicos.email_tecnico)='".$this->session-
>correo."'))";

    // Si se proporciona un texto, se aplica un filtro adicional sobre la cadena descriptiva
    if($texto) {
        $query .= " having CadenaDescripcion like '%$texto%'";
    }

    $res = $this->MLConexion->Consultar($query);

    // Devuelve los resultados en formato JSON si existen
    if($res){
        echo json_encode($res);
    }
}

function ajax_getRSItems(){

```

```

$id_factura = $this->input->post('id_factura');

// Consulta para obtener los ítems asociados a una factura específica
$res = $this->Conexion->Consultar("SELECT * from rsitems_facturas where id_factura =
$id_factura");

// Devuelve los resultados en formato JSON si existen
if($res){
    echo json_encode($res);
}
}

function ajax_setComentarios(){
// Decodifica el comentario recibido vía POST en formato JSON
$comentario = json_decode($this->input->post('comentario'));
    $comentario->mail_us = $this->session->correo;
    $funciones = array('fecha' => 'CURRENT_TIMESTAMP()');
    $res = $this->MLConexion->insertar('comentarios_wo', $comentario, $funciones);

// Devuelve "1" si la operación fue exitosa, vacío si falló
if($res > 0) {
    echo "1";
} else {
    echo "";
}
}

function ajax_getComentarios(){
    $id = $this->input->post('id');

// Consulta los comentarios de la orden de trabajo junto al nombre del técnico
$query = "SELECT cm.*, ct.nombre FROM comentarios_wo cm JOIN catalogo_tecnicos ct on
cm.mail_us=ct.email_tecnico where 1 = 1";

// Si se recibe un ID de orden de trabajo, se filtra por él
if($id) {
    $query .= " and cm.id_wo = '$id'";
}
}

```

```

$res = $this->MLConexion->consultar($query);

// Devuelve los resultados en formato JSON si existen, vacío en caso contrario
if($res) {
    echo json_encode($res);
} else {
    echo "";
}
}

function ajax_getRequisitores(){
    $id = $this->input->post('id');

    $query = "SELECT U.id, concat(U.nombre, ' ', U.paterno) as Nombre, P.puesto as Puesto
from usuarios U inner join puestos P on U.puesto = P.id inner join privilegios PR on PR.usuario =
U.id where U.activo = 1 and PR.solicitar_facturas = 1";

    if($id)
    {
        $query .= " and U.id = '$id'";
    }

    $res = $this->Conexion->consultar($query, $id);
    if($res)
    {
        echo json_encode($res);
    }
    else
    {
        echo "";
    }
}

function ajax_getVFPData(){
    $modelo = $this->input->post('modelo');
    $res = shell_exec("C:/xampp/htdocs/MASMetrologia/vfp_reader/vfp_reader.exe
\"$modelo\"");

```



```

    echo $res;
}

function archivo_impresion(){
$pdf = new PDFMerger;
ob_start();
error_reporting(E_ALL & ~E_NOTICE);
ini_set('display_errors', 0);
ini_set('log_errors', 1);
/* ...
Resto del código que genera el PDF
... */
/* Limpiamos la salida del búfer y lo desactivamos */
ob_end_clean();
$id = $this->input->post('id');
$codigo = $this->input->post('codigo');

        ob_start();
error_reporting(E_ALL & ~E_NOTICE);
ini_set('display_errors', 0);
ini_set('log_errors', 1);
/* ...
Resto del código que genera el PDF
... */
/* Limpiamos la salida del búfer y lo desactivamos */

$pdf = new PDFMerger();
$q="SELECT SF.* from solicitudes_facturas SF where SF.id = $id";
$res = $this->Conexion->consultar($q, TRUE);

for ($i=0; $i < strlen($codigo); $i++) {
    // Determina el tipo de documento según el carácter leído

    switch (strtoupper($codigo[$i])) {

        case 'F':
            $campo = 'f_factura';
            break;

```

```

    case 'R':
        $campo = 'f_remision';
        break;

    case 'O':
        $campo = 'f_orden_compra';
        break;

    case 'A':
        $campo = 'f_acuse';
        break;

    case 'P':
        $campo = 'OPINION';
        break;

    case 'S':
        $campo = 'EMISI ON';
        break;

    default:
        $campo = null;
        break;
}

// Si se reconoció un campo válido
if($campo != null) {

    // Si es un archivo binario guardado en base de datos (prefijo "f_")
    if(substr($campo, 0, 2) == "f_") {

        $file = $res->$campo; // Recupera archivo desde el objeto
        $fichero = sys_get_temp_dir() . '/' . $campo . '.pdf'; // Ruta temporal
        file_put_contents($fichero, $file); // Escribe archivo temporal
        $pdf->addPDF($fichero, 'all'); // Agrega PDF generado al combinador

    } else {

```

```

        // Si es archivo precargado del sistema de archivos
        $fichero = "data/empresas/documentos_globales/" . $campo . "_000001.pdf";
        $pdf->addPDF($fichero, 'all'); // Agrega PDF externo
    }
}
}

ob_end_clean();
$pdf->merge('browser');
}

function ajax_getClientes(){

    $query = "SELECT C.id, C.nombre, C.razon_social, C.foto, C.opinion_positiva, C.emision_sua
from empresas C where C.cliente = 1";

    $res = $this->Conexion->consultar($query);
    if($res)
    {
        echo json_encode($res);
    }
    else
    {
        echo "";
    }
}

function ajax_getClientesSolicitudes(){
    $texto = $this->input->post('texto');

    // Consulta para obtener clientes y cantidad de solicitudes asociadas
    $query = "SELECT E.id, E.nombre, count(S.id) as NumSol from solicitudes_facturas S inner join
empresas E on E.id = S.cliente";

    // Aplica filtro si se ingresó texto de búsqueda
    if($texto)
    {

```

```

        $query .= " where E.nombre like '%$texto%'";
    }

    // Agrupa por ID de empresa para contar solicitudes
    $query .= " group by E.id;";

    $res = $this->Conexion->consultar($query);

    if($res)
    {
        echo json_encode($res); // Devuelve los resultados en formato JSON
    }
}

function ajax_getEjecutivosSolicitudes(){
    $texto = $this->input->post('texto');

    // Consulta para obtener ejecutivos y cantidad de solicitudes que gestionaron
    $query = "SELECT U.id, concat(U.nombre, ' ', U.paterno) as Ejecutivo, count(S.id) as NumSol
from solicitudes_facturas S inner join usuarios U on U.id = S.ejecutivo";

    // Aplica filtro por nombre si se proporcionó
    if($texto)
    {
        $query .= " where concat(U.nombre, ' ', U.paterno) like '%$texto%'";
    }

    // Agrupa por ID de usuario para contar solicitudes
    $query .= " group by U.id;";

    $res = $this->Conexion->consultar($query);

    if($res)
    {
        echo json_encode($res); // Devuelve los resultados en formato JSON
    }
}

```

```

function ajax_getDocumentosGlobales(){
// Consulta los documentos globales registrados (solo uno, con id = 1)
$query = "SELECT id, opinion_positiva, emision_sua from documentos_globales where id = 1";

$res = $this->Conexion->consultar($query);
if($res)
{
    echo json_encode($res); // Devuelve los resultados en formato JSON
}
else
{
    echo ""; // Respuesta vacía si no hay datos
}
}

function ajax_filesExists($id){
    $this->load->helper('file'); // Carga helper para funciones de archivos

    // Rellena con ceros a la izquierda para obtener un ID de 6 dígitos
    $id = str_pad($id, 6, "0", STR_PAD_LEFT);

    // Verifica si existen los archivos PDF de acuse y emisión para la empresa
    $acuse = read_file(base_url("data/empresas/documentos_facturacion/ACUSE_" . $id . ".pdf"));
    ? "1" : "0";
    $emision = read_file(base_url("data/empresas/documentos_facturacion/EMISION_" . $id .
    ".pdf")); ? "1" : "0";

    // Devuelve un arreglo indicando si cada archivo existe ("1") o no ("0")
    echo json_encode(array($acuse, $emision));
}

function ajax_readXML(){
// Crea una instancia del objeto DOMDocument para leer el XML
$dom = new DomDocument;
$dom->preserveWhiteSpace = FALSE;

// Carga el contenido del archivo XML subido
$dom->loadXML(file_get_contents($_FILES['f_X']['tmp_name']));

```

```

// Obtiene el nodo <Comprobante> del XML
$comp = $dom->getElementsByTagName('Comprobante');
$ext = 1; // Bandera para validar si el folio ya existe
$data = array(); // Arreglo para guardar los datos extraídos del XML

// Recorre los atributos del nodo <Comprobante>
foreach ($comp[0]->attributes as $elem)
{
    // Si el atributo es Serie, Folio o SubTotal, lo guarda en $data
    if($elem->name == "Serie" | $elem->name == "Folio" | $elem->name == "SubTotal")
    {
        $e = array($elem->name => $elem->value);
        array_push($data, $e);
    }

    // Si el atributo es Folio, verifica si ya existe una solicitud con ese folio
    if($elem->name == "Folio")
    {
        $folio = $elem->value;
        $res = $this->Conexion->consultar("SELECT count(*) as existe FROM solicitudes_facturas
where folio = '$folio'", TRUE);
        $ext = $res->existe;
    }
}

// Si el folio no existe en la base de datos, devuelve los datos del XML
if($ext == 0)
{
    echo json_encode($data);
}
else
{
    // Si el folio ya existe, retorna "0" indicando duplicado
    echo "0";
}
}

```

```

function ajax_setDocumentoFacturacion(){
$file = $this->input->post('file');
$documento = $this->input->post('documento');
$id = $this->input->post('empresa');
$id = str_pad($id, 6, "0", STR_PAD_LEFT); // Completa el ID a 6 dígitos con ceros a la izquierda

if($file != "undefined")
{
    // Configura la carga del archivo PDF
    $config['upload_path'] = 'data/empresas/documentos_facturacion/';
    $config['allowed_types'] = 'pdf';
    $config['overwrite'] = TRUE;
    $config['file_name'] = $documento . '_' . $id;

    $this->load->library('upload', $config);

    // Intenta subir el archivo
    if ($this->upload->do_upload('file'))
    {
        $where['id'] = $id;

        // Asigna el campo correspondiente según el tipo de documento
        switch($documento)
        {
            case "EMISION":
                $campo = "emision_sua";
                break;
            case "OPINION":
                $campo = "opinion_positiva";
                break;
        }

        // Guarda el nombre del archivo en la base de datos
        $data[$campo] = $this->upload->data('file_name');
        $this->Conexion->modificar('empresas', $data, null, $where);
        echo "1"; // Indica éxito al frontend
    }
}
}

```

```

}

function ajax_deleteDocumentoFacturacion(){
    $documento = $this->input->post('documento');
    $id = $this->input->post('empresa');
    $id = str_pad($id, 6, "0", STR_PAD_LEFT); // Asegura que el ID tenga 6 dígitos

    // Elimina físicamente el archivo PDF correspondiente
    unlink('data/empresas/documentos_facturacion/' . $documento . '_' . $id . '.pdf');

    // Determina el campo que se debe actualizar como vacío
    $where['id'] = $id;
    switch($documento)
    {
        case "EMISION":
            $campo = "emision_sua";
            break;
        case "OPINION":
            $campo = "opinion_positiva";
            break;
    }

    // Actualiza el campo a vacío en la base de datos
    $data[$campo] = "";
    $this->Conexion->modificar('empresas', $data, null, $where);
}

function ajax_setDocumentoGlobal(){
    $file = $this->input->post('file');
    $documento = $this->input->post('documento');
    $id = $this->input->post('empresa');
    $id = str_pad($id, 6, "0", STR_PAD_LEFT); // Asegura que el ID tenga 6 dígitos

    if($file != "undefined")
    {
        // Configuración de la subida del archivo
        $config['upload_path'] = 'data/empresas/documentos_globales/';
        $config['allowed_types'] = 'pdf';
    }
}

```



```

$config['overwrite'] = TRUE;
$config['file_name'] = $documento . '_' . $id;

$this->load->library('upload', $config);

// Intenta subir el archivo
if ($this->upload->do_upload('file'))
{
    $where['id'] = $id;

    // Determina el campo que se debe actualizar
    switch($documento)
    {
        case "EMISION":
            $campo = "emision_sua";
            break;
        case "OPINION":
            $campo = "opinion_positiva";
            break;
    }

    // Guarda el nombre del archivo en la base de datos
    $data[$campo] = $this->upload->data('file_name');
    $this->Conexion->modificar('documentos_globales', $data, null, $where);
    echo "1"; // Notifica éxito al frontend
}
}
}

function ajax_deleteDocumentoGlobal(){
    $documento = $this->input->post('documento');
    $id = $this->input->post('empresa');
    $id = str_pad($id, 6, "0", STR_PAD_LEFT); // Asegura que el ID tenga 6 dígitos con ceros a la
    izquierda

    // Elimina físicamente el archivo PDF correspondiente del sistema de archivos
    unlink('data/empresas/documentos_globales/' . $documento . '_' . $id . '.pdf');
}

```

```
// Determina el campo correspondiente que se debe actualizar como vacío en la base de
datos
```

```
$where['id'] = $id;
switch($documento)
{
    case "EMISION":
        $campo = "emision_sua";
        break;
    case "OPINION":
        $campo = "opinion_positiva";
        break;
}
```

```
// Limpia el campo en la base de datos que hacía referencia al archivo eliminado
$data[$campo] = "";
$this->Conexion->modificar('documentos_globales', $data, null, $where);
}
```

```
function ajax_enviarCorreo(){
    $id = $this->input->post('id');
    $body = $this->input->post('body');
    $subject = $this->input->post('subject');
    $para = $this->input->post('para');
    $cc = $this->input->post('cc');
```

```
$campos = json_decode($this->input->post('campos'));
```

```
$archivos = [];
```

```
// Obtiene los datos de la solicitud de factura por ID
$res = $this->Conexion->consultar("SELECT SF.* from solicitudes_facturas SF where SF.id =
$id", TRUE);
```

```
foreach ($campos as $value) {
    // Si el campo es un archivo binario de la tabla (f_pdf, f_xml, etc.)
    if(substr($value, 0, 2 ) == "f_") {
        $file = $res->$value;
        $fichero = sys_get_temp_dir(). '/' . $value . ($value == "f_xml" ? '.xml' : '.pdf');
```

```

        file_put_contents($fichero, $file); // Guarda el archivo temporalmente
    } else {
        // Para documentos globales con nombres codificados
        switch ($value) {
            case 'opinion_positiva':
                $value = 'OPINION';
                break;
            case 'emision_sua':
                $value = 'EMISION';
                break;
        }
        $fichero = "data/empresas/documentos_globales/" . $value . "_000001.pdf";
    }

    array_push($archivos, $fichero); // Añade el archivo al array de adjuntos
}

// Prepara el array con todos los datos del correo
$datos['id'] = $id;
$datos['para'] = $para;
$datos['cc'] = $cc;
$datos['subject'] = $subject;
$datos['body'] = $body;
$datos['campos'] = $campos;
$datos['archivos'] = $archivos;

// Llama a la librería de envío de correos
$this->correos_facturacion->enviarCorreo($datos);
}

//////////////////////////////////// FACTURAS //////////////////////////////////////
function ajax_getFacturas(){
    $id = $this->input->post('id');

    // Consulta principal para obtener detalles de las facturas aceptadas
    $query = "SELECT F.id, F.fecha, F.usuario, F.cliente, F.contacto, F.reporte_servicio,
F.orden_compra, F.forma_pago, F.pagada, F.conceptos, F.notas, F.estatus_factura,
F.documentos_requeridos, F.serie, F.folio, F.codigo_impresion, (SELECT count(id) from

```

```

recorrido_conceptos where id_concepto = F.id and tipo = 'FACTURA') as Recorridos, (SELECT
count(id) from envios_factura where factura = F.id) as Envios, E.nombre as Cliente,
concat(U.nombre, ' ', U.paterno) as User, U.correo, ifnull(EC.correo, 'N/A') as CorreoContacto
from solicitudes_facturas F inner join empresas E on E.id = F.cliente inner join usuarios U on U.id
= F.usuario left join empresas_contactos EC on EC.id = F.contacto";
$query .= " where F.folio > 0 and F.estatus = 'ACEPTADO'";

```

```

// Filtra por ID si se proporciona
if($id) {
    $query .= " and F.id = '$id'";
}

```

```

// Filtro adicional opcional por estatus
if(isset($_POST['estatus'])) {
    $estatus = $this->input->post('estatus');
    $query .= " and F.estatus = '$estatus'";
}

```

```

$res = $this->Conexion->consultar($query, $id);
if($res) {
    echo json_encode($res);
}
}

```

```

//////////////////// LOGISTICA //////////////////////

```

```

function ajax_getMensajeros(){
    // Consulta para obtener todos los usuarios con privilegio de mensajero activos
    $query = "SELECT U.id, U.nombre, U.paterno, U.materno, U.no_empleado, U.puesto,
U.correo, U.ultima_sesion, U.departamento, U.activo, U.jefe_directo, U.autorizador_compras,
U.autorizador_compras_venta, concat(U.nombre, ' ', U.paterno) as User, concat(U.nombre, ' ',
U.paterno, ' ', U.materno) as CompleteName from usuarios U inner join privilegios P on
P.usuario = U.id where U.activo = '1'";
    $query .= " and P.mensajero = '1'";
    $query .= " order by User";
}

```

```

$res = $this->Conexion->consultar($query);
if($res) {

```

```

        echo json_encode($res);
    }
}

function ajax_setRecorrido(){
    $mensajero = $this->input->post('mensajero');
    $fecha = $this->input->post('fecha');
    $recorrido = json_decode($this->input->post('recorrido'));

    // Inserta cabecera del recorrido
    $data['mensajero'] = $mensajero;
    $data['fecha_recorrido'] = $fecha;
    $recorrido_id = $this->Conexion->insertar('recorridos', $data);

    // Por cada factura en el recorrido, crea detalle y actualiza su estatus
    foreach ($recorrido as $value) {
        $data2['recorrido'] = $recorrido_id;
        $data2['factura'] = $value[1];
        $data2['accion'] = $value[0];
        $data2['estatus'] = "EN RECORRIDO";

        $this->Conexion->insertar('recorrido_conceptos', $data2);
        $this->Conexion->modificar('solicitudes_facturas', array('estatus' => $data2['estatus']), null,
array('id' => $data2['factura']));
    }
}

function ajax_getRecorridos(){
    $pendientes = $this->input->post('pendientes');
    $factura = $this->input->post('factura');

    // Consulta de todos los recorridos y sus detalles, con subconsultas para comentarios y
    pendientes
    $query = "SELECT RF.*, R.mensajero, R.fecha_recorrido, (SELECT count(RC.id) from
    recorrido_comentarios RC where RC.recorrido_factura = RF.id) as Comentarios, (SELECT
    count(RF2.id) from recorrido_facturas RF2 where RF2.recorrido = RF.recorrido and RF2.estatus =
    'EN RECORRIDO') as Pendientes, (SELECT E.nombre from solicitudes_facturas F inner join
    empresas E on E.id = F.cliente where F.id = RF.factura) as Cliente, ifnull(concat(M.nombre, ' ',

```

```
M.paterno), 'N/A') as Mensajero from recorrido_facturas RF inner join recorridos R on R.id = RF.recorrido left join usuarios M on M.id = R.mensajero where 1 = 1";
```

```
// Filtra por factura específica si se envía
if(isset($_POST['factura'])) {
    $query .= " and RF.factura = $factura";
}

// Filtra para mostrar solo recorridos con facturas pendientes
if($pendientes == "1") {
    $query .= " having Pendientes > 0";
}

// Ordena resultados por fecha de recorrido, ID del recorrido y ID de la factura dentro del
recorrido
$query .= " order by R.fecha_recorrido, R.id, RF.id asc";

$res = $this->Conexion->consultar($query);
echo json_encode($res);
}

function ajax_updateRecorrido(){
    $recorrido = json_decode($this->input->post('recorrido')); // Decodifica los datos del
recorrido
    $recolecta = $this->input->post('recolecta'); // Indica si el estatus final es recolección
    $comentario = $this->input->post('comentario'); // Comentario del usuario

    // Actualiza los datos del recorrido en la tabla correspondiente
    $this->Conexion->modificar('recorrido_facturas', $recorrido, null, array('id' => $recorrido-
>id));

    // Si el estatus inicia con "NO", se considera como pendiente
    if(substr($recorrido->estatus, 0, 2) == "NO") {
        $stat = "PENDIENTE " . $recorrido->accion; // Marca como pendiente la acción (ej.
ENTREGA o RECOLECCIÓN)
        $this->Conexion->modificar('solicitudes_facturas', array('estatus' => $stat), null, array('id'
=> $recorrido->factura));
    } else {
```

```

        // Si recolecta está activa, el nuevo estatus es "PENDIENTE RECOLECTA", si no, es
"CERRADO"
        $estat = $recolecta == "1" ? "PENDIENTE RECOLECTA" : "CERRADO";
        $this->Conexion->modificar('solicitudes_facturas', array('estatus' => $estat), null, array('id'
=> $recorrido->factura));
    }

```

```

    // Si se ingresó un comentario, se guarda en la tabla de comentarios del recorrido
    if($comentario) {
        // Define color del comentario: rojo para negativos, verde para positivos
        $color = substr($recorrido->estatus, 0, 2) == "NO" ? "red" : "green";
        $comentario = '<font color=' . $color . '><b>' . $recorrido->estatus . ':</b></font> ' .
$comentario;

```

```

        $data_com['recorrido_factura'] = $recorrido->id;
        $data_com['usuario'] = $this->session->id;
        $data_com['comentario'] = $comentario;
        $func_com['fecha'] = "CURRENT_TIMESTAMP()";

        // Inserta el comentario en la base de datos
        $this->Conexion->insertar('recorrido_comentarios', $data_com, $func_com);
    }
}

```

```

function ajax_getComentariosRecorrido(){
    // Se recibe el ID del recorrido de la factura mediante POST
    $id = $this->input->post('id');

    // Consulta los comentarios asociados al recorrido de factura, junto con el nombre del usuario
    que los hizo
    $query = "SELECT C.*, concat(U.nombre, ' ', U.paterno) as User from recorrido_comentarios C
    inner join usuarios U on U.id = C.usuario where 1 = 1";

    // Si se proporcionó un ID específico, se filtra por ese recorrido
    if($id) {
        $query .= " and C.recorrido_factura = '$id'";
    }
}

```

```

// Ordena los resultados cronológicamente por fecha
$query .= " order by C.fecha";

$res = $this->Conexion->consultar($query);
if($res) {
    echo json_encode($res);
}
}

function ver_POD($id){
    //Se obtiene la información principal de la solicitud
    $query = "SELECT sf.id, sf.folio,sf.serie, sf.reporte_servicio, sf.fecha, sf.orden_compra,
concat(u.nombre, ' ', u.paterno) as responsable, e.razon_social, concat(e.calle, ' ',e.numero, ' CP
',e.cp) as direccion,e.ciudad, e.estado, e.rfc,e.colonia, ec.nombre as contacto, ec.correo FROM
solicitudes_facturas sf join usuarios u on sf.ejecutivo = u.id JOIN empresas e on sf.cliente = e.id
JOIN empresas_contactos ec on sf.contacto = ec.id WHERE sf.id =$id";
    $factura = $this->Conexion->consultar($query, TRUE);

    //Se consultan los equipos relacionados a la factura para mostrarlos en el documento
    $query2 = "SELECT rs.descripcion, rs.Equipo_ID, rs.Fec_CalibracionMT,
s.DescripcionDeServicio, concat(rs.descripcion, if(isnull(rs.Fabricante), '', concat(' ',
rs.Fabricante)), if(isnull(rs.Modelo), '', concat(' ', rs.Modelo)), if(isnull(rs.Serie), '', concat(' Serie:
', rs.Serie))) as CadenaDescripcion from rsitems rs JOIN catalogo_servicios s on s.servicio_id =
rs.item_servicio_id WHERE rs.Solicitud_ID = ".$id." and rs.Factura = ".$factura->folio."";
    $rs = $this->MLConexion->consultar($query2);
    $total=0;
    foreach($rs as $row){
        $total++;
    }
    ini_set('display_errors', 0);
    $this->load->library('pdfview');

    $pdf = new TCPDF(PDF_PAGE_ORIENTATION, PDF_UNIT, PDF_PAGE_FORMAT, true, 'UTF-8',
false);

    $f=$factura->serie . "-" . $factura->folio;
    //Configura el encabezado con folio, responsable y fecha

```



```

$pdf->SetCreator(PDF_CREATOR);
$pdf->SetAuthor('AleksOrtiz');
$pdf->SetTitle('Masmetrologia');
$pdf->SetSubject('Formato Cotización');
$spc = "      ";
$head = "$spc      Prueba de Entrega      $spc Folio: $id / Factura: $f";
$txt = "      Proof of delivery      Responsable: " . $factura-
>responsable;
$txt .= "\n      Fec. de elaboración:: " . $factura-
>fecha;

```

```

$pdf->SetHeaderData(PDF_HEADER_LOGO_ORIGINAL, '40', $head, $txt);
$pdf->setHeaderFont(Array(PDF_FONT_NAME_MAIN, "", 10));
$pdf->setFooterFont(Array(PDF_FONT_NAME_DATA, "", PDF_FONT_SIZE_DATA));
$pdf->SetDefaultMonospacedFont(PDF_FONT_MONOSPACED);
$pdf->SetMargins(8, PDF_MARGIN_TOP, 8);
$pdf->SetHeaderMargin(PDF_MARGIN_HEADER);
$pdf->SetFooterMargin(PDF_MARGIN_FOOTER);
$pdf->SetAutoPageBreak(TRUE, PDF_MARGIN_BOTTOM);
$pdf->setImageScale(PDF_IMAGE_SCALE_RATIO);

```

```

if (@file_exists(dirname(__FILE__).'/lang/eng.php')) {
    require_once(dirname(__FILE__).'/lang/eng.php');
    $pdf->setLanguageArray($l);
}

```

```

$pdf->SetFont('helvetica', "", 8);

```

```

$pdf->AddPage();
$pdf->SetFillColor(255, 255, 255);
$pdf->SetFont('helvetica', "", 10);

```

//Construccion de la tabla HTML con los datos del cliente, dirección, contacto y total de equipos

```

$tbl = <<<EOD
<br>
<table border="0">
    <tr>

```

```

        <td>
            <b>Cliente/Customer:</b><br>
            $factura->razon_social<br>
            $factura->direccion<br>
            $factura->colonia<br>
            $factura->ciudad, $factura->estado<br>
            RFC: $factura->rfc<br>

        </td>
    <td>
EOD;

```

```

    $tbl .= <<<EOD
    <b>Orden de Compra: </b>
    $factura->orden_compra<br>
    <b>Contacto / Contact: </b><br>
    $factura->contacto<br>
    $factura->correo<br>
    <br>
    Total de Equipo(s): $total<br>
    </td>
    </tr>
</table>
EOD;

```

```

$pdf->writeHTML($tbl, false, false, false, false, "");
$w = array(8, 125, 12, 24, 24);

```

```

$pdf->SetFont('helvetica', 'B', 9);
$pdf->SetTextColor(255);
$pdf->Ln();$pdf->Ln();
$w = array(20, 20, 125, 20);

```

```

$pdf->SetTextColor(0);
$pdf->SetFont('helvetica', '', 10);
    $pdf->Ln();

```

```

$tabla_items="";

$tabla_items .= '<table style=" ">
    <thead>
        <tr>
            <th style="border-bottom: 1px solid #000; text-align: center; font-weight:
bold; width: 15%;">Servicio</th>
            <th style="border-bottom: 1px solid #000; text-align: center; font-weight:
bold; width: 15%;">Equipo ID</th>
            <th style="border-bottom: 1px solid #000; text-align: center; font-weight:
bold; width: 55%;">Descripcion</th>
            <th style="border-bottom: 1px solid #000; text-align: center; font-weight:
bold; width: 15%;">Realizado</th>
        </tr>
    </thead>
    <tbody>';
foreach($rs as $row){
    $date=date_create($row->Fec_CalibracionMT);
    $tabla_items .= '
        <tr>
            <td style="border-bottom: 1px solid #000; text-align: center; width: 15%; tr:nth-
child:background: #F8F8F8;">'.$row->DescripcionDeServicio .'</td>
            <td style="border-bottom: 1px solid #000; text-align: center; width: 15%; tr:nth-
child:background: #F8F8F8;">'.$row->Equipo_ID.'</td>
            <td style="border-bottom: 1px solid #000; text-align:center ; width: 55%; tr:nth-
child:background: #F8F8F8;">'.$row->CadenaDescripcion.'</td>
            <td style="border-bottom: 1px solid #000; text-align: center;width: 15%; tr:nth-
child:background: #F8F8F8;">'.date_format($date,'d/M/Y').'</td>
        </tr>';
    }
$tabla_items .= '</tbody>
</table>
<br>
<br>
<br>
<br>
<br>
<br>';

```

```

$pdf->writeHTML($tabla_items, true, false, false, false, "");
$pdf->Ln();
$pdf->Ln();
$pdf->SetFont('helvetica', "", 10);
$pdf->MultiCell(150, 8, "Recibí el servicio a los equipos arriba listados", 0, 'L', 1, 0, "", "", true,
0, false, true, 0);
$pdf->SetFont('helvetica', 'B', 10);
$pdf->writeHTML("_____", true, false, false, false, "");
$pdf->MultiCell(150, 8, "Firma de recibido", 0, 'L', 1, 0, "", "", true, 0, false, true, 0);
$pdf->Ln();
$name = "POD ".$id." - PO ".$factura->orden_compra.'.pdf';

$pdf->Ln();

//El PDF generado se envía al navegador para su visualización.
$pdf->Output(sys_get_temp_dir().'/'.$name, 'I');
}

```

```

function ajax_enviarCorreoPOD(){
    //Recepción de parámetros del formulario
    $id = $this->input->post('id');
    $body = $this->input->post('body');
    $subject = $this->input->post('subject');
    $para = $this->input->post('para');
    $cc = $this->input->post('cc');
    $certificados=[];
    $pdfname=null;
    $xmlname =null;
    $campos = json_decode($this->input->post('campos'));
    $archivos = [];
    $name_files = [];
    //Consulta principal de la solicitud de factura
    $res = $this->Conexion->consultar("SELECT SF.* from solicitudes_facturas SF where SF.id =
$id", TRUE);
    $pdfname=$res->name_factura;
    $xmlname=$res->name_xml;
    foreach ($campos as $value) {

```

```

if(substr($value, 0, 2 ) == "f_")
{
    $file = $res->$value;

    $fichero = sys_get_temp_dir(). '/' . $value . ($value == "f_xml" ? '.xml' : '.pdf');
    $name =$value == "f_xml" ? $res->name_xml : $res->name_factura;
    file_put_contents($fichero, $file);
}
else
{
    switch ($value) {
        case 'opinion_positiva':
            $value = 'OPINION';
            break;

        case 'emision_sua':
            $value = 'EMISION';
            break;
    }
    $fichero = "data/empresas/documentos_globales/" . $value . "_000001.pdf";
}

array_push($archivos, $fichero);

}

//Consulta de datos de cliente y factura
$query = "SELECT sf.id, sf.folio,sf.serie, sf.reporte_servicio, sf.fecha, sf.orden_compra,
concat(u.nombre, ' ', u.paterno) as responsable, e.razon_social, concat(e.calle, ' ',e.numero, ' CP
',e.cp) as direccion,e.ciudad, e.estado, e.rfc,e.colonia, ec.nombre as contacto, ec.correo FROM
solicitudes_facturas sf join usuarios u on sf.ejecutivo = u.id JOIN empresas e on sf.cliente = e.id
JOIN empresas_contactos ec on sf.contacto = ec.id WHERE sf.id =$id";
$factura = $this->Conexion->consultar($query, TRUE);

//Consulta de equipos incluidos en la factura
$query2 = "SELECT rs.descripcion, rs.Equipo_ID, rs.documento_id, rs.Fec_CalibracionMT,
s.DescripcionDeServicio, concat(if(isnull(rs.Fabricante), '', concat(' ', rs.Fabricante)),
if(isnull(rs.Modelo), '', concat(' ', rs.Modelo)), if(isnull(rs.Serie), '', concat(' Serie: ', rs.Serie))) as

```

```

CadenaDescripcion from rsitems rs JOIN catalogo_servicios s on s.servicio_id =
rs.item_servicio_id WHERE rs.Solicitud_ID = ".$id." and rs.Factura = ".$factura->folio."";
    $rs = $this->MLConexion->consultar($query2);
    $total=0;
    foreach($rs as $row){
        $total++;
    }

    foreach ($rs as $key) {
        array_push($certificados, $key->documento_id);
    }

    ini_set('display_errors', 0);
    $this->load->library('pdfview');

    //Iniciación y configuración del PDF
    $pdf = new TCPDF(PDF_PAGE_ORIENTATION, PDF_UNIT, PDF_PAGE_FORMAT, true, 'UTF-8',
false);

```

```

    $f=$factura->serie . "-" . $factura->folio;
    $pdf->SetCreator(PDF_CREATOR);
    $pdf->SetAuthor('AleksOrtiz');
    $pdf->SetTitle('Masmetrologia');
    $pdf->SetSubject('Formato Cotización');
    $spc = "          ";
    $head = "$spc          Prueba de Entrega          $spc Folio: $id / Factura: $f";
    $txt = "          Proof of delivery          Responsable: " . $factura-
>responsable;
    $txt .= "\n          Fec. de elaboración:: " . $factura-
>fecha;

```

```

    $pdf->SetHeaderData(PDF_HEADER_LOGO_ORIGINAL, '40', $head, $txt);

```

```

    $pdf->setHeaderFont(Array(PDF_FONT_NAME_MAIN, "", 10));

```

```

$pdf->setFooterFont(Array(PDF_FONT_NAME_DATA, "", PDF_FONT_SIZE_DATA));
$pdf->SetDefaultMonospacedFont(PDF_FONT_MONOSPACED);
$pdf->SetMargins(8, PDF_MARGIN_TOP, 8);
$pdf->SetHeaderMargin(PDF_MARGIN_HEADER);
$pdf->SetFooterMargin(PDF_MARGIN_FOOTER);
$pdf->SetAutoPageBreak(TRUE, PDF_MARGIN_BOTTOM);
$pdf->setImageScale(PDF_IMAGE_SCALE_RATIO);

if (@file_exists(dirname(__FILE__).'/lang/eng.php')) {
    require_once(dirname(__FILE__).'/lang/eng.php');
    $pdf->setLanguageArray($l);
}

$pdf->SetFont('helvetica', "", 8);

$pdf->AddPage();
$pdf->SetFillColor(255, 255, 255);
$pdf->SetFont('helvetica', "", 10);

//Construcción del contenido del PDF
$tbl = <<<EOD
<br>
<table border="0">
    <tr>
        <td>
            <b>Cliente/Client:</b><br>
            $factura->razon_social<br>
            $factura->direccion<br>
            $factura->colonia<br>
            $factura->ciudad, $factura->estado<br>
            $factura->rfc<br>

        </td>
        <td>

EOD;

$tbl .= <<<EOD

```

```

<b>Orden de Compra: </b>
$factura->orden_compra<br>
<b>Contacto / Contact: </b><br>
$factura->contacto<br>
$factura->correo<br>
<br>
Total de Equipo(s): $total<br>
</td>
</tr>
</table>
EOD;

```

```

$pdf->writeHTML($tbl, false, false, false, false, "");
$w = array(8, 125, 12, 24, 24);

```

```

$pdf->SetFont('helvetica', 'B', 9);
$pdf->SetTextColor(255);
$pdf->Ln();$pdf->Ln();
$w = array(20, 20, 125, 20);

```

```

$pdf->SetTextColor(0);
$pdf->SetFont('helvetica', '', 10);
    $pdf->Ln();
$tabla_items="";

```

```

$tabla_items .= '<table style=" ">
    <thead>
        <tr>
            <th style="border-bottom: 1px solid #000; text-align: center; font-weight:
bold; width: 15%;">Servicio</th>
            <th style="border-bottom: 1px solid #000; text-align: center; font-weight:
bold; width: 15%;">Equipo ID</th>
            <th style="border-bottom: 1px solid #000; text-align: center; font-weight:
bold; width: 55%;">Descripcion</th>
            <th style="border-bottom: 1px solid #000; text-align: center; font-weight:
bold; width: 15%;">Realizado</th>
        </tr>

```



```

        </thead>
        <tbody>';
foreach($rs as $row){
    $date=date_create($row->Fec_CalibracionMT);
    $tabla_items.='
        <tr>
            <td style="border-bottom: 1px solid #000; text-align: center; width: 15%; tr:nth-
child:background: #F8F8F8;">'.$row->DescripcionDeServicio.'</td>
            <td style="border-bottom: 1px solid #000; text-align: center; width: 15%; tr:nth-
child:background: #F8F8F8;">'.$row->Equipo_ID.'</td>
            <td style="border-bottom: 1px solid #000; text-align:center ; width: 55%; tr:nth-
child:background: #F8F8F8;">'.$row->CadenaDescripcion.'</td>
            <td style="border-bottom: 1px solid #000; text-align: center;width: 15%; tr:nth-
child:background: #F8F8F8;">'.date_format($date,'d/M/Y').</td>
        </tr>';
    }
    $tabla_items.='</tbody>
</table>
<br>
<br>
<br>
<br>
<br>
<br>';
$pdf->writeHTML($tabla_items, true, false, false, false, "");
$pdf->Ln();
$pdf->Ln();
$pdf->SetFont('helvetica', "", 10);
$pdf->MultiCell(150, 8,"Recibí el servicio a los equipos arriba listados", 0, 'L', 1, 0, "", "", true,
0, false, true, 0);
$pdf->SetFont('helvetica', 'B', 10);
$pdf->writeHTML("_____", true, false, false, false, "");
$pdf->MultiCell(150, 8,"Firma de recibido", 0, 'L', 1, 0, "", "", true, 0, false, true, 0);
$pdf->Ln();
$name = "POD ".$id." - PO ".$factura->orden_compra.'.pdf';

$pdf->Ln();
//Generación del PDF

```

```

$pdf->Output(sys_get_temp_dir().'/'.$name, 'F');
$datos['pod']=sys_get_temp_dir().'/'.$name;

$datos['id'] = $id;
$datos['para'] = $para;
$datos['cc'] = $cc;
$datos['subject'] = "POD ".$id." - PO ".$factura->orden_compra;
$datos['body'] = $body;
$datos['campos'] = $campos;
$datos['archivos'] = $archivos;
$datos['certificados'] = $certificados;
$datos['pdfname']=$pdfname;
$datos['xmlname']=$xmlname;
$datos['po']=$factura->orden_compra;
$this->correos_facturacion->archivos_facturacion($datos);
}

public function validar_archivos(){
    // Obtener el ID de la solicitud enviado por POST
    $id=$this->input->post('ID');
    $factura = $this->Conexion->consultar("SELECT id, folio from solicitudes_facturas where
id= ".$id, true);

    $query = " select item_id, documento_id,Fec_CalibracionMT from rsitems where factura =
".$factura->folio." and Solicitud_ID";
    $rs = $this->MLConexion->consultar($query);

    if($rs)
    {
        echo json_encode($rs);
    }
    else{
        // Si no se encontraron resultados, regresar una cadena vacía
        echo "";
    }
}

function ver_wo_pdf($id)
{

```

```
$query = "SELECT wo.*, wd.*, rs.folio_id, rs.Localizacion, ei.Status_Descripcion as
estatus_item, ew.Status_Descripcion as estatus_wo, concat(rs.descripcion,
if(isnull(rs.Fabricante), '', concat(' ', rs.Fabricante)), if(isnull(rs.Modelo), '', concat(' ', rs.Modelo)),
if(isnull(rs.Serie), '', concat(' Serie: ', rs.Serie)), if(isnull(rs.Equipo_ID), '', concat(' ID: ',
rs.Equipo_ID))) as CadenaDescripcion from WO_Master wo JOIN WO_Detail wd on
wo.WorkOrder_ID=wd.WorkOrder_ID JOIN rsitems rs ON rs.item_id=wd.Item_Id JOIN
tblStatusWO ei on ei.Status_id = wd.Item_Status JOIN tblStatusWO ew on
ew.Status_ID=wo.WorkOrder_Status where wo.WorkOrder_ID = ".$id;
```

```
$wo_detail=$this->MLConexion->consultar($query);
```

```
$query2 = "SELECT wo.*, wd.*, rsn.Notas, we.Status_Descripcion, ct.nombre, rs.folio_id,
rh.Empresa, rh.Direccion1, rh.Contacto, rh.TelefonoContacto, rh.CelularContacto from
WO_Master wo JOIN WO_Detail wd on wo.WorkOrder_ID=wd.WorkOrder_ID JOIN tblStatusWO
we on we.Status_ID=wo.WorkOrder_Status JOIN catalogo_tecnicos ct on
ct.email_tecnico=wo.correo_us join rsitems rs on rs.item_id =wd.Item_Id join rsheaders rh on
rh.folio_id=rs.folio_id join rsheadersnotas rsn on wo.rs=rsn.Folio_ID where wo.WorkOrder_ID=
".$id;
```

```
// Obtener información general de la orden de trabajo: empresa, contacto, técnico
responsable y notas.
```

```
$wo=$this->MLConexion->consultar($query2, TRUE);
```

```
$comentario="SELECT cm.*, ct.nombre FROM comentarios_wo cm JOIN catalogo_tecnicos
ct on cm.mail_us=ct.email_tecnico where 1 = 1 and cm.id_wo = ".$id;
```

```
// Obtener comentarios registrados por el técnico para esta orden de trabajo.
```

```
$cm=$this->MLConexion->consultar($comentario);
```

```
$qr = '';
```

```
ini_set('display_errors', 0);
```

```
$this->load->library('pdfview');
```

```
$pdf = new TCPDF(PDF_PAGE_ORIENTATION, PDF_UNIT, PDF_PAGE_FORMAT, true, 'UTF-8',
false);
```

```
$pdf->SetCreator(PDF_CREATOR);
```

```
$pdf->SetAuthor('AleksOrtiz');
```

```
$pdf->SetTitle('Masmetrologia');
```

```
$pdf->SetSubject('Formato Cotización');
```

```
$spc = " ";
```

```

$head = "$spc          Orden De Servicio   $spc Folio: $id";
$txt = "$spc$spc Work Order          $spc  Responsable: " . $wo->nombre;
$txt .= "\n    $spc  $spc                                Fec. de Programada: " .
$wo->FProgramado;
$txt .= "\n    $spc  $spc                                RS: " . $wo->folio_id;

```

```

$pdf->SetHeaderData(PDF_HEADER_LOGO_ORIGINAL, '40', $head, $txt);

```

```

$pdf->setHeaderFont(Array(PDF_FONT_NAME_MAIN, "", 10));
$pdf->setFooterFont(Array(PDF_FONT_NAME_DATA, "", PDF_FONT_SIZE_DATA));
$pdf->SetDefaultMonospacedFont(PDF_FONT_MONOSPACED);
$pdf->SetMargins(8, PDF_MARGIN_TOP, 8);
$pdf->SetHeaderMargin(PDF_MARGIN_HEADER);
$pdf->SetFooterMargin(PDF_MARGIN_FOOTER);
$pdf->SetAutoPageBreak(TRUE, PDF_MARGIN_BOTTOM);
$pdf->setImageScale(PDF_IMAGE_SCALE_RATIO);

```

```

if (@file_exists(dirname(__FILE__).'/lang/eng.php')) {
    require_once(dirname(__FILE__).'/lang/eng.php');
    $pdf->setLanguageArray($l);
}

```

```

$pdf->SetFont('helvetica', "", 8);

```

```

$pdf->AddPage('L', array('format' => 'A4'));

```

```

$pdf->SetFillColor(255, 255, 255);
$pdf->SetFont('helvetica', "", 10);

```

```

//Construcción del contenido del PDF

```

```

$tbl = <<<EOD

```

```

<br>

```

```

<table>

```

```

<thead>

```

```

<tr>

```

```

<th>

```

```

        <b>Cliente/Customer:</b><br>
        $wo->Empresa<br>
        $wo->Direccion1<br>
        $wo->Contacto<br>
        Telefono: $wo->TelefonoContacto Celular: $wo->CelularContacto<br>
    </th>
    <th>
    </th>

    <th>
    $qr
    </th>
</tr>
</thead>
</table>

```

EOD;

```
$pdf->writeHTML($tbl, true, true, true, true, "");
```

```

$w = array(8, 125, 12, 24, 24);
$pdf->SetFont('helvetica', 'B', 9);
$pdf->SetTextColor(255);
$pdf->Ln();$pdf->Ln();
$w = array(20, 20, 125, 20);
$pdf->SetTextColor(0);
$pdf->SetFont('helvetica', "", 10);
    $pdf->Ln();
$tabla_items="";

```

```

$tabla_items .= '<table >
    <thead>
        <tr>
            <th style="border: 1px solid #000; text-align: center; font-weight:
bold; width: 10%;>RS</th>
            <th style="border: 1px solid #000; text-align: center; font-weight:
bold; width: 10%;>Item</th>

```

```

                <th style="border: 1px solid #000; text-align: center; font-weight:
bold;width: 20%;" >Descripcion</th>
                <th style="border: 1px solid #000; text-align: center; font-weight:
bold;width: 10%;" >Loc.</th>
                <th style="border: 1px solid #000; text-align: center; font-weight:
bold;width: 10%;" >Venc.</th>
                <th style="border: 1px solid #000; text-align: center; font-weight:
bold;width: 10%;" >Cal.</th>
                <th style="border: 1px solid #000; text-align: center; font-weight:
bold;width: 10%;" >Tipo</th>
                <th style="border: 1px solid #000; text-align: center; font-weight:
bold;width: 10%;" >Motivo</th>
                <th style="border: 1px solid #000; text-align: center; font-weight:
bold;width: 10%;" >Estatus</th>
            </tr>
        </thead>
        <tbody>';
        foreach ($wo_detail as $key){

            $tabla_items .=
            <tr>
                <td style="border: 1px solid #000; text-align: center; tr:nth-child:background:
#F8F8F8; width: 10%;">'. $key->folio_id.'</td>
                <td style="border: 1px solid #000; text-align: center; tr:nth-child:background:
#F8F8F8; width: 10%;">'. $key->Item_Id.'</td>
                <td style="border: 1px solid #000; text-align: center; tr:nth-child:background:
#F8F8F8; width: 20%;">'. $key->CadenaDescripcion.'</td>
                <td style="border: 1px solid #000; text-align: center; tr:nth-child:background:
#F8F8F8; width: 10%;">'. $key->Localizacion.'</td>
                <td style="border: 1px solid #000; text-align: center; tr:nth-child:background:
#F8F8F8; width: 10%;">'. $key->vencimiento . '</td>
                <td style="border: 1px solid #000; text-align: center; tr:nth-child:background:
#F8F8F8; width: 10%;">'. $key->calibrado.'</td>
                <td style="border: 1px solid #000; text-align: center; tr:nth-child:background:
#F8F8F8; width: 10%;">'. $key->tipo_cal.'</td>
                <td style="border: 1px solid #000; text-align: center; tr:nth-child:background:
#F8F8F8; width: 10%;">'. $key->motivo.'</td>

```

```

        <td style="border: 1px solid #000; text-align: center; tr:nth-child:background:
#F8F8F8; width: 10%;">'. $key->estatus_item.'</td>
    </tr>';
}
$tabla_items .= '</tbody>
</table>
<br>
<br>
<br>';

$pdf->SetFont('helvetica', '', 9);
$pdf->writeHTML($tabla_items, true, false, false, false, "");
$pdf->Ln();
$pdf->SetFont('helvetica', '', 10);
$pdf->writeHTML("_____ ", true, false, false, false,
");
$pdf->MultiCell(150, 8, "Nombre y firma de quien recibe el servicio.", 0, 'L', 1, 0, "", true, 0,
false, true, 0);
$pdf->Ln();
$pdf->Ln();
$lista = "";
$lista .= "<h2>Comentarios:</h2>";
foreach ($cm as $ul) {
    $lista .= "
    <li>

        <span>
            ".$ul->nombre."<small> ".$ul->fecha."</small>
        </span>
        <span >".$ul->comentario."</span>
        <br>

    </li>";
}
$pdf->writeHTML($lista, true, false, false, false, "");

$pdf->Ln();
$notas = "";

```

```

$notas.="<h3>Notas:</h3>";
$notas.="
    <span>
        ".$wo->Notas."</span>
    </span>";
$pdf->writeHTML($notas, true, false, false, false, "");
$name = "WO".$id.'.pdf';

$pdf->Ln();
//Generación del PDF
$pdf->Output(sys_get_temp_dir().'/'.$name, 'I');
}

function ajax_getClientesCotizaciones(){
    $texto = $this->input->post('texto');

    // Consulta clientes con nombre corto válido, y si se proporciona texto, se filtra por
    coincidencia parcial
    $query = "SELECT Cust_ID, `Nombre Corto` as nombre FROM catalogo_clientes where
    `Nombre Corto` is not null and `Nombre Corto` != '.'";

    if($texto)
    {
        $query.=" and `Nombre Corto` like '%$texto%'";
    }

    $res = $this->MLConexion->consultar($query);

    if($res)
    {
        echo json_encode($res);
    }
}

function ver_foto($id){
    // Recupera y muestra la foto asociada a la orden de trabajo como imagen PNG

```



```

    $photo = $this->MLConexion->consultar('select * from WO_Master where WorkOrder_ID =
'. $id);
    if($photo)
    {
        header("Content-type: image/png");
        echo $photo->foto;
    }
    else {
        echo "ERROR";
    }
}
}

```

Privilegios.php

```
<?php
```

```
defined('BASEPATH') OR exit('No direct script access allowed');
```

```
class Privilegios extends CI_Controller {
```

```

    function __construct() {
        parent::__construct();
    }

```

```
    // Carga del modelo que maneja privilegios y de la conexión genérica
```

```
    $this->load->model('privilegios_model');
```

```
    $this->load->model('conexion_model','Conexion');
```

```
}
```

```
function index() {
```

```
    // Obtiene listado de puestos para mostrarlos en la vista de roles
```

```
    $datos['privilegios'] = $this->privilegios_model->listadoPuestos();
```

```
    $this->load->view('header');
```

```
    $this->load->view('privilegios/roles', $datos);
```

```
}
```

```
public function administrar($id_privilegio) {
```

```
    // Obtiene los privilegios específicos del rol seleccionado
```

```

    $datos['privilegio'] = $this->privilegios_model->getPrivilegios($id_privilegio);
    $this->load->view('header');
    $this->load->view('cambiar_privilegios', $datos);
}

function agregar() {
    $ACIERTOS = array();
    $ERRORES = array();

    // Obtiene el nombre del puesto desde el formulario y lo convierte a mayúsculas
    $nombre_puesto = trim(strtoupper($this->input->post('puesto')));
    $datos = array('puesto' => $nombre_puesto);

    // Intenta agregar el nuevo rol y guarda el resultado en la sesión
    if($this->privilegios_model->agregarPuesto($datos)) {
        $sacerto = array('titulo' => $nombre_puesto, 'detalle' => 'Se han agregado Rol');
        array_push($ACIERTOS, $sacerto);
    } else {
        $error = array('titulo' => 'ERROR', 'detalle' => 'Error al agregar Rol');
        array_push($ERRORES, $error);
    }

    $this->session->aciertos = $ACIERTOS;
    $this->session->errores = $ERRORES;

    // Redirige de nuevo a la sección de privilegios
    redirect(base_url('privilegios'));
}

public function modificar() {
    $ACIERTOS = array(); $ERRORES = array();

    // Se capturan los valores enviados desde el formulario
    $aprobador_compras = $this->input->post('opAprobadorCompra');
    $aprobador_compras_venta = $this->input->post('opAprobadorCompra_venta');
    $aprobador_cotizacion = $this->input->post('opAprobadorCotizacion');
    $usuario = $this->input->post('usuario');

```

```

// Determina quién es el aprobador de TC (puede ser el propio usuario si se seleccionó
autorizarTC)
if (filter_var($this->input->post('autorizarTC'), FILTER_VALIDATE_BOOLEAN) == true) {
    $aprobadorTC = $usuario;
} else {
    $aprobadorTC = $this->input->post('autorizadorTC');
}

// Actualiza los campos de autorizadores directamente por SQL
$query = "UPDATE usuarios set autorizador_compras = $aprobador_compras,
autorizador_compras_venta = $aprobador_compras_venta, autorizador_cotizacion =
$aprobador_cotizacion, autorizadorTC=$aprobadorTC where id=$usuario";
$this->Conexion->comando($query);

// Se arma el arreglo de privilegios a modificar
$datos = array(
    'usuario' => $usuario,
    'administrar_usuarios' => filter_var($this->input->post('administrar_usuarios'),
FILTER_VALIDATE_BOOLEAN),
    'administrar_empresas' => filter_var($this->input->post('administrar_empresas'),
FILTER_VALIDATE_BOOLEAN),
    'generar_tickets' => filter_var($this->input->post('generar_tickets'),
FILTER_VALIDATE_BOOLEAN),
    'tickets_it_soporte' => filter_var($this->input->post('tickets_it_soporte'),
FILTER_VALIDATE_BOOLEAN),
    'tickets_at_soporte' => filter_var($this->input->post('tickets_at_soporte'),
FILTER_VALIDATE_BOOLEAN),
    'tickets_ed_soporte' => filter_var($this->input->post('tickets_ed_soporte'),
FILTER_VALIDATE_BOOLEAN),
    'crear_qr_interno' => filter_var($this->input->post('crear_qr_interno'),
FILTER_VALIDATE_BOOLEAN),
    'crear_qr_venta' => filter_var($this->input->post('crear_qr_venta'),
FILTER_VALIDATE_BOOLEAN),
    'editar_qr' => filter_var($this->input->post('editar_qr'), FILTER_VALIDATE_BOOLEAN),
    'revisar_qr' => filter_var($this->input->post('revisar_qr'), FILTER_VALIDATE_BOOLEAN),
    'liberar_qr' => filter_var($this->input->post('liberar_qr'), FILTER_VALIDATE_BOOLEAN),
    'cancelar_pr' => filter_var($this->input->post('cancelar_pr'), FILTER_VALIDATE_BOOLEAN),
    'aprobar_pr' => filter_var($this->input->post('aprobar_pr'), FILTER_VALIDATE_BOOLEAN),

```

```

'aprobar_compra' => filter_var($this->input->post('aprobar_compra'),
FILTER_VALIDATE_BOOLEAN),
'qr_critico' => filter_var($this->input->post('qr_critico'), FILTER_VALIDATE_BOOLEAN),
'retroceder_qr' => filter_var($this->input->post('retroceder_qr'),
FILTER_VALIDATE_BOOLEAN),
'retroceder_po' => filter_var($this->input->post('retroceder_po'),
FILTER_VALIDATE_BOOLEAN),
'administrar_servicios' => filter_var($this->input->post('administrar_servicios'),
FILTER_VALIDATE_BOOLEAN),
'evaluar_requerimientos' => filter_var($this->input->post('evaluar_requerimientos'),
FILTER_VALIDATE_BOOLEAN),
'asignar_recursos' => filter_var($this->input->post('asignar_recursos'),
FILTER_VALIDATE_BOOLEAN),
'gestionar_recursos' => filter_var($this->input->post('gestionar_recursos'),
FILTER_VALIDATE_BOOLEAN),
'solicitar_facturas' => filter_var($this->input->post('solicitar_facturas'),
FILTER_VALIDATE_BOOLEAN),
'reponder_facturas' => filter_var($this->input->post('responder_facturas'),
FILTER_VALIDATE_BOOLEAN),
'documentacion_cliente' => filter_var($this->input->post('documentacion_cliente'),
FILTER_VALIDATE_BOOLEAN),
'documentacion_global' => filter_var($this->input->post('documentacion_global'),
FILTER_VALIDATE_BOOLEAN),
'bitacora_autos' => filter_var($this->input->post('bitacora_autos'),
FILTER_VALIDATE_BOOLEAN),
'generar_cotizaciones' => filter_var($this->input->post('generar_cotizaciones'),
FILTER_VALIDATE_BOOLEAN),
'administrar_cotizaciones' => filter_var($this->input->post('administrar_cotizaciones'),
FILTER_VALIDATE_BOOLEAN),
'aprobar_cotizacion' => filter_var($this->input->post('aprobar_cotizacion'),
FILTER_VALIDATE_BOOLEAN),
'compras_dashboard' => filter_var($this->input->post('compras_dashboard'),
FILTER_VALIDATE_BOOLEAN),
'administrar_equipos_it' => filter_var($this->input->post('administrar_equipos_it'),
FILTER_VALIDATE_BOOLEAN),
'mensajero' => filter_var($this->input->post('mensajero'), FILTER_VALIDATE_BOOLEAN),
'administrar_parametros_cotizacion' => filter_var($this->input-
>post('administrar_parametros_cotizacion'), FILTER_VALIDATE_BOOLEAN),

```

```

        'administrar_empresas_facturacion' => filter_var($this->input-
>post('administrar_empresas_facturacion'), FILTER_VALIDATE_BOOLEAN),
        'administrar_empresas_logistica' => filter_var($this->input-
>post('administrar_empresas_logistica'), FILTER_VALIDATE_BOOLEAN),
        'administrar_empresas_proveedor' => filter_var($this->input-
>post('administrar_empresas_proveedor'), FILTER_VALIDATE_BOOLEAN),
        'editar_facturas' => filter_var($this->input->post('editar_facturas'),
FILTER_VALIDATE_BOOLEAN),
        'autorizar_facturas' => filter_var($this->input->post('autorizar_facturas'),
FILTER_VALIDATE_BOOLEAN),
        'enviar_facturas_logistica' => filter_var($this->input->post('enviar_facturas_logistica'),
FILTER_VALIDATE_BOOLEAN),
        'reloj' => filter_var($this->input->post('reloj'), FILTER_VALIDATE_BOOLEAN),
        'produTC'=> filter_var($this->input->post('produTC'), FILTER_VALIDATE_BOOLEAN),
        'crearPedidosTC'=> filter_var($this->input->post('crearPedidosTC'),
FILTER_VALIDATE_BOOLEAN),
        'autorizarTC'=> filter_var($this->input->post('autorizarTC'), FILTER_VALIDATE_BOOLEAN),
        'movimientosTC'=> filter_var($this->input->post('movimientosTC'),
FILTER_VALIDATE_BOOLEAN),
        'cotDashboard'=>filter_var($this->input->post('dashboardCot'),
FILTER_VALIDATE_BOOLEAN),
        'cotCalendario'=>filter_var($this->input->post('calCot'), FILTER_VALIDATE_BOOLEAN),
        'ticketsDash'=>filter_var($this->input->post('ticketsDash'), FILTER_VALIDATE_BOOLEAN),
        'equiposCal'=>filter_var($this->input->post('equiposCal'), FILTER_VALIDATE_BOOLEAN),
        'editarPO'=>filter_var($this->input->post('editarPO'), FILTER_VALIDATE_BOOLEAN),
        'aprobar_po'=>filter_var($this->input->post('aprobar_po'), FILTER_VALIDATE_BOOLEAN),
        'adminCompras'=>filter_var($this->input->post('adminCompras'),
FILTER_VALIDATE_BOOLEAN),
        'solicitudesPago'=>filter_var($this->input->post('solicitudesPago'),
FILTER_VALIDATE_BOOLEAN),
        'responderPago'=>filter_var($this->input->post('responderPago'),
FILTER_VALIDATE_BOOLEAN),
        'cafeteria'=>filter_var($this->input->post('cafeteria'), FILTER_VALIDATE_BOOLEAN),
        'crear_wo'=>filter_var($this->input->post('crear_wo'), FILTER_VALIDATE_BOOLEAN),
        'cerrar_wo'=>filter_var($this->input->post('cerrar_wo'), FILTER_VALIDATE_BOOLEAN),
        'recibirNotPO'=>filter_var($this->input->post('recibirNotPO'), FILTER_VALIDATE_BOOLEAN),
    );

```

```

// Guarda los privilegios y actualiza sesión si aplica
if ($this->privilegios_model->setPrivilegios($datos, $usuario)) {
    if ($this->session->id == $usuario) {
        $this->load->model('usuarios_model');
        $this->session->privilegios = $this->usuarios_model->getPrivilegios($this->session->id);
    }
    $acierto = array('titulo' => 'Privilegios', 'detalle' => 'Se han modificado Privilegios');
    array_push($ACIERTOS, $acierto);
} else {
    $error = array('titulo' => 'ERROR', 'detalle' => 'Error al modificar Privilegios');
    array_push($ERRORES, $error);
}

// Guarda los mensajes y redirige a la vista del usuario
$this->session->aciertos = $ACIERTOS;
$this->session->errores = $ERRORES;
redirect(base_url('usuarios/ver/') . $usuario);
}
}

```

Recursos.php

```
<?php
```

```
defined('BASEPATH') OR exit('No direct script access allowed');
```

```
class Recursos extends CI_Controller {
```

```

    function __construct() {
        parent::__construct();
        // La siguiente línea fue eliminada porque estaba comentada e inactiva
        // $this->load->library('aos_funciones');
    }

```

```

// Carga la vista para asignación de recursos, incluyendo la tasa USD y su fecha de actualización
function asignar_recursos(){
    $usd = $this->aos_funciones->getUSD();
    $data['usd'] = $usd[0];
}

```

```

$data['usd_actualizacion'] = $usd[1];

$this->load->view('header');
$this->load->view('recursos/asignar_recursos', $data);
}

// Carga la vista para mostrar la forma de pago de una orden específica
function ver_forma_pago(){
    if(isset($_POST["id"])){
        $id = $this->input->post('id');
    } else {
        redirect(base_url('inicio'));
    }

    $data["id"] = $id;

    $this->load->view('header');
    $this->load->view('recursos/ver_forma_pago', $data);
}

// Devuelve por AJAX la lista de órdenes de compra relacionadas a recursos pendientes de pago
function ajax_getPO_recursos(){
    $metodo = 0; $date_sort = 0;

    if(isset($_POST['metodo'])) {
        $metodo = $this->input->post('metodo');
    }

    if(isset($_POST['date_sort'])) {
        $date_sort = $this->input->post('date_sort');
    }

    // Se consultan únicamente órdenes con estatus válidos y que aún no han sido marcadas
    como "PAGADO"
    $query = "SELECT PO.*, concat(U.nombre, ' ', U.paterno) as User from ordenes_compra PO
inner join usuarios U on U.id = PO.usuario where 1 = 1";
    $query .= " and PO.recurso != 'PAGADO'";

```

```
$query .= " and (PO.estatus = 'AUTORIZADA' or PO.estatus = 'ORDENADA' or PO.estatus = 'RECIBIDA' or PO.estatus = 'CERRADA')";
```

```
if($metodo != 0){  
    $query .= " and PO.metodo_pago = $metodo";  
}
```

```
if($date_sort != 0){  
    $query .= " order by PO.fecha_cobro desc";  
}
```

```
$res = $this->Conexion->consultar($query);
```

```
if($res){  
    echo json_encode($res);  
} else {  
    echo "";  
}  
}
```

```
// Consulta los métodos de pago activos junto con su saldo disponible (ingresos - egresos)  
function ajax_getMetodosPago(){  
    $query = "SELECT MP.*, (SELECT (ifnull(sum(MPM.monto), 0)) as Ingreso from  
empresa_metodos_pago_movimientos MPM where MPM.metodo = MP.id) -  
(ifnull(sum(PO.total * PO.tipo_cambio), 0)) as Saldo from empresa_metodos_pago MP left join  
ordenes_compra PO on PO.metodo_pago = MP.id and PO.recurso != 'PENDIENTE' and  
(PO.estatus != 'RECHAZADA' and PO.estatus != 'CANCELADA') where MP.activo = '1' group by  
MP.id";
```

```
$res = $this->Conexion->consultar($query);  
if($res){  
    echo json_encode($res);  
} else {  
    echo "";  
}  
}
```



```

// Marca una orden de compra como provisionada, registra la fecha y dispara una notificación
por bajo saldo
function ajax_setPO(){
    $this->load->library('recursos_funciones');
    $po = json_decode($this->input->post('PO'));

    $res = $this->Conexion->modificar('ordenes_compra', $po, array('fecha_provision' =>
'CURRENT_TIMESTAMP()), array('id' => $po->id));
    if($res > 0){
        $this->recursos_funciones->NotificacionMinimo($po->metodo_pago);
        echo "1";
    }
}

// Inserta un nuevo fondeo (abono) a un método de pago y actualiza su estado de fondeo
pendiente
function ajax_setFondeo(){
    $registro = json_decode($this->input->post('registro'));
    $registro->usuario = $this->session->id;

    $res = $this->Conexion->insertar('empresa_metodos_pago_movimientos', $registro,
array('fecha' => 'CURRENT_TIMESTAMP()));
    if($res > 0){
        $this->Conexion->modificar('empresa_metodos_pago', array('fondear' => 0), null, array('id'
=> $registro->metodo));
        echo "1";
    }
}

// Obtiene todos los movimientos de un método de pago: fondeos (abonos) y pagos (ordenes
de compra)
function ajax_getMovimientos(){
    $id = $this->input->post('id');

    $query = "SELECT EMM.id, EMM.monto, '1' as tipo_cambio, EMM.fecha, 'ABONO' as TipoM,
EM.nombre, EM.tipo from empresa_metodos_pago_movimientos EMM inner join
empresa_metodos_pago EM on EMM.metodo = EM.id where EMM.metodo = $id";
    $query .= " UNION ";

```

```
$query := "SELECT PO.id, PO.total, PO.tipo_cambio, PO.fecha_provision, PO.recurso,
M.nombre, M.tipo from ordenes_compra PO inner join empresa_metodos_pago M on
PO.metodo_pago = M.id where PO.metodo_pago = $id and PO.recurso != 'PENDIENTE' and
(PO.estatus != 'RECHAZADA' and PO.estatus != 'CANCELADA') order by fecha";
```

```
$res = $this->Conexion->consultar($query);
if($res){
    echo json_encode($res);
} else {
    echo "";
}
}
}
```

Reloj_checador.php

```
<?php
```

```
defined('BASEPATH') OR exit('No direct script access allowed');
```

```
class Reloj_checador extends CI_Controller {
```

```
    function __construct() {
        parent::__construct();
        $this->load->model('reloj_model'); // Modelo que maneja la lógica del checador
    }
```

```
function index() {
    // Vista principal del checador con valores predeterminados
    $data['usuario'] = 0;
    $data['idusuario'] = 0;
    $idusuario = $this->session->id;
    if (isset($idusuario)) {
        $data['idusuario'] = $idusuario;
    }
    $this->load->view('reloj_checador/checador', $data);
}
```

```

function checar() {
    // Vista del checador cuando ya se tiene un usuario en sesión
    $data['usuario'] = $this->session->no_empleado;
    $data['idusuario'] = $this->session->id;
    $this->load->view('reloj_checador/checador', $data);
}

function reporte() {
    // Vista del reporte semanal del checador
    $datos['semanaActual'] = $this->aos_funciones->no_semana(); // Obtiene número de semana
    actual
    $this->load->view('header');
    $this->load->view('reloj_checador/reporte', $datos);
}

function comprobar_usuario_ajax() {
    // Verifica si el número de empleado ingresado existe
    $no_empleado = strtoupper(trim($this->input->post('no_empleado')));
    $user = $this->reloj_model->getUsuario($no_empleado);
    if ($user) {
        echo json_encode($user);
    }
}

function checar_ajax() {
    // Registra una checada de entrada o salida
    $this->load->model('usuarios_model');
    $data['foto'] = $this->input->post('src');
    $data['usuario'] = $this->input->post('usuario');
    $data['tipo'] = $this->input->post('tipo');

    $res = $this->reloj_model->checarEntrada($data); // Guarda la checada en la base de datos
    if ($res) {
        $checada = $this->reloj_model->getEntrada($res); // Obtiene la hora registrada
        $this->usuarios_model->ultimaSesion($data['usuario']); // Actualiza última sesión del
        usuario
        $data['hora'] = $checada->hora;
        echo json_encode($data);
    }
}

```

```

    } else {
        echo "";
    }
}

function checadas_ajax() {
    // Devuelve las checadas del usuario actual
    $idusuario = $this->input->post('idusuario');
    $res = $this->reloj_model->getChecadas($idusuario);
    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

function reporte_ajax() {
    // Genera el reporte de checadas por semana/año
    $year = $this->input->post('year');
    $semana = $this->input->post('semana');

    $fecha = $this->aos_funciones->fecha_semana($year, $semana, $semana); // Devuelve
    fechas inicio/fin
    $data['inicio'] = $fecha[0];
    $data['final'] = $fecha[1];

    $res = $this->reloj_model->getReporte($data);
    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}
}

```

Requerimientos.php

<?php

```
defined('BASEPATH') OR exit('No direct script access allowed');
```

```
class Requerimientos extends CI_Controller {
```

```
    function __construct() {
        parent::__construct();
        $this->load->model('conexion_model', 'Conexion'); // Modelo de conexión general a la base
        de datos
        $this->load->library('correos_servicios'); // Librería para envío de correos relacionados a
        servicios
    }
```

```
    function index() {
        // Vista principal del catálogo de requerimientos
        $this->load->view('header');
        $this->load->view('requerimientos/catalogo');
    }
```

```
    function crear_requerimiento() {
        // Vista para crear un nuevo requerimiento
        $data["id"] = 0;
        $this->load->view('header');
        $this->load->view('requerimientos/crear_requerimiento', $data);
    }
```

```
    function editar($id = 0) {
        // Vista para editar un requerimiento existente
        if (isset($_POST["id"])) {
            $id = $this->input->post('id');
        } else if ($id == 0) {
            redirect(base_url('inicio')); // Redirección si no hay ID válido
        }
    }
```

```
    $data["id"] = $id;
```

```

$this->load->view('header');
$this->load->view('requerimientos/crear_requerimiento', $data);
}

```

```

function ver($id = 0) {
    // Vista para ver detalles de un requerimiento
    if (isset($_POST['id'])) {
        $id = $this->input->post('id');
    } else if ($id == 0) {
        redirect(base_url('inicio')); // Redirección si no hay ID válido
    }
}

```

```

$data['id'] = $id;
$this->load->view('header');
$this->load->view('requerimientos/ver', $data);
}

```

```

function ajax_getRequerimientos() {
    $texto = $this->input->post('texto');
    $parametro = $this->input->post('parametro');
}

```

```

$query = "SELECT R.*, concat(U.nombre, ' ', U.paterno) as User from requerimientos R inner
join usuarios U on R.usuario = U.id where 1 = 1";

```

// Filtros deshabilitados actualmente. Se puede agregar lógica basada en \$parametro y \$texto si es necesario.

```

$query .= " order by R.fecha desc";

```

```

$res = $this->Conexion->consultar($query);
if ($res) {
    echo json_encode($res);
}
}

```

```

function ajax_getRequerimiento() {
    $id = $this->input->post('id');
}

```

```
$query = "SELECT R.*, concat(U.nombre, ' ', U.paterno) as User, concat(UC.nombre, ' ', UC.paterno) as Cerrador from requerimientos R inner join usuarios U on R.usuario = U.id left join usuarios UC on R.despachador = UC.id where 1 = 1 and R.id = '$id'";
```

```
$res = $this->Conexion->consultar($query, TRUE);
if ($res) {
    echo json_encode($res);
} else {
    echo "";
}
}
```

```
function ajax_setRequerimiento() {
    $requerimiento = json_decode($this->input->post('requerimiento'));

    // Si el requerimiento no tiene ID, es un nuevo registro
    if (!isset($requerimiento->id)) {
        $requerimiento->usuario = $this->session->id;
        $func["fecha"] = "CURRENT_TIMESTAMP()";
        $res = $this->Conexion->insertar('requerimientos', $requerimiento, $func);
        $requerimiento->id = $res;
        $res = $res > 0;
    } else {
        // Si es modificación, verificar si se debe registrar fecha de cierre
        $fc = $requerimiento->estatus == "EVALUADO" | $requerimiento->estatus == "NO
PROCEDE";
        $res = $this->Conexion->modificar('requerimientos', $requerimiento, $fc ?
array('fecha_cierre' => 'CURRENT_TIMESTAMP()') : null, array('id' => $requerimiento->id)) >= 0;
    }
}
```

```
// Determinar destinatario de correo según estatus
$evaluador = json_decode($requerimiento->evaluadores);
switch ($requerimiento->estatus) {
    case "ABIERTO":
    case "CANALIZADO":
        $u = array_pop($evaluador);
        $res = $this->Conexion->consultar("SELECT ifnull(correo, 'N/A') as Mail from usuarios
where id = $u", TRUE);
```

```

        $requerimiento->correo = $res->Mail;
        break;

        case "RECHAZADO":
        case "EVALUADO":
        case "NO PROCEDE":
            $res = $this->Conexion->consultar("SELECT ifnull(corr, 'N/A') as Mail from usuarios
where id = $requerimiento->id", TRUE);
            $requerimiento->correo = $res->Mail;
            break;
    }

    // Envío de notificación por correo
    $this->correos_servicios->requerimiento($requerimiento);

    if ($res) {
        echo "1";
    }
}

function ajax_getEvaluadores() {
    // Consulta a usuarios activos que tienen privilegio para evaluar requerimientos
    $query = "SELECT U.id, concat(U.nombre,' ',U.paterno) as User from usuarios U inner join
privilegios P on U.id = P.usuario where U.activo = '1' and P.evaluar_requerimientos = '1'";

    $res = $this->Conexion->consultar($query);
    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

function ajax_getServicios() {
    // Servicios catalogados relacionados con un requerimiento específico por fabricante y
    modelo
    $fabricante = $this->input->post('fabricante');
    $modelo = $this->input->post('modelo');

```



```
$query = "SELECT S.*, S.descripcion as DescripcionServicio, R.descripcion, R.id as IdReq from
requerimientos R inner join servicios S on S.id = R.servicio where R.catalogado = '1' and
upper(trim(R.fabricante)) = '$fabricante' and upper(trim(R.modelo)) = '$modelo' group by S.id";
```

```
$res = $this->Conexion->consultar($query);
if ($res) {
    echo json_encode($res);
} else {
    echo "";
}
}
```

```
function ajax_getRequerimientoArchivos() {
    // Archivos asociados a un requerimiento junto con el nombre del usuario que los subió
    $id = $this->input->post('id');
```

```
$query = "SELECT RA.id, RA.fecha, RA.nombre, RA.comentarios, concat(U.nombre, ' ',
U.paterno) as User from requerimiento_archivos RA inner join usuarios U on U.id = RA.usuario
where RA.requerimiento = '$id'";
```

```
$res = $this->Conexion->consultar($query);
if ($res) {
    echo json_encode($res);
} else {
    echo "";
}
}
```

```
function ajax_subirArchivo() {
    // Recoge y guarda un archivo subido relacionado a un requerimiento, incluyendo el
    contenido binario
    $datos['usuario'] = $this->session->id;
    $datos['requerimiento'] = $this->input->post('requerimiento');
    $datos['nombre'] = $this->input->post('nombre');
    $datos['comentarios'] = $this->input->post('comentarios');
    $datos['archivo'] = file_get_contents($_FILES['file']['tmp_name']); // contenido del archivo
```

```

$func['fecha'] = "CURRENT_TIMESTAMP()";

$this->Conexion->insertar("requerimiento_archivos", $datos, $func);
}

function test() {
    // Prueba rápida para verificar hora del servidor
    echo date("Y-m-d h:i:s");
}

function ajax_setRequerimientoComentario() {
    // Agrega un nuevo comentario (como entrada JSON) al campo 'comentarios' del
    requerimiento
    $id = $this->input->post('id');
    $comentario = $this->input->post('comentario');
    $id_user = $this->session->id;

    if ($this->Conexion->comando("UPDATE requerimientos set comentarios =
JSON_MERGE(comentarios, JSON_ARRAY(JSON_ARRAY($id_user, CURRENT_TIMESTAMP(),
'$comentario')))) where id = $id")) {
        echo "1";
    }

    // Esta línea imprime la consulta, posiblemente para depuración
    echo "UPDATE requerimientos set comentarios = JSON_MERGE(comentarios,
JSON_ARRAY(JSON_ARRAY($id_user, CURRENT_TIMESTAMP(), '$comentario')))) where id = $id";
}
}

```

Seguridad.php

```
<?php
```

```
defined('BASEPATH') OR exit('No direct script access allowed');
```

```
class Seguridad extends CI_Controller {
```

```
    function __construct() {
```

```

    parent::__construct();
}

function vencimiento_password() {
    // Vista para advertencia de vencimiento de contraseña
    $this->load->view('seguridad/password');
}

function recuperacion_password() {
    // Vista para recuperación de contraseña mediante correo
    $this->load->view('seguridad/recuperacion_password');
}

function reiniciar_password($link) {
    // Validación del enlace de recuperación de contraseña y reinicio de sesión temporal
    date_default_timezone_set('America/Chihuahua');
    $res = $this->Conexion->consultar("SELECT R.vencimiento, U.id, concat(U.nombre, ' ',
U.paterno) as Name, U.activo, U.password from recuperacion_password R inner join usuarios U
on U.id = R.usuario where R.link = '$link'", TRUE);

    $today = date("y-m-d H:i", strtotime("now"));
    $fecha = date("y-m-d H:i", strtotime($res->vencimiento));

    if ($today > $fecha) {
        echo "LIGA VENCIDA"; // El enlace ya expiró
    } else {
        // Se establece sesión temporal para permitir el cambio de contraseña
        $this->session->id = $res->id;
        $this->session->activo = $res->activo;
        $this->session->nombre = $res->Name;
        $this->session->password = $res->password;
        $this->session->vencimiento_password = date("y-m-d H:i", strtotime("last month"));

        redirect(base_url('inicio'));
    }
}

function ajax_changePass() {

```

```

// Cambia la contraseña del usuario si es distinta de la anterior
$pass = sha1($this->input->post('password'));
if (strtoupper($this->session->password) != strtoupper($pass)) {
    $this->session->sess_destroy(); // Cierra sesión para evitar que continúe con sesión vieja
    echo $this->Conexion->modificar('usuarios', array('password' => $pass),
array('vencimiento_password' => 'CURRENT_TIMESTAMP() + INTERVAL 30 day'), array('id' =>
$this->session->id));
    }
}

function ajax_recoverPass() {
    // Proceso de recuperación de contraseña: valida usuario, genera link único, guarda y envía
correo
    $noempleado = $this->input->post('noempleado');
    $correo = $this->input->post('correo');

    $res = $this->Conexion->consultar("SELECT id, correo, concat(nombre, ' ', paterno) as Name
from usuarios where activo = 1 and no_empleado='$noempleado' and correo = '$correo' limit
1", TRUE);
    if ($res) {
        $data['usuario'] = $res->id;
        $data['link'] = uniqid();
        $func['fecha'] = "CURRENT_TIMESTAMP()";
        $func['vencimiento'] = "CURRENT_TIMESTAMP() + interval 6 hour";
        $this->Conexion->insertar('recuperacion_password', $data, $func);

        $this->load->library('email');
        $logo = base_url('template/images/logo.png');
        $url = base_url('seguridad/reiniciar_password/') . $data['link'];

        $mensaje = "
        <img width='400' src='$logo'><br>
        <h1><font face='Times'>SIGA-MAS</font></h1>
        <h2>Recuperación de Contraseña</h2>
        <p>Ingresa a la liga debajo para reestablecer tu contraseña</p>
        <br>
        <a href='$url' class='btn btn-primary'>Reestablecer contraseña</a>";
    }
}

```

```

        $this->email->from('tickets@masmetrologia.mx', 'Soporte SIGA-MAS');
        $this->email->to($res->correo);
        $this->email->subject('Recuperación de Contraseña');
        $this->email->message($mensaje);
        $this->email->send();

        echo json_encode($res);
    }
}

function ajax_md5() {
    // Devuelve el hash MD5 del texto recibido por POST
    echo md5($this->input->post('text'));
}

function md5($texto) {
    // Devuelve el hash MD5 del texto recibido por parámetro
    echo md5($texto);
}
}

```

Servicios.php

```

<?php

defined('BASEPATH') OR exit('No direct script access allowed');

class Servicios extends CI_Controller {

    function __construct() {
        parent::__construct();
        $this->load->model('conexion_model', 'Conexion');
    }

    function index() {
        // Carga la vista principal del catálogo de servicios
        $this->load->view('header');
        $this->load->view('servicios/catalogo');
    }
}

```

```
}
```

```
// Función para obtener servicios según distintos filtros (id, código, texto, tipo, etc.)
```

```
function ajax_getServicios() {
```

```
    $codigo = $this->input->post('codigo');
```

```
    $id = $this->input->post('id');
```

```
    $texto = $this->input->post('texto');
```

```
    $parametro = $this->input->post('parametro');
```

```
    $tipo = json_decode($this->input->post('tipo'));
```

```
    // Consulta base para obtener servicios y sus precios asociados
```

```
    $query = "SELECT S.*, CP.bajo_a as bajo, CP.alto_a as alto from servicios S inner join  
claves_precio CP on S.clave_precio = CP.id where 1 = 1";
```

```
    // Filtro por código exacto
```

```
    if (isset($codigo)) {
```

```
        $query .= " and S.codigo = '$codigo'";
```

```
    }
```

```
    // Filtro por ID exacto
```

```
    else if (isset($id)) {
```

```
        $query .= " and S.id = '$id'";
```

```
    }
```

```
    // Filtros dinámicos por tipo de búsqueda (contenido, magnitud, código parcial, etc.)
```

```
    else {
```

```
        if (!empty($texto)) {
```

```
            if ($parametro == "id") {
```

```
                $query .= " and S.id = '$texto'";
```

```
            }
```

```
            if ($parametro == "codigo") {
```

```
                $query .= " and S.codigo like '$texto%'";
```

```
            }
```

```
            if ($parametro == "contenido") {
```

```
                $query .= " and S.descripcion like '%$texto%'";
```

```
            }
```

```
            if ($parametro == "magnitud") {
```

```
                $query .= " and S.magnitud = '$texto'";
```

```
            }
```

```
            if ($parametro == "codigo_contenido") {
```

```

        $query .= " and (S.descripcion like '%$texto%' or S.codigo like '$texto%')";
    }
}

// Filtro por tipo de servicio si se especifica (pueden ser múltiples)
if (isset($tipo)) {
    if (count($tipo) > 0) {
        $query .= " and ( 1 = 0 ";
        foreach ($tipo as $key => $value) {
            $query .= " or S.tipo = '$value'";
        }
        $query .= " )";
    }
}

$query .= " order by S.codigo";

$row = isset($codigo) || isset($id); // Si se busca un solo resultado
$res = $this->Conexion->consultar($query, $row);
if ($res) {
    echo json_encode($res);
}
}

function ajax_getServicio() {
    $id = $this->input->post('id');

    // Consulta para obtener un servicio por ID, junto con su servicio estándar (si aplica)
    $query = "SELECT S.*, ifnull(SE.codigo,'') as CodeEst, ifnull(SE.id,0) as IdEst from servicios S
left join servicios SE on S.estandar = SE.id where 1 = 1 and S.id = $id";

    $res = $this->Conexion->consultar($query, TRUE);
    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

```

```
}
```

```
function ajax_setServicio() {
    $servicio = json_decode($this->input->post('servicio'));

    // Datos comunes para insertar o actualizar un servicio
    $datos['tipo'] = $servicio->tipo;
    $datos['tipo_calibracion'] = $servicio->tipo_calibracion;
    $datos['descripcion'] = $servicio->descripcion;
    $datos['observaciones'] = $servicio->observaciones;
    $datos['sitio'] = $servicio->sitio;
    $datos['interno'] = $servicio->interno;
    $datos['proveedor'] = $servicio->proveedor;
    $datos['tags'] = $servicio->tags;
    $datos['clave_precio'] = $servicio->clave_precio;
    $datos['activo'] = $servicio->activo;
    $datos['estandar'] = $servicio->estandar;

    $res = FALSE;

    // Si es un nuevo servicio
    if ($servicio->id == 0) {
        $func['codigo'] = "(SELECT concat('$servicio->prefijo', LPAD(count(S.prefijo) + 1, 4, 0)) from
servicios S where S.prefijo = '$servicio->prefijo')";
        $datos['magnitud'] = $servicio->magnitud;
        $datos['prefijo'] = $servicio->prefijo;
        $res = $this->Conexion->insertar('servicios', $datos, $func) > 0;
    } else {
        // Si es una actualización
        $where['id'] = $servicio->id;
        $res = $this->Conexion->modificar('servicios', $datos, null, $where) >= 0;

        // También actualiza los servicios que lo tienen como estándar
        $this->Conexion->modificar('servicios', array('activo' => $servicio->activo), null,
array('estandar' => $servicio->id));
    }

    if ($res) {
```



```

        echo "1";
    }
}

function ajax_deleteServicio() {
    $id = $this->input->post('id');
    $where['id'] = $id;

    // Elimina el servicio si existe
    if ($this->Conexion->eliminar('servicios', $where)) {
        echo "1";
    }
}

function ajax_codeExists() {
    $codigo = $this->input->post('codigo');

    // Verifica si ya existe un código de servicio igual
    $query = "SELECT count(S.codigo) as CodeCount from servicios S where 1 = 1 and codigo = '$codigo'";

    $res = $this->Conexion->consultar($query, TRUE);
    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}
}
}

```

Solicitudes_pago.php

```
<?php
```

```
defined('BASEPATH') OR exit('No direct script access allowed');
```

```
class Solicitudes_pago extends CI_Controller {
```

```

function __construct() {
    parent::__construct();
    $this->load->library('correos');
    $this->load->model('pago_model');
    $this->load->model('descargas_model');
}

function generar_pago() {
    $this->load->view('header');
    $this->load->view('solicitudes_pago/generar_pago');
}

function catalogo_solicitudes() {
    $this->load->view('header');
    $this->load->view('solicitudes_pago/catalogo_solicitudes');
}

function construccion_pago($idtemp) {
    // Consulta los datos temporales de un pago junto con información de proveedor, orden de
    compra y empresa
    $query = "SELECT pt.*, po.proveedor, po.total, po.estatus as estatusPO, pr.monto_credito,
    pr.moneda_credito, pr.terminos_pago, e.nombre from pago_temp pt join ordenes_compra po
    on po.id=pt.po JOIN proveedores pr on po.proveedor = pr.empresa join empresas e on e.id =
    pr.empresa WHERE pt.idtemp ='$idtemp'";

    $res = $this->Conexion->consultar($query);
    $data['idTemp'] = $idtemp;
    $data['pago'] = $res;

    $this->load->view('header');
    $this->load->view('solicitudes_pago/construccion_pago', $data);
}

function ver_pago($id) {
    $data['id'] = $id;

    // Obtiene comentarios, fotos y contacto relacionados al pago
    $data['comentarios'] = $this->pago_model->verPago_comentarios($id);

```

```

$data['comentarios_fotos'] = $this->pago_model->verPago_comentarios_fotos($id);
$data['contacto'] = $this->pago_model->contactos_pago($id);

// Información general del pago
$query = "SELECT s.*, p.monto_credito, p.empresa as idEmpresa, p.moneda_credito,
p.terminos_pago, e.nombre, concat(u.nombre, ' ', u.paterno) as user from solicitudes_pago s
JOIN proveedores p on s.idprov = p.empresa JOIN empresas e on p.empresa = e.id join usuarios
u on s.idus = u.id WHERE s.id = '$id'";

// Detalle de órdenes de compra vinculadas al pago
$q = "SELECT ps.*, concat(u.nombre, ' ', u.paterno) as requisitor, po.tipo, po.total, po.estatus,
concat(ua.nombre, ' ', ua.paterno) as aprobador FROM `Po_solicitudes` ps join ordenes_compra
po on po.id = ps.idPO join usuarios u on u.id=po.usuario JOIN usuarios ua on
ua.id=po.aprobador WHERE idPago = '$id'";

$data['pago'] = $this->Conexion->consultar($query, TRUE);
$data['pos'] = $this->Conexion->consultar($q);

$this->load->view('header');
$this->load->view('solicitudes_pago/editar_pago', $data);
}

function ajax_getPoPagos() {
    $texto = $this->input->post('texto');
    $parametro = $this->input->post('parametro');
    $id_proveedor = $this->input->post('id_proveedor');
    $tipo = $this->input->post('tipo');
    $moneda = $this->input->post('moneda');

    // Consulta órdenes de compra pendientes de pago con filtros dinámicos
    $query = "SELECT PO.id, E.id as IdProv, E.nombre as Prov, PO.tipo, PO.total, PO.moneda,
PO.estatus_pago from ordenes_compra PO inner join empresas E on PO.proveedor = E.id where
1=1";

    if($id_proveedor > 0) {
        $query .= " and E.id = '$id_proveedor' and PO.moneda = '$moneda' and PO.tipo = '$tipo'";
    }
}

```

```

$query = " and PO.estatus_pago = 'PENDIENTE'";

if(!empty($texto)) {
    if($parametro == "folio") {
        $query = " and PO.id = '$texto'";
    }
    if($parametro == "proveedor") {
        $query = " having Prov like '%$texto%'";
    }
}

$res = $this->Conexion->consultar($query);
if($res) {
    echo json_encode($res);
} else {
    echo "";
}
}

function ajax_setTempPago() {
    $idtemp = uniqid();

    // Decodifica las órdenes de compra seleccionadas y las inserta en tabla temporal
    $po = json_decode($this->input->post('pos'));
    foreach($po as $elem) {
        $data['idtemp'] = $idtemp;
        $data['us'] = $this->session->id;
        $data['po'] = $elem;
        $this->Conexion->insertar('pago_temp', $data, null);
    }
    echo $idtemp;
}

function ajax_getTempPago() {

    // Consulta productos relacionados a las órdenes seleccionadas

```

```

$query = "SELECT PR.*, ifnull(JSON_UNQUOTE(PR.atributos->'$.modelo'), '') as Modelo,
ifnull(JSON_UNQUOTE(PR.atributos->'$.marca'), '') as Marca,
ifnull(JSON_UNQUOTE(PR.atributos->'$.serie'), '') as Serie, QRP.costos, QRP.factor, P.entrega from
prs PR inner join qr_proveedores QRP on PR.qr_proveedor = QRP.id inner join proveedores P on
P.empresa = QRP.empresa where 1 != 1";
foreach ($pos as $elem) {
    $query .= " or PR.id = '$elem'";
}

$res = $this->Conexion->consultar($query);
if($res) {
    echo json_encode($res);
} else {
    echo "";
}
}

function generarSolicitud($id) {
    $total = null;

    // Consulta la orden de compra relacionada al ID temporal
    $query = "SELECT pt.*, po.proveedor, po.total FROM pago_temp pt JOIN ordenes_compra po
on pt.po = po.id where pt.idtemp = '$id'";
    $r = $this->Conexion->consultar($query, TRUE);
    $res = $this->Conexion->consultar($query);

    foreach ($res as $elem) {
        $total += $elem->total;
    }

    // Inserta la solicitud de pago con su información base
    $data['idus'] = $this->session->id;
    $data['idprov'] = $r->proveedor;
    $data['estatus'] = 'PENDIENTE';
    $data['estatus_factura'] = 'PENDIENTE';
    $data['estatus_complemento'] = 'PENDIENTE';
    $data['total'] = $total;

```

```

$idPago = $this->Conexion->insertar('solicitudes_pago', $data, null);

// Relaciona las órdenes con la solicitud de pago
foreach ($res as $elem) {
    $pago['idPago'] = $idPago;
    $pago['idPO'] = $elem->po;
    $this->Conexion->insertar('Po_solicitudes', $pago, null);

    $dato['estatus_pago'] = "SOLICITADA";
    $where['id'] = $elem->po;
    $this->Conexion->modificar('ordenes_compra', $dato, null, $where);
}

redirect(base_url('solicitudes_pago/ver_pago/' . $idPago));
}

function agregarComentarioPago() {
// Inserta un comentario asociado a una solicitud de pago
$id = $this->input->post('id');
$comentario = $this->input->post('comentario');

$data = array(
    'idpago' => $id,
    'usuario' => $this->session->id,
    'comentario' => $comentario,
);

$funciones['fecha'] = 'current_timestamp()';

$this->Conexion->insertar('pago_comentarios', $data, $funciones);

redirect(base_url('solicitudes_pago/ver_pago/' . $id));
}

function uploadPreFactura() {
// Guarda el archivo PDF de la prefactura en la solicitud de pago
$id = $this->input->post('id');
$datos['prefactura'] = file_get_contents($_FILES['file']['tmp_name']);

```

```

    $this->pago_model->updateSolicitudPago($id, $datos);
}

function uploadFactura() {
    // Guarda el archivo PDF de la factura en la solicitud de pago y registra fecha
    $date = date('Y-m-d h:i:s');
    $id = $this->input->post('id');
    $datos['factura'] = file_get_contents($_FILES['file']['tmp_name']);
    $datos['fecha_factura'] = $date;
    $this->pago_model->updateSolicitudPago($id, $datos);

    $query = "SELECT factura, xml from solicitudes_pago where id=" . $id;
    $res = $this->Conexion->consultar($query, TRUE);
}

function uploadXML() {
    // Guarda el archivo XML en la solicitud de pago y verifica si debe cambiar estatus
    $date = date('Y-m-d H:i:s');
    $id = $this->input->post('id');
    $datos['xml'] = file_get_contents($_FILES['file']['tmp_name']);
    $this->pago_model->updateSolicitudPago($id, $datos);

    $query = "SELECT factura, xml from solicitudes_pago where id=" . $id;
    $res = $this->Conexion->consultar($query, TRUE);
}

function uploadComprobante() {
    // Carga el archivo de comprobante de pago, marca la solicitud como PAGADA y envía correo
    al solicitante
    $date = date('Y-m-d H:i:s');
    $id = $this->input->post('id');
    $datos['comprobante_pago'] = file_get_contents($_FILES['file']['tmp_name']);
    $datos['estatus'] = 'PAGADO';
    $datos['fecha_comprobante'] = $date;

    $this->pago_model->updateSolicitudPago($id, $datos);
}

```

```

$query = 'SELECT u.correo from usuarios u join solicitudes_pago p on p.idus = u.id WHERE
p.id = ' . $id;
$res = $this->Conexion->consultar($query, TRUE);

$mail['id'] = $id;
$mail['correo'] = $res->correo;
$mail['date'] = $date;
$this->correos->correos_solictudes_pagada($mail);

// Actualiza estatus de pago de las órdenes asociadas
$po = $this->Conexion->consultar('select * from Po_solicitudes where idPago = ' . $id);
foreach ($po as $elem) {
    $dato['estatus_pago'] = 'APROBADO';
    $where['id'] = $elem->idPO;
    $this->Conexion->modificar('ordenes_compra', $dato, null, $where);
}
}

function uploadComplemento() {
    // Carga el archivo de complemento de pago y notifica a los usuarios con permiso de
    responder pagos
    $date = date('Y-m-d H:i:s');
    $id = $this->input->post('id');
    $datos['complemento'] = file_get_contents($_FILES['file']['tmp_name']);
    $datos['fecha_factura'] = $date;
    $this->pago_model->updateSolicitudPago($id, $datos);

    $query = 'SELECT u.correo from usuarios u join privilegios p on p.usuario = u.id where
    p.responderPago=1 and u.activo=1';
    $res = $this->Conexion->consultar($query);
    $correo = [];
    foreach ($res as $elem) {
        array_push($correo, $elem->correo);
    }

    $mail['id'] = $id;
    $mail['correo'] = $correo;
    $this->correos->correos_solictudes_complemento($mail);

```



```
}
```

```
function getprePDF($id) {  
    // Devuelve el archivo PDF de la prefactura directamente al navegador  
    $row = $this->descargas_model->getFile($id, 'solicitudes_pago');  
    $file = $row->prefactura;  
  
    header('Content-type: application/pdf');  
    header('Content-Transfer-Encoding: binary');  
    header('Accept-Ranges: bytes');  
  
    echo $file;  
}
```

```
function getPDF($id) {  
    // Devuelve el archivo PDF de la factura directamente al navegador  
    $row = $this->descargas_model->getFile($id, 'solicitudes_pago');  
    $file = $row->factura;  
  
    header('Content-type: application/pdf');  
    header('Content-Transfer-Encoding: binary');  
    header('Accept-Ranges: bytes');  
  
    echo $file;  
}
```

```
function getXML($id) {  
    // Devuelve el archivo XML como si fuera PDF para visualización directa en navegador  
    $row = $this->descargas_model->getFile($id, 'solicitudes_pago');  
    $file = $row->xml;  
  
    header('Content-type: application/pdf');  
    header('Content-Transfer-Encoding: binary');  
    header('Accept-Ranges: bytes');  
  
    echo $file;  
}
```

```

function solicitarPago() {
    // Actualiza el tipo de pago y factura de la solicitud, y redirige a la vista del pago
    $id = $this->input->post('id');
    $pago = $this->input->post('pago');
    $tipoFactura = $this->input->post('tipoFactura');

    $data['tipo_pago'] = $pago;
    $data['tipo_factura'] = $tipoFactura;
    $where['id'] = $id;

    $this->Conexion->modificar('solicitudes_pago', $data, null, $where);
    redirect(base_url('solicitudes_pago/ver_pago/' . $id));
}

function ajax_getContactos() {
    // Devuelve los contactos activos de una empresa en formato JSON
    $id = $this->input->post('id');
    $query = 'SELECT * from empresas_contactos where activo = 1 and empresa = ' . $id;
    $res = $this->Conexion->consultar($query);

    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

function seleccionarContacto() {
    // Inserta la relación de un contacto con una solicitud de pago
    $id = $this->input->post('id');
    $idContacto = $this->input->post('idContacto');
    $contacto = $this->input->post('contacto');

    $data['idpago'] = $id;
    $data['idcontacto'] = $idContacto;
    $data['tipo'] = $contacto;

    $this->Conexion->insertar('contactos_pago', $data, null);
}

```

```

}

function enviarSolicitud() {
    // Cambia estatus de la solicitud a 'SOLICITADO' y notifica a usuarios con permiso de
    responder pagos
    $id = $this->input->post('id');

    $data['estatus'] = 'SOLICITADO';
    $data['estatus_factura'] = 'EN REVISION';
    $where['id'] = $id;
    $this->Conexion->modificar('solicitudes_pago', $data, null, $where);

    $query = 'SELECT u.correo from usuarios u join privilegios p on p.usuario = u.id where
    p.responderPago=1 and u.activo=1';
    $res = $this->Conexion->consultar($query);
    $correo = [];
    foreach ($res as $elem) {
        array_push($correo, $elem->correo);
    }

    $mail['id'] = $id;
    $mail['correo'] = $correo;

    $this->correos->correos_solicitud($mail);
}

function aceptarSolicitud() {
    // Cambia estatus a 'APROBADA', registra aprobador y fecha, y notifica al usuario que solicitó
    el pago
    $date = date('Y-m-d h:i:s');
    $id = $this->input->post('id');

    $po = $this->Conexion->consultar('select * from Po_solicitudes where idPago = ' . $id);

    foreach ($po as $elem) {
        $dato['estatus_pago'] = 'APROBADO';
        $where['id'] = $elem->idPO;
        $this->Conexion->modificar('ordenes_compra', $dato, null, $where);
    }
}

```

```

}

$data['estatus'] = 'APROBADA';
$data['estatus_factura'] = 'ACEPTADA';
$data['aprobador_factura'] = $this->session->id;
$data['fecha_aprobacion_factura'] = $date;
$where['id'] = $id;
$this->Conexion->modificar('solicitudes_pago', $data, null, $where);

$query = 'SELECT u.correo from usuarios u join solicitudes_pago p on p.idus = u.id WHERE
p.id = ' . $id;
$res = $this->Conexion->consultar($query, TRUE);

$mail['id'] = $id;
$mail['correo'] = $res->correo;

$this->correos->correos_solicitudes_aceptada($mail);
}

function programar_pago() {
// Cambia el estatus del pago a "PROGRAMADA" y notifica al usuario responsable
$id = $this->input->post('id');
$date = $this->input->post('date');
$po = $this->Conexion->consultar('select * from Po_solicitudes where idPago = ' . $id);

foreach ($po as $elem) {
    $dato['estatus_pago'] = 'PROGRAMADO';
    $where['id'] = $elem->idPO;
    $this->Conexion->modificar('ordenes_compra', $dato, null, $where);
}

$data['estatus'] = 'PROGRAMADA';
$data['fecha_programada_pago'] = $date;
$where['id'] = $id;
$this->Conexion->modificar('solicitudes_pago', $data, null, $where);

$query = 'SELECT u.correo from usuarios u join solicitudes_pago p on p.idus = u.id WHERE
p.id = ' . $id;

```

```

$res = $this->Conexion->consultar($query, TRUE);

$mail['id'] = $id;
$mail['correo'] = $res->correo;
$mail['date'] = $date;
$this->correos->correos_solicitud_programada($mail);
}

function solicitar_complemento() {
    // Cambia el estatus del complemento a "SOLICITADO"
    $id = $this->input->post('id');
    $data['estatus_complemento'] = 'SOLICITADO';
    $where['id'] = $id;
    $this->Conexion->modificar('solicitudes_pago', $data, null, $where);
}

function aceptar_complemento() {
    // Cambia el estatus del complemento a "ACEPTADO" y notifica al usuario
    $id = $this->input->post('id');
    $data['estatus_complemento'] = 'ACEPTADO';
    $where['id'] = $id;
    $this->Conexion->modificar('solicitudes_pago', $data, null, $where);

    $query = 'SELECT u.correo from usuarios u join solicitudes_pago p on p.idus = u.id WHERE
p.id = ' . $id;
    $res = $this->Conexion->consultar($query, TRUE);

    $mail['id'] = $id;
    $mail['correo'] = $res->correo;
    $mail['date'] = date('Y-m-d H:i:s'); // Se añadió para evitar variable indefinida
    $this->correos->correos_solicitud_complemento_aceptado($mail);
}

function cancelar_pago() {
    // Cambia el estatus del pago a "CANCELADO" y notifica al usuario
    $id = $this->input->post('id');
    $data['estauss'] = 'CANCELADO'; // Nota: probable typo, debería ser 'estatus'
    $where['id'] = $id;

```

```

$this->Conexion->modificar('solicitudes_pago', $data, null, $where);

$query = 'SELECT u.correo from usuarios u join solicitudes_pago p on p.idus = u.id WHERE
p.id = ' . $id;
$res = $this->Conexion->consultar($query, TRUE);

$mail['id'] = $id;
$mail['correo'] = $res->correo;
$this->correos->correos_solictudes_cancelado($mail);
}

```

```

function rechazar_factura() {
    // Cambia el estatus de la factura a "RECHAZADA" y notifica al usuario
    $id = $this->input->post('id');
    $data['estatus_factura'] = 'RECHAZADA';
    $where['id'] = $id;
    $this->Conexion->modificar('solicitudes_pago', $data, null, $where);
}

```

```

$query = 'SELECT u.correo from usuarios u join solicitudes_pago p on p.idus = u.id WHERE
p.id = ' . $id;
$res = $this->Conexion->consultar($query, TRUE);

$mail['id'] = $id;
$mail['correo'] = $res->correo;
$this->correos->correos_solictudes_rechazar_factura($mail);
}

```

```

function rechazar_complemento() {
    // Cambia el estatus del complemento a "RECHAZADO" y notifica al usuario correspondiente
    $id = $this->input->post('id');
    $data['estatus_complemento'] = 'RECHAZADO';
    $where['id'] = $id;
    $this->Conexion->modificar('solicitudes_pago', $data, null, $where);

    $query = 'SELECT u.correo from usuarios u join solicitudes_pago p on p.idus = u.id WHERE
p.id = ' . $id;
    $res = $this->Conexion->consultar($query, TRUE);
}

```

```

$mail['id'] = $id;
$mail['correo'] = $res->correo;
$this->correos->correos_solicitud_rechazar_complemento($mail);
}

function ajax_getSolicitudes() {
    // Devuelve lista de solicitudes de pago con filtros opcionales por estatus, folio, usuario o
    proveedor
    $query = "SELECT e.nombre, concat(u.nombre, ' ', u.paterno) as requisitor, p.id,
    p.fecha_creacion, p.estatus from solicitudes_pago p JOIN proveedores pr on
    pr.empresa=p.idprov JOIN empresas e on e.id = pr.empresa JOIN usuarios u on u.id=p.idus
    where 1 = 1";

    if (isset($_POST['estatus'])) {
        $estatus = $this->input->post('estatus');
        if ($estatus != "TODO") {
            $query .= " and p.estatus = '$estatus'";
        }
    }

    if (isset($_POST['texto'])) {
        $texto = $this->input->post('texto');
        $parametro = $this->input->post('parametro');
        if (!empty($texto)) {
            if ($parametro == "folio") {
                $query .= " and p.id = '$texto'";
            }
            if ($parametro == "usuario") {
                $query .= " having requisitor like '%$texto%'";
            }
            if ($parametro == "proveedor") {
                $query .= " having nombre like '%$texto%'";
            }
        }
    }

    $query .= " order by p.id desc";
    $res = $this->Conexion->consultar($query);
}

```

```

    if ($res) {
        echo json_encode($res);
    }
}

function validarContacto() {
    // Valida si ya existe un contacto relacionado al pago y tipo dado
    $id = $this->input->post('id');
    $idContacto = $this->input->post('idContacto');
    $contacto = $this->input->post('contacto');

    $query = "SELECT * from contactos_pago where tipo ='$contacto' and idpago = " . $id . " and
idcontacto = " . $idContacto;
    $res = $this->Conexion->consultar($query);

    if ($res) {
        echo json_encode($res);
    }
}
}

```

Test.php

```

<?php defined('BASEPATH') OR exit('No direct script access allowed');

class Test extends CI_Controller {
    public function __construct() {
        // Constructor básico del controlador, hereda configuración del padre
        parent::__construct();
    }

    function sms() {
        // Carga la vista de prueba de envío SMS
        $this->load->view('test/sms');
    }

    function index() {

```



```

$this->load->model('compras_model','Modelo');
$qqr = $this->Modelo->getDetalleQR(11);
$datos['correos'] = array($qqr->correo);

$Notificar = json_decode($qqr->notificaciones);

$query = "SELECT U.correo from usuarios U where 1 != 1";
foreach ($Notificar as $value) {
    $query .= " or U.id = $value";
}

$res = $this->Conexion->consultar($query);
foreach ($res as $key => $value) {
    // array_push($datos['correos'], $value->correo);
}

echo json_encode($datos['correos']);
}

function fecha(){
    switch(date('N', strtotime("2020-05-28")))
    {
        case 1:
            $dia = "Lunes";
            break;

        case 2:
            $dia = "Martes";
            break;

        case 3:
            $dia = "Miercoles";
            break;

        case 4:
            $dia = "Jueves";
            break;
    }
}

```

```

case 5:
    $dia = "Viernes";
    break;

case 6:
    $dia = "Sabado";
    break;

case 7:
    $dia = "Domingo";
    break;
}

switch(date('n', strtotime("2020-05-28")))
{
    case 1:
        $mes = "Enero";
        break;

    case 2:
        $mes = "Febrero";
        break;

    case 3:
        $mes = "Marzo";
        break;

    case 4:
        $mes = "Abril";
        break;

    case 5:
        $mes = "Mayo";
        break;

    case 6:
        $mes = "Junio";
        break;

```

```

case 7:
    $mes = "Julio";
    break;

case 8:
    $mes = "Agosto";
    break;

case 9:
    $mes = "Septiembre";
    break;

case 10:
    $mes = "Octubre";
    break;

case 11:
    $mes = "Noviembre";
    break;

case 12:
    $mes = "Diciembre";
    break;
}

echo $dia . " " . date('j', strtotime("2020-05-28")) . " de " . $mes;
}

function getLocation(){
    if(!empty($_POST['latitude']) && !empty($_POST['longitude'])){
        $url =
'https://maps.googleapis.com/maps/api/geocode/json?latlng=' . trim($_POST['latitude']) . ',' . trim(
$_POST['longitude']) . '&key=AlzaSyBLE6J6TWRVoQ4PbrMxTr7Y2K8QsVtCuBM';
        $json = @file_get_contents($url);
        $data = json_decode($json);
        $status = $data->status;
        if($status=="OK"){

```

```

        $location = $data->results[0]->formatted_address;
    } else {
        $location = "";
    }
    echo $location;
}
}
}

```

Tickets_AT.php

```
<?php
```

```
defined('BASEPATH') OR exit('No direct script access allowed');
```

```
class Tickets_AT extends CI_Controller {
```

```

    function __construct() {
        parent::__construct();
        $this->load->model('tickets_AT_model','Modelo'); // Modelo principal para tickets de autos
        $this->load->model('conexion_model', 'Conexion'); // Modelo para consultas generales
        $this->load->library('correos'); // Librería para envío de correos
    }

```

```

    public function generar() {
        $this->load->model('autos_model');
        $this->load->model('Tickets_AT_model');

```

```
        $auto = $this->input->post('auto');
```

```
        // Si se redirige desde generar_qr, se toma el auto de la sesión
```

```

        if (isset($this->session->POST_auto)) {
            $auto = $this->session->POST_auto;
            $this->session->unset_userdata('POST_auto');
        }

```

```

        if (isset($auto)) {
            $datosCoche = $this->autos_model->getAuto($auto);

```

```

if ($datosCoche) {
    // Datos del auto para mostrar en la vista de generación de ticket
    $datos['foto'] = $datosCoche->foto;
    $datos['marca'] = $datosCoche->marca;
    $datos['combustible'] = $datosCoche->combustible;
    $datos['placas'] = $datosCoche->placas;
    $datos['auto'] = $auto;
    $this->load->view('header');
    $this->load->view('generar_ticket_auto', $datos);
} else {
    redirect(base_url('inicio')); // Auto no encontrado
}
} else {
    // Si no hay auto, se muestra el catálogo para seleccionar
    $datos['autos'] = $this->autos_model->getCatalogo();
    $this->load->view('header');
    $this->load->view('tickets_autos/seleccion', $datos);
}
}

public function generar_qr($id){
    // Guarda temporalmente el auto y redirige al generador de tickets
    $this->session->POST_auto = $id;
    redirect(base_url('tickets_AT/generar'));
}

public function mantenimiento($auto) {
    $this->load->model('autos_model');
    $datosCoche = $this->autos_model->getAuto($auto);

    // Datos del auto para vista de generación de ticket de mantenimiento
    $datos['auto'] = $auto;
    $datos['foto'] = $datosCoche->foto;
    $datos['marca'] = $datosCoche->marca;
    $datos['combustible'] = $datosCoche->combustible;
    $datos['placas'] = $datosCoche->placas;
    $datos['aumento'] = $datosCoche->combustible == 'DIESEL' ? 15000 : 10000;
    $datos['kilometraje'] = $datosCoche->kilometraje;

```

```

$datos['proximo_mtto'] = $datosCoche->proximo_mtto;

$this->load->view('header');
$this->load->view('tickets_autos/generar_ticket_mtto', $datos);
}

public function mtto_solucionado() {
    $this->load->model('autos_model');

    $idTicket = $this->input->post('id_ticket');
    $id_auto = $this->input->post('auto');
    $proxMtto = $this->input->post('proxMtto');
    $kilometraje = $this->input->post('kilometraje');

    // Actualiza datos del auto después del mantenimiento
    $carData = array(
        'kilometraje' => $kilometraje,
        'ultimo_mtto' => $kilometraje,
        'proximo_mtto' => str_replace(',', '', $proxMtto),
        'ticket_mtto' => '0',
    );
    $this->autos_model->updateAuto($id_auto, $carData);

    // Actualiza estatus del ticket y notifica por correo
    $Res = $this->Modelo->getUsuarioTicket($idTicket);
    $correo = $Res->correo;
    $usuario = $Res->User;

    $data = array('estatus' => 'MTTO');
    $this->Modelo->update($idTicket, $data);

    if (isset($correo)) {
        $datosCorreo['id'] = $idTicket;
        $datosCorreo['prefijo'] = substr($this->router->fetch_class(), 8);
        $datosCorreo['fecha'] = date('d/m/Y h:i A');
        $datosCorreo['usuario'] = $usuario;
        $datosCorreo['tecnico'] = $this->session->nombre;
        $datosCorreo['correo'] = $correo;
    }
}

```

```

        $this->correos->ticketSolucionado($datosCorreo);
    }

    redirect(base_url('tickets_AT/ver/' . $idTicket));
}

function administrar($estatus) {
    $count = $this->Modelo->getTicketsCount();

    // Conteo general por estatus para dashboard
    $datos['c_activos'] = $count->activos;
    $datos['c_solucionados'] = $count->solucionados;
    $datos['c_cerrados'] = $count->cerrados;
    $datos['c_cancelados'] = $count->cancelados;
    $datos['c_revision'] = $count->revision;
    $datos['c_todos'] = $count->todos;
    $datos['c_detenidos'] = $count->detenidos;

    $datos['filtro'] = $estatus;
    $datos['tickets'] = $this->Modelo->getTickets($estatus);
    $datos['controlador'] = 'tickets_AT';

    $this->load->view('header');
    $this->load->view('tickets_sistemas', $datos);
}

public function registrar() {
    $auto = $this->input->post('auto');
    $tipo = $this->input->post('tipo');

    // Datos del nuevo ticket
    $data = array(
        'usuario' => $this->session->id,
        'auto' => $auto,
        'tipo' => $tipo,
        'titulo' => $this->input->post('titulo'),
        'descripcion' => $this->input->post('descripcion'),
        'estatus' => 'ABIERTO',
    );
}

```

```

        'cierre' => '0',
    );

    $last_id = $this->Modelo->crear_ticket($data);

    // Si es mantenimiento, registrar en el auto
    if ($tipo == 'MANTENIMIENTO') {
        $this->load->model('autos_model');
        $this->autos_model->updateAuto($auto, array('ticket_mtto' => $last_id));
    }

    // Envío de correo al crear ticket
    $datosCorreo['id'] = $last_id;
    $datosCorreo['prefijo'] = substr($this->router->fetch_class(), 8);
    $datosCorreo['titulo'] = $data['titulo'];
    $datosCorreo['fecha'] = date('d/m/Y h:i A');
    $datosCorreo['usuario'] = $this->session->nombre;
    $datosCorreo['correo'] = $this->session->correo;
    $this->correos->creacionTicket($datosCorreo);

    redirect(base_url('tickets_AT/archivos/') . $last_id);
}

function archivos($id_ticket) {
    $datos['id_ticket'] = $id_ticket;
    $datos['controlador'] = 'tickets_AT';
    $this->load->view('header');
    $this->load->view('subir_archivos', $datos);
}

public function ver($id) {
    $this->load->model('autos_model');

    // Consulta principal del ticket
    $Renglon = $this->Modelo->verTicket($id);
    $datos['ticket'] = $Renglon->row();
    $datos['comentarios'] = $this->Modelo->verTicket_comentarios($id);
    $datos['comentarios_fotos'] = $this->Modelo->verTicket_comentarios_fotos($id);

```



```

$datos['archivos'] = $this->Modelo->verTicketArchivos($id);
$datos['controlador'] = $this->router->fetch_class();

// Info del auto asociado al ticket
$datosCoche = $this->autos_model->getAuto($datos['ticket']->auto);
$datos['auto_id'] = $datosCoche->id;
$datos['auto_foto'] = $datosCoche->foto;
$datos['auto_marca'] = $datosCoche->marca;
$datos['auto_combustible'] = $datosCoche->combustible;
$datos['auto_placas'] = $datosCoche->placas;
$datos['auto_mttoActual'] = $datosCoche->proximo_mtto;
$datos['auto_kilometraje'] = $datosCoche->kilometraje;

$this->load->view('header');
$this->load->view('tickets_autos/ver_ticket', $datos);
}

```

```

public function agregarComentario() {
    $idTicket = $this->input->post('idticket');
    $data = array(
        'ticket' => $idTicket,
        'usuario' => $this->session->id,
        'comentario' => $this->input->post('comentario'),
    );
    $this->Modelo->agregar_comentario($data);
    redirect(base_url('tickets_AT/ver/' . $idTicket));
}

```

```

public function estatus($idTicket, $estatus) {
    // Asignar estatus según código recibido
    switch ($estatus) {
        case '1':
            $Stat = 'ABIERTO';
            break;
        case '2':
            $Stat = 'EN CURSO';
            break;
        case '3':

```

```

        $Stat = 'DETENIDO';
        break;
    case '4':
        $Stat = 'CANCELADO';
        break;
    case '5':
        $Stat = 'SOLUCIONADO';
        // Obtener info del usuario para enviar correo
        $Res = $this->Modelo->getUsuarioTicket($idTicket);
        $correo = $Res->correo;
        $usuario = $Res->User;
        break;
    case '6':
        $Stat = 'CERRADO';
        break;
    default:
        redirect(base_url('inicio'));
        exit();
        break;
}

$data = array('estatus' => $Stat);
$this->Modelo->update($idTicket, $data);

// Enviar correo si se cambió a SOLUCIONADO
if (isset($correo)) {
    $datosCorreo['id'] = $idTicket;
    $datosCorreo['prefijo'] = substr($this->router->fetch_class(), 8);
    $datosCorreo['fecha'] = date('d/m/Y h:i A');
    $datosCorreo['usuario'] = $usuario;
    $datosCorreo['tecnico'] = $this->session->nombre;
    $datosCorreo['correo'] = $correo;
    $this->correos->ticketSolucionado($datosCorreo);
}

redirect(base_url('tickets_AT/ver/' . $idTicket));
}

```

```

function subir_archivos() {
    $idTicket = $this->input->post('id_ticket');

    // Subir múltiples archivos asociados al ticket
    for ($i = 0; $i < count($_FILES['file']['tmp_name']); $i++) {
        $datos = array(
            'ticket' => $idTicket,
            'nombre' => $_FILES['file']['name'][$i],
            'archivo' => file_get_contents($_FILES['file']['tmp_name'][$i])
        );
        if (!$this->Modelo->subir_archivos($datos)) {
            trigger_error("Error al subir archivo", E_USER_ERROR);
        }
    }
}

function ver_foto($id) {
    // Mostrar imagen si existe
    $photo = $this->Modelo->getFoto($id);
    if ($photo) {
        header("Content-type: image/png");
        echo $photo->archivo;
    } else {
        echo "ERROR";
    }
}

function ajax_cerrarTicket() {
    $id = $this->input->post('id');
    $comentario = $this->input->post('comentario');

    // Cerrar ticket y registrar calificación
    $t->calificacion = $this->input->post('calificacion');
    $t->estatus = "CERRADO";

    $this->Conexion->modificar('tickets_autos', $t, null, array('id' => $id));

    // Guardar comentario final si fue ingresado

```

```

if (!empty($comentario)) {
    $data = array(
        'ticket' => $id,
        'usuario' => $this->session->id,
        'comentario' => $comentario
    );
    $this->Modelo->agregar_comentario($data);
}
}

function reporte() {
    $this->load->view('header');
    $this->load->view('tickets/reporte_at');
}

function reporte_at() {
    // Captura de filtros del formulario
    $estatus = $this->input->post('estatus');
    $texto = $this->input->post('texto');
    $parametro = $this->input->post('parametro');
    $fecha1 = $this->input->post('fecha1');
    $fecha2 = $this->input->post('fecha2');
    $f1 = strval($fecha1) . ' 00:00:00';
    $f2 = strval($fecha2) . ' 23:59:59';

    // Generación dinámica del query
    $query = "SELECT t.*, concat(u.nombre, ' ',u.paterno) as User from tickets_autos t JOIN
    usuarios u on t.usuario = u.id where 1=1";

    if ($estatus != 'TODO') {
        $query .= " and t.estatus = '$estatus'";
    }

    if (!empty($texto)) {
        if ($parametro == "folio") {
            $query .= " and t.id = '$texto'";
        }
        if ($parametro == "usuario") {

```

```

        $query .= " and concat(u.nombre, ' ', u.paterno) like '%$texto%'";
    }
}

if (!empty($fecha1) && !empty($fecha2)) {
    $query .= " and t.fecha BETWEEN '" . $f1 . "' AND '" . $f2 . "'";
}

$query .= " order by t.id desc";

$res = $this->Conexion->consultar($query);
if ($res) {
    echo json_encode($res);
} else {
    echo "";
}
}

function excel_AT(){
    $estatus= $this->input->post('opEstatus');
    $texto= $this->input->post('txtBusqueda');
    $fecha1 = $this->input->post('fecha1');
    $fecha2 = $this->input->post('fecha2');
    $parametro = $this->input->post('rbBusqueda');
    $f1=strval($fecha1).' 00:00:00';
    $f2=strval($fecha2).' 23:59:59';

    // Consulta principal para obtener los tickets con comentarios y usuario
    $query = "SELECT t.id as No_Ticket,t.fecha as Fecha_Ticket,t.tipo as Tipo,t.titulo as
Titulo,t.descripcion as Descripcion,t.estatus as Estatus, tc.fecha as
fecha_comentario,tc.comentario as Comentario, concat(u.nombre,' ', u.paterno) as
Creador_Ticket FROM tickets_autos t JOIN tickets_autos_comentarios tc on t.id=tc.ticket JOIN
usuarios u on t.usuario = u.id where 1=1 ";

    // Filtros según estatus, texto y fechas
    if($estatus != 'TODO')
    {
        $query .= " and t.estatus = '$estatus'";
    }
}

```

```

}
if(!empty($texto))
{
    if($parametro == "folio")
    {
        $query .= " and t.id = '$texto'";
    }
    if($parametro == "usuario")
    {
        $query .= " and concat(u.nombre, ' ', u.paterno) like '%$texto%'";
    }
}
if (!empty($fecha1) && !empty($fecha2)) {
    $query .= " and t.fecha BETWEEN '" . $f1 . "' AND '" . $f2 . "'";
}

$query .= " order by t.id desc";
$result= $this->Conexion->consultar($query);

// Inicio de tabla HTML para exportar
$salida="";
$salida .= '<table style="border: 1px solid black; border-collapse: collapse;">
    <thead>
        <tr>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">No_Ticket</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Fecha_Ticket</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Tipo</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Titulo</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Descripcion</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Estatus</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Fecha_Comentario</th>

```

```

        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Comentario</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Creador Ticket</th>
    </tr>
</thead>
<tbody>';

// Cuerpo de la tabla con resultados
foreach($result as $row){
    $salida .='
        <tr>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">'.$row-
>No_Ticket.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">'.$row-
>Fecha_Ticket.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">'.$row-
>Tipo.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">'.$row-
>Titulo.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">'.$row-
>Descripcion.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">'.$row-
>Estatus.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">'.$row-
>fecha_comentario.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">'.$row-
>Comentario.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">'.$row-
>Creador_Ticket.'</td>
        </tr>';
    }

    $salida .='</tbody>
</table>';

// Encabezados para forzar descarga como archivo Excel
$timestamp = date('m/d/Y', time());

```

```

$filename='Tickets_'. $timestamp.'.xls';
header("Content-Type: application/vnd.ms-excel");
header("Content-Disposition: attachment; filename=\"".$filename.\"");
header("Pragma: no-cache");
header("Expires: 0");
header('Content-Transfer-Encoding: binary');
echo $salida;
}
}

```

Tickets_ED.php

```
<?php
```

```
defined('BASEPATH') OR exit('No direct script access allowed');
```

```
class Tickets_ED extends CI_Controller {
```

```

    function __construct() {
        parent::__construct();
        $this->load->model('tickets_ED_model','Modelo'); // Modelo principal de tickets para el
        módulo de Edificio
        $this->load->model('conexion_model', 'Conexion'); // Modelo para conexión a base de datos
        personalizada
        $this->load->library('correos'); // Librería personalizada para envío de correos
        $this->load->library('AOS_funciones'); // Librería adicional para funciones AOS
    }

```

```

    public function generar() {
        // Carga la vista para generar un nuevo ticket de edificio
        $this->load->view('header');
        $this->load->view('generar_ticket_edificio');
    }

```

```

function administrar($estatus) {
    // Obtiene conteo de tickets por estatus
    $count = $this->Modelo->getTicketsCount();
    $datos['c_activos'] = $count->activos;
}

```



```

$datos['c_solucionados'] = $count->solucionados;
$datos['c_cerrados'] = $count->cerrados;
$datos['c_cancelados'] = $count->cancelados;
$datos['c_revision'] = $count->revision;
$datos['c_todos'] = $count->todos;
$datos['c_detenidos'] = $count->detenidos;

$datos['filtro'] = $estatus; // Filtro actual aplicado
$datos['tickets'] = $this->Modelo->getTickets($estatus); // Tickets filtrados por estatus
$datos['controlador'] = 'tickets_ED'; // Nombre del controlador para identificar módulo
$this->load->view('header');
$this->load->view('tickets_sistemas', $datos); // Vista compartida de tickets
}

```

```

public function registrar() {
    // Datos del formulario para crear nuevo ticket
    $data = array(
        'usuario' => $this->session->id,
        'tipo' => $this->input->post('tipo'),
        'titulo' => $this->input->post('titulo'),
        'descripcion' => $this->input->post('descripcion'),
        'estatus' => 'ABIERTO',
        'cierre' => '0',
    );

    $last_id = $this->Modelo->crear_ticket($data); // Guarda ticket en BD y obtiene ID

    // Preparación de datos para enviar correo de confirmación
    $datosCorreo['id'] = $last_id;
    $datosCorreo['prefijo'] = substr($this->router->fetch_class(), 8);
    $datosCorreo['titulo'] = $data['titulo'];
    $datosCorreo['fecha'] = date('d/m/Y h:i A');
    $datosCorreo['usuario'] = $this->session->nombre;
    $datosCorreo['correo'] = $this->session->correo;
    $this->correos->creacionTicket($datosCorreo); // Envía correo

    redirect(base_url('tickets_ED/archivos/') . $last_id); // Redirige a subir archivos
}

```

```

function archivos($id_ticket) {
    // Carga vista para subir archivos al ticket
    $datos['id_ticket'] = $id_ticket;
    $datos['controlador'] = 'tickets_ED';
    $this->load->view('header');
    $this->load->view('subir_archivos', $datos);
}

public function ver($id) {
    // Consulta información completa de un ticket
    $Renglon = $this->Modelo->verTicket($id);
    $datos['ticket'] = $Renglon->row(); // Ticket individual
    $datos['comentarios'] = $this->Modelo->verTicket_comentarios($id); // Comentarios del
ticket
    $datos['comentarios_fotos'] = $this->Modelo->verTicket_comentarios_fotos($id); //
Comentarios con fotos
    $datos['archivos'] = $this->Modelo->verTicketArchivos($id); // Archivos adjuntos al ticket
    $datos['controlador'] = $this->router->fetch_class(); // Nombre del controlador actual
    $this->load->view('header');
    $this->load->view('ver_ticket', $datos); // Carga vista para ver el ticket completo
}

public function agregarComentario() {
    // Registra un nuevo comentario para el ticket correspondiente
    $idTicket = $this->input->post('idticket');
    $data = array(
        'ticket' => $idTicket,
        'usuario' => $this->session->id,
        'comentario' => $this->input->post('comentario'),
    );
    $this->Modelo->agregar_comentario($data); // Inserta comentario en base de datos
    redirect(base_url('tickets_ED/ver/' . $idTicket)); // Redirige a vista del ticket
}

public function estatus($idTicket, $estatus) {
    // Cambia el estatus de un ticket dependiendo del parámetro recibido
    switch ($estatus) {

```

```

case '1':
    $Stat = 'ABIERTO';
    break;

case '2':
    $Stat = 'EN CURSO';
    break;

case '3':
    $Stat = 'DETENIDO';
    break;

case '4':
    $Stat = 'CANCELADO';
    break;

case '5':
    $Stat = 'SOLUCIONADO';
    $Res = $this->Modelo->getUsuarioTicket($idTicket); // Se obtiene información del
usuario dueño del ticket
    $correo = $Res->correo;
    $usuario = $Res->User;
    break;

case '6':
    $Stat = 'CERRADO';
    break;

default:
    redirect(base_url('inicio')); // Redirección si el estatus no es válido
    exit();
    break;
}

$data = array(
    'estatus' => $Stat,
);

```

```

$this->Modelo->update($idTicket, $data); // Actualiza el estatus del ticket en BD

// Si se ha definido $correo (solo cuando es SOLUCIONADO), se envía correo al usuario
if (isset($correo)) {
    $datosCorreo['id'] = $idTicket;
    $datosCorreo['prefijo'] = substr($this->router->fetch_class(), 8);
    $datosCorreo['fecha'] = date('d/m/Y h:i A');
    $datosCorreo['usuario'] = $usuario;
    $datosCorreo['tecnico'] = $this->session->nombre;
    $datosCorreo['correo'] = $correo;
    $this->correos->ticketSolucionado($datosCorreo); // Envío de correo de solución
}

redirect(base_url('tickets_ED/ver/' . $idTicket)); // Redirige a vista del ticket
}

function subir_archivos() {
    // Sube uno o más archivos al ticket
    $idTicket = $this->input->post('id_ticket');
    for ($i=0; $i < count($_FILES['file']['tmp_name']); $i++) {
        $datos = array('ticket' => $idTicket, 'nombre' => $_FILES['file'][$i]['name'], 'archivo' =>
file_get_contents($_FILES['file'][$i]['tmp_name'][$i]));
        if(!$this->Modelo->subir_archivos($datos)) {
            trigger_error("Error al subir archivo", E_USER_ERROR); // Muestra error si la carga falla
        }
    }
}

function ver_foto($id) {
    // Muestra la imagen almacenada en la base de datos
    $photo = $this->Modelo->getFoto($id);
    if($photo) {
        header("Content-type: image/png"); // Define el tipo de contenido
        echo $photo->archivo; // Imprime el contenido de la imagen
    } else {
        echo "ERROR"; // Mensaje de error si no se encuentra la foto
    }
}

```

```

}

function ajax_cerrarTicket() {
    // Cierra un ticket vía Ajax y registra un comentario si se proporciona
    $id = $this->input->post('id');
    $comentario = $this->input->post('comentario');
    $t->calificacion = $this->input->post('calificacion');
    $t->estatus = "CERRADO";

    $this->Conexion->modificar('tickets_edificio', $t, null, array('id' => $id)); // Actualiza ticket

    if(!empty($comentario)) {
        $data = array(
            'ticket' => $id,
            'usuario' => $this->session->id,
            'comentario' => $comentario
        );
        $this->Modelo->agregar_comentario($data); // Agrega comentario si se escribió
    }
}

function reporte() {
    // Muestra la vista del reporte general
    $this->load->view('header');
    $this->load->view('tickets/reporte_ed');
}

function reporte_ed() {
    // Genera reporte en base a filtros recibidos por POST y responde en JSON
    $estatus = $this->input->post('estatus');
    $texto = $this->input->post('texto');
    $parametro = $this->input->post('parametro');
    $fecha1 = $this->input->post('fecha1');
    $fecha2 = $this->input->post('fecha2');
    $f1=strval($fecha1).' 00:00:00';
    $f2=strval($fecha2).' 23:59:59';

```

```
$query = "SELECT t.*, concat(u.nombre, ' ', u.paterno) as User from tickets_edificio t JOIN usuarios u on t.usuario = u.id where 1=1";
```

```
if($estatus != 'TODO') {  
    $query .= " and t.estatus = '$estatus'";  
}
```

```
if(!empty($texto)) {  
    if($parametro == "folio") {  
        $query .= " and t.id = '$texto'";  
    }  
    if($parametro == "usuario") {  
        $query .= " and concat(u.nombre, ' ', u.paterno) like '%$texto%'";  
    }  
}
```

```
if (!empty($fecha1) && !empty($fecha2)) {  
    $query .= " and t.fecha BETWEEN '" . $f1 . "' AND '" . $f2 . "'";  
}
```

```
$query .= " order by t.id desc";
```

```
$res = $this->Conexion->consultar($query);
```

```
if($res) {  
    echo json_encode($res); // Devuelve resultado en formato JSON  
} else {  
    echo ""; // Vacío si no hay resultados  
}  
}
```

```
function excel_ED() {  
    // Recibe los filtros del formulario  
    $estatus = $this->input->post('opEstatus');  
    $texto = $this->input->post('txtBusqueda');  
    $fecha1 = $this->input->post('fecha1');  
    $fecha2 = $this->input->post('fecha2');  
    $parametro = $this->input->post('rbBusqueda');
```

```

$f1 = strval($fecha1) . ' 00:00:00';
$f2 = strval($fecha2) . ' 23:59:59';

// Construcción de la consulta SQL
$query = "SELECT t.id as No_Ticket, t.fecha as Fecha_Ticket, t.tipo as Tipo, t.titulo as Titulo,
t.descripcion as Descripcion, t.estatus as Estatus, tc.fecha as fecha_comentario, tc.comentario
as Comentario, concat(u.nombre, ' ', u.paterno) as Cerador_Ticket FROM tickets_edificio t JOIN
tickets_edificio_comentarios tc on t.id=tc.ticket JOIN usuarios u on t.usuario = u.id WHERE 1=1";

if ($estatus != 'TODO') {
    $query .= " and t.estatus = '$estatus'";
}

if (!empty($texto)) {
    if ($parametro == "folio") {
        $query .= " and t.id = '$texto'";
    }
    if ($parametro == "usuario") {
        $query .= " and concat(u.nombre, ' ', u.paterno) like '%$texto%'";
    }
}

if (!empty($fecha1) && !empty($fecha2)) {
    $query .= " and t.fecha BETWEEN '$f1' AND '$f2'";
}

$query .= " order by t.id desc";

$result = $this->Conexion->consultar($query); // Consulta ejecutada

// Preparación de la tabla en HTML para exportar como Excel
$salida = "";
$salida .= '<table style="border: 1px solid black; border-collapse: collapse;">
    <thead>
        <tr>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">No_Ticket</th>

```

```

        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Fecha_Ticket</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Tipo</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Titulo</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Descripcion</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Estatus</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Fecha_Comentario</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Comentario</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Creador Ticket</th>
    </tr>
</thead>
<tbody>';
foreach($result as $row){

    $salida .= '
        <tr>
            <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'.$row->No_Ticket.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'.$row->Fecha_Ticket.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'.$row->Tipo.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'.$row->Titulo.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'.$row->Descripcion.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'.$row->Estatus.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'.$row->fecha_comentario.'</td>

```



```

                <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'.$row->Comentario.'</td>
                <td style="color: $444; border: 1px solid black; border-collapse:
collapse">'.$row->Cerador_Ticket.'</td>
            </tr>';
        }

        $salida .= '</tbody>
</table>';

        $timestamp = date('m/d/Y', time());
        $filename = 'Tickets_' . $timestamp . '.xls';

        header("Content-Type: application/vnd.ms-excel");
        header("Content-Disposition: attachment; filename=\"$filename\"");
        header("Pragma: no-cache");
        header("Expires: 0");
        header('Content-Transfer-Encoding: binary');

        echo $salida; // Se imprime la tabla como Excel
    }
}

```

Tickets_IT.php

```

<?php

defined('BASEPATH') OR exit('No direct script access allowed');

class Tickets_IT extends CI_Controller {

    function __construct() {
        parent::__construct();
        $this->load->library('unit_test'); // Carga biblioteca para pruebas unitarias
        $this->load->model('tickets_IT_model', 'Modelo'); // Modelo principal para reportes
        $this->load->model('conexion_model', 'Conexion'); // Modelo para gestión de conexión
        $this->load->library('correos'); // Biblioteca para envío de correos
        $this->load->library('AOS_funciones'); // Biblioteca de funciones personalizadas
    }
}

```

```

}

function reporte() {
    // Carga vistas de encabezado y vista de reporte IT
    $this->load->view('header');
    $this->load->view('tickets/reporte_it');
}

function reporte_usuarios_ajax() {
    // Recibe fechas desde el formulario por POST
    $datos[0] = $this->input->post('inicio');
    $datos[1] = $this->input->post('final');

    $res = $this->Modelo->getReporteUsuarios($datos);
    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

function reporte_tipo_ajax() {
    // Recibe fechas desde el formulario por POST
    $datos[0] = $this->input->post('inicio');
    $datos[1] = $this->input->post('final');

    $res = $this->Modelo->getReporteTipo($datos);
    if ($res) {
        echo json_encode($res);
    } else {
        echo "";
    }
}

function reporte_tickets_ajax() {
    // Recibe fechas desde el formulario por POST y las asigna con formato clave-valor
    $datos['TS.fecha >='] = $this->input->post('inicio');
    $datos['TS.fecha <='] = $this->input->post('final');
}

```

```

$res = $this->Modelo->getReporteTickets($datos);
if ($res) {
    echo json_encode($res);
} else {
    echo "";
}
}

function generar() {
    // Carga vistas para generación de ticket del área de sistemas
    $this->load->view('header');
    $this->load->view('generar_ticket_sistemas');
}

function administrar($estatus) {
    $this->load->model('privilegios_model');
    $count = $this->Modelo->getTicketsCount();

    $datos['usuarios'] = $this->privilegios_model->listadoJefes();

    // Conteo de tickets por estado
    $datos['c_activos'] = $count->activos;
    $datos['c_detenidos'] = $count->detenidos;
    $datos['c_solucionados'] = $count->solucionados;
    $datos['c_cerrados'] = $count->cerrados;
    $datos['c_cancelados'] = $count->cancelados;
    $datos['c_revision'] = $count->revision;
    $datos['c_todos'] = $count->todos;

    $datos['filtro'] = $estatus;
    $datos['tickets'] = $this->Modelo->getTickets($estatus);
    $datos['controlador'] = 'tickets_IT';

    // Carga las vistas con datos para administrar tickets
    $this->load->view('header');
    $this->load->view('tickets_sistemas', $datos);
}

```

```

function registrar() {
    // Datos para crear un nuevo ticket
    $data = array(
        'usuario' => $this->session->id,
        'tipo' => $this->input->post('opCategoria'),
        'titulo' => $this->input->post('titulo'),
        'descripcion' => $this->input->post('descripcion'),
        'estatus' => 'ABIERTO',
        'cierre' => '0',
    );

    $last_id = $this->Modelo->crear_ticket($data);

    // Preparar datos para enviar correo de notificación
    $datosCorreo['id'] = $last_id;
    $datosCorreo['prefijo'] = substr($this->router->fetch_class(), 8);
    $datosCorreo['titulo'] = $data['titulo'];
    $datosCorreo['fecha'] = date('d/m/Y h:i A');
    $datosCorreo['usuario'] = $this->session->nombre;
    $datosCorreo['correo'] = $this->session->correo;

    $this->correos->creacionTicket($datosCorreo);

    // Redireccionar para subir archivos al ticket creado
    redirect(base_url('tickets_IT/archivos/') . $last_id);
}

```

```

function archivos($id_ticket) {
    $datos['id_ticket'] = $id_ticket;
    $datos['controlador'] = 'tickets_IT';

    // Carga vista para subir archivos al ticket
    $this->load->view('header');
    $this->load->view('subir_archivos', $datos);
}

```

```

public function ver($id) {

```

```

// Obtiene datos completos del ticket seleccionado
$Renglon = $this->Modelo->verTicket($id);
$datos['ticket'] = $Renglon->row(); // Solo un renglón del ticket
$datos['comentarios'] = $this->Modelo->verTicket_comentarios($id);
$datos['comentarios_fotos'] = $this->Modelo->verTicket_comentarios_fotos($id);
$datos['archivos'] = $this->Modelo->verTicketArchivos($id);
$datos['controlador'] = $this->router->fetch_class();

// Carga vistas para mostrar el ticket y sus detalles
$this->load->view('header');
$this->load->view('ver_ticket', $datos);
}

public function agregarComentario() {
    $idTicket = $this->input->post('idticket');

    $data = array(
        'ticket' => $idTicket,
        'usuario' => $this->session->id,
        'comentario' => $this->input->post('comentario'),
    );

    // Agrega comentario y redirecciona a la vista del ticket
    $this->Modelo->agregar_comentario($data);
    redirect(base_url('tickets_IT/ver/' . $idTicket));
}

public function estatus($idTicket, $estatus) {
    // Obtiene datos del usuario asociado al ticket
    $Res = $this->Modelo->getUsuarioTicket($idTicket);

    // Mapea el estatus recibido a su texto y obtiene correo y usuario
    switch ($estatus) {
        case '1':
            $Stat = 'ABIERTO';
            $correo = $Res->correo;
            $usuario = $Res->User;
            break;

```

```
case '2':  
    $Stat = 'EN CURSO';  
    $correo = $Res->correo;  
    $usuario = $Res->User;  
    break;
```

```
case '3':  
    $Stat = 'DETENIDO';  
    $correo = $Res->correo;  
    $usuario = $Res->User;  
    break;
```

```
case '4':  
    $Stat = 'CANCELADO';  
    $correo = $Res->correo;  
    $usuario = $Res->User;  
    break;
```

```
case '5':  
    $Stat = 'SOLUCIONADO';  
    $correo = $Res->correo;  
    $usuario = $Res->User;  
    break;
```

```
case '6':  
    $Stat = 'CERRADO';  
    $correo = $Res->correo;  
    $usuario = $Res->User;  
    break;
```

```
case '7':  
    $Stat = 'EN REVISION';  
    $correo = $Res->correo;  
    $usuario = $Res->User;  
    break;
```

```
default:
```

```

        // Si estatus no válido, redirige al inicio
        redirect(base_url('inicio'));
        exit();
    }

    // Actualiza el estatus del ticket
    $data = array(
        'estatus' => $Stat,
    );
    $this->Modelo->update($idTicket, $data);

    // Si se tiene correo, envía notificación del cambio de estatus
    if (isset($correo)) {
        $datosCorreo['id'] = $idTicket;
        $datosCorreo['prefijo'] = substr($this->router->fetch_class(), 8);
        $datosCorreo['fecha'] = date('d/m/Y h:i A');
        $datosCorreo['usuario'] = $usuario;
        $datosCorreo['tecnico'] = $this->session->nombre;
        $datosCorreo['estatus'] = $Stat;
        $datosCorreo['correo'] = $correo;

        $this->correos->ticketSolucionado($datosCorreo);
    }

    // Redirige a la vista del ticket actualizado
    redirect(base_url('tickets_IT/ver/' . $idTicket));
}

function subir_archivos() {
    $idTicket = $this->input->post('id_ticket');

    // Procesa múltiples archivos enviados
    for ($i = 0; $i < count($_FILES['file']['tmp_name']); $i++) {
        $datos = array(
            'ticket' => $idTicket,
            'nombre' => $_FILES['file']['name'][$i],
            'archivo' => file_get_contents($_FILES['file']['tmp_name'][$i])
        );
    }
}

```

```

        // Intenta subir archivo, lanza error si falla
        if (!$this->Modelo->subir_archivos($datos)) {
            trigger_error("Error al subir archivo", E_USER_ERROR);
        }
    }
}

function ver_foto($id) {
    // Obtiene la foto del ticket por ID
    $photo = $this->Modelo->getFoto($id);

    if ($photo) {
        // Envía encabezado de imagen PNG y muestra el contenido binario
        header("Content-type: image/png");
        echo $photo->archivo;
    } else {
        echo "ERROR";
    }
}

function ajax_cerrarTicket() {
    $id = $this->input->post('id');
    $comentario = $this->input->post('comentario');

    // Construye objeto con nueva calificación y estatus cerrado
    $t->calificacion = $this->input->post('calificacion');
    $t->estatus = "CERRADO";

    // Actualiza tabla tickets_sistemas para el ticket específico
    $this->Conexion->modificar('tickets_sistemas', $t, null, array('id' => $id));

    // Si hay comentario, se agrega como comentario del ticket
    if (!empty($comentario)) {
        $data = array(
            'ticket' => $id,
            'usuario' => $this->session->id,
            'comentario' => $comentario
        );
    }
}

```



```

    );
    $this->Modelo->agregar_comentario($data);
}
}

```

```

function ajax_TicketsSimultaneos() {
    $categoria = $this->input->post("categoria");

    // Consulta tickets activos de la categoría que no están cerrados ni cancelados
    $res = $this->Conexion->consultar(
        "SELECT TS.*, concat(U.nombre, ' ', U.paterno) as User
        FROM tickets_sistemas TS
        INNER JOIN usuarios U ON U.id = TS.usuario
        WHERE TS.tipo = '$categoria'
        AND TS.estatus != 'CERRADO'
        AND TS.estatus != 'CANCELADO'"
    );

    if ($res) {
        // Devuelve resultado en formato JSON
        echo json_encode($res);
    }
}

```

```

function excel(){
    // Obtener filtros enviados por POST
    $estatus= $this->input->post('estatus');
    $user= $this->input->post('user');
    $fecha1 = $this->input->post('fecha1');
    $fecha2 = $this->input->post('fecha2');

    // Preparar fechas para filtro en formato datetime
    $f1=strval($fecha1).' 00:00:00';
    $f2=strval($fecha2).' 23:59:59';

    // Construcción base del query con joins para traer info del ticket, comentarios y usuarios
    involucrados

```

```

$query = " SELECT t.id as No_Ticket, t.fecha as Fecha_Ticket, t.tipo as Tipo, t.titulo as Titulo,
t.descripcion as Descripcion, t.estatus as Estatus, t.fecha_cierre,
CONCAT(s.nombre, ' ', s.paterno) AS solucionador,
GROUP_CONCAT(
    CONCAT(DATE_FORMAT(tc.fecha, '%d/%m/%Y %H:%i'), ' - ', tc.comentario)
    SEPARATOR '\n'
) AS Comentarios,
CONCAT(u.nombre, ' ', u.paterno) AS Creador_Ticket
FROM tickets_sistemas t
LEFT JOIN tickets_sistemas_comentarios tc ON t.id = tc.ticket
LEFT JOIN usuarios s ON t.cierre = s.id
JOIN usuarios u ON t.usuario = u.id
WHERE 1=1 ";

```

```

// Filtros por estatus, agregando condiciones específicas para cada caso
if ($estatus) {
    if ($estatus=='activos') {
        $query.=" and (t.estatus = 'EN CURSO' OR t.estatus = 'ABIERTO')";
    } else if ($estatus=='revision') {
        $query.=" and t.estatus = 'EN REVISION'";
    }
    else if ($estatus=='solucionados') {
        $query.=" and t.estatus = 'SOLUCIONADO'";
    }
    else if ($estatus=='cerrados') {
        $query.=" and t.estatus = 'CERRADO'";
    }
    else if ($estatus=='cancelados') {
        $query.=" and t.estatus = 'CANCELADO'";
    }
    else if ($estatus=='detenidos') {
        $query.=" and t.estatus = 'DETENIDO'";
    } else if ($estatus=='todos') {
        $query.=" and t.estatus IN ('EN CURSO', 'ABIERTO', 'EN REVISION', 'SOLUCIONADO',
'CERRADO', 'CANCELADO', 'DETENIDO')";
    }
}
}

```

```

// Filtro por usuario creador del ticket
if (!empty($user)) {
    $query .= " and t.usuario = '$user'";
}

// Filtro por rango de fechas
if (!empty($fecha1) && !empty($fecha2)) {
    $query .= " and t.fecha BETWEEN '". $f1. "' AND '". $f2. "'";
}

// Agrupar resultados por id de ticket para evitar duplicados por comentarios múltiples
$query .= " GROUP BY t.id";

// Ordenar resultados por id ascendente
$query .= " ORDER BY t.id";

// Ejecutar consulta
$result= $this->Conexion->consultar($query);

// Inicializar salida HTML para tabla Excel
$salida="";

// Cabecera de tabla con estilos inline para borde y fondo
$salida .= '<table style="border: 1px solid black; border-collapse: collapse;">
    <thead>
        <tr>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">No_Ticket</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Fecha_Ticket</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Tipo</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Titulo</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Descripcion</th>
            <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Estatus</th>

```

```

        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Fecha Cierre</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Solucionado</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Comentario</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Creador Ticket</th>
    </tr>
</thead>
<tbody>';

```

```
// Verificar que la consulta devolvió resultados válidos
```

```

if (!$result || !is_array($result)) {
    echo "No se encontraron resultados o hubo un error en la consulta.";
    return;
}

```

```
// Recorrer cada fila para armar el contenido de la tabla
```

```
foreach($result as $row){
```

```
    $salida .='
```

```

        <tr>
            <td style="color: #444; border: 1px solid black; border-collapse: collapse">'. $row-
>No_Ticket.'</td>
            <td style="color: #444; border: 1px solid black; border-collapse: collapse">'. $row-
>Fecha_Ticket.'</td>
            <td style="color: #444; border: 1px solid black; border-collapse: collapse">'. $row-
>Tipo.'</td>
            <td style="color: #444; border: 1px solid black; border-collapse: collapse">'. $row-
>Titulo.'</td>
            <td style="color: #444; border: 1px solid black; border-collapse: collapse">'. $row-
>Descripcion.'</td>
            <td style="color: #444; border: 1px solid black; border-collapse: collapse">'. $row-
>Estatus.'</td>
            <td style="color: #444; border: 1px solid black; border-collapse: collapse">'. $row-
>fecha_cierre.'</td>

```

```
  |
```

```

// Cierre de tabla HTML
$salida .= '</tbody>
</table>';

```

```

// Nombre del archivo Excel con fecha actual
$timestamp = date('m/d/Y', time());
$filename='Tickets_'. $timestamp.'.xls';

```

```

// Headers para forzar descarga como archivo Excel
header("Content-Type: application/vnd.ms-excel");
header("Content-Disposition: attachment; filename=\"{$filename}\"");
header("Pragma: no-cache");
header("Expires: 0");
header('Content-Transfer-Encoding: binary');

```

```

// Enviar contenido al navegador
echo $salida;
}

```

```

function reporte_it()
{
// Recibir datos enviados vía POST
$status = $this->input->post('estatus');
$texto = $this->input->post('texto');
$parametro = $this->input->post('parametro');
$fecha1 = $this->input->post('fecha1');
$fecha2 = $this->input->post('fecha2');

// Preparar fechas con horas para filtro entre rangos

```

```

$fecha1=$fecha1.' 00:00:00';
$fecha2=$fecha2.' 23:59:59';

// Query base para obtener tickets con nombre completo del usuario que los creó
$query = "SELECT t.*, concat(u.nombre, ' ', u.paterno) as User from tickets_sistemas t JOIN
usuarios u on t.usuario = u.id where 1=1";

// Filtrar por estatus si no es TODO (todos)
if($estatus != 'TODO')
{
    $query .= " and t.estatus = '$estatus'";
}

// Filtros de búsqueda según texto y parámetro (folio o usuario)
if(!empty($texto))
{
    if($parametro == "folio")
    {
        $query .= " and t.id = '$texto'";
    }
    if($parametro == "usuario")
    {
        $query .= " and concat(u.nombre, ' ', u.paterno) like '%$texto%'";
    }
}

// Filtro por rango de fechas, si se especifican ambas fechas
if (!empty($fecha1) && !empty($fecha2)) {
    $query .= " and t.fecha BETWEEN '$fecha1' AND '$fecha2'";
}

// Ordenar resultados de más recientes a más antiguos
$query .= " order by t.id desc";

// Ejecutar consulta
$res= $this->Conexion->consultar($query);

// Devolver resultados en formato JSON si existen, sino devolver cadena vacía

```

```

if($res)
{
    echo json_encode($res);
}
else{
    echo "";
}
}

function excel_IT(){
// Obtener datos enviados vía POST
$estatus= $this->input->post('opEstatus');
$texto= $this->input->post('txtBusqueda');
$fecha1 = $this->input->post('fecha1');
$fecha2 = $this->input->post('fecha2');
$parametro = $this->input->post('rbBusqueda');
$f1=strval($fecha1).' 00:00:00';
$f2=strval($fecha2).' 23:59:59';

// Consulta para obtener tickets con comentarios y creador
$query = "SELECT t.id as No_Ticket,t.fecha as Fecha_Ticket,t.tipo as Tipo,t.titulo as
Titulo,t.descripcion as Descripcion,t.estatus as Estatus, tc.fecha as
fecha_comentario,tc.comentario as Comentario, concat(u.nombre,' ', u.paterno) as
Creador_Ticket FROM tickets_sistemas t JOIN tickets_sistemas_comentarios tc on t.id=tc.ticket
JOIN usuarios u on t.usuario = u.id where 1=1 ";

// Filtro por estatus, excepto cuando es TODO
if($estatus != 'TODO')
{
    $query .= " and t.estatus = '$estatus'";
}
// Filtros por texto y parámetro (folio o usuario)
if(!empty($texto))
{
    if($parametro == "folio")
    {
        $query .= " and t.id = '$texto'";
    }
}

```

```

        if($parametro == "usuario")
        {
            $query .= " and concat(u.nombre, ' ', u.paterno) like '%$texto%'";
        }
    }

    // Filtro por rango de fechas
    if (!empty($fecha1) && !empty($fecha2)) {
        $query .= " and t.fecha BETWEEN '". $f1. "' AND '". $f2. "'";
    }

    // Ordenar resultados de forma descendente por ID
    $query .= " order by t.id desc";

    // Ejecutar consulta en base de datos
    $result= $this->Conexion->consultar($query);

    // Inicializar variable para salida HTML de tabla
    $salida="";

    // Crear encabezados de tabla con estilos para Excel
    $salida .= '<table style="border: 1px solid black; border-collapse: collapse;">
        <thead>
            <tr>
                <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">No_Ticket</th>
                <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Fecha_Ticket</th>
                <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Tipo</th>
                <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Titulo</th>
                <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Descripcion</th>
                <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Estatus</th>
                <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Fecha_Comentario</th>

```



```

        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Comentario</th>
        <th style="background-color: #F3F1F1; color: black; border: 1px solid black;
border-collapse: collapse">Creador Ticket</th>
    </tr>
</thead>
<tbody>';

// Recorrer cada resultado y agregar fila a tabla
foreach($result as $row){
    $salida .='
        <tr>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">'.$row-
>No_Ticket.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">'.$row-
>Fecha_Ticket.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">'.$row-
>Tipo.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">'.$row-
>Titulo.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">'.$row-
>Descripcion.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">'.$row-
>Estatus.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">'.$row-
>fecha_comentario.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">'.$row-
>Comentario.'</td>
            <td style="color: $444; border: 1px solid black; border-collapse: collapse">'.$row-
>Creador_Ticket.'</td>
        </tr>';
    }

    $salida .='</tbody>
</table>';

// Preparar nombre de archivo con fecha actual
$timestamp = date('m/d/Y', time());

```

```

$filename='Tickets_'. $timestamp.'.xls';

// Encabezados para descarga del archivo Excel
header("Content-Type: application/vnd.ms-excel");
header("Content-Disposition: attachment; filename=\"$filename\"");
header("Pragma: no-cache");
header("Expires: 0");
header('Content-Transfer-Encoding: binary');

// Mostrar contenido para descarga
echo $salida;
}

function buscar_tickets(){
// Obtener datos enviados vía POST
$status = $this->input->post('estatus');
$user = $this->input->post('user');
$fecha1 = $this->input->post('fecha1');
$fecha2 = $this->input->post('fecha2');

// Asignar fechas solo si no están vacías
$f1 = (!empty($fecha1)) ? $fecha1 : null;
$f2 = (!empty($fecha2)) ? $fecha2 : null;

// Llamar al modelo para obtener tickets según filtros
$res = $this->Modelo->getTickets($status, $user, $f1, $f2);

// Retornar resultado en formato JSON (arreglo vacío si no hay resultados)
echo json_encode($res ?: []);
}

public function prueba_buscar_tickets()
{
// Simulación de datos POST para pruebas unitarias
$_POST['estatus'] = 'activos';

// Obtener variables simuladas desde POST
$status = $this->input->post('estatus');

```

```

$user = $this->input->post('user');
$fecha1 = $this->input->post('fecha1');
$fecha2 = $this->input->post('fecha2');

// Asignar fechas solo si no están vacías
$f1 = (!empty($fecha1)) ? $fecha1 : null;
$f2 = (!empty($fecha2)) ? $fecha2 : null;

// Ejecutar función del modelo
$res = $this->Modelo->getTickets($estatus, $user, $f1, $f2);

// Validar que la respuesta sea un arreglo
echo $this->unit->run(is_array($res), TRUE, 'getTickets devuelve un arreglo');

// Validar estructura si hay resultados
if (!empty($res)) {
    echo $this->unit->run(isset($res[0]['id']), TRUE, 'El ticket tiene ID');
    echo $this->unit->run(isset($res[0]['estatus']), TRUE, 'El ticket tiene estatus');
} else {
    // No hay tickets pero la respuesta es válida
    echo $this->unit->run(true, true, 'No hay tickets, pero respuesta válida');
}
}
}

```

Tickets.php

```
<?php
```

```
defined('BASEPATH') OR exit('No direct script access allowed');
```

```
class Tickets extends CI_Controller {
```

```

    function __construct() {
        parent::__construct();
        // Cargar modelos necesarios
        $this->load->model('tickets_model');
    }
}

```

```

$this->load->model('conexion_model', 'Conexion');
// Cargar biblioteca para envíos de correos de tickets
$this->load->library('correos_tickets');
}

public function index() {
    // Vista principal para generar tickets
    $datos['modulo'] = 'generar';
    $this->load->view('header');
    $this->load->view('tickets', $datos);
}

public function administrar() {
    // Vista para administrar tickets activos
    $datos['modulo'] = 'administrar/activos';
    $this->load->view('header');
    $this->load->view('tickets', $datos);
}

public function mis_tickets() {
    // Vista para mostrar tickets asignados al usuario logueado
    $datos['tickets'] = $this->tickets_model->getMis_tickets($this->session->id);
    $this->load->view('header');
    $this->load->view('tickets/mis_tickets', $datos);
}

public function reportes() {
    // Vista para reportes de tickets
    $datos['modulo'] = 'reporte';
    $this->load->view('header');
    $this->load->view('tickets', $datos);
}

public function cafeteria() {
    // Vista para módulo cafetería
    $datos['modulo'] = 'cafeteria';
    $this->load->view('header');
    $this->load->view('tickets', $datos);
}

```

```

}

function dashboard(){
    // Vista del dashboard de tickets
    $this->load->view('header');
    $this->load->view('tickets/dashboard');
}

function camaras(){
    // Vista para cámaras relacionadas a tickets
    $this->load->view('header');
    $this->load->view('tickets/camaras');
}

////////// F U N C I O N E S   A J A X   //////////

function ajax_getTicketsSolucionados() {
    // Obtener datos del POST
    $user = $this->input->post('usuario');
    $tipo = $this->input->post('tipo');

    // Determinar tabla y campo usuario según el tipo
    $tabla = null;
    $usuario = 'usuario';

    if ($tipo == 'IT') {
        $tabla = 'tickets_sistemas';
    } else if ($tipo == 'AT') {
        $tabla = 'tickets_autos';
    } else if ($tipo == 'ED') {
        $tabla = 'tickets_edificio';
    } else if ($tipo == 'cafeteria') {
        $tabla = 'comentarios_cafeteria';
        $usuario = 'id_user';
    }

    // Consultar el conteo de tickets solucionados del usuario

```

```

    $res = $this->Conexion->consultar("SELECT count(*) as Conteo FROM $tabla where $usuario =
    '$user' and estatus = 'SOLUCIONADO'", TRUE);
    if ($res) {
        echo json_encode($res);
    }
}

```

```

function ajax_sendTicketNotification() {
    // Consulta tickets pendientes por tipo y estatus (ABIERTO o EN CURSO)
    $query = "SELECT 'IT' as tipo, T.id, T.usuario, concat(U.nombre,' ',U.paterno) as User, U.correo,
    T.estatus from tickets_sistemas T inner join usuarios U on U.id = T.usuario where T.estatus =
    'ABIERTO'"
        . " union "
        . "SELECT 'IT' as tipo, T.id, T.usuario, concat(U.nombre,' ',U.paterno) as User, U.correo,
    T.estatus from tickets_sistemas T inner join usuarios U on U.id = T.usuario where T.estatus = 'EN
    CURSO'"
        . " union "
        . "SELECT 'AT', T.id, T.usuario, concat(U.nombre,' ',U.paterno) as User, U.correo, T.estatus
    from tickets_autos T inner join usuarios U on U.id = T.usuario where T.estatus = 'ABIERTO'"
        . " union "
        . "SELECT 'AT', T.id, T.usuario, concat(U.nombre,' ',U.paterno) as User, U.correo, T.estatus
    from tickets_autos T inner join usuarios U on U.id = T.usuario where T.estatus = 'EN CURSO'"
        . " union "
        . "SELECT 'ED', T.id, T.usuario, concat(U.nombre,' ',U.paterno) as User, U.correo, T.estatus
    from tickets_edificio T inner join usuarios U on U.id = T.usuario where T.estatus = 'ABIERTO'"
        . " union "
        . "SELECT 'ED', T.id, T.usuario, concat(U.nombre,' ',U.paterno) as User, U.correo, T.estatus
    from tickets_edificio T inner join usuarios U on U.id = T.usuario where T.estatus = 'EN CURSO'
    order by usuario";
}

```

```

    $arr = array();
    $res = $this->Conexion->consultar($query);
}

```

```

// Agrupar tickets por usuario para enviar resumen por correo
if ($res) {
    foreach ($res as $key => $value) {
        if (!isset($arr[$value->usuario])) {
            $arr[$value->usuario]['nombre'] = $value->User;
        }
    }
}

```

```

        $arr[$value->usuario]['correo'] = $value->correo;
        $arr[$value->usuario]['tickets'] = array();
    }
    array_push($arr[$value->usuario]['tickets'], $value->tipo . str_pad($value->id, 6, "0",
STR_PAD_LEFT));
    }
}

```

```

// Enviar notificaciones por correo de los tickets pendientes
$this->correos_tickets->ticketsPendientes($arr);
}

```

```

function ajax_getTicktesIT() {
    // Obtener el período desde POST
    $periodo_it = $this->input->post('periodo_it');
    $periodo = null;

    // Si no es "TODO", filtramos por fecha
    if ($periodo_it != 'TODO') {
        $periodo = ' and fecha like "%' . $periodo_it . '%"';
    }

    // Consulta de resumen por estatus en tickets_sistemas
    $query = 'SELECT count(*) as Total,
        (SELECT count(*) FROM tickets_sistemas where estatus = "ABIERTO"' . $periodo . ') as
Abiertos,
        (SELECT max(fecha) FROM tickets_sistemas where estatus = "ABIERTO"' . $periodo . ') as
ultAbiertos,
        (SELECT count(*) FROM tickets_sistemas where estatus = "EN CURSO"' . $periodo . ') as
Curso,
        (SELECT max(fecha) FROM tickets_sistemas where estatus = "EN CURSO"' . $periodo . ') as
ultCurso,
        (SELECT count(*) FROM tickets_sistemas where estatus = "EN REVISION"' . $periodo . ') as
Revision,
        (SELECT max(fecha) FROM tickets_sistemas where estatus = "EN REVISION"' . $periodo . ')
as ultRevision,
        (SELECT count(*) FROM tickets_sistemas where estatus = "DETENIDO"' . $periodo . ') as
Detenido,

```

```

        (SELECT max(fecha) FROM tickets_sistemas where estatus = "DETENIDO" . $periodo . ') as
ultDetenido,
        (SELECT count(*) FROM tickets_sistemas where estatus = "SOLUCIONADO" . $periodo . ') as
Solucionado,
        (SELECT max(fecha) FROM tickets_sistemas where estatus = "SOLUCIONADO" . $periodo . ')
as ultSolucionado,
        (SELECT count(*) FROM tickets_sistemas where estatus = "CERRADO" . $periodo . ') as
Cerrado,
        (SELECT max(fecha) FROM tickets_sistemas where estatus = "CERRADO" . $periodo . ') as
ultCerrado,
        (SELECT count(*) FROM tickets_sistemas where estatus = "CANCELADO" . $periodo . ') as
Cancelado,
        (SELECT max(fecha) FROM tickets_sistemas where estatus = "CANCELADO" . $periodo . ') as
ultCancelado
        FROM tickets_sistemas where 1=1 ' . $periodo;

$res = $this->Conexion->consultar($query, TRUE);
echo json_encode($res);
}

```

```

function ajax_getTicktesAutos() {
    // Obtener el período desde POST
    $periodo_at = $this->input->post('periodo_at');
    $periodo = null;

    // Filtro por fecha
    if ($periodo_at != 'TODO') {
        $periodo = ' and fecha like "%' . $periodo_at . '%"';
    }

    // Consulta de resumen por estatus en tickets_autos
    $query = 'SELECT count(*) as Total,
        (SELECT count(*) FROM tickets_autos where estatus = "ABIERTO" . $periodo . ') as
Abiertos,
        (SELECT max(fecha) FROM tickets_autos where estatus = "ABIERTO" . $periodo . ') as
ultAbiertos,
        (SELECT count(*) FROM tickets_autos where estatus = "EN CURSO" . $periodo . ') as Curso,

```



```

        (SELECT max(fecha) FROM tickets_autos where estatus = "EN CURSO" . $periodo . ') as
ultCurso,
        (SELECT count(*) FROM tickets_autos where estatus = "DETENIDO" . $periodo . ') as
Detenido,
        (SELECT max(fecha) FROM tickets_autos where estatus = "DETENIDO" . $periodo . ') as
ultDetenido,
        (SELECT count(*) FROM tickets_autos where estatus = "SOLUCIONADO" . $periodo . ') as
Solucionado,
        (SELECT max(fecha) FROM tickets_autos where estatus = "SOLUCIONADO" . $periodo . ') as
ultSolucionado,
        (SELECT count(*) FROM tickets_autos where estatus = "CERRADO" . $periodo . ') as
Cerrado,
        (SELECT max(fecha) FROM tickets_autos where estatus = "CERRADO" . $periodo . ') as
ultCerrado,
        (SELECT count(*) FROM tickets_autos where estatus = "CANCELADO" . $periodo . ') as
Cancelado,
        (SELECT max(fecha) FROM tickets_autos where estatus = "CANCELADO" . $periodo . ') as
ultCancelado
    FROM tickets_autos where 1=1 ' . $periodo;

```

```

$res = $this->Conexion->consultar($query, TRUE);
echo json_encode($res);
}

```

```

// Función para obtener estadísticas de tickets de mantenimiento de edificio
function ajax_getTicktesEdificio() {

```

```

    // Obtener el periodo enviado por POST
    $periodo_ed = $this->input->post('periodo_ed');
    $periodo = null;

```

```

    // Si se seleccionó un periodo específico, se agrega condición para filtrar por fecha
    if ($periodo_ed != 'TODO') {
        $periodo = ' and fecha like "%' . $periodo_ed . '%"';
    }

```

```

    // Consulta SQL para obtener el total y las cantidades por estatus, además de la última fecha
    por cada estatus

```

```

    $query = 'SELECT count(*) as Total,

```

```

        (SELECT count(*) FROM tickets_edificio where estatus = "ABIERTO" . $periodo . ') as
Abiertos,
        (SELECT max(fecha) FROM tickets_edificio where estatus = "ABIERTO" . $periodo . ') as
ultAbiertos,
        (SELECT count(*) FROM tickets_edificio where estatus = "EN CURSO" . $periodo . ') as
Curso,
        (SELECT max(fecha) FROM tickets_edificio where estatus = "EN CURSO" . $periodo . ') as
ultCurso,
        (SELECT count(*) FROM tickets_edificio where estatus = "DETENIDO" . $periodo . ') as
Detenido,
        (SELECT max(fecha) FROM tickets_edificio where estatus = "DETENIDO" . $periodo . ') as
ultDetenido,
        (SELECT count(*) FROM tickets_edificio where estatus = "SOLUCIONADO" . $periodo . ') as
Solucionado,
        (SELECT max(fecha) FROM tickets_edificio where estatus = "SOLUCIONADO" . $periodo . ')
as ultSolucionado,
        (SELECT count(*) FROM tickets_edificio where estatus = "CERRADO" . $periodo . ') as
Cerrado,
        (SELECT max(fecha) FROM tickets_autos where estatus = "CERRADO" . $periodo . ') as
ultCerrado, -- bug: consulta desde otra tabla
        (SELECT count(*) FROM tickets_edificio where estatus = "CANCELADO" . $periodo . ') as
Cancelado,
        (SELECT max(fecha) FROM tickets_edificio where estatus = "CANCELADO" . $periodo . ') as
ultCancelado
    FROM tickets_edificio where 1=1 ' . $periodo;

```

```

// Ejecuta la consulta y devuelve el resultado en formato JSON
$res = $this->Conexion->consultar($query, TRUE);
echo json_encode($res);
}

```

```

// Función para obtener estadísticas de comentarios o tickets de cafetería
function ajax_getTicktesCafeteria() {
    $periodo_ca = $this->input->post('periodo_ca');
    $periodo = null;

    // Filtro por periodo si se indicó alguno
    if ($periodo_ca != 'TODO') {

```

```

    $periodo = ' and fecha like "%' . $periodo_ca . '%"' ;
}

// Consulta SQL similar a las demás, adaptada para comentarios_cafeteria
$query = 'SELECT count(*) as Total,
        (SELECT count(*) FROM comentarios_cafeteria where estatus = "ABIERTO"' . $periodo . ') as
Abiertos,
        (SELECT max(fecha) FROM comentarios_cafeteria where estatus = "ABIERTO"' . $periodo . ')
as ultAbiertos,
        (SELECT count(*) FROM comentarios_cafeteria where estatus = "EN CURSO"' . $periodo . ')
as Curso,
        (SELECT max(fecha) FROM comentarios_cafeteria where estatus = "EN CURSO"' . $periodo .
') as ultCurso,
        (SELECT count(*) FROM comentarios_cafeteria where estatus = "DETENIDO"' . $periodo . ')
as Detenido,
        (SELECT max(fecha) FROM comentarios_cafeteria where estatus = "DETENIDO"' . $periodo .
') as ultDetenido,
        (SELECT count(*) FROM comentarios_cafeteria where estatus = "SOLUCIONADO"' .
$periodo . ') as Solucionado,
        (SELECT max(fecha) FROM comentarios_cafeteria where estatus = "SOLUCIONADO"' .
$periodo . ') as ultSolucionado,
        (SELECT count(*) FROM comentarios_cafeteria where estatus = "CERRADO"' . $periodo . ')
as Cerrado,
        (SELECT max(fecha) FROM comentarios_cafeteria where estatus = "CERRADO"' . $periodo .
') as ultCerrado,
        (SELECT count(*) FROM comentarios_cafeteria where estatus = "CANCELADO"' . $periodo .
') as Cancelado,
        (SELECT max(fecha) FROM comentarios_cafeteria where estatus = "CANCELADO"' .
$periodo . ') as ultCancelado
        FROM comentarios_cafeteria where 1=1' . $periodo;

$res = $this->Conexion->consultar($query, TRUE);
echo json_encode($res);
}

```

```

// Función para obtener la lista de cámaras registradas
function ajax_getCamaras() {
    $res = $this->Conexion->consultar("SELECT * FROM camaras");
}

```

```

    if ($res) {
        echo json_encode($res);
    }
}

// Función para registrar una nueva cámara
function registrar_camara() {
    $data = json_decode($this->input->post('data')); // decodifica los datos JSON enviados
    $this->Conexion->insertar('camaras', $data); // inserta en la tabla 'camaras'
}
}

```

Toolcrib.php

```

<?php

defined('BASEPATH') OR exit('No direct script access allowed');

class Toolcrib extends CI_Controller {
    function __construct() {
        parent::__construct();
        $this->load->library('correos_po');
        $this->load->model('Tool_model');
    }

    function tool_crib() {
        $this->load->model('tool_model');
        $this->load->model('privilegios_model');

        // Obtiene productos solicitados para el módulo ToolCrib
        $datos['productos'] = $this->tool_model->ProdPedidos();

        // Lista de usuarios con rol de jefe para asignaciones
        $datos['usuarios'] = $this->privilegios_model->listadoJefes();

        // Información de venta temporal (posible carrito o pre-pedido)
        $datos['venta'] = $this->tool_model->ventatemp();
    }
}

```

```

$this->load->view('header');
$this->load->view('toolcrib/toolcrib', $datos);
}

function pedidos() {
    $this->load->model('tool_model');

    // Obtiene pedidos registrados en el sistema
    $datos['pedido'] = $this->tool_model->pedidos();

    $this->load->view('header');
    $this->load->view('toolcrib/pedido', $datos);
}

function producto_nuevo() {
    $this->load->model('tool_model');
    $this->load->view('header');
    $this->load->view('toolcrib/productoNuevo');
}

function inventario() {
    $this->load->model('tool_model');

    // Consulta todos los productos registrados en inventario
    $data['productos'] = $this->tool_model->productos();

    $this->load->view('header');
    $this->load->view('toolcrib/inventario', $data);
}

function movimientos() {
    $this->load->model('tool_model');

    // Historial de movimientos de productos (entradas/salidas)
    $data['movimientos'] = $this->tool_model->movimientos();
    $this->load->view('header');
    $this->load->view('toolcrib/movimientos', $data);
}

```

```

function verProducto($idp) {
    $this->load->model('tool_model');

    // Datos principales del producto seleccionado
    $datosProd = $this->tool_model->getProds($idp);
    $data['codigo'] = $datosProd->codigo;
    $data['producto'] = $datosProd->producto;

    // Detalles adicionales: movimientos, cantidad actual y ubicación
    $data['productos'] = $this->tool_model->getProd($idp);
    $data['movimientos'] = $this->tool_model->movimientosProd($idp);
    $data['qty'] = $this->tool_model->prodCant($idp);
    $data['ubicacion'] = $this->tool_model->ubicaciones($idp);

    $this->load->view('header');
    $this->load->view('toolcrib/ver_Prod', $data);
}

function cancelarProducto($idp) {
    $this->load->model('tool_model');

    // Cancela un producto del inventario por ID
    $this->tool_model->cancelarProducto($idp);

    // Redirige al listado principal
    redirect(base_url('toolcrib/tool_crib'));
}

function modificarProd($idp) {
    $this->load->model('tool_model');

    // Obtiene datos actuales del producto
    $datosProd = $this->tool_model->getProds($idp);
    $data['codigo'] = $datosProd->codigo;
    $data['producto'] = $datosProd->producto;

    // Detalles para modificación: historial, ubicación

```

```

$data['productos'] = $this->tool_model->getProd($idp);
$data['movimientos'] = $this->tool_model->movimientosProdCom($idp);
$data['ubicacion'] = $this->tool_model->ubicaciones($idp);

$this->load->view('header');
$this->load->view('toolcrib/modProd', $data);
}

function EditarPedido($idp) {
    $this->load->model('tool_model');

    // Obtiene detalle del pedido a editar
    $datosProd = $this->tool_model->getDetalleVent($idp);
    $data['producto'] = $datosProd->idProd;
    $data['producto'] = $datosProd->cantidad;
    $idProd = $datosProd->idProd;

    // Carga detalle del pedido y ubicaciones disponibles para ajuste
    $data['productos'] = $this->tool_model->DetalleVent($idp);
    $data['ubi'] = $this->tool_model->ubicacionAjuste($idProd);

    $this->load->view('header');
    $this->load->view('toolcrib/editarPedido', $data);
}

function EditarProd($idp) {
    $this->load->model('tool_model');

    $datosub = $this->tool_model->ubicacion($idp);

    // Obtiene datos del producto y ubicaciones para edición
    $data['productos'] = $this->tool_model->getProd($idp);
    $data['ubicaciones'] = $this->tool_model->locacion($idp);

    $this->load->view('header');
    $this->load->view('toolcrib/editar_Prod', $data);
}

```

```

function verPedido($idVenta) {
    $this->load->model('tool_model');
    $this->load->model('privilegios_model');

    // Prepara datos para visualización detallada de pedido
    $data['privilegios'] = $this->privilegios_model->listadoPuestos();
    $venta = $this->tool_model->getventaDet($idVenta);
    $data['idVenta'] = $venta->idToolCrib;
    $data['pedido'] = $this->tool_model->getPedido($idVenta);

    $this->load->view('header');
    $this->load->view('toolcrib/verPedido', $data);
}

```

```

function aprobarPedido($idVenta) {
    $this->load->model('tool_model');

    // Recupera información necesaria para aprobación de pedido
    $venta = $this->tool_model->getventaDet($idVenta);
    $data['idVenta'] = $venta->idToolCrib;
    $data['estatus'] = $venta->estatus;
    $data['pedido'] = $this->tool_model->getPedido($idVenta);

    $this->load->view('header');
    $this->load->view('toolcrib/aprobar', $data);
}

```

```

function reporte() {
    $this->load->model('tool_model');
    // Genera reporte de pedidos
    $data['pedidos'] = $this->tool_model->reporte();
    $this->load->view('header');
    $this->load->view('toolcrib/reporte', $data);
}

```

```

function registrarVenta() {
    $this->load->model('tool_model');

```



```

// Prepara y registra venta temporal
$datos = array(
    'idUs' => $this->session->id,
    'idProd' => $this->input->post('producto'),
    'cantidad' => $this->input->post('cantidad'),
);

$res = $this->tool_model->registrarVentaTemp($datos);
redirect(base_url('toolcrib/tool_crib'));
}

function registrarProducto() {
    $this->load->model('tool_model');

    // Registra nuevo producto con datos del formulario
    $data = array(
        'codigo' => $this->input->post('codigo'),
        'categoria' => $this->input->post('categoria'),
        'producto' => $this->input->post('producto'),
        'descripcion' => $this->input->post('descripcion'),
        'proveedor' => $this->input->post('proveedor'),
        'marca' => $this->input->post('marca'),
        'modelo' => $this->input->post('modelo'),
        'precio' => $this->input->post('precio'),
        'um' => $this->input->post('um'),
        'cantMax' => $this->input->post('cantMax'),
        'cantMin' => $this->input->post('cantMin'),
        'estatus' => '1',
    );

    $id_inserted = $this->tool_model->registrarProducto($data);
    redirect(base_url('toolcrib/producto_nuevo'));
}

function actualizarProducto() {
    // Actualiza producto y registra movimiento de modificación
    $idp = $this->input->post('idp');
    $check = $this->input->post('estatus');

```

```

// Determina estatus basado en checkbox
$status = ($check == 1) ? '1' : '0';

// Prepara datos de actualización
$data = array(
    'codigo' => $this->input->post('codigo'),
    'categoria' => $this->input->post('categoria'),
    'producto' => $this->input->post('producto'),
    'descripcion' => $this->input->post('descripcion'),
    'proveedor' => $this->input->post('proveedor'),
    'marca' => $this->input->post('marca'),
    'modelo' => $this->input->post('modelo'),
    'precio' => $this->input->post('precio'),
    'um' => $this->input->post('um'),
    'cantMax' => $this->input->post('cantMax'),
    'cantMin' => $this->input->post('cantMin'),
    'estatus' => $status,
);

$this->load->model('tool_model');
$this->tool_model->updateProd($idp, $data);

// Registra movimiento de modificación
$mov = array(
    'idProd' => $idp,
    'idus' => $this->session->id,
    'cantidad' => '0',
    'local' => 'N/A',
    'tipo' => 'MODIFICACION',
    'comentario' => $this->input->post('comentario'),
    'fecha' => date('Y-m-d'),
);
$this->tool_model->registrarMov($mov);

redirect(base_url('toolcrib/inventario'));
}

```

```

function editarProducto() {
    $this->load->model('tool_model');
    $this->load->model('conexion_model', 'Conexion');
    $idp = $this->input->post('idProducto');
    $qty = $this->input->post('cantidad');
    $ubicacion = $this->input->post('ubicacion');
    $stock = $this->input->post('stock');
    $qtyF = $stock + $qty;

    // Verifica si la ubicación ya está registrada para este producto
    $query = "SELECT * from ubiProds where ubicacion ='" . $ubicacion . "' and idProd='" . $idp . "'";
    $result = $this->db->query($query)->result_array();

    if ($result) {
        // Muestra alerta si la ubicación ya existe
        echo '<script type="text/javascript">';
        echo 'alert("Ubicacion ya ha sido registraad");';
        echo 'window.location = "verProducto/" . $idp . ""';
        echo '</script>';
    } else {
        // Registra nueva ubicación para el producto
        $data = array(
            'idProd' => $this->input->post('idProducto'),
            'ubicacion' => $this->input->post('ubicacion'),
            'cantidad' => $this->input->post('cantidad'),
        );
        $this->tool_model->registrarubicacion($data);

        // Registra movimiento de entrada
        $mov = array(
            'idProd' => $idp,
            'idUs' => $this->session->id,
            'cantidad' => $qty,
            'local' => $ubicacion,
            'tipo' => 'ENTRADA',
            'comentario' => $this->input->post('comentario'),
            'fecha' => date('Y-m-d'),
        );
    }
}

```

```

        $this->tool_model->registrarMov($mov);

        redirect(base_url('toolcrib/verProducto/'.$$idp));
    }
}

function updateProducto() {
    $this->load->model('tool_model');
    $idp = $this->input->post('idProducto');
    $stock = $this->input->post('stock');
    $qty = $this->input->post('cantidad');
    $qtyF = $stock + $qty;
    $ubi = $this->input->post('ubicacion');

    // Actualiza cantidad en ubicación existente
    $data = array(
        'cantidad' => $qtyF,
    );
    $id_inserted = $this->tool_model->updateProducto($idp, $ubi, $data);
    // Registra movimiento de entrada
    $mov = array(
        'idProd' => $idp,
        'idUs' => $this->session->id,
        'cantidad' => $qty,
        'local' => $ubi,
        'tipo' => 'ENTRADA',
        'comentario' => $this->input->post('comentario'),
        'fecha' => date('Y-m-d'),
    );
    $this->tool_model->registrarMov($mov);

    redirect(base_url('toolcrib/verProducto/'.$$idp));
}

function eliminarProducto() {
    // Elimina producto solo si su cantidad es cero
    $idp = trim($this->input->post('idProducto'));
    $datosP = $this->tool_model->getProds($idp);

```

```

$qty = $data['cantidad'] = $datosP->cantidad;

// Verifica si el producto tiene cantidad cero
if ($qty == 0) {
    $data = array(
        'idProducto' => $idp,
    );

    $res = $this->tool_model->deleteProducto($data);

    // Retorna respuesta para AJAX
    if ($res) {
        echo "1";
    } else {
        echo "";
    }
}
}

function registrarPedido() {
    $this->load->model('tool_model');
    $this->load->model('privilegios_model');
    $this->load->library('correos');

    // Maneja registro de pedido con diferentes flujos de aprobación
    $op = $this->input->post('apro');
    $apro = $this->input->post('aprobador');
    $fecha = date("Y/m/d");

    // Usuario con privilegios de autorización
    if ($this->session->privilegios['autorizarTC']) {
        $data = array(
            'idUs' => $this->session->id,
            'aprobador' => $this->session->id,
            'estatus' => "APROBADO",
            'fecha' => $fecha,
        );
    }
}

```

```

        $id_inserted = $this->tool_model->registrarDetVenta($data);
    } else {
        // Usuario sin privilegios: requiere aprobación externa
        $consulta = 'SELECT u.autorizadorTC, a.correo from usuarios u JOIN usuarios a on
u.autorizadorTC=a.id WHERE u.id='.$this->session->id.'';
        $r = $this->Conexion->consultar($consulta, TRUE);

        $data = array(
            'idUs' => $this->session->id,
            'aprobador' => $r->autorizadorTC,
            'estatus' => "PENDIENTE",
            'fecha' => $fecha,
        );
        $id_inserted = $this->tool_model->registrarDetVenta($data);

        // Envía correo de notificación
        $correo = array(
            'nombre' => $this->session->nombre,
            'mail' => $r->correo,
            'fecha' => $fecha,
            'tool' => $id_inserted,
        );
        $this->correos->registrar_tool($correo);
    }

    // Recupera ID del pedido recién creado
    $consulta = 'SELECT * from VentToolCrib WHERE idToolCrib =(SELECT MAX(idToolCrib) from
VentToolCrib WHERE idUs='.$this->session->id.')';
    $resV = $this->Conexion->consultar($consulta);
    foreach ($resV as $valV) {
        $idToolCrib = $valV->idToolCrib;
    }

    // Transfiere items temporales al pedido permanente
    $query = 'SELECT * from VentTCTemp where idUs='.$this->session->id;
    $res = $this->Conexion->consultar($query);

    foreach ($res as $val) {

```

```

        $data = array(
            'idVenta' => $idToolCrib,
            'idUs' => $val->idUs,
            'idProd' => $val->idProd,
            'cantidad' => $val->cantidad,
            'estatus' => 'PENDIENTE'
        );
        $id_inserted = $this->tool_model->registrarVentaDet($data);
        $this->tool_model->delVentTemp();
    }

    redirect(base_url('toolcrib/tool_crib'));
}

```

```

function entregarPedido() {
    $this->load->library('correos');
    $this->load->model('tool_model');
}

```

// Proceso de entrega de pedido con notificación por correo

```

$idVenta = $this->input->post('idVenta');
$prod = $this->input->post('prod');
$cant = $this->input->post('cant');
$idVD = $this->input->post('idVenta');
$idP = $this->input->post('idProd');

```

```

$venta = $this->tool_model->getventaDet($idVenta);
$p = $data['idVenta'] = $venta->idToolCrib;
$idUs = $data['idVenta'] = $venta->idUs;
$correo = $this->tool_model->correoJefe($idUs, $p);
$correoJefe = $data['correo'] = $correo->correo;

```

```

$pedido = $this->tool_model->getPedido($idVenta);

```

```

$mail['pedido'] = $p;
$mail['pedidos'] = $this->tool_model->getPedido($idVenta);
$mail['correo'] = $correoJefe;

```

// Envía notificación por correo electrónico

```

$this->correos->entregarPedido($mail);

// Actualiza estado del pedido a ENTREGADO
$data = array('estatus' => 'ENTREGADO');
$res = $this->tool_model->updateVenta($idVenta, $data);

// Retorna respuesta JSON y redirecciona
if ($res) {
    echo json_encode($data);
} else {
    echo "";
}
redirect(base_url('toolcrib/pedidos'));
}

function excel() {
// Genera reporte de inventario en formato Excel
$txt = $this->input->post('txtBuscar');
$opc = $this->input->post('rbBusqueda');

$query = 'SELECT p.idProducto as CODIGO, p.producto as PODUCTO, p.descripcion AS
DESCRIPCION, p.proveedor as PROVEEDOR, p.marca AS MARCA, p.modelo AS MODELO, p.precio
AS PRECIO, p.um AS UNIDAD_DE_MEDIDA,u.ubicacion as LOCAL, u.cantidad AS STOCK FROM
productos p JOIN ubiProds u on p.idProducto=u.idProd';

// Construye consulta con filtros dinámicos
if (!empty($txt)) {
    if ($opc == "prod") {
        $query .= " where producto like '$txt'";
    } else if ($opc == "marca") {
        $query .= " where marca like '$txt'";
    } else if ($opc == "prov") {
        $query .= " where proveedor like '$txt'";
    }
}
$result = $this->db->query($query)->result_array();

// Prepara y descarga archivo Excel

```



```

$timestamp = date('m/d/Y', time());
$filename = 'inventario_'. $timestamp. '.xls';
header("Content-Type: application/vnd.ms-excel");
header("Content-Disposition: attachment; filename=\"$filename\"");

$sisPrintHeader = false;

// Imprime filas de datos
foreach ($result as $row) {
    if (!$sisPrintHeader) {
        echo implode("\t", array_keys($row)). "\n";
        $sisPrintHeader = true;
    }
    echo implode("\t", array_values($row)). "\n";
}

exit;
}

function getProds() {
// Obtiene productos filtrados por criterios de búsqueda
$texto = $this->input->post('texto');
$texto = trim($texto);
$parametro = $this->input->post('parametro');

$query = "SELECT * from productos";

// Construye consulta con filtros dinámicos
if (!empty($texto)) {
    if ($parametro == "prod") {
        $query .= " where producto like '$texto'";
    } else if ($parametro == "marca") {
        $query .= " where marca like '$texto'";
    } else if ($parametro == "prov") {
        $query .= " where proveedor like '$texto'";
    } else if ($parametro == "modelo") {
        $query .= " where modelo like '$texto'";
    }
}

```

```

}

// Ejecuta consulta y devuelve resultados en JSON
$res = $this->Conexion->consultar($query);
if ($res) {
    echo json_encode($res);
} else {
    echo "";
}
}

function productos() {
// Obtiene productos con stock total agrupado
$texto = $this->input->post('texto');
$texto = trim($texto);
$parametro = $this->input->post('parametro');

// Consulta base con suma de cantidades por ubicación
$query = "SELECT p.*, SUM(u.cantidad) as cantidad FROM productos p JOIN ubiProds u ON
p.idProducto=u.idProd ";

// Aplica filtros de búsqueda si existen
if (!empty($texto)) {
    if ($parametro == "prod") {
        $query .= " where producto like '$texto' ";
    } else if ($parametro == "codigo") {
        $query .= " where codigo like '$texto' ";
    }
}
$query .= " GROUP BY p.idProducto";

// Ejecuta consulta y devuelve resultados en JSON
$res = $this->Conexion->consultar($query);
if ($res) {
    echo json_encode($res);
} else {
    echo "";
}
}

```

```
}
```

```
function getRep() {  
    // Genera reporte de ventas con múltiples criterios de filtrado  
    $texto = $this->input->post('texto');  
    $texto = trim($texto);  
    $parametro = $this->input->post('parametro');  
    $fecha1 = $this->input->post('fecha1');  
    $fecha2 = $this->input->post('fecha2');  
  
    // Consulta base con joins entre tablas  
    $query = "SELECT dv.*, v.fecha, p.marca, p.proveedor, p.producto, p.precio, concat(u.nombre,  
' ', u.paterno) as nombre, u.no_empleado FROM detalleVentTC dv JOIN usuarios u ON  
dv.idUs=u.id JOIN VentToolCrib v ON dv.idVenta=v.idToolCrib JOIN productos p ON  
dv.idProd=p.idProducto ";  
  
    // Aplica filtros según parámetros seleccionados  
    if ($parametro == "date") {  
        $query .= " where v.fecha BETWEEN '". $fecha1. "' and '". $fecha2. "'";  
    }  
    if (!empty($texto)) {  
        if ($parametro == "prod") {  
            $query .= " where p.producto like '$texto'";  
        } else if ($parametro == "marca") {  
            $query .= " where p.marca like '$texto'";  
        } else if ($parametro == "prov") {  
            $query .= " where p.proveedor like '$texto'";  
        } else if ($parametro == "user") {  
            $query .= " where u.nombre like '$texto'";  
        }  
    }  
}  
  
    // Ejecuta consulta y devuelve resultados en JSON  
    $res = $this->Conexion->consultar($query);  
    if ($res) {  
        echo json_encode($res);  
    } else {  
        echo "";
```

```

    }
}
function mov()
{
    /* $texto = $this->input->post('texto');
    $texto = trim($texto);
    $parametro = $this->input->post('parametro');*/
    $fecha1 = $this->input->post('fecha1');
    $fecha2 = $this->input->post('fecha2');
    $f1=strval($fecha1);
    $f2=strval($fecha2);

    $query = "SELECT m.*, concat(u.nombre, ' ', u.paterno) as nombre, p.producto FROM
movimientosTool m JOIN usuarios u ON u.id=m.idus JOIN productos p ON
p.idProducto=m.idProd ";
    if (!empty($fecha1) && !empty($fecha2)) {
        $query .= "where m.fecha BETWEEN '". $f1. "' AND '". $f2. "' ";
    }
    $query .= " ORDER BY m.fecha DESC";

    $res = $this->Conexion->consultar($query);
    if($res)
    {
        echo json_encode($res);
        //echo $query;
    }
    else{
        echo "";
    }
}
function excelRep() {
// Genera reporte de ventas en formato Excel con múltiples filtros
$texto = $this->input->post('txtBuscar');
$parametro = $this->input->post('rbBusqueda');
$fecha1 = $this->input->post('fecha1');
$fecha2 = $this->input->post('fecha2');
$f1 = strval($fecha1);
$f2 = strval($fecha2);

```

```
// Consulta base para reporte de ventas detallado
$query = "SELECT dv.*, v.fecha, p.marca, p.proveedor, p.producto, p.precio, concat(u.nombre,
', u.paterno) as nombre, u.no_empleado FROM detalleVentTC dv JOIN usuarios u ON
dv.idUs=u.id JOIN VentToolCrib v ON dv.idVenta=v.idToolCrib JOIN productos p ON
dv.idProd=p.idProducto ";
```

```
// Aplica filtros según parámetros seleccionados
if (!empty($texto)) {
    if ($parametro == "prod") {
        $query .= " where p.producto like '$texto'";
    } else if ($parametro == "marca") {
        $query .= " where p.marca like '$texto'";
    } else if ($parametro == "prov") {
        $query .= " where p.proveedor like '$texto'";
    } else if ($parametro == "user") {
        $query .= " where u.nombre like '$texto'";
    }
} else if ($parametro = "date") {
    $query .= " where v.fecha BETWEEN '$f1' and '$f2'";
}
```

```
// Prepara y descarga archivo Excel
$result = $this->db->query($query)->result_array();
$timestamp = date('m/d/Y', time());
$filename = 'Reporte_'. $timestamp .'.xls';
header("Content-Type: application/vnd.ms-excel");
header("Content-Disposition: attachment; filename=\"$filename\"");
```

```
$isPrintHeader = false;
```

```
// Imprime filas de datos
foreach ($result as $row) {
    if (!$isPrintHeader) {
        echo implode("\t", array_keys($row)). "\n";
        $isPrintHeader = true;
    }
    echo implode("\t", array_values($row)). "\n";
}
```

```

    }

    exit;
}

function ajusteInventario() {
    $this->load->model('tool_model');

    // Realiza ajustes de inventario por entrega de productos
    $idV = $this->input->post('idV');
    $idDV = $this->input->post('idDV');
    $idProd = $this->input->post('idProd');
    $producto = $this->input->post('producto');
    $ubi = $this->input->post('ubicacion');
    $qty = $this->input->post('cantidad');

    // Obtiene stock actual en ubicación
    $datos = $this->tool_model->getUbi($idProd, $ubi);
    $cant = $datos->cantidad;
    $qtyF = $cant - $qty;

    // Verifica disponibilidad suficiente
    if ($qty > $cant) {
        echo '<script type="text/javascript">';
        echo 'alert("No hay suficientes productos en la local, elija otra local o modifique la cantidad a entregar");';
        echo 'window.history.back();';
        echo '</script>';
    } else {
        // Actualiza detalle de venta a ENTREGADO
        $data = array(
            'cantidad' => $qty,
            'estatus' => 'ENTREGADO'
        );
        $this->tool_model->ajusteDetalle($idDV, $data);

        // Actualiza stock en ubicación
        $data = array('cantidad' => $qtyF);
    }
}

```

```

$this->tool_model->ajuste($idProd, $ubi, $data);

// Registra movimiento de salida
$mov = array(
    'idProd' => $idProd,
    'idUs' => $this->session->id,
    'cantidad' => $qty,
    'local' => $ubi,
    'tipo' => 'SALIDA',
    'comentario' => $this->input->post('comentario'),
    'fecha' => date('Y-m-d'),
);
$this->tool_model->registrarMov($mov);

redirect(base_url('toolcrib/verPedido/'.$idV));
}
}

function aprobPedido() {
$this->load->model('tool_model');

// Aprueba un pedido cambiando su estado a APROBADO
$idVenta = $this->input->post('idVenta');
$prod = $this->input->post('prod');
$cant = $this->input->post('cant');
$idVD = $this->input->post('idVenta');
$idP = $this->input->post('idProd');

// Prepara datos de actualización
$data = array(
    'estatus' => 'APROBADO'
);
$res = $this->tool_model->aprobar($idVenta, $data);

// Retorna respuesta JSON y redirecciona
if ($res) {
    echo json_encode($data);
} else {

```

```

        echo "";
    }
    redirect(base_url('inicio'));
}

function excelMov() {
    // Genera reporte de movimientos en formato Excel filtrado por fechas
    $fecha1 = $this->input->post('fecha1');
    $fecha2 = $this->input->post('fecha2');
    $f1 = strval($fecha1);
    $f2 = strval($fecha2);

    // Consulta base para movimientos de inventario
    $query = "SELECT m.tipo, concat(u.nombre, ' ', u.paterno) as nombre, p.producto, m.local,
m.cantidad, m.fecha, m.comentario FROM movimientosTool m JOIN usuarios u ON u.id=m.idus
JOIN productos p ON p.idProducto=m.idProd ";

    // Aplica filtro de rango de fechas si está presente
    if (!empty($fecha1) && !empty($fecha2)) {
        $query .= " where m.fecha BETWEEN '$f1' and '$f2'";
    }

    // Prepara y descarga archivo Excel
    $result = $this->db->query($query)->result_array();
    $timestamp = date('m/d/Y', time());
    $filename = 'movimientos_'. $timestamp. '.xls';
    header("Content-type: application/vnd-ms-xls; name='excel'");
    header('Content-Disposition: attachment; filename='.$filename);

    $isPrintHeader = false;

    // Imprime filas de datos con retorno de carro
    foreach ($result as $row) {
        if (!$isPrintHeader) {
            echo implode("\t", array_keys($row)). "\r\n";
            $isPrintHeader = true;
        }
        echo implode("\t", array_values($row)). "\r\n";
    }
}

```



```

    }

    exit;
}

function autorizadores() {
    $this->load->model('conexion_model', 'Conexion');

    // Obtiene lista de usuarios autorizadores activos
    $query = "SELECT U.id, concat(U.nombre,' ',U.paterno,' ',U.materno) as Name from usuarios U
inner join privilegios P on P.usuario = U.id where U.activo = 1 and P.autorizarTC = 1";

    $res = $this->Conexion->consultar($query);
    if ($res) {
        echo json_encode($res);
    }
}
}

```

Usuarios.php

```

<?php

defined('BASEPATH') OR exit('No direct script access allowed');

class Usuarios extends CI_Controller {

    function __construct() {
        parent::__construct();
        $this->load->model('usuarios_model');
    }

    function index() {
        // Carga vista principal de catálogo de usuarios
        $this->load->view('header');
        $this->load->view('usuarios/catalogo');
    }
}

```

```

function puestos($idPuesto) {
// Obtiene usuarios por puesto específico
$datos['usuarios'] = $this->usuarios_model->getUsuarios_Puesto($idPuesto);

$this->load->view('header');
$this->load->view('usuarios/catalogo', $datos);
}

function ver($id = 0) {
$this->load->model('privilegios_model');

// Maneja ID de usuario desde POST si está presente
if (isset($_POST['id'])) {
    $id = $this->input->post('id');
}

// Obtiene datos de usuario y sus privilegios
$datos['usuario'] = $this->usuarios_model->getUsuario($id);
$datos['privilegio'] = $this->privilegios_model->getPrivilegios($id);

$this->load->view('header');
$this->load->view('usuarios/ver', $datos);
}

function alta() {
$this->load->model('privilegios_model');

// Prepara datos para formulario de alta de usuario
$datos['puestos'] = $this->privilegios_model->listadoPuestos();

$this->load->view('header');
$this->load->view('usuarios/alta_usuario', $datos);
}

function registrar() {
$this->load->model('privilegios_model');
$ACIERTOS = array();
$ERRORES = array();

```

```

// Prepara datos para nuevo usuario con valores normalizados
$data = array(
    'no_empleado' => trim(strtoupper($this->input->post('no_empleado'))),
    'nombre' => trim(strtoupper($this->input->post('nombre'))),
    'paterno' => trim(strtoupper($this->input->post('paterno'))),
    'materno' => trim(strtoupper($this->input->post('materno'))),
    'departamento' => trim(strtoupper($this->input->post('departamento'))),
    'puesto' => $this->input->post('puesto'),
    'correo' => $this->input->post('correo'),
    'password' => sha1(trim(strtoupper($this->input->post('no_empleado')))),
    'activo' => '1',
    'foto' => file_get_contents(base_url('template/images/avatar.png')),
    'password_correo' => '',
);

// Registra nuevo usuario y maneja resultado
$id_inserted = $this->usuarios_model->crear_usuario($data);

if ($id_inserted) {
    // Asigna privilegios básicos al nuevo usuario
    $this->privilegios_model->agregarPrivilegio(array('usuario' => $id_inserted));
    $acierto = array('titulo' => 'Agregar Usuario', 'detalle' => 'Se ha agregado Usuario con
Éxito');
    array_push($ACIERTOS, $acierto);
} else {
    $error = array('titulo' => 'ERROR', 'detalle' => 'Error al agregar Usuario');
    array_push($ERRORES, $error);
}

// Almacena mensajes en sesión y redirecciona
$this->session->aciertos = $ACIERTOS;
$this->session->errores = $ERRORES;
redirect(base_url('usuarios'));
}

function subirfoto() {
    // Procesa imagen en base64 y actualiza foto de perfil

```

```

$foto = $this->input->post('foto');
$foto = str_replace('data:image/png;base64,', '', $foto);
$foto = base64_decode($foto);

$data = array(
    'foto' => $foto,
);

// Actualiza foto en base de datos y sesión
if ($this->usuarios_model->updateFoto($data)) {
    $this->session->foto = $foto;
}
redirect(base_url('inicio'));
}

function foto() {
    // Carga vista para subir nueva foto de perfil
    $this->load->view('header');
    $this->load->view('subir_foto');
}

function modificar_contraseña() {
    $OLD = sha1($this->input->post('oldpass'));
    $NEW = sha1($this->input->post('newpass'));
    $NEW1 = sha1($this->input->post('newpass1'));
    $ACIERTOS = array();
    $ERRORES = array();

    $PSW_CORRECTO = $OLD == $this->session->password;
    $PSW_COINCIDEN = $NEW == $NEW1;

    //VALIDACION
    if (!$PSW_CORRECTO) {
        $error = array('titulo' => 'Contraseña Incorrecta', 'detalle' => 'La contraseña antigua es incorrecta');
        array_push($ERRORES, $error);
    }
    if (!$PSW_COINCIDEN) {

```

```

        $error2 = array('titulo' => 'Contraseñas NO Coinciden', 'detalle' => 'No coinciden
contraseñas');
        array_push($ERRORES, $error2);
    }

    //VALIDACION
    if ($PSW_CORRECTO && $PSW_COINCIDEN) {
        $this->db->set('password', $NEW);
        $this->db->where('id', $this->session->id);
        if ($this->db->update('usuarios')) {
            $this->session->password = $NEW;
            $acierto = array('titulo' => 'Contraseña Modificada', 'detalle' => 'Se ha modificado su
contraseña con Éxito');
            array_push($ACIERTOS, $acierto);
        } else {
            $error3 = array('titulo' => 'ERROR', 'detalle' => 'Error al modificar contraseña');
            array_push($ERRORES, $error3);
        }
    }

    $this->session->aciertos = $ACIERTOS;
    $this->session->errores = $ERRORES;
    redirect(base_url('inicio'));
}

/// AJAX FUNCTIONS ///

function ajax_getUsuarios() {
    // Construye consulta base para obtener usuarios
    $query = "SELECT U.id, U.nombre, U.paterno, U.materno, U.no_empleado, U.puesto as
id_puesto, U.correo, U.ultima_sesion, U.departamento, U.activo, U.jefe_directo,
U.autorizador_compras, U.autorizador_compras_venta, concat(U.nombre, ' ', U.paterno) as
User, concat(U.nombre, ' ', U.paterno, ' ', U.materno) as CompleteName, U.password_correo,
P.puesto from usuarios U inner join puestos P on P.id = U.puesto where 1 = 1";

    // Filtro por estado activo
    if (isset($_POST['activo'])) {
        $activo = $this->input->post('activo');
        if ($activo == "0") {

```

```

        $query .= " and U.activo = '1'";
    }
} else {
    $query .= " and U.activo = '1'";
}

// Filtros por texto y parámetro
if (isset($_POST['texto']) && $_POST['texto']) {
    $texto = $this->input->post('texto');

    if (isset($_POST['parametro'])) {
        $parametro = $this->input->post('parametro');

        if ($parametro == "nombre") {
            $query .= " having User like '%$texto%'";
        } else if ($parametro == "id") {
            $query .= " and U.id = '$texto'";
        } else if ($parametro == "no_empleado") {
            $query .= " and U.no_empleado = '$texto'";
        } else if ($parametro == "correo") {
            $query .= " and U.correo like '%$texto%'";
        }
    } else {
        $query .= " and U.activo = 1 having User like '%$texto%' or P.puesto like '%$texto%'";
    }
}

$query .= " order by User";

$res = $this->Conexion->consultar($query);
if ($res) {
    echo json_encode($res);
}
}

function ajax_guardarDatos() {
// Actualiza datos de usuario desde solicitud AJAX
$usuario = json_decode($this->input->post('usuario'));

```

```

    echo $this->Conexion->modificar('usuarios', $usuario, null, array('id' => $usuario->id));
}

function ajax_getPuestos() {
    // Obtiene todos los puestos disponibles
    $query = "SELECT * from puestos where 1 = 1";
    $res = $this->Conexion->consultar($query);
    echo json_encode($res);
}

// FUNCIONES DE RETORNO RÁPIDO //
function name($id) {
    // Devuelve nombre completo de usuario por ID
    $res = $this->Conexion->consultar("SELECT ifnull(concat(U.nombre, ' ', U.paterno), '') as User
from usuarios U where U.id = $id", TRUE);
    echo $res->User;
}

function photo($id) {
    // Devuelve foto de perfil en formato PNG
    $res = $this->Conexion->consultar("SELECT ifnull(U.foto, '') as Photo from usuarios U where
U.id = $id", TRUE);
    header("Content-type: image/png");
    echo $res->Photo;
}

function uploadFoto() {
    // Actualiza foto de firma desde archivo subido
    $id = $this->input->post('id');
    $datos['foto_firma'] = file_get_contents($_FILES['file']['tmp_name']);
    $this->usuarios_model->updateFirma($id, $datos);
}

function get_foto($id) {
    // Obtiene foto de usuario por ID (posible error en implementación)
    $photo = $this->Conexion->consultar("SELECT ifnull(U.foto, '') as Photo from usuarios U
where U.id = $id", TRUE);
    if ($photo) {

```

```
    header("Content-type: image/png");  
    echo $photo->file; // Nota: debería ser $photo->Photo  
} else {  
    echo "ERROR";  
}  
  
}
```